Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 1: Introduction

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

Course Outline (To be covered in 30 lectures)

1. Introduction, Prime Number Generation, Shannon's Theory of Perfect Secrecy (5)
2. Asymmetric Key Cryptosystem and related issues (3)
3. Public Key Cryptography and related concepts/methodologies (4)
4. Cryptographic Hash Functions design and implementation issues. (4)
5. Digital Signatures and related issues (5)
6. E-Mail, IP and Web security (6)
7. Malicious Programs & Firewall(3)

Text Books

1. Modern Cryptography : Theory and Practice by W Mao
2. Applied cryptography by Bruce Schiener
3. “Cryptography: Theory & Practice” D R Stinson,
4. Introduction to cryptography by Johannes A Buchmann
5. Network Security and Cryptography by Bernard Menezes

Course Outcome(s):

After learning all the units of the course, the student can

1. Understand terms related to Cryptography, Attack types
2. Different Encryption Techniques to be used
3. Asymmetric key algorithm,
4. Public key cryptography
5. Mathematical foundation of cryptography,
6. Message integrity, message authentication and authentication protocols.
7. Digital Signature Mechanism
8. Advanced topics of Cryptography.

Program Outcome(s):

- PO1: Apply knowledge of mathematics, science and algorithm in solving complex Computer engineering problems.
- PO2: Generate solutions by programming and applying techniques to analyse and interpret data.
- PO3: Design component, or processes to meet the needs with in realistic constraints.
- PO4: Comprehend professional and ethical responsibility in computing profession.
- PO5: Express effective communication skills.
- PO6: Recognize the need for, and an ability to engage in life-long learning.
- PO7: Knowledge of contemporary issues and emerging developments in computing profession.
- PO 8: Design research problems and conduct research in computing environment.

Lets Get Started !!!!

History and Overview of Cryptography

- The Concise Oxford English Dictionary defines cryptography as “the art of writing or solving codes.”
- Cryptography: The Science of creating coded messages
- Cryptanalysis: The art of breaking coded messages
- Clear text/Plaintext: The original Message
- Cipher text: The encoded message
- Key: Input to the cryptographic algorithm
- “Cryptography: Practice of the enciphering and deciphering of messages in secret code in order to render them unintelligible to all but the intended receiver

History and Overview of Cryptography

- But cryptography nowadays encompasses much more than this: it deals with
  - mechanisms for ensuring integrity,
  - techniques for exchanging secret keys,
  - protocols for authenticating users,
  - electronic auctions and elections,
  - digital cash, and more.
- Modern cryptography involves the study of mathematical techniques for securing digital information, systems, and distributed computations against adversarial attacks.

Early Cryptography

- 3500 BC: Sumerians: Cuneiform writings



Source: Internet

Early Cryptography

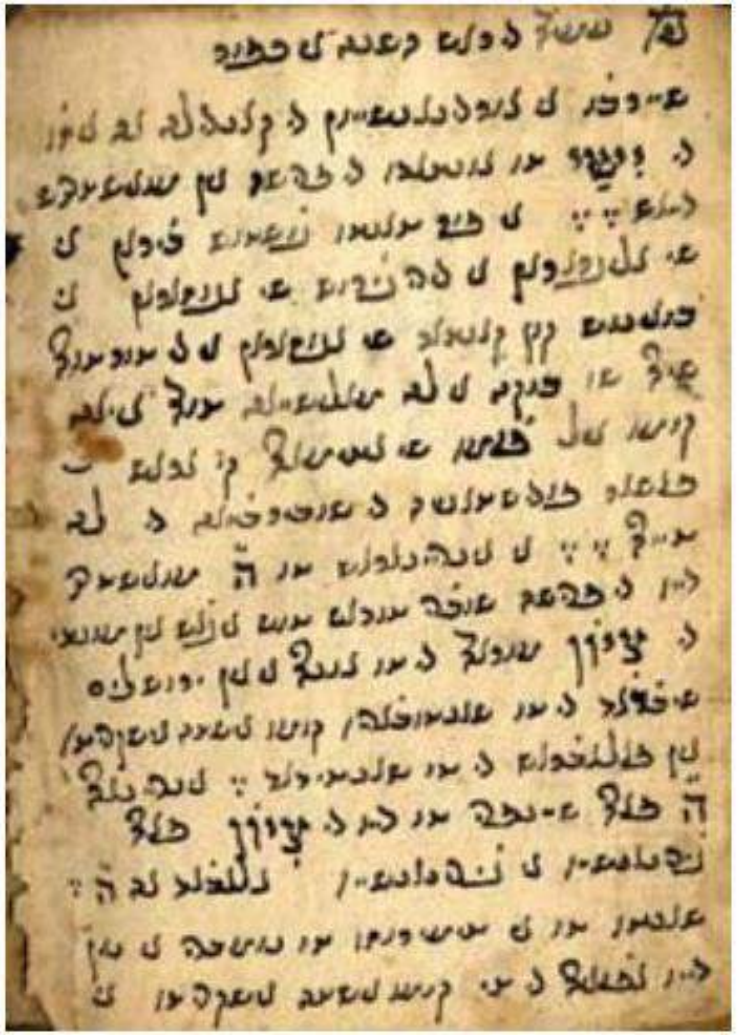
- 1900 BC: Egypt: First known use of cryptography



Source: Internet

Early Cryptography

- 500 – 600 BC: ATBASH Cipher
- Used by Hebrew scribes – Substitution cipher (reversed alphabet)



Source: Internet

Early Cryptography

- 486 BC: Greece
- σκυτάλη – skytale



Source: Internet

Early Cryptography

- 60 – 50 BC: Julius Caesar – substitution cipher – Shift letters by X positions:
- E.g. X = 3: A -> D, B -> E, C -> F,....
- Weakness?
  - Frequency analysis (1000 AD)



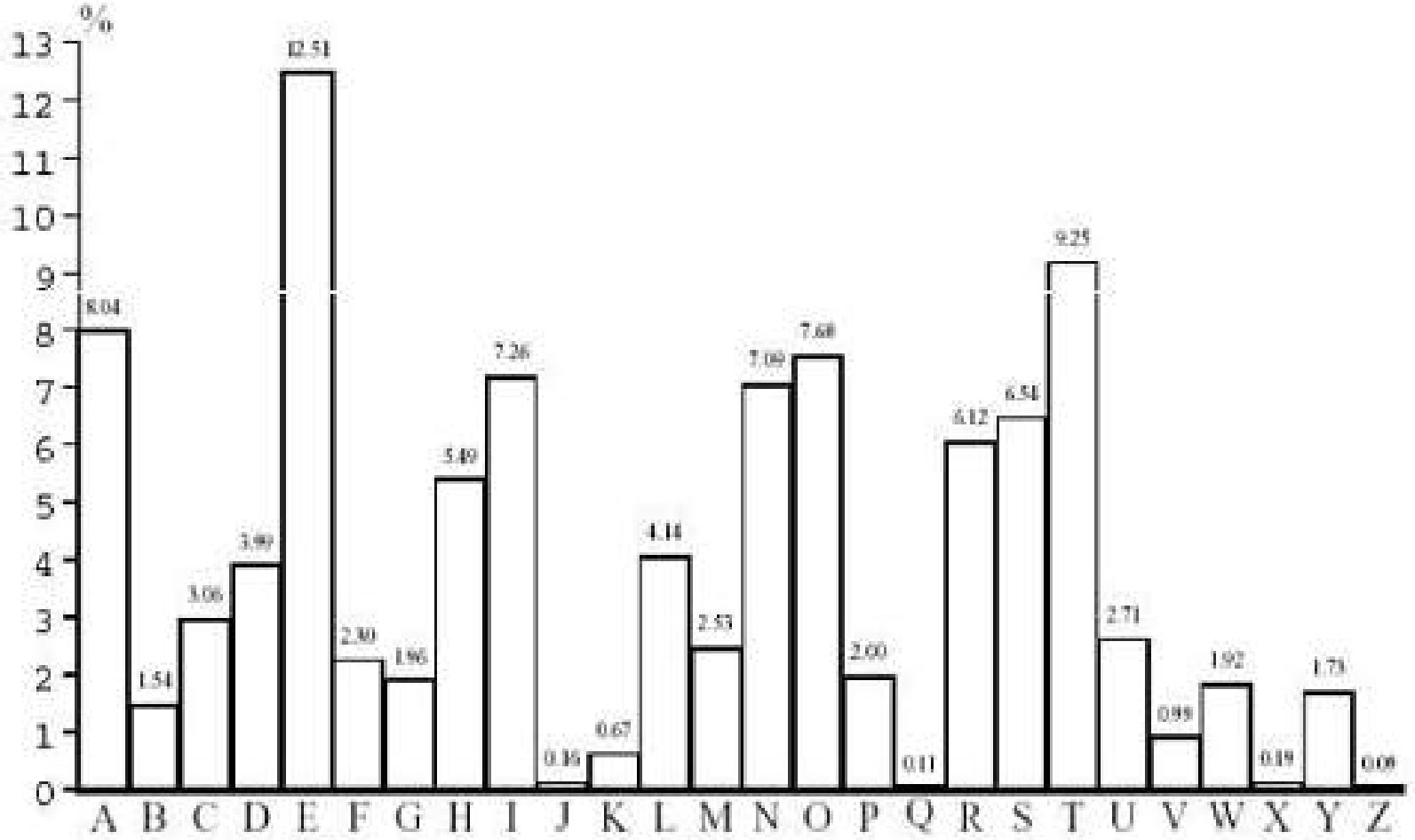
Source: Internet

Early Cryptography

- Mono-alphabetic substitution
  - letters of the plain text alphabet are mapped on to unique letters throughout the entire message text
  - cipher can be trivially broken because

    i. The language of the plain text is easily recognizable. (frequency distribution- unigram statistics- next slide)
    ii. There are only s= |A| keys (e.g. for Roman alphabet, only 25 keys – 1 to 25) to search exhaustively
    - Exhaustive key search is always possible <make it practically infeasible is the goal>

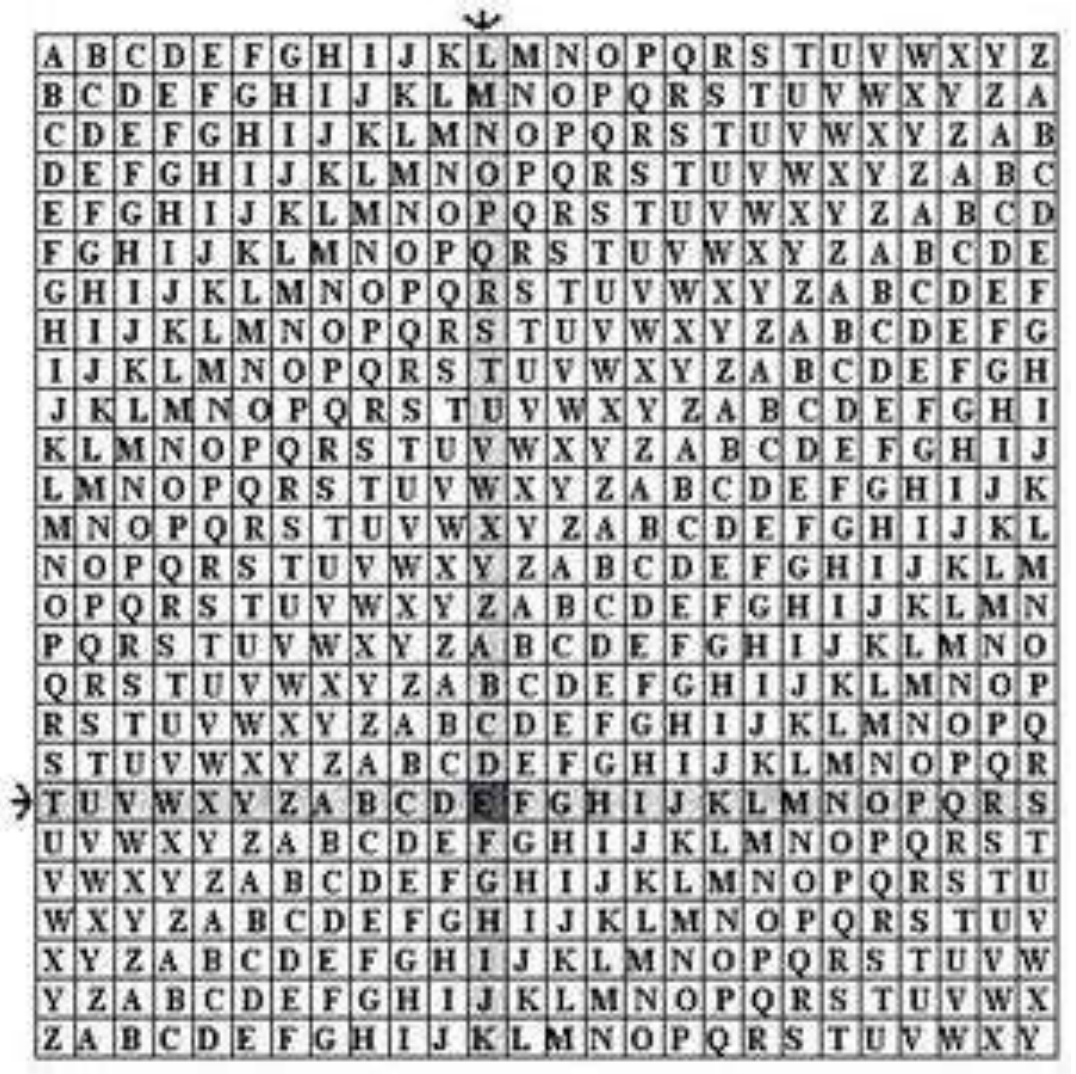
Frequency of single characters in English text



Medieval Cryptography

- 1587: Vigenère Cipher
- Polyalphabetic: one to many relationship
- Example

plaintext:	T	O	B	E	O	R	N	O	T	T	O	B	E
key:	L	T	O	B	E	O	R	N	O	T	T	O	B
ciphertext:	E	H	P	F	S	F	E	B	H	M	H	P	F



Medieval Cryptography
Vigenère Cipher

Example:
Message: "What fools these mortals be"

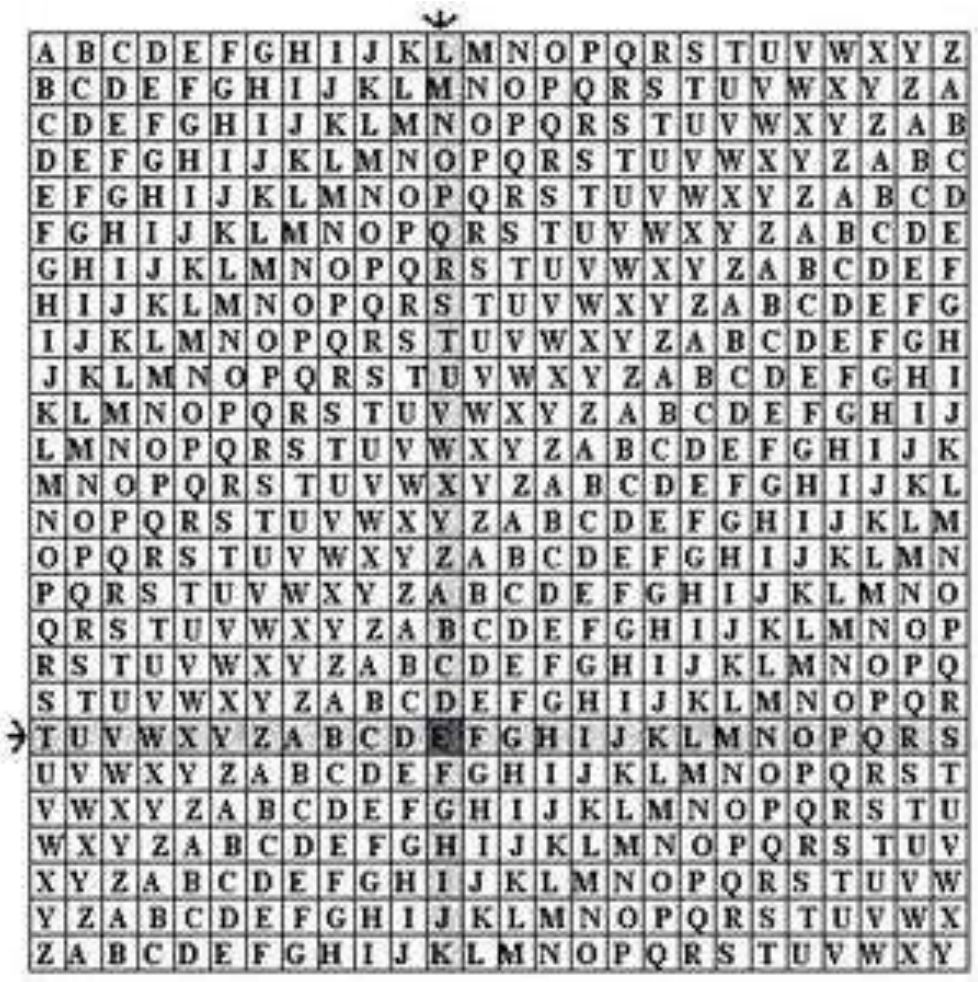
Keyword: Puck

plaintext:	W	H	A	T	F		O	O	L	S	T		H	E	S	E	M		O	R	T	A	L		S	B	E
key:	P	U	C	K	P		U	C	K	P	U		C	K	P	U	C		K	P	U	C	K		P	U	C
ciphertext:	L	B	C	D	U		I	Q	V	H	N		J	O	H	Y	O		Y	G	N	C	V		H	V	G

Example:
Message: LBCDU IQVHN JOHYO YGNCV HVG

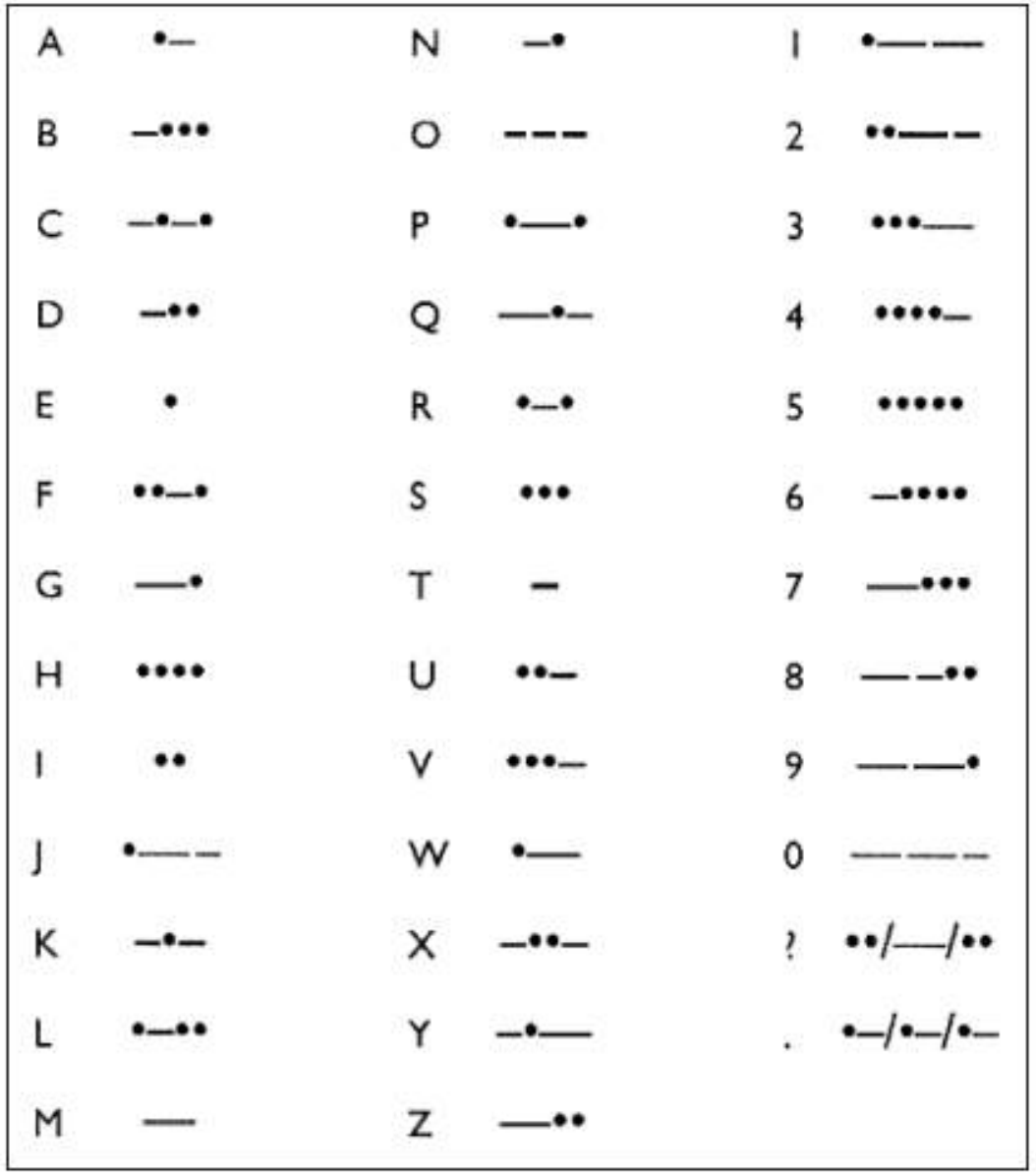
Keyword: Puck

key:	P	U	C	K	P		U	C	K	P	U		C	K	P	U	C		K	P	U	C	K		P	U	C
ciphertext:	L	B	C	D	U		I	Q	V	H	N		J	O	H	Y	O		Y	G	N	C	V		H	V	G
plaintext:	W	H	A	T	F		O	O	L	S	T		H	E	S	E	M		O	R	T	A	L		S	B	E



Modern Cryptography

- 1845: Morse Code –
- Representation by code signal
- States (on and off) composed into 5 symbols



A	•–	N	–•	1	•——
B	–•••	O	–––	2	••—–
C	–•–•	P	•––•	3	•••—–
D	–••	Q	––•–	4	••••–
E	•	R	•–•	5	•••••
F	••–•	S	•••	6	–••••
G	––•	T	–	7	––•••
H	••••	U	••–	8	––––••
I	••	V	•••–	9	––––•
J	•–––	W	•––	0	–––––
K	–•–	X	–••–	?	••/––/••
L	•–••	Y	–•––	.	•–/•–/•–
M	––	Z	––••		

Modern Cryptography

- 1863: Friedrich Kasiski breaks Vigenere: –
  - By examining repeated strings of characters in the cipher text, which could indicate the length of the secret key.
  - Find length of keyword
  - Once the length of the secret key is known, the cipher text is rewritten into a corresponding number of columns, with a column for each letter of the key.
  - Each column is then made up of plaintext that’s been encrypted by one Caesar cipher.
  - Use frequency analysis to solve these

Modern Cryptography

- 1918: ADFGVX Cipher – Used in the German army in WWI

Encrypt the plaintext "attack at 1200am"
using keywords *147 regiment* and *privacy*

	A	D	F	G	V	X
A	1	4	7	R	E	G
D	I	M	N	T	A	B
F	C	D	F	H	J	K
G	L	O	P	Q	S	U
V	V	W	X	Y	Z	0
X	2	3	5	6	8	9

a	t	t	a	c	k	a	t	1	2	0	0	a	m
DV	DG	DG	DV	FA	FX	DV	DG	AA	XA	VX	VX	DV	DD

P	R	I	V	A	C	Y
4	5	3	6	1	2	7
D	V	D	G	D	G	D
V	F	A	F	X	D	V
D	G	A	A	X	A	V
X	V	X	D	V	D	D

Cipher Text: "DXXV GDAD DAAX DVDX VFGV GFAD DVVD".

Modern Cryptography

- 1918: ADFGVX Cipher – Used in the German army in WWI

Decrypt the ciphertext "ADDDF DDAXF XAGGF DXXAX FGXFG G" which was encrypted using keywords *monkeys* and *zebras*

By reading off each row we get the intermediate text "GXFGAX XFDFDA FXDDDX GAAXDF GG".

Z	E	B	R	A	S
6	3	2	4	1	5
G	X	F	G	A	X
X	F	D	F	D	A
F	X	D	D	D	X
G	A	A	X	D	F
G	G				

	A	D	F	G	X
A	M	O	N	K	E
D	Y	S	A	B	C
F	D	F	G	H	I
G	L	P	Q	R	T
X	U	V	W	X	Z

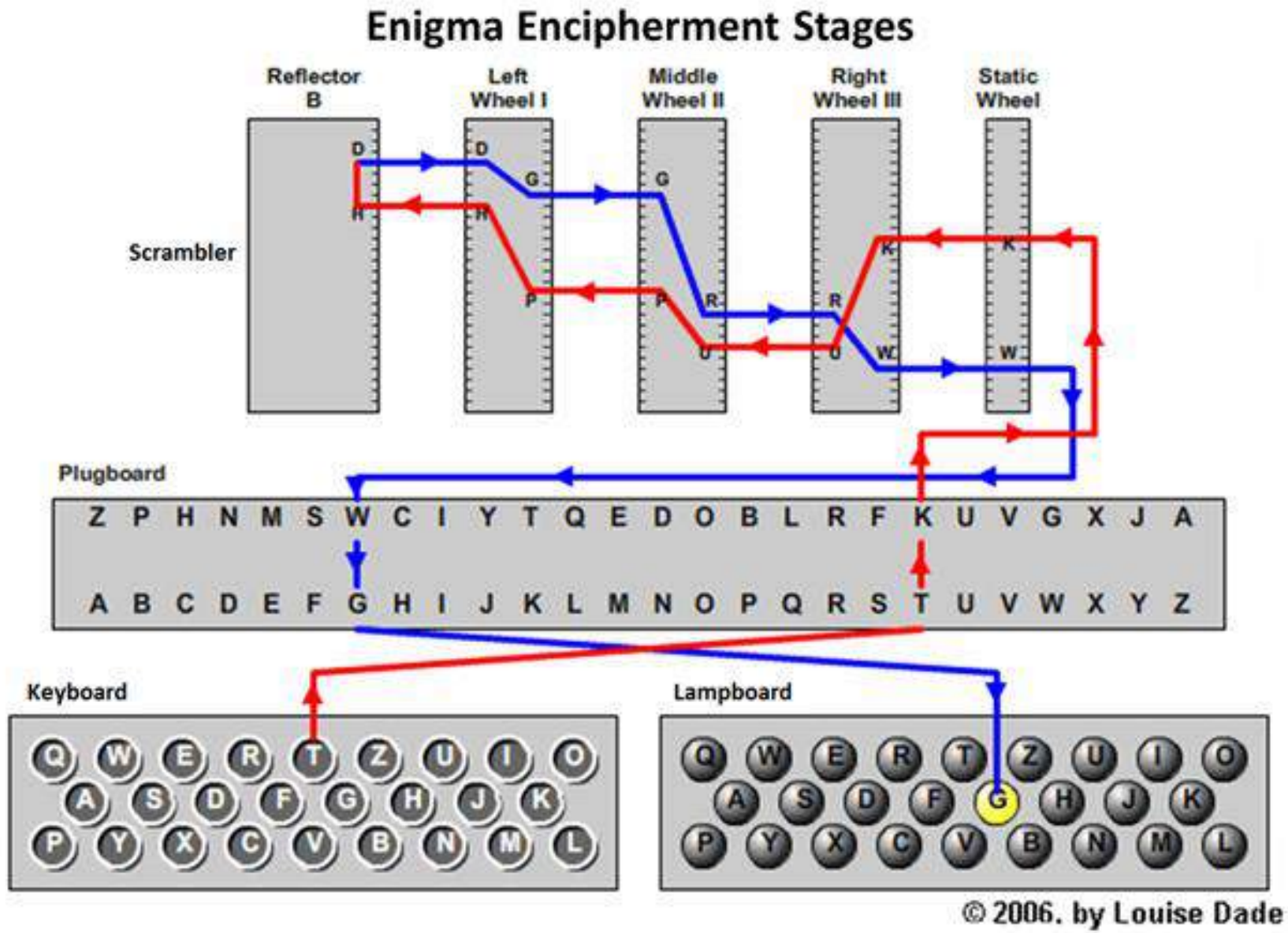
The plaintext is then retrieved as "the way is clear".

Modern Cryptography

- 1918: The Enigma – Arthur Scherbius
- Business: confidential docs
- No codebooks
- Rotors -> multi substitution
- Wireing changes as-youtype
- German forces in WWII

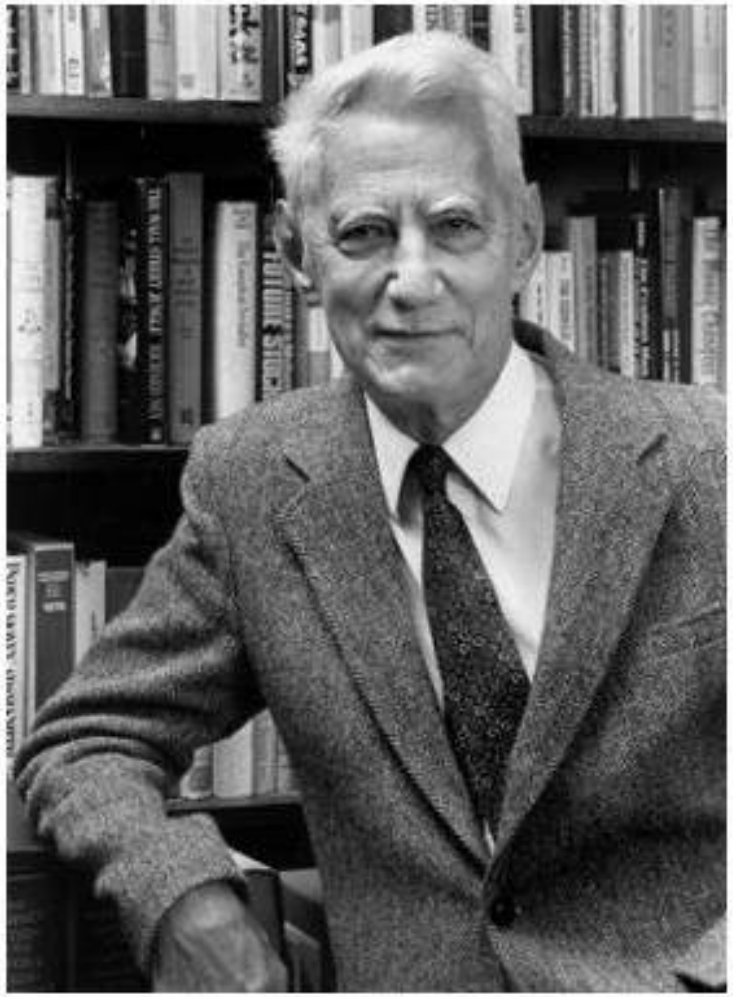


Modern Cryptography: The Enigma



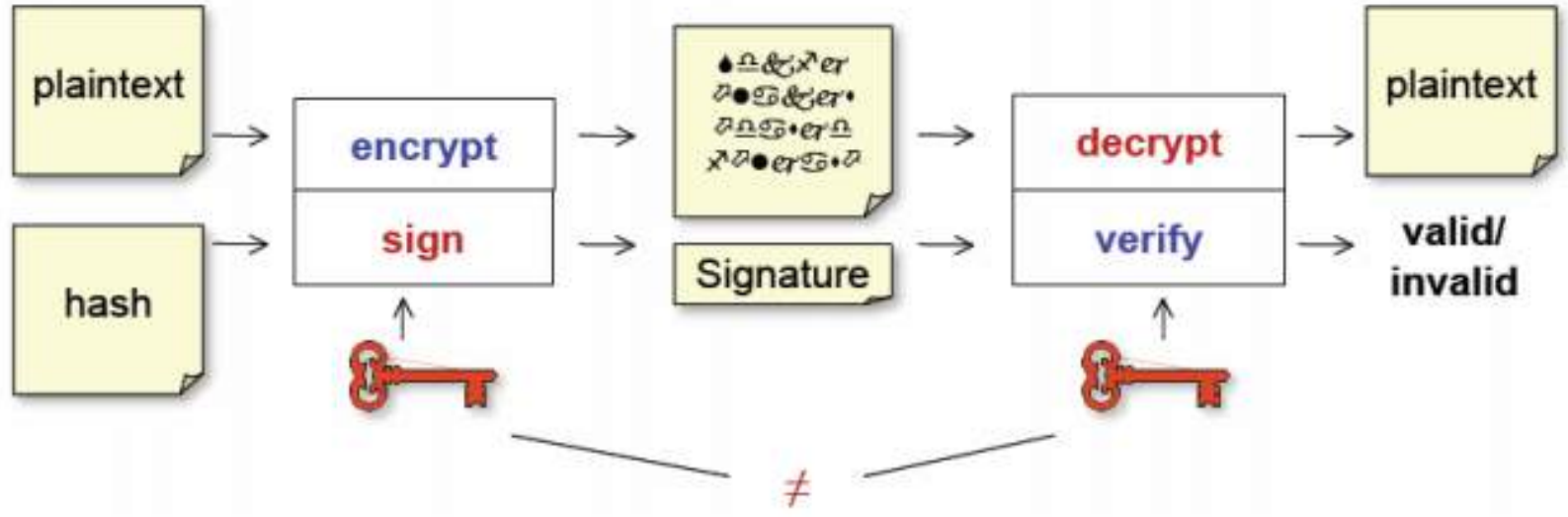
Modern Cryptography

- 1949: Shannon: –
- Communication Theory of Secret Systems
- Proved: One time pad unbreakable



Modern Cryptography

- 1976: Diffie – Hellman Key Exchange
- Public Key Crypto –
  - Key exchange problem –
  - Asymmetric key algorithm – E.g: RSA, MIT, 1977



Assignment #0

- Watch Movie “The Imitation Game”

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

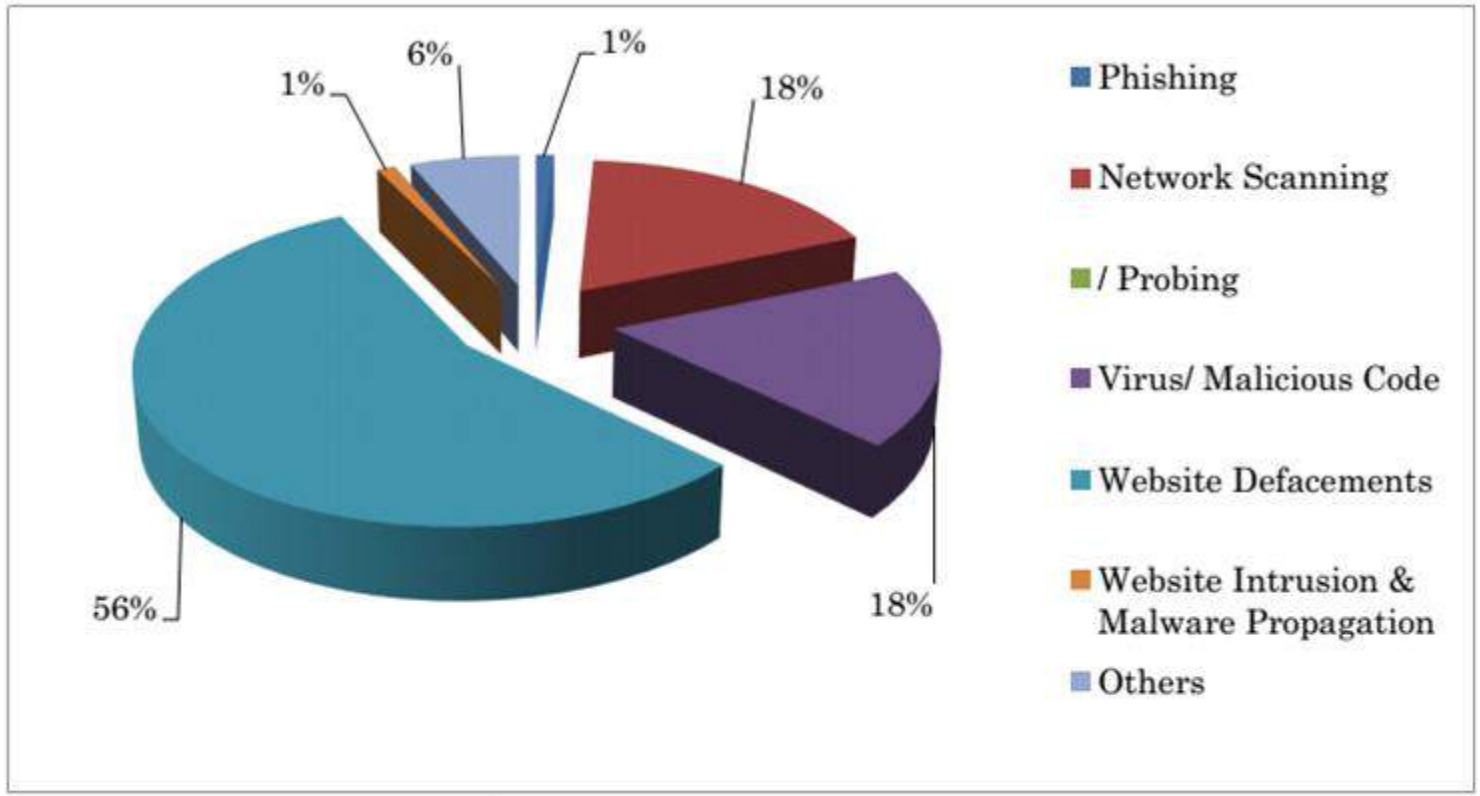
Lecture – 2a: Security Basics

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

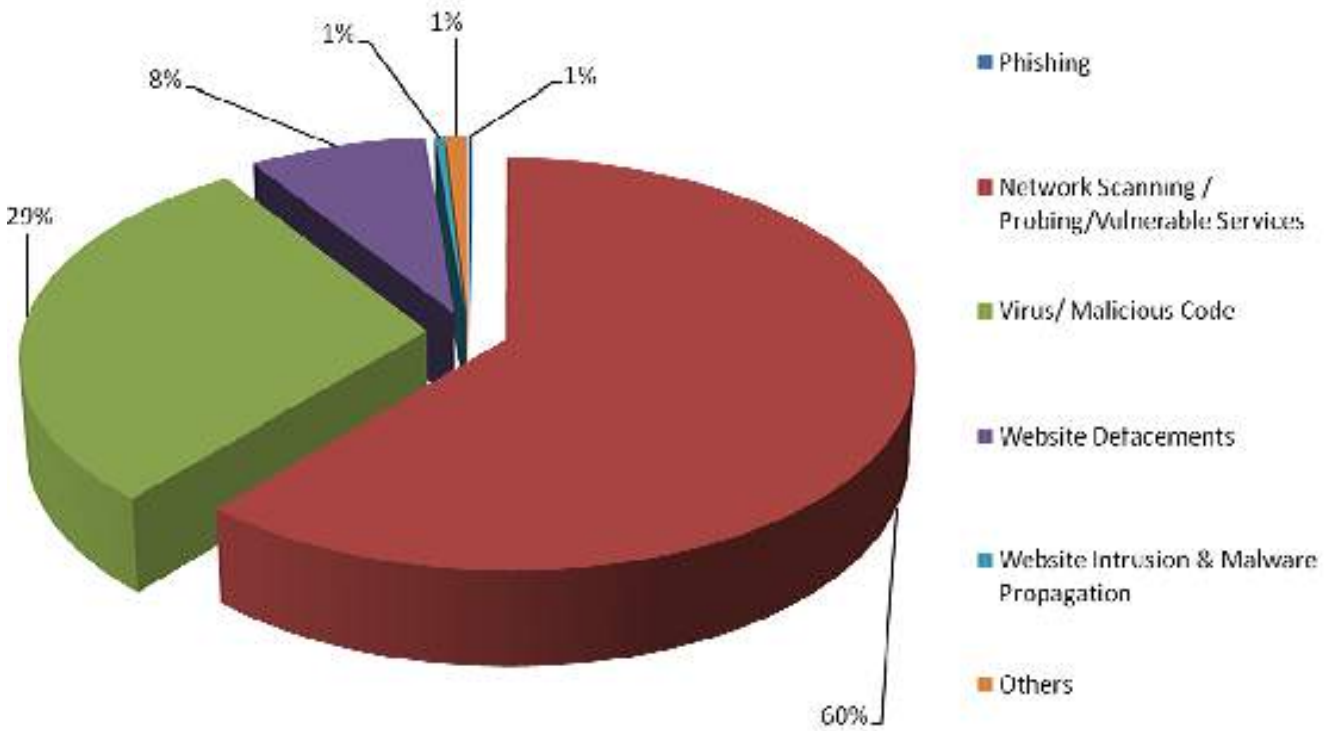
Security Trends



Summary of incidents handled by CERT-In during 2017

Security Incidents	2017
Phishing	552
Network Scanning / Probing	9383
Virus/ Malicious Code	9750
Website Defacements	29518
Website Intrusion & Malware Propagation	563
Others	3351
Total	53117

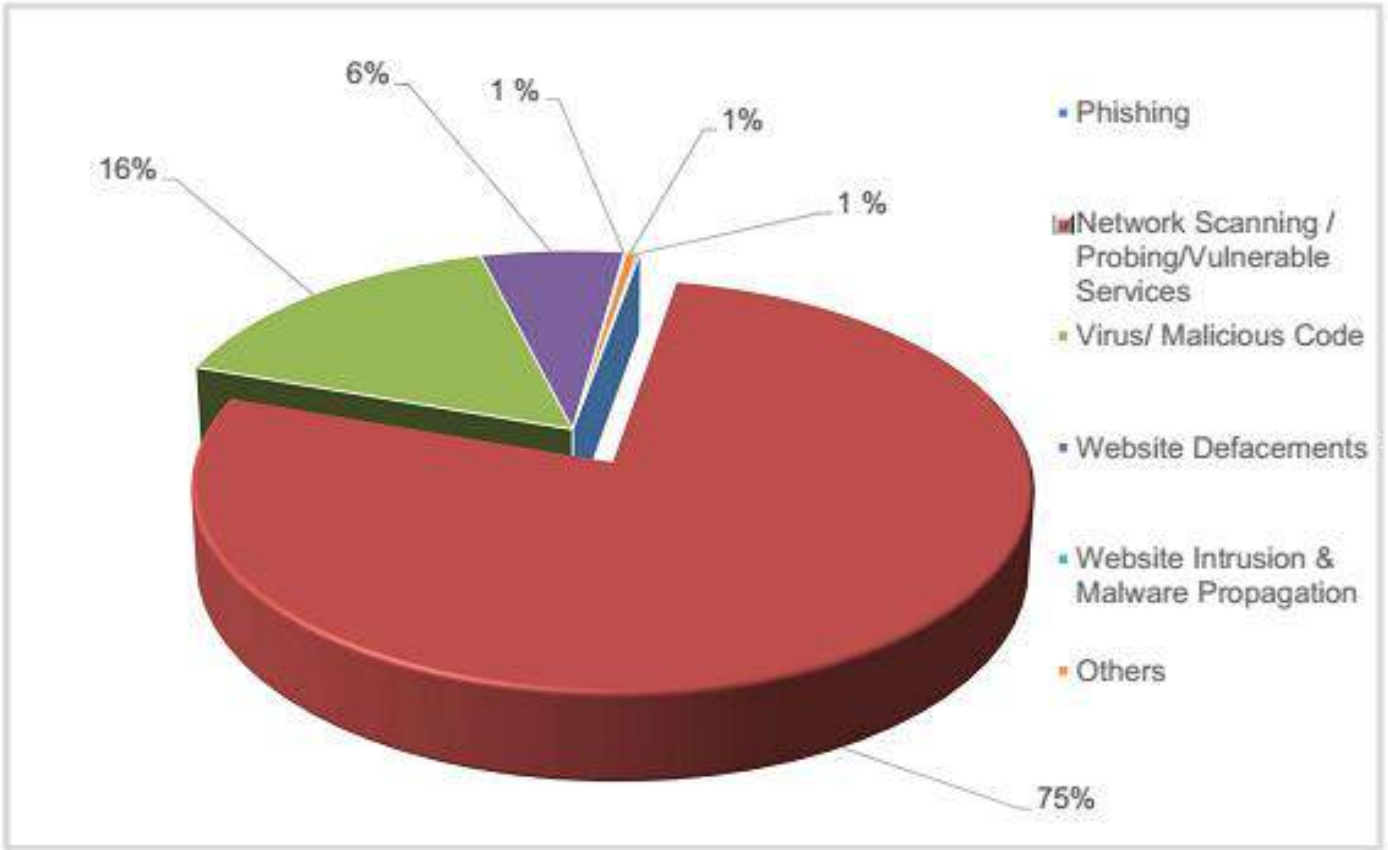
Security Trends



Summary of incidents handled by CERT-In during 2018

Security Incidents	2018
Phishing	454
Network Scanning / Probing/Vulnerable Services	127481
Virus/ Malicious Code	61055
Website Defacements	16655
Website Intrusion & Malware Propagation	905
Others	1906
Total	208456

Security Trends



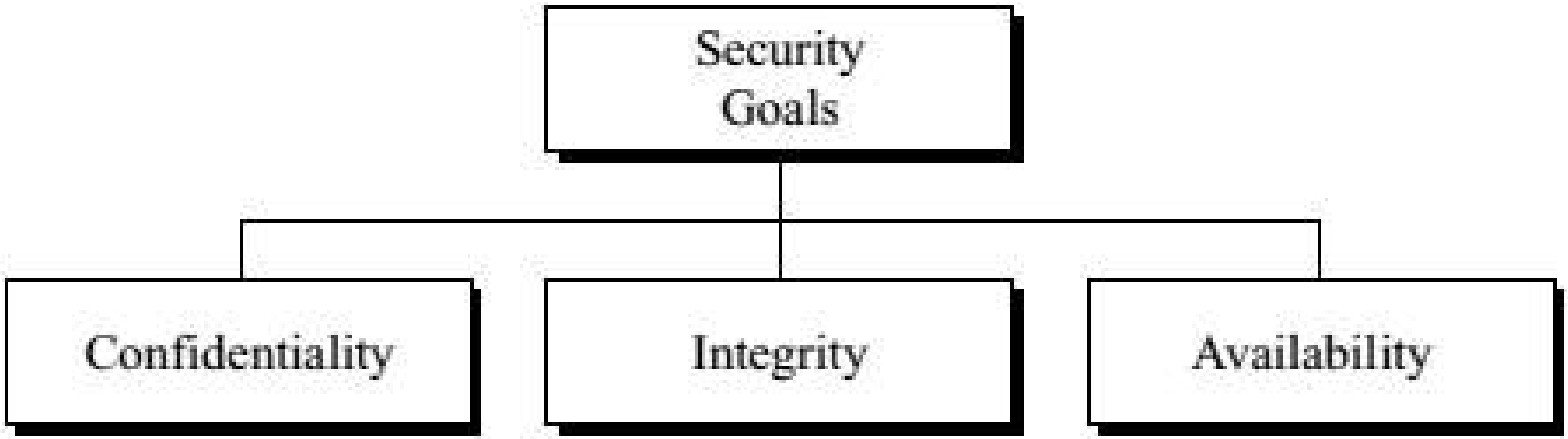
Summary of incidents handled by CERT-In during 2019

Security Incidents	2019
Phishing	472
Unauthorized Network Scanning /Probing/Vulnerable Services	305276
Virus/ Malicious Code	62163
Website Defacements	24366
Website Intrusion & Malware Propagation	417
Others	1805
Total	394499

Security Goals

- We are living in the information age.
- We need to keep information about every aspect of our lives.
- In other words, information is an asset that has a value like any other asset.
- As an asset, information needs to be secured from attacks.
- To be secured, information needs to be hidden from unauthorized access (confidentiality), protected from unauthorized change (integrity), and available to an authorized entity when it is needed (availability).

SECURITY GOALS



SECURITY GOALS





- Confidentiality
  - Need to protect our confidential information.
  - An organization needs to guard against those malicious actions that endanger the confidentiality of its information
  - In the military, concealment of sensitive information is the major concern.
  - In industry, hiding some information from competitors is crucial to the operation of the organization.
  - In banking, customers' accounts need to be kept secret.
  - Confidentiality not only applies to the storage of the information, it also applies to the transmission of information.
  - When we send a piece of information to be stored in a remote computer or when we retrieve a piece of information from a remote computer, we need to conceal it during transmission

SECURITY GOALS



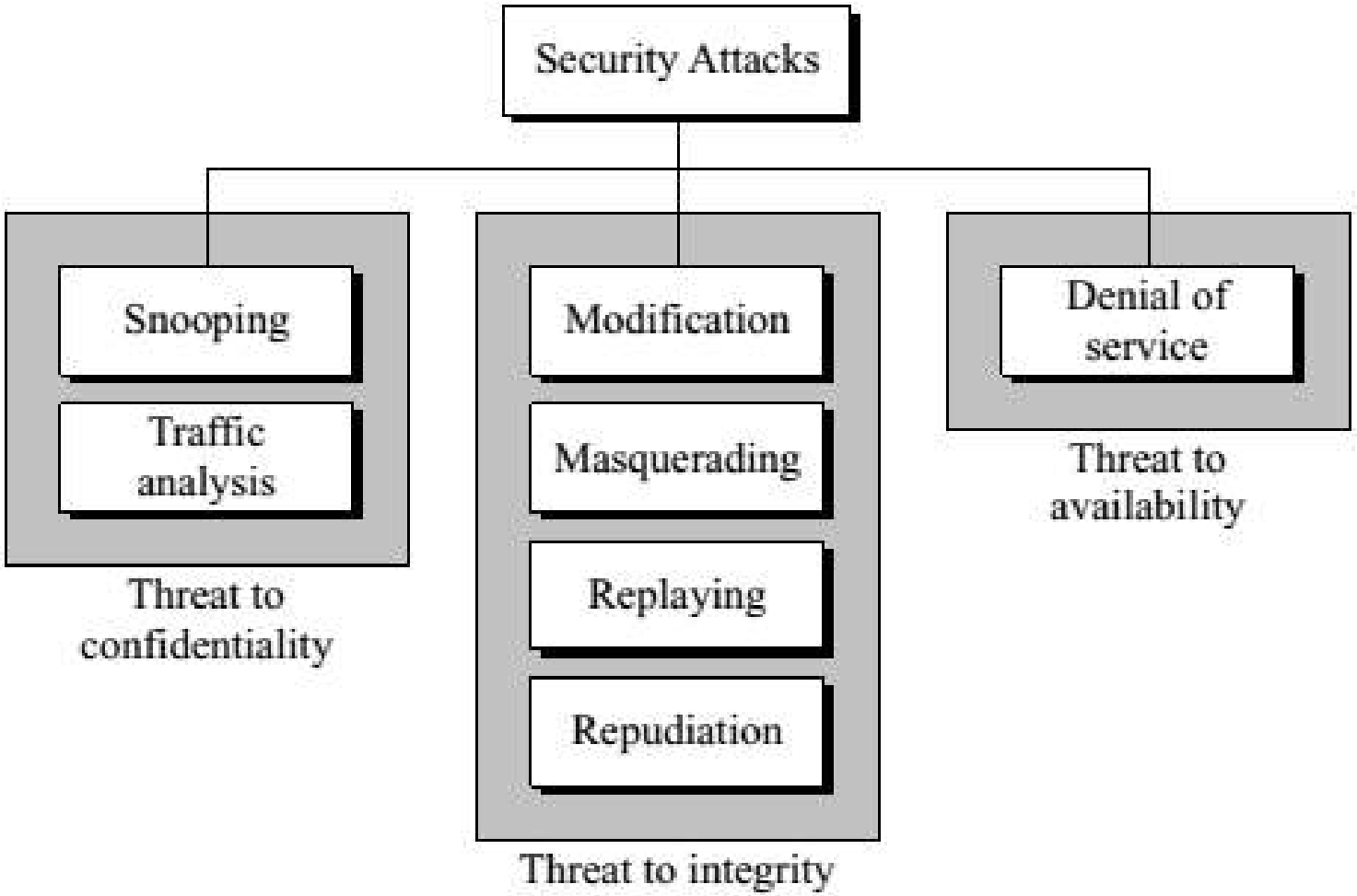
- Integrity
  - Information needs to be changed constantly.
  - In a bank, when a customer deposits or withdraws money, the balance of her account needs to be changed.
  - Integrity means that changes need to be done only by authorized entities and through authorized mechanisms.
  - Integrity violation is not necessarily the result of a malicious act.
  - An interruption in the system, such as a power surge, may also create unwanted changes in some information.

SECURITY GOALS

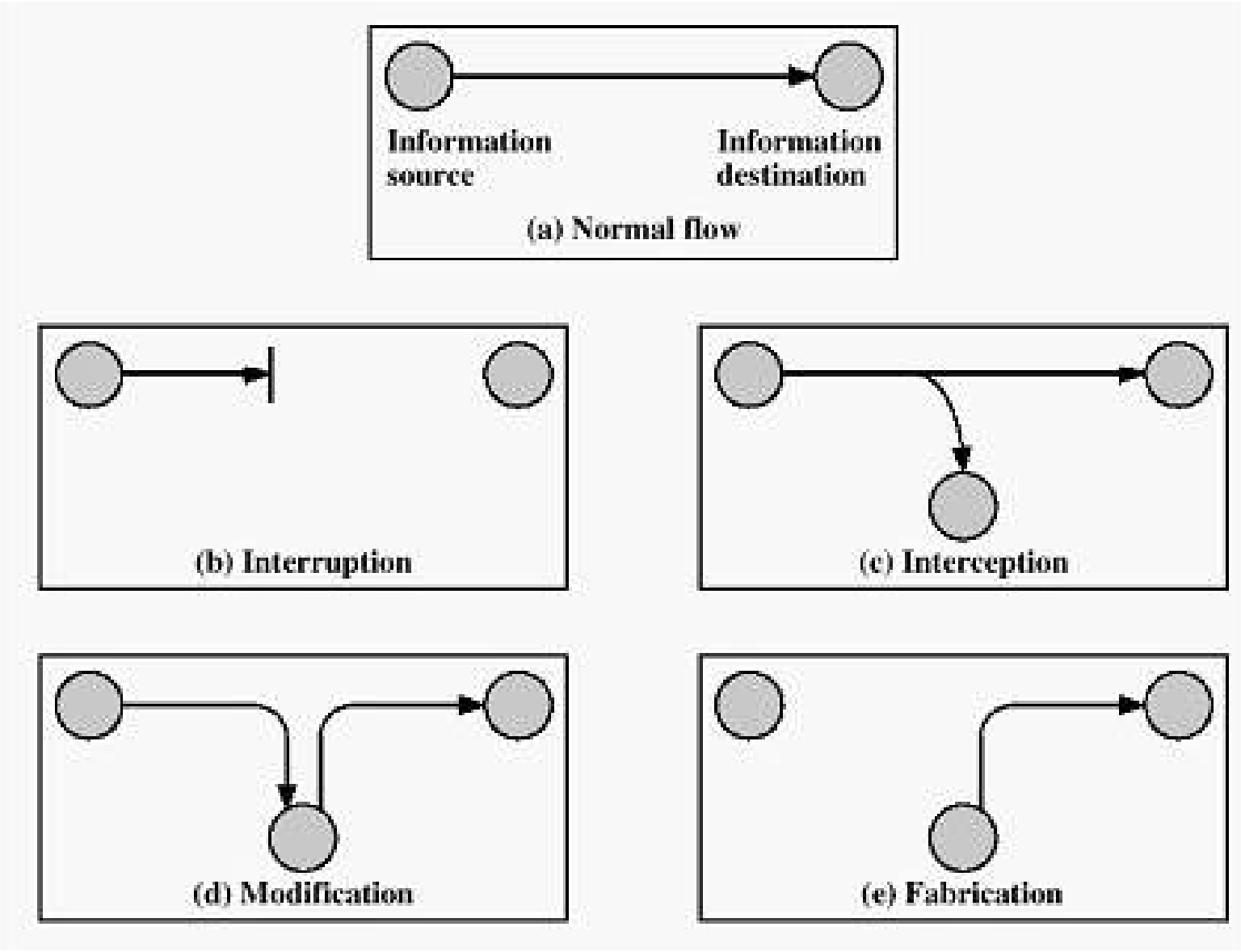


- Availability
  - The information created and stored by an organization needs to be available to authorized entities.
  - Information is useless if it is not available.
  - Information needs to be constantly changed, which means it must be accessible to authorized entities.
  - The unavailability of information is just as harmful for an organization as the lack of confidentiality or integrity.
  - Imagine what would happen to a bank if the customers could not access their accounts for transactions.

ATTACKS

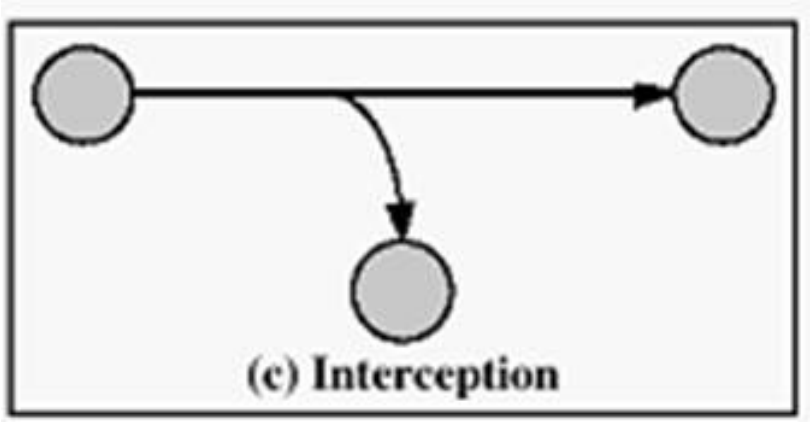

Security Attacks
Snooping
Traffic analysis
Threat to confidentiality
Modification
Masquerading
Replaying
Repudiation
Threat to integrity
Denial of service
Threat to availability

ATTACKS


Information source
Information destination
(a) Normal flow
(b) Interruption
(c) Interception
(d) Modification
(e) Fabrication

Attacks Threatening Confidentiality

- Snooping
  - Snooping refers to unauthorized access to or interception of data.
  - For example, a file transferred through the Internet may contain confidential information.
  - An unauthorized entity may intercept the transmission and use the contents for her own benefit.
  - To prevent snooping, the data can be made non-intelligible to the intercepter by using encipherment techniques



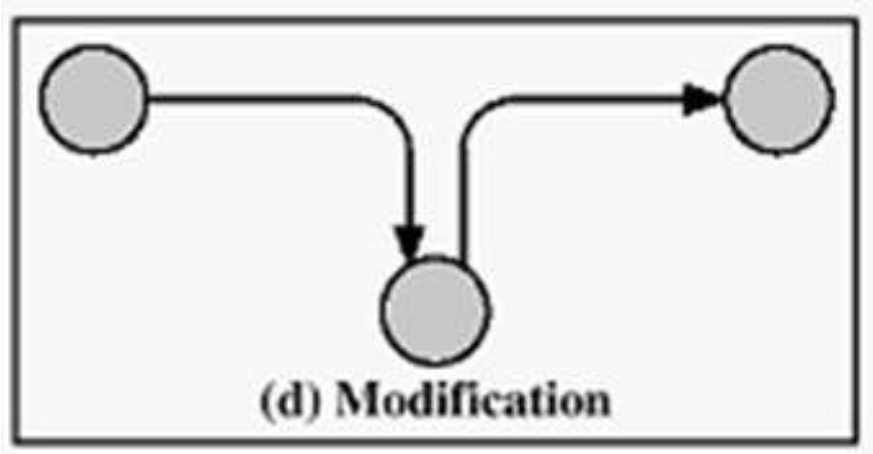
Attacks Threatening Confidentiality

- Traffic Analysis
  - Although encipherment of data may make it nonintelligible for the intercepter, one can obtain some other type information by monitoring online traffic.
  - For example, he/she can find the electronic address (such as the e-mail address) of the sender or the receiver.
  - He/She can collect pairs of requests and responses to help him/her guess the nature of transaction.



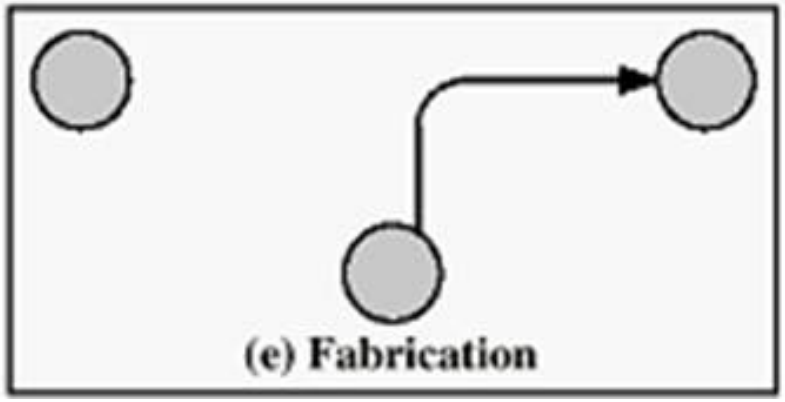
Attacks Threatening Integrity

- Modification
  - After intercepting or accessing information, the attacker modifies the information to make it beneficial to herself.
  - For example, a customer sends a message to a bank to do some transaction.
  - The attacker intercepts the message and changes the type of transaction to benefit herself.
  - Note that sometimes the attacker simply deletes or delays the message to harm the system or to benefit from it.



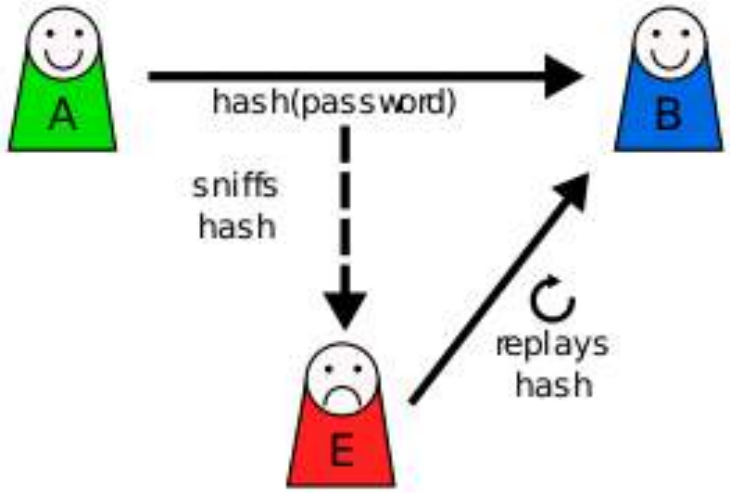
Attacks Threatening Integrity

- Masquerading
  - Masquerading, or spoofing, happens when the attacker impersonates somebody else.
  - For example, an attacker might steal the bank card and PIN of a bank customer and pretend that she is that customer.
  - Sometimes the attacker pretends instead to be the receiver entity.
  - For example, a user tries to contact a bank, but another site pretends that it is the bank and obtains some information from the user.



Attacks Threatening Integrity

- Replaying
  - Replaying is another attack.
  - The attacker obtains a copy of a message sent by a user and later tries to replay it.
  - For example, a person sends a request to her bank to ask for payment to the attacker, who has done a job for her.
  - The attacker intercepts the message and sends it again to receive another payment from the bank.

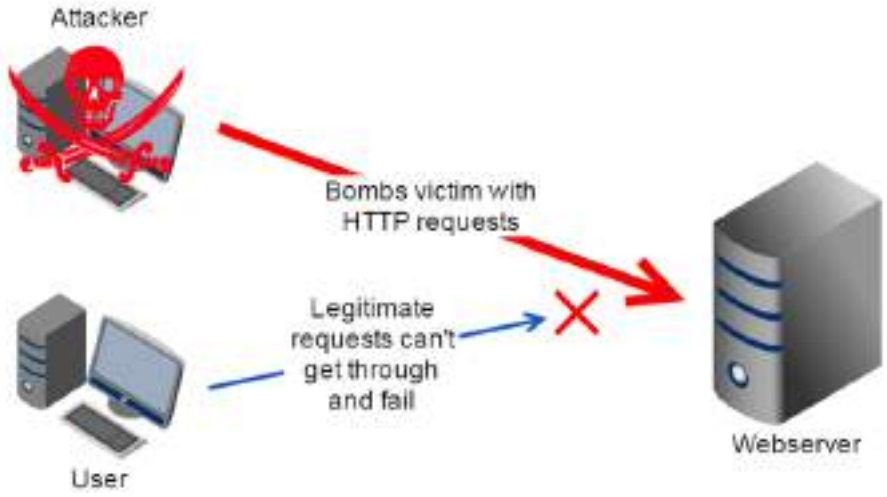


Attacks Threatening Integrity

- Repudiation
  - This type of attack is different from others because it is performed by one of the two parties in the communication: the sender or the receiver.
  - The sender of the message might later deny that she has sent the message;
  - The receiver of the message might later deny that he has received the message.
  - An example of denial by the sender would be a bank customer asking her bank to send some money to a third party but later denying that she has made such a request.
  - An example of denial by the receiver could occur when a person buys a product from a manufacturer and pays for it electronically, but the manufacturer later denies having received the payment and asks to be paid.

Attacks Threatening Availability

- Denial of Service
  - Denial of service (DoS) is a very common attack.
  - It may slow down or totally interrupt the service of a system.
  - The attacker can use several strategies to achieve this.
  - He/She might send so many bogus requests to a server that the server crashes because of the heavy load.
  - The attacker might intercept and delete a server's response to a client, making the client to believe that the server is not responding.
  - The attacker may also intercept requests from the clients, causing the clients to send requests many times and overload the system.



Passive Versus Active Attacks

- Passive Attacks
  - In a passive attack, the attacker's goal is just to obtain information.
  - This means that the attack does not modify data or harm the system.
  - However, the attack may harm the sender or the receiver of the message.
  - Attacks that threaten confidentiality, snooping and traffic analysis, are passive attacks.
  - The revealing of the information may harm the sender or receiver of the message, but the system is not affected.
  - For this reason, it is difficult to detect this type of attack until the sender or receiver finds out about the leaking of confidential information.
  - Passive attacks, however, can be prevented by encipherment of the data.

Passive Versus Active Attacks

- Active Attacks
  - An active attack may change the data or harm the system.
  - Attacks that threaten the integrity and availability are active attacks.
  - Active attacks are normally easier to detect than to prevent, because an attacker can launch them in a variety of ways.

Passive Versus Active Attacks

Attacks	*Passive/Active*	*Threatening*
Snooping Traffic analysis	Passive	Confidentiality
Modification Masquerading Replaying Repudiation	Active	Integrity
Denial of service	Active	Availability

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 2b: Introduction to Cryptography Mathematics

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

Integer Arithmetic

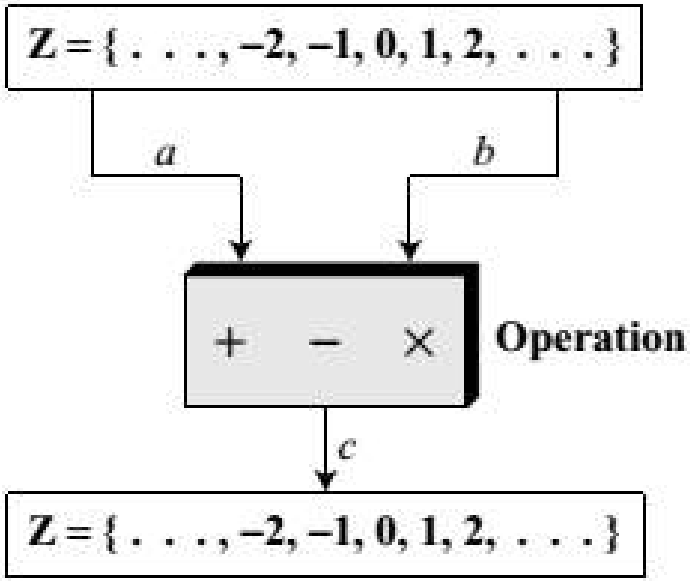
- In integer arithmetic, we use a set and a few operations.
- Reviewed here to create a background for modular arithmetic.
- The set of integers, denoted by Z, contains all integral numbers (with no fraction) from negative infinity to positive infinity

$$\mathbf{Z} = \{ \ldots, -2, -1, 0, 1, 2, \ldots \}$$

The set of integers

Binary Operations

- In cryptography, we are interested in three binary operations applied to the set of integers.
- A binary operation takes two inputs and creates one output.



Example #1

Add:	$5 + 9 = 14$	$(-5) + 9 = 4$	$5 + (-9) = -4$	$(-5) + (-9) = -14$
Subtract:	$5 - 9 = -4$	$(-5) - 9 = -14$	$5 - (-9) = 14$	$(-5) - (-9) = +4$
Multiply:	$5 \times 9 = 45$	$(-5) \times 9 = -45$	$5 \times (-9) = -45$	$(-5) \times (-9) = 45$

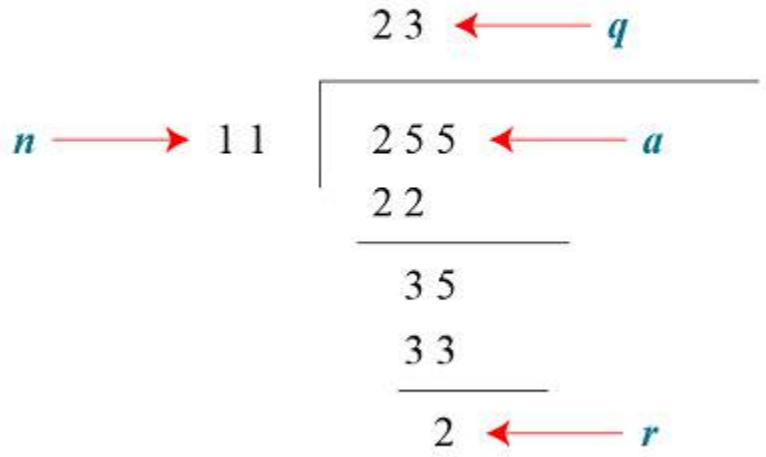
Integer Division

- In integer arithmetic, if we divide a by n, we can get q and r.
- The relationship between these four integers can be shown as

$$a = q \times n + r$$

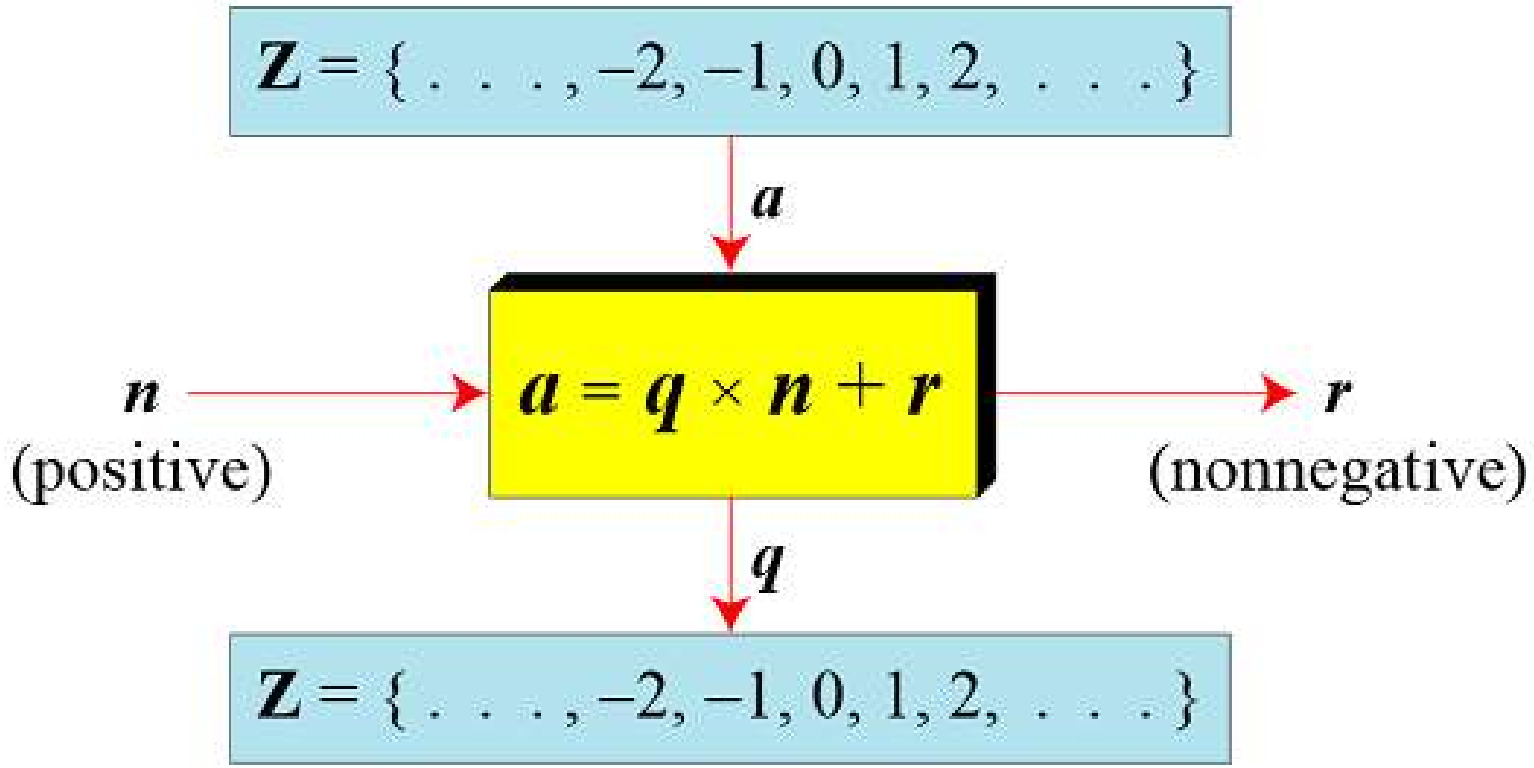
Example #2

Assume that a = 255 and n = 11. We can find q = 23 and r = 2 using the division algorithm.



Finding the quotient and the remainder

Integer Division



Division algorithm for integers

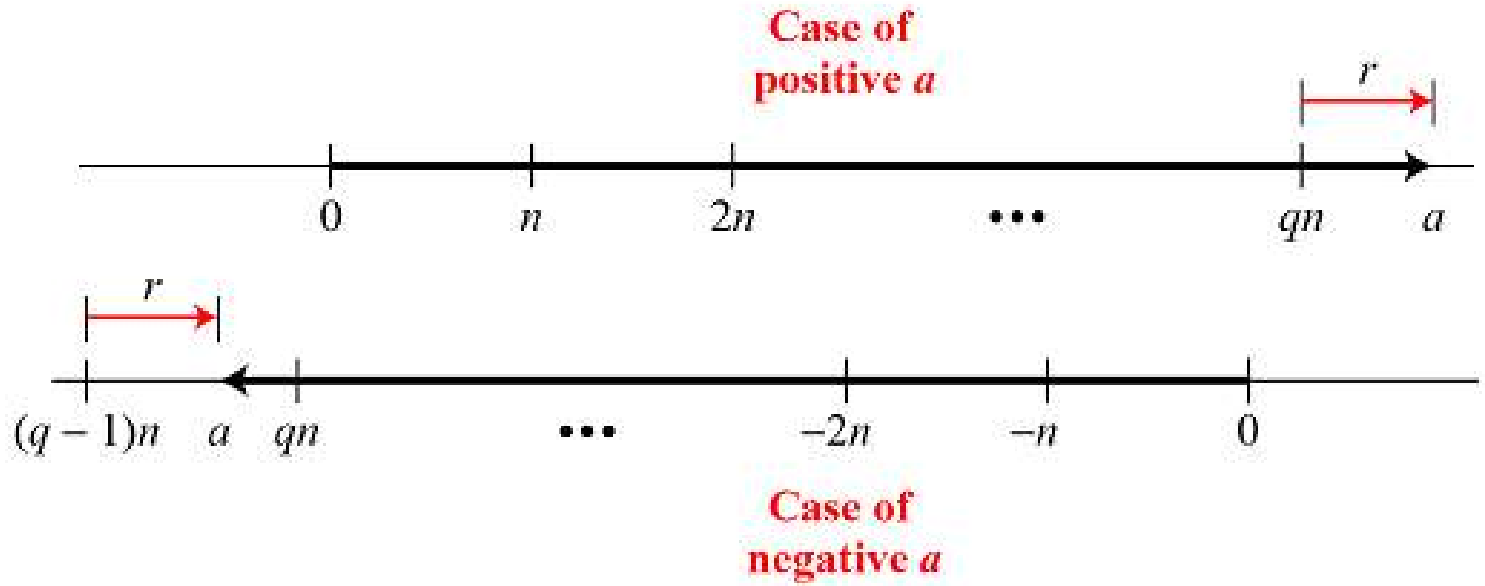
Integer Division

- When we use a computer or a calculator, r and q are negative when a is negative.
- How can we apply the restriction that r needs to be positive?
- The solution is simple, we decrement the value of q by 1 and we add the value of n to r to make it positive.

$$-255 = (\mathbf{-23} \times 11) + (\mathbf{-2}) \quad \leftrightarrow \quad -255 = (\mathbf{-24} \times 11) + \mathbf{9}$$

Integer Division

- Graph of division algorithm



Divisibility

- If a is not zero and we let r = 0 in the division relation, we get

$$a = q \times n$$

- If the remainder is zero, $n \mid a$
- If the remainder is not zero, $n \nmid a$

Example #3

a. The integer 4 divides the integer 32 because $\mathbf{32} = 8 \times \mathbf{4}$. We show this as $4|32$.

b. The number 8 does not divide the number 42 because $\mathbf{42} = 5 \times \mathbf{8} + 2$. There is a remainder, the number 2, in the equation. We show this as $8 \nmid 42$.

Divisibility

- Properties

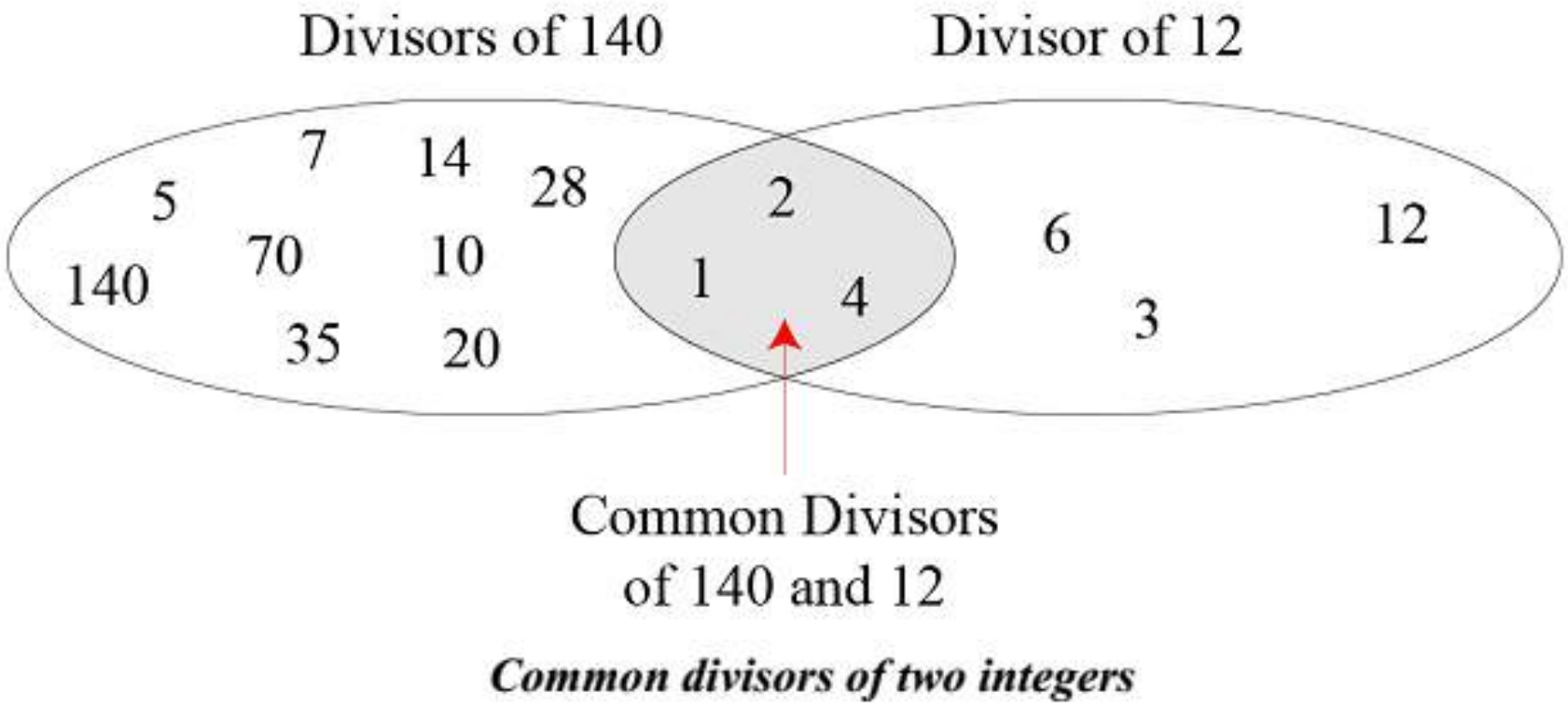
Property 1: if $a|1$, then $a = \pm 1$.

Property 2: if $a|b$ and $b|a$, then $a = \pm b$.

Property 3: if $a|b$ and $b|c$, then $a|c$.

Property 4: if $a|b$ and $a|c$, then $a|(m \times b + n \times c)$, where m and n are arbitrary integers

Divisibility



Common divisors of two integers

Divisibility

Greatest Common Divisor

The greatest common divisor of two positive integers is the largest integer that can divide both integers.

Euclidean Algorithm

Fact 1: gcd (a, 0) = a
Fact 2: gcd (a, b) = gcd (b, r), where r is the remainder of dividing a by b

Divisibility

• For example, to calculate the gcd(36,10), we use following steps:

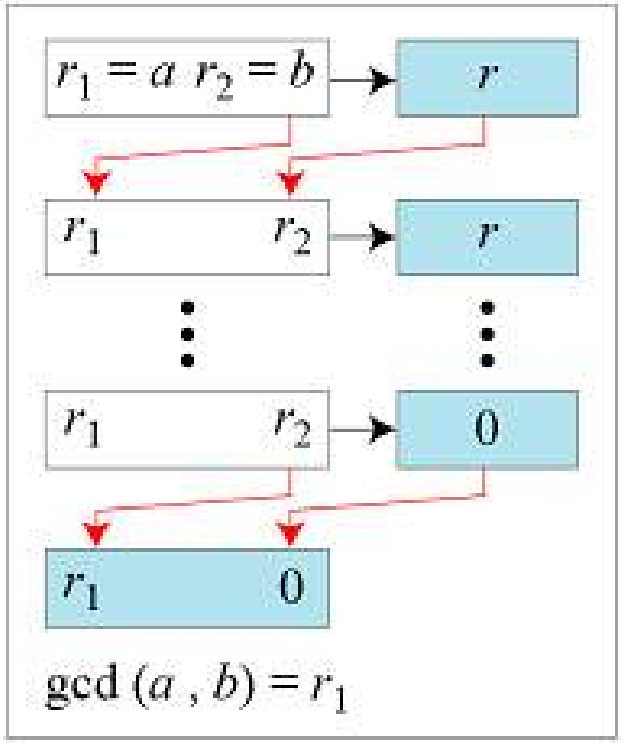
gcd (36, 10) = gcd (10, 6).....by fact 2
gcd (10, 6) = gcd (6, 4).....by fact 2
gcd (6, 4) = gcd (4, 2).....by fact 2
gcd (4, 2) = gcd (2, 0).....by fact 2
gcd (2, 0) = 2.....by fact 1

Hence, Answer = 2

Example #4

Divisibility

Euclidean Algorithm



a. Process

$r_1 \leftarrow a;$ $r_2 \leftarrow b;$ (Initialization)

while $(r_2 > 0)$

{

$q \leftarrow r_1 / r_2;$

$r \leftarrow r_1 - q \times r_2;$

$r_1 \leftarrow r_2;$ $r_2 \leftarrow r;$

}

gcd $(a, b) \leftarrow r_1$

b. Algorithm

When gcd (a, b) = 1, we say that a and b are relatively prime.

Divisibility

Find the greatest common divisor of 2740 and 1760.

q	r_1	r_2	r
1	2740	1760	980
1	1760	980	780
1	980	780	200
3	780	200	180
1	200	180	20
9	180	20	0
	20	0	

Example #5

Answer: gcd (2740, 1760) = 20.

Divisibility

Class Exercise #1

Find the greatest common divisor of 25 and 60.

Divisibility

- **Extended Euclidean Algorithm**

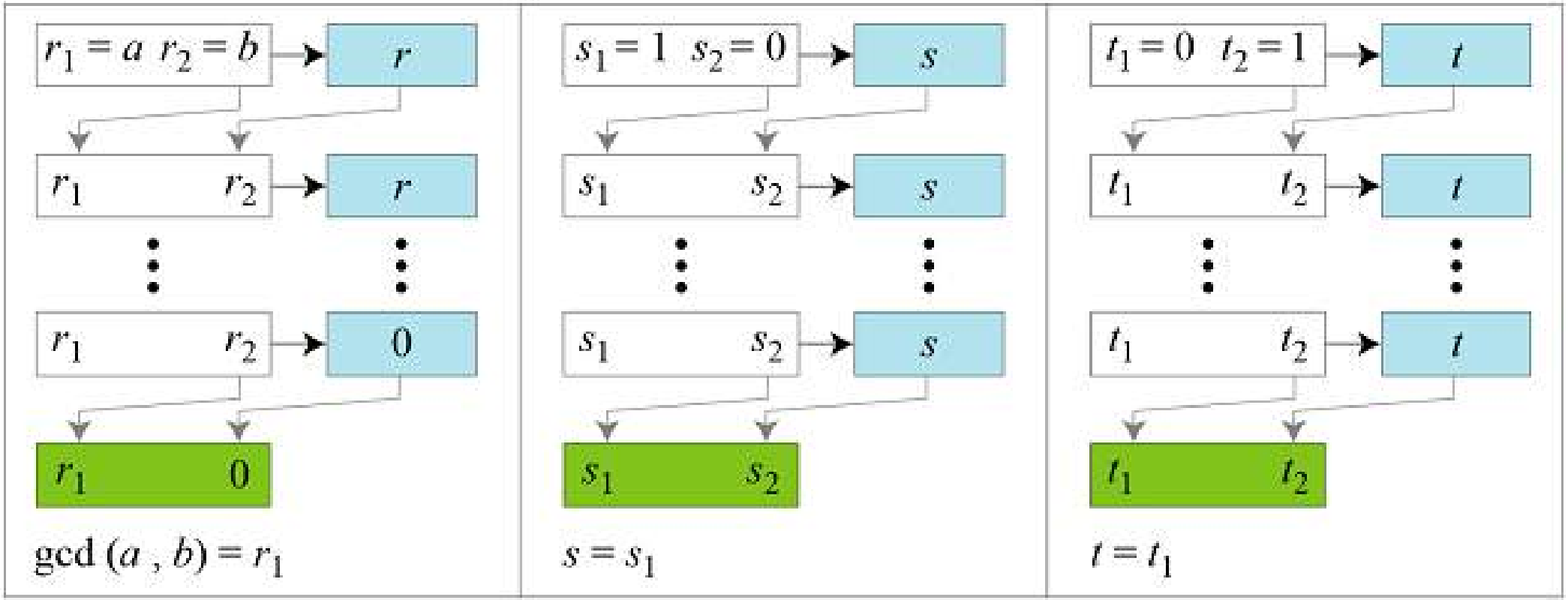
  Given two integers *a* and *b*, we often need to find other two integers, *s* and *t*, such that

$$s \times a + t \times b = \gcd(a, b)$$

  The extended Euclidean algorithm can calculate the gcd (a, b) and at the same time calculate the value of s and t.

Divisibility

Extended Euclidean algorithm, part a



a. Process

Divisibility

Extended Euclidean algorithm, part b

$r_1 \leftarrow a$; $r_2 \leftarrow b$;
$s_1 \leftarrow 1$; $s_2 \leftarrow 0$;
$t_1 \leftarrow 0$; $t_2 \leftarrow 1$;

(Initialization)

while ($r_2 > 0$)

{

$q \leftarrow r_1 / r_2$;

$r \leftarrow r_1 - q \times r_2$;
$r_1 \leftarrow r_2$; $r_2 \leftarrow r$;

(Updating r's)

$s \leftarrow s_1 - q \times s_2$;
$s_1 \leftarrow s_2$; $s_2 \leftarrow s$;

(Updating s's)

$t \leftarrow t_1 - q \times t_2$;
$t_1 \leftarrow t_2$; $t_2 \leftarrow$ t;

(Updating t's)

}

gcd $(a, b) \leftarrow r_1$; $s \leftarrow s_1$; $t \leftarrow t_1$

b. Algorithm

Divisibility

Given $a = 161$ and $b = 28$, find gcd (a, b) and the values of s and t.

Solution

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
5	161	28	21	1	0	1	0	1	−5
1	28	21	7	0	1	−1	1	−5	6
3	21	7	0	1	−1	4	−5	6	−23
	7	0		**−1**	4		**6**	−23	

We get gcd (161, 28) = 7, $s = -1$ and $t = 6$.

Divisibility

Given $a = 17$ and $b = 0$, find gcd (a, b) and the values of s and t.

Solution

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
	17	0		**1**	**0**		**0**	**1**	

We get gcd (17, 0) = 17, $s = 1$, and $t = 0$

Divisibility

Given $a = 0$ and $b = 45$, find gcd (a, b) and the values of s and t.

Solution

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
0	0	45	0	1	0	1	0	1	0
	45	0		0	1		**1**	0	

We get gcd (0, 45) = 45, $s = 0$, and $t = 1$.

Divisibility

Exercise:

Given $a = 84$ and $b = 320$, find gcd (a, b) and the values of s and t.

Solution:

gcd(84,320) = 4, s = -19, t = 5

Linear Diophantine Equations

- **A linear Diophantine equation of two variables is,**
  $$ax + by = c.$$
- **We want to find integer values for x and y that satisfy the equation.**
- **Either no solution or an infinite number of solutions**
- **Let d = gcd(a,b); if $d \nmid c$, the equation has no solution.**
- **If $d \mid c$, the equation has infinite number of solutions : one of them is particular and the rest are general**

Linear Diophantine Equations(cont.)

Particular solution:

$x_0 = (c/d)s$ and $y_0 = (c/d)t$

General solutions:

$x = x_0 + k\,(b/d)$ and $y = y_0 - k(a/d)$

where k is an integer

Linear Diophantine Equations(cont.)

Example:

Find the particular and general solutions for the equation

$21x + 14y = 35$.

Linear Diophantine Equations(cont.)

- d = gcd (21, 14) = 7.
- Since 7| 35, the equation has an infinite number of solutions.
- We can divide both sides by 7 to find the equation 3x + 2y = 5.
- Using the extended Euclidean algorithm, we find s and t such as 3s + 2t = 1. We have s = 1 and t = −1.

Particular: $x_0 = 5 \times 1 = 5$ and $y_0 = 5 \times (-1) = -5$ since 35/7 = 5
General: $x = 5 + k \times 2$ and $y = -5 - k \times 3$ where k is an integer

Therefore, the solutions are (5, −5), (7, −8), (9, −11), . . . We can easily test that each of these solutions satisfies the original equation.

Linear Diophantine Equations(cont.)

Example:

Imagine we want to cash a Rs.100 cheque and get some Rs.20 notes and some Rs.5 notes.

Find out the possible choices if any exist for the given problem

Linear Diophantine Equations(cont.)

$d = \gcd(20, 5) = 5$ and $5 \mid 100$, infinite number of solutions.

$$4s + t = 1.$$

The particular solutions are $x_0 = 0 \times 20 = 0$ and $y_0 = 1 \times 20 = 20$.

$$(0, 20), (1, 16), (2, 12), (3, 8), (4, 4), (5, 0).$$

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 3: Introduction to Cryptography Mathematics- Part-2

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

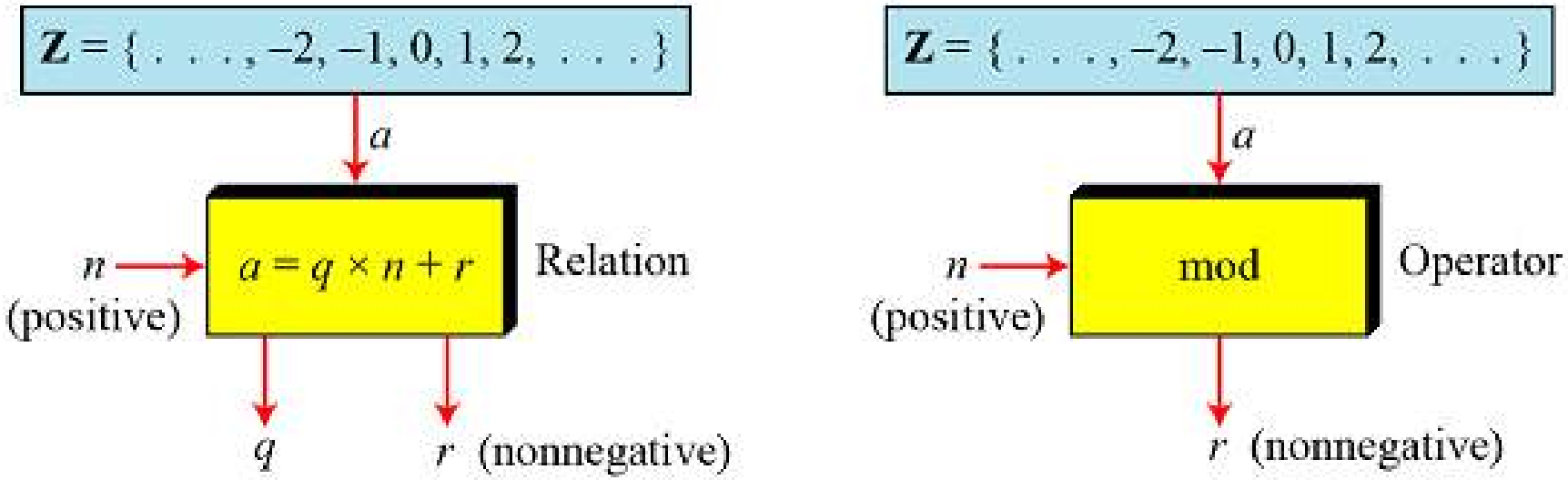
Modular Arithmetic

Preliminary

- The division relationship ($a = q \times n + r$) discussed in the previous section has two inputs (a and n) and two outputs (q and r).
- In modular arithmetic, we are interested in only one of the outputs, the remainder r.
- We use modular arithmetic in our daily life;
  - for example, we use a clock to measure time.
  - Our clock system uses modulo 12 arithmetic. However, instead of a 0 we use the number 12

Modulo Operator

- The modulo operator is shown as mod.
- The second input (n) is called the modulus.
- The output r is called the residue.



Division algorithm and modulo operator

Modulo Operator(cont.)

- Find the result of the following operations:
  a. 27 mod 5
  b. 36 mod 12
  c. −18 mod 14
  d. −7 mod 10
- Solution
  a. Dividing 27 by 5 results in r = 2
  b. Dividing 36 by 12 results in r = 0
  c. Dividing −18 by 14 results in r = −4. After adding the modulus r = 10
  d. Dividing −7 by 10 results in r = −7. After adding the modulus to −7, r = 3

Set of Residues

- The modulo operation creates a set, which in modular arithmetic is referred to as the set of least residues modulo n, or Z_n.

$$\mathbf{Z}_n = \{ 0, 1, 2, 3, \ldots, (n-1) \}$$

$$\mathbf{Z}_2 = \{ 0, 1 \} \qquad \mathbf{Z}_6 = \{ 0, 1, 2, 3, 4, 5 \} \qquad \mathbf{Z}_{11} = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 \}$$

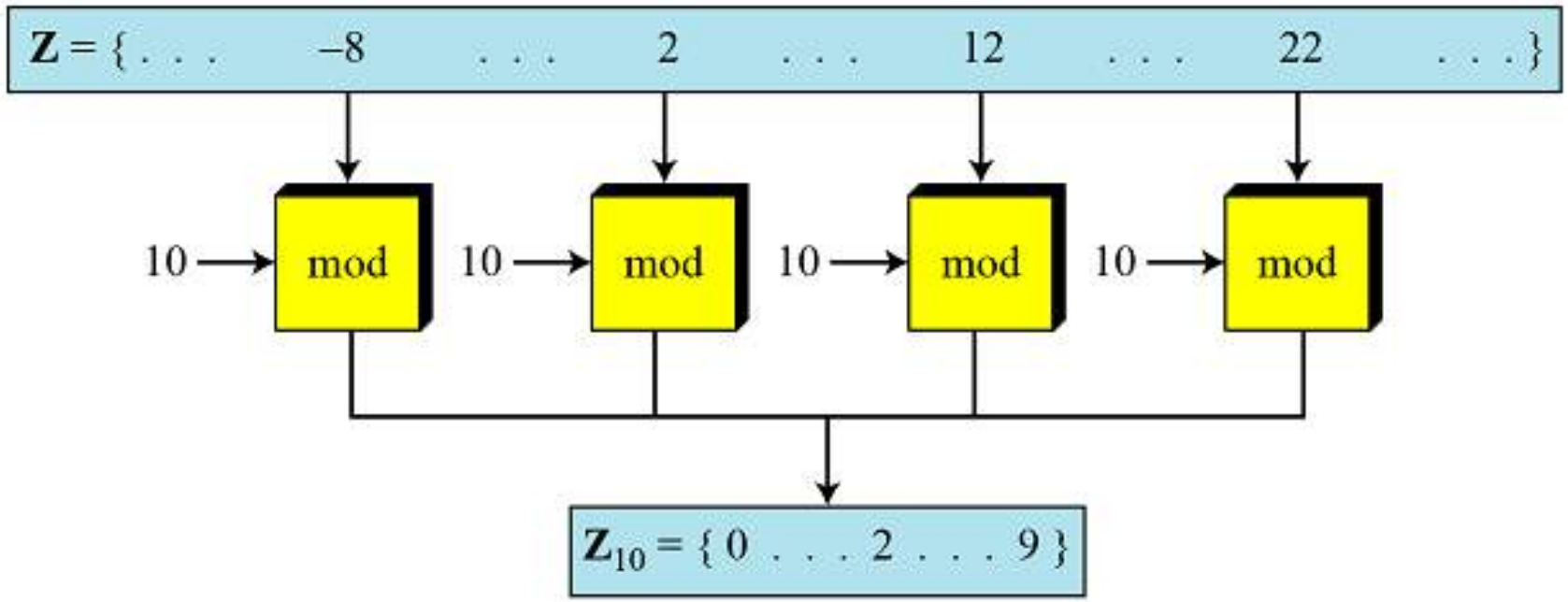
Some Z_n sets

Congruence

- To show that two integers are congruent, we use the congruence operator (≡).
- For example, we write:

$$2 \equiv 12 \pmod{10} \qquad 13 \equiv 23 \pmod{10}$$
$$3 \equiv 8 \pmod{5} \qquad 8 \equiv 13 \pmod{5}$$

Congruence



$$-8 \equiv 2 \equiv 12 \equiv 22 \pmod{10}$$

Congruence Relationship

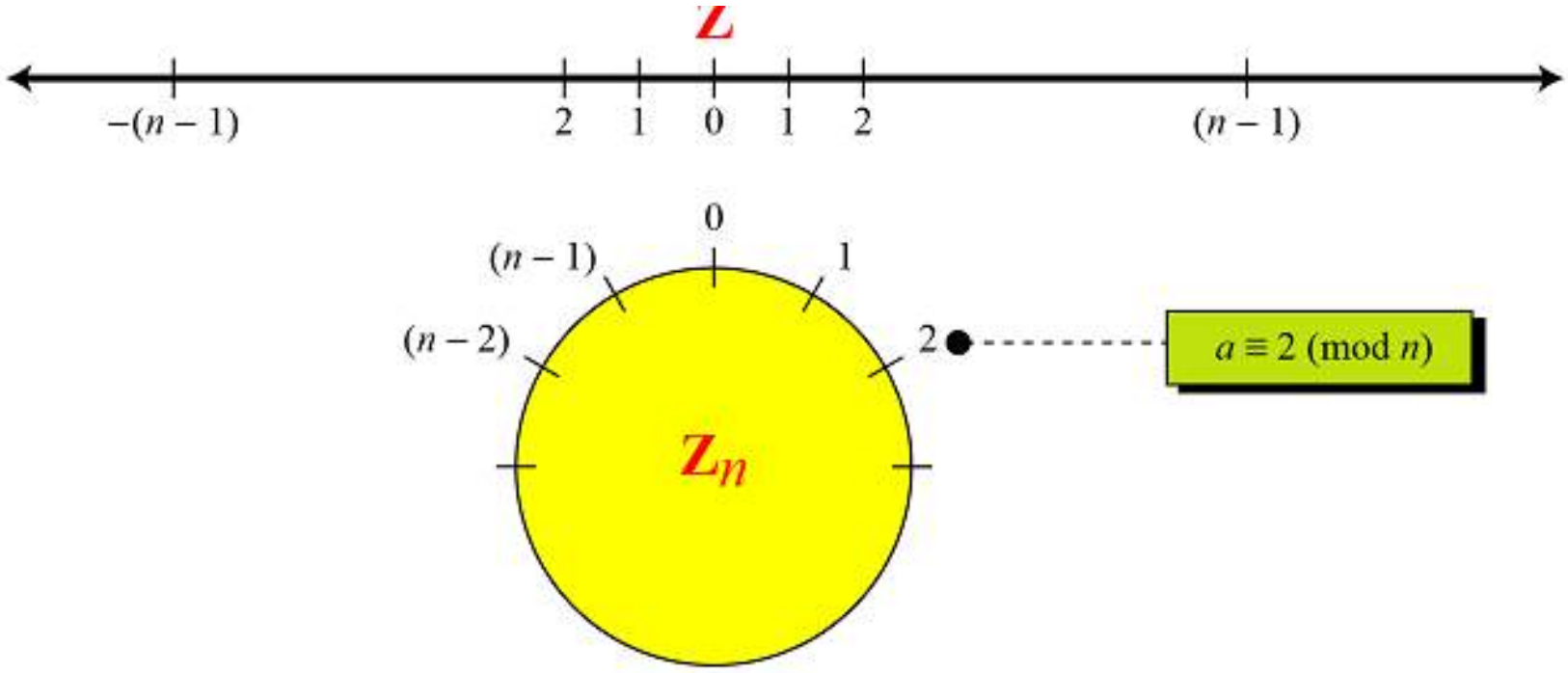
Congruence

- Residue Classes
  - A residue class [a] or $[a]_n$ is the set of integers congruent modulo n.
  - It is the set of all integers such that x=a(mod)n
  - E.g. for n=5, we have five sets as shown below:

$$
\begin{aligned}
[0] &= \{\ldots, -15, -10, -5, 0, 5, 10, 15, \ldots\} \\
[1] &= \{\ldots, -14, -9, -4, 1, 6, 11, 16, \ldots\} \\
[2] &= \{\ldots, -13, -8, -3, 2, 7, 12, 17, \ldots\} \\
[3] &= \{\ldots, -12, -7, -5, 3, 8, 13, 18, \ldots\} \\
[4] &= \{\ldots, -11, -6, -1, 4, 9, 14, 19, \ldots\}
\end{aligned}
$$

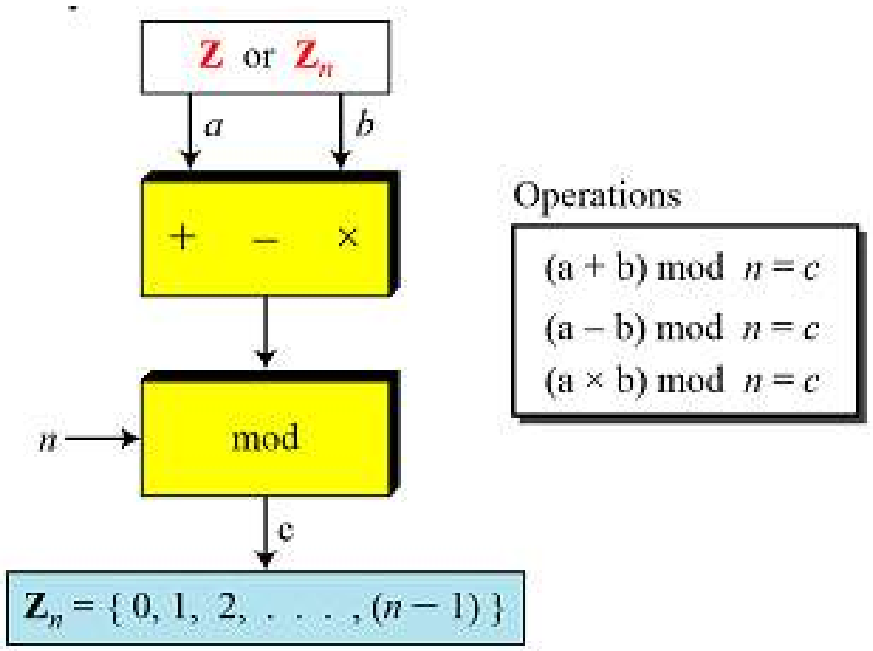
Congruence

- Comparison of Z and Z_n using graphs



Operation in Z_n

- The three binary operations that we discussed for the set Z can also be defined for the set Z_n.
- The result may need to be mapped to Z_n using the mod operator.



Operation in Z_n

- Perform the following operations (the inputs come from Z_n):
  a. Add 7 to 14 in Z_{15}.
  b. Subtract 11 from 7 in Z_{13}.
  c. Multiply 11 by 7 in Z_{20}.
- Solution

$$(14 + 7) \bmod 15 \rightarrow (21) \bmod 15 = 6$$
$$(7 - 11) \bmod 13 \rightarrow (-4) \bmod 13 = 9$$
$$(7 \times 11) \bmod 20 \rightarrow (77) \bmod 20 = 17$$

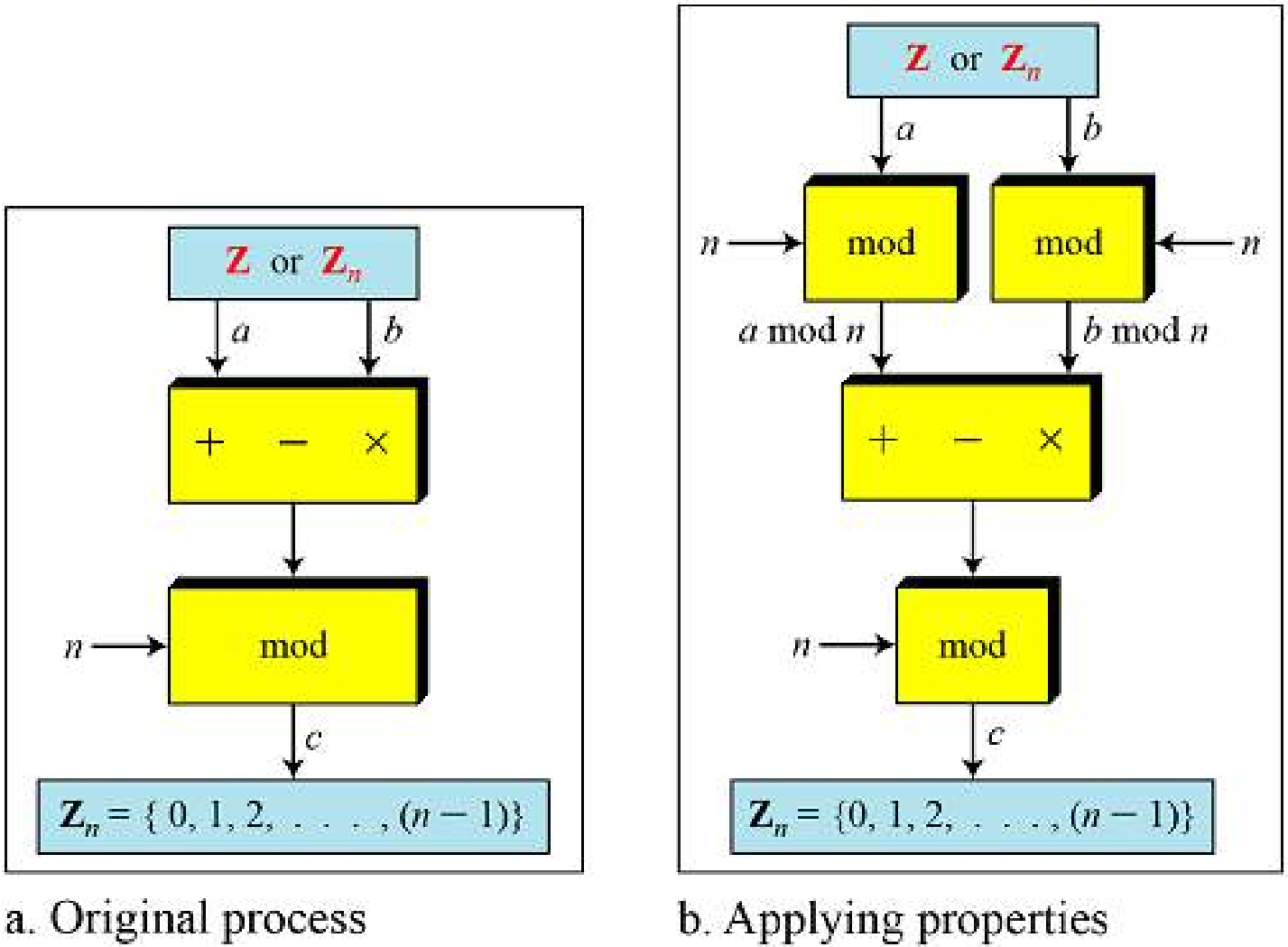
Operation in Z_n

- Perform the following operations (the inputs come from either Z or Z_n):
  a. Add 17 to 27 in Z_{14}.
  b. Subtract 43 from 12 in Z_{13}.
  c. Multiply 123 by −10 in Z_{19}.
- Solution
  a. Add 17 to 27 in Z_{14}: (17+27)mod 14 = 2
  - Subtract 43 from 12 in Z_{13} : (12-43)mod 13 = 8
  - Multiply 123 by −10 in Z_{19}. : (123 x (-10)) mod 19 = 5

Operation in Z_n(cont.)

First Property:	$(a + b) \bmod n = [(a \bmod n) + (b \bmod n)] \bmod n$
Second Property:	$(a - b) \bmod n = [(a \bmod n) - (b \bmod n)] \bmod n$
Third Property:	$(a \times b) \bmod n = [(a \bmod n) \times (b \bmod n)] \bmod n$

Operation in Z_n(cont.)



a. Original process

b. Applying properties

Operation in Z_n(cont.)

- The following shows the application of the above properties:

  1. $(1{,}723{,}345 + 2{,}124{,}945) \bmod 11 = (8 + 9) \bmod 11 = 6$

  2. $(1{,}723{,}345 - 2{,}124{,}945) \bmod 11 = (8 - 9) \bmod 11 = 10$

  3. $(1{,}723{,}345 \times 2{,}124{,}945) \bmod 11 = (8 \times 9) \bmod 11 = 6$

Operation in Z_n(cont.)

- In arithmetic, we often need to find the remainder of powers of 10 when divided by an integer.

$$10^n \bmod x = (10 \bmod x)^n \qquad \text{Applying the third property } n \text{ times.}$$

$$\begin{aligned}
10 \bmod 3 = 1 \quad &\rightarrow \quad 10^n \bmod 3 = (10 \bmod 3)^n = 1 \\
10 \bmod 9 = 1 \quad &\rightarrow \quad 10^n \bmod 9 = (10 \bmod 9)^n = 1 \\
10 \bmod 7 = 3 \quad &\rightarrow \quad 10^n \bmod 7 = (10 \bmod 7)^n = 3^n \bmod 7
\end{aligned}$$

Inverses

- When we are working in modular arithmetic, we often need to find the inverse of a number relative to an operation.
- We are normally looking for an additive inverse (relative to an addition operation) or a multiplicative inverse (relative to a multiplication operation).

Additive Inverses

- In Z_n, two numbers a and b are additive inverses of each other if

$$a + b \equiv 0 \pmod{n}$$

In modular arithmetic, each integer has an additive inverse. The sum of an integer and its additive inverse is congruent to 0 modulo n.

Additive Inverses

- Find all additive inverse pairs in Z_{10}.
- Solution
  – The six pairs of additive inverses are (0, 0), (1, 9), (2, 8), (3, 7), (4, 6), and (5, 5).

Multiplicative Inverses

- In Z_n, two numbers a and b are the multiplicative inverse of each other if,

$$a \times b \equiv 1 \pmod{n}$$

In modular arithmetic, an integer may or may not have a multiplicative inverse.
When it does, the product of the integer and its multiplicative inverse is congruent to 1 modulo n.

Multiplicative Inverses

- Find the multiplicative inverse of 8 in Z_{10}.
  - There is no multiplicative inverse because gcd (10, 8) = 2 ≠ 1.
  - In other words, we cannot find any number between 0 and 9 such that when multiplied by 8, the result is congruent to 1.
- Find all multiplicative inverses in Z_{10}.
  - There are only three pairs: (1, 1), (3, 7) and (9, 9).
  - The numbers 0, 2, 4, 5, 6, and 8 do not have a multiplicative inverse.

Multiplicative Inverses

- Find all multiplicative inverse pairs in Z_{11}.
- Solution
  - We have seven pairs: (1, 1), (2, 6), (3, 4), (5, 9), (7, 8), and (10, 10).

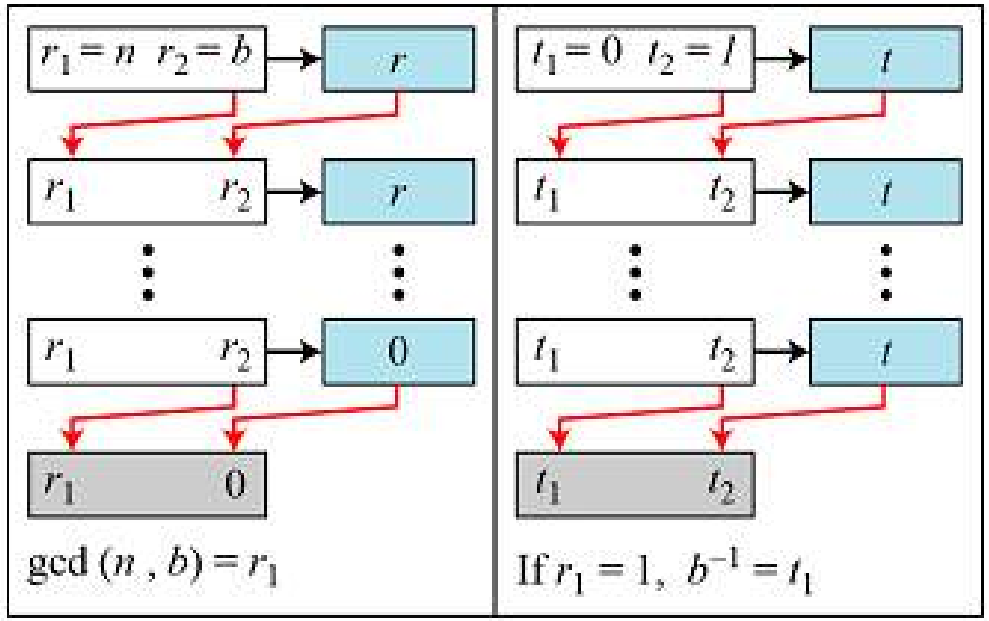
Multiplicative Inverses

- The extended Euclidean algorithm finds the multiplicative inverses of b in Z_n when n and b are given and gcd (n, b) = 1.
- The multiplicative inverse of b is the value of t after being mapped to Z_n.
- If the multiplicative inverse of b exists, gcd (n, b) must be 1.

$$(s \times n) + (b \times t) = 1$$

$$(s \times n + b \times t) \bmod n = 1 \bmod n$$
$$[(s \times n) \bmod n] + [(b \times t) \bmod n] = 1 \bmod n$$
$$0 + [(b \times t) \bmod n] = 1$$
$$(b \times t) \bmod n = 1 \qquad \rightarrow \text{This means } t \text{ is the multiplicative inverse of } b \text{ in } Z_n$$

Multiplicative Inverses



a. Process

$r_1 \leftarrow$ n; $r_2 \leftarrow b$;
$t_1 \leftarrow 0$; $t_2 \leftarrow 1$;

while $(r_2 > 0)$
{
$q \leftarrow r_1 / r_2$;

$r \leftarrow r_1 - q \times r_2$;
$r_1 \leftarrow r_2$; $r_2 \leftarrow r$;

$t \leftarrow t_1 - q \times t_2$;
$t_1 \leftarrow t_2$; $t_2 \leftarrow t$;

}
if $(r_1 = 1)$ then $b^{-1} \leftarrow t_1$

b. Algorithm

Using extended Euclidean algorithm to find multiplicative inverse

Multiplicative Inverses

- Find the multiplicative inverse of 11 in Z_{26}.

Solution

q	r_1	r_2	r	t_1	t_2	t
2	26	11	4	0	1	−2
2	11	4	3	1	−2	5
1	4	3	1	−2	5	−7
3	3	1	0	5	−7	26
	1	0		−7	26	

The gcd (26, 11) is 1; the inverse of 11 is -7 or 19.

Multiplicative Inverses

- Find the multiplicative inverse of 23 in Z_{100}.

Solution

q	r_1	r_2	r	t_1	t_2	t
4	100	23	8	0	1	−4
2	23	8	7	1	−4	19
1	8	7	1	−4	9	−13
7	7	1	0	9	−13	100
	1	0		−13	100	

The gcd (100, 23) is 1; the inverse of 23 is -13 or 87.

Multiplicative Inverses

- Find the inverse of 12 in Z_{26}.

Solution

q	r_1	r_2	r	t_1	t_2	t
2	26	12	2	0	1	−2
6	12	2	0	1	−2	13
	2	0		−2	13	

The gcd (26, 12) is 2; the inverse does not exist.

Addition and Multiplication Tables

- Addition and multiplication table for Z_{10}

	0	1	2	3	4	5	6	7	8	9
0	**0**	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9	**0**
2	2	3	4	5	6	7	8	9	**0**	1
3	3	4	5	6	7	8	9	**0**	1	2
4	4	5	6	7	8	9	**0**	1	2	3
5	5	6	7	8	9	**0**	1	2	3	4
6	6	7	8	9	**0**	1	2	3	4	5
7	7	8	9	**0**	1	2	3	4	5	6
8	8	9	**0**	1	2	3	4	5	6	7
9	9	**0**	1	2	3	4	5	6	7	8

Addition Table in $\mathbf{Z}_{10}$

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	**1**	2	3	4	5	6	7	8	9
2	0	2	4	6	8	0	2	4	6	8
3	0	3	6	9	2	5	8	**1**	4	7
4	0	4	8	2	6	0	4	8	2	6
5	0	5	0	5	0	5	0	5	0	5
6	0	6	2	8	4	0	6	2	8	4
7	0	7	4	**1**	8	0	2	9	6	3
8	0	8	6	4	2	0	8	6	4	2
9	0	9	8	7	6	5	4	3	2	**1**

Multiplication Table in $\mathbf{Z}_{10}$

Different Sets

- Some Z_n and Z_n^* sets

$\mathbf{Z}_6 = \{0, 1, 2, 3, 4, 5\}$

$\mathbf{Z}_6^* = \{1, 5\}$

$\mathbf{Z}_7 = \{0, 1, 2, 3, 4, 5, 6\}$

$\mathbf{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$

$\mathbf{Z}_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$\mathbf{Z}_{10}^* = \{1, 3, 7, 9\}$

We need to use Zn when additive inverses are needed; we need to use Zn* when multiplicative inverses are needed.

Two More Sets

- Cryptography often uses two more sets: Z_p and Z_p*.
- The modulus in these two sets is a primenumber.

$$Z_{13} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$$
$$Z_{13}^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$$

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 4a: Introduction to Cryptography Mathematics- Part-3

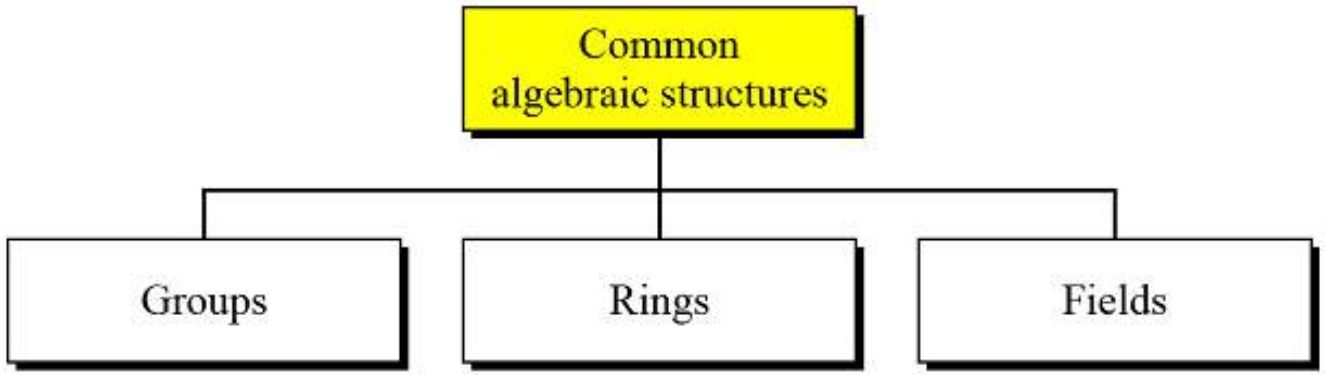
Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

ALGEBRAIC STRUCTURES

- Cryptography requires sets of integers and specific operations that are defined for those sets.
- The combination of the set and the operations that are applied to the elements of the set is called an algebraic structure.
- Three common algebraic structures:
  - Groups
  - Rings, and
  - Fields.



Groups

- A group (G) is a set of elements with a binary operation (•) that satisfies four properties (or axioms).
- Closure
  - If a and b are elements of G, then c = a•b is also an element of G.
- Associativity
  - If a, b and c are elements of G, then (a•b) •c=a•(b•c)
- Existence of identity
  - For all a in G, there exist an element e, called the identity element, such that e•a=a•e=a
- Existence of inverse
  - For each a in G, there exists an element a', called the inverse of a, such that a•a'=a'•a=e

Groups

- A Commutative group (Abelian group) is group in which the operator satisfies four properties plus an extra property that is commutativity.
  - For all a and b in G, we have a • b = b • a
- Application
  - Although a group involves a single operation, the properties imposed on the operation allow the use of a pair of operations!!!!
- Example
  - *The set of residue integers with the addition operator,*
    $G = < Z_n , + >$,
    is a commutative group.
    Check the properties….

Groups

Let us check the properties.

1. Closure is satisfied. The result of adding two integers in Zn is another integer in Z_n.

2. Associativity is satisfied. The result of 4 + (3 + 2) is the same as (4 + 3) + 2.

3. Commutativity is satisfied. We have 3 + 5 = 5 + 3.

4. The identify element is 0. We have 3 + 0 = 0 + 3 = 3.

5. Every element has an additive inverse. The inverse of an element is its complement. For example, the inverse of 3 is −3 (n − 3 in Zn) and the inverse of−3 is 3. The inverse allows us to perform subtraction on the set.

Groups

- The set Z_n^* with the multiplication operator, $G = \langle Z_n^*, \times \rangle$, is also an abelian group.
- We can perform multiplication and division on the elements of this set without moving out of the set.
- It is easy to check the first three properties.
- The identity element is 1.
- Each element has an inverse that can be found according to the extended Euclidean algorithm.

Groups

- Let us define a set G = < {a, b, c, d}, •> and the operation

•	*a*	*b*	*c*	*d*
a	*a*	*b*	*c*	*d*
b	*b*	*c*	*d*	*a*
c	*c*	*d*	*a*	*b*
d	*d*	*a*	*b*	*c*

Check for properties….

- Is the group abelian????

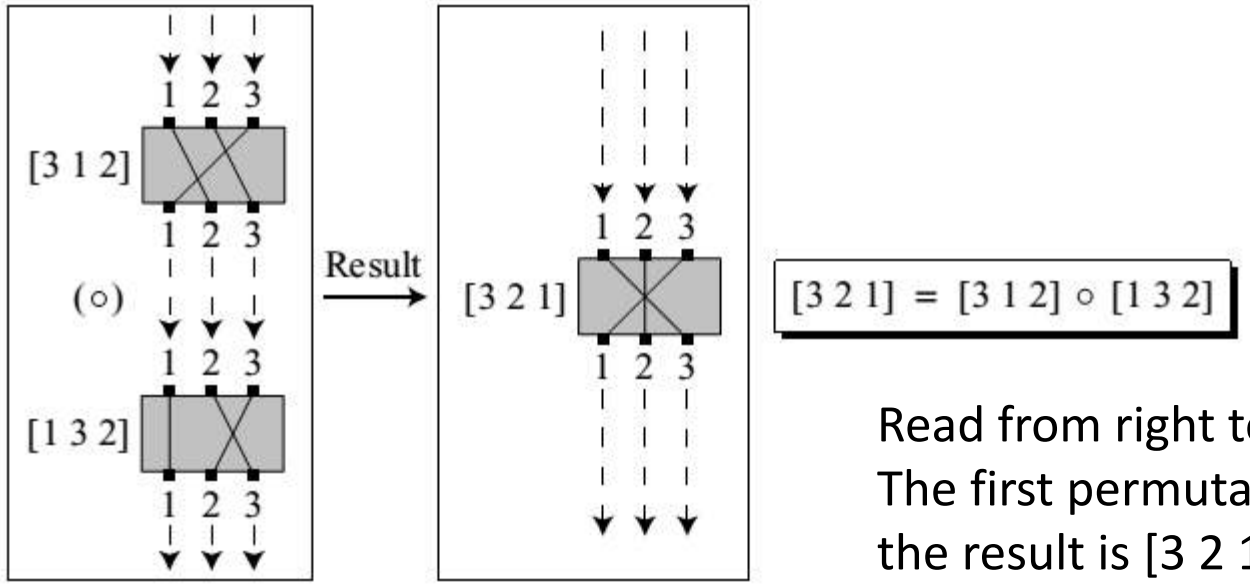
Groups

•	a	b	c	d
a	a	b	c	d
b	b	c	d	a
c	c	d	a	b
d	d	a	b	c

- This is an abelian group. All five properties are satisfied:
  1. Closure is satisfied. Applying the operation on any pair of elements result in another elements in the set.
  2. Associativity is also satisfied. To prove this we need to check the property for any combination of three elements. For example, (a + b) + c = a + (b + c) = d.
  3. The operation is commutative. We have a + b = b + a.
  4. The group has an identity element, which is a.
  5. Each element has an inverse. The inverse pairs can be found by finding the identity in each row (shaded). The pairs are (a, a), (b, d), (c, c).

Groups

- A very interesting group is the permutation group.
- The set is the set of all permutations, and the operation is composition: applying one permutation after another.



Read from right to left:
The first permutation is [1 3 2] followed by [3 1 2];
the result is [3 2 1].

Operation table for permutation group

°	[1 2 3]	[1 3 2]	[2 1 3]	[2 3 1]	[3 1 2]	[3 2 1]
[1 2 3]	[1 2 3]	[1 3 2]	[2 1 3]	[2 3 1]	[3 1 2]	[3 2 1]
[1 3 2]	[1 3 2]	[1 2 3]	[2 3 1]	[2 1 3]	[3 2 1]	[3 1 2]
[2 1 3]	[2 1 3]	[3 1 2]	[1 2 3]	[3 2 1]	[1 3 2]	[2 3 1]
[2 3 1]	[2 3 1]	[3 2 1]	[1 3 2]	[3 1 2]	[1 2 3]	[2 1 3]
[3 1 2]	[3 1 2]	[2 1 3]	[3 2 1]	[1 2 3]	[2 3 1]	[1 3 2]
[3 2 1]	[3 2 1]	[2 3 1]	[3 1 2]	[1 3 2]	[2 1 3]	[1 2 3]

Check for properties….

- Is the group abelian????

Permutation group

- Closure is satisfied.
- Associativity is also satisfied.
- The commutative property is not satisfied.
- The set has an identity element, which is [1 2 3] (no permutation).
- Each element has an inverse.
- Set of permutations with the composition operation is a group.
- This implies that using two permutations one after another cannot strengthen the security of a cipher, because we can always find a permutation that can do the same job because of the closure property.

Groups

- Finite Group
  - If the set has a finite number of elements; otherwise, it is an infinite group.
- Order of a Group |G|
  - The number of elements in the group.
  - If the group is finite, its order is finite
- Subgroups
  - A subset H of a group G is a subgroup of G if H itself is a group with respect to the operation on G
  - If G=<S, •> is a group, H=<T, •> is a group under the same operation, and T is a nonempty subset of S, then H is a subgroup of G
  - If a and b are members of both groups, then c=a•b is also member of both groups
  - The group share the same identity element
  - If a is a member of both groups, the inverse of a is also a member of both groups
  - The group made of the identity element of G, H=<{e}, •>, is a subgroup of G
  - Each group is a subgroup of itself

Groups

- Finite Group
  - If the set has a finite number of elements; otherwise, it is an infinite group.
- Order of a Group |G|
  - The number of elements in the group.
  - If the group is finite, its order is finite
- Subgroups
  - A subset H of a group G is a subgroup of G if H itself is a group with respect to the operation on G
  - If G=<S, •> is a group, H=<T, •> is a group under the same operation, and T is a nonempty subset of S, then H is a subgroup of G
  - If a and b are members of both groups, then c=a•b is also member of both groups
  - The group share the same identity element
  - If a is a member of both groups, the inverse of a is also a member of both groups
  - The group made of the identity element of G, H=<{e}, •>, is a subgroup of G
  - Each group is a subgroup of itself

Groups

- Is the group H = $\langle Z_{10}, + \rangle$ a subgroup of the group G = $\langle Z_{12}, + \rangle$?
- The answer is no. Although H is a subset of G, the operations defined for these two groups are different. The operation in H is addition modulo 10; the operation in G is addition modulo 12.

Groups

- Cyclic subgroups
  - If a subgroup of a group can be generated using the power of an element, the subgroup is called the cyclic subgroup.

$$a^n \rightarrow a \bullet a \bullet \ldots \bullet a \quad (n \text{ times})$$

Groups

- Four cyclic subgroups can be made from the group G = $\langle Z_6, + \rangle$.
- They are $H_1 = \langle \{0\}, + \rangle$, $H_2 = \langle \{0, 2, 4\}, + \rangle$, $H_3 = \langle \{0, 3\}, + \rangle$, and $H_4 = G$.

$$0^0 \bmod 6 = 0$$

$$1^0 \bmod 6 = 0$$
$$1^1 \bmod 6 = 1$$
$$1^2 \bmod 6 = (1 + 1) \bmod 6 = 2$$
$$1^3 \bmod 6 = (1 + 1 + 1) \bmod 6 = 3$$
$$1^4 \bmod 6 = (1 + 1 + 1 + 1) \bmod 6 = 4$$
$$1^5 \bmod 6 = (1 + 1 + 1 + 1 + 1) \bmod 6 = 5$$

$$2^0 \bmod 6 = 0$$
$$2^1 \bmod 6 = 2$$
$$2^2 \bmod 6 = (2 + 2) \bmod 6 = 4$$

$$3^0 \bmod 6 = 0$$
$$3^1 \bmod 6 = 3$$

$$4^0 \bmod 6 = 0$$
$$4^1 \bmod 6 = 4$$
$$4^2 \bmod 6 = (4 + 4) \bmod 6 = 2$$

$$5^0 \bmod 6 = 0$$
$$5^1 \bmod 6 = 5$$
$$5^2 \bmod 6 = 4$$
$$5^3 \bmod 6 = 3$$
$$5^4 \bmod 6 = 2$$
$$5^5 \bmod 6 = 1$$

Groups

- Exercise:
  – Find out the cyclic subgroups for group G = $<Z_{10}^*, \times>$.

Groups

- Three cyclic subgroups can be made from the group G = $\langle Z_{10}^*, \times \rangle$.
- G has only four elements: 1, 3, 7, and 9.
- The cyclic subgroups are $H_1 = \langle \{1\}, \times \rangle$, $H_2 = \langle \{1, 9\}, \times \rangle$, and $H_3 = G$.

$$1^0 \bmod 10 = 1$$

$$3^0 \bmod 10 = 1$$
$$3^1 \bmod 10 = 3$$
$$3^2 \bmod 10 = 9$$
$$3^3 \bmod 10 = 7$$

$$7^0 \bmod 10 = 1$$
$$7^1 \bmod 10 = 7$$
$$7^2 \bmod 10 = 9$$
$$7^3 \bmod 10 = 3$$

$$9^0 \bmod 10 = 1$$
$$9^1 \bmod 10 = 9$$

Groups

- Cyclic group
  – A cyclic group is a group that is its own cyclic subgroup.

$$\{e, g, g^2, \ldots, g^{n-1}\}, \text{ where } g^n = e$$

- Example:
  - Three cyclic subgroups can be made from the group G = $\langle Z_{10}^*, \times \rangle$.
  - The cyclic subgroups are $H_1 = \langle \{1\}, \times \rangle$, $H_2 = \langle \{1, 9\}, \times \rangle$, and $H_3 = G$.
  - The group G = $\langle Z_{10}^*, \times \rangle$ is a cyclic group with two generators, $g = 3$ and $g = 7$.
  - The group G = $\langle Z_6, + \rangle$ is a cyclic group with two generators, $g = 1$ and $g = 5$.

Lagrange's Theorem

- Assume that G is a group, and H is a subgroup of G.
- If the order of G and H are |G| and |H|, respectively, then, based on this theorem, |H| divides |G| .
- Order of an Element
  – The order of an element is the order of the cyclic group it generates.
- Example:
  - In the group G = $\langle Z_6, + \rangle$, the orders of the elements are:
    ord(0) = 1, ord(1) = 6, ord(2) = 3, ord(3) = 2, ord(4) = 3, ord(5) = 6.
  - In the group G = $\langle Z_{10}^*, \times \rangle$, the orders of the elements are:
    ord(1) = 1, ord(3) = 4, ord(7) = 4, ord(9) = 2.

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

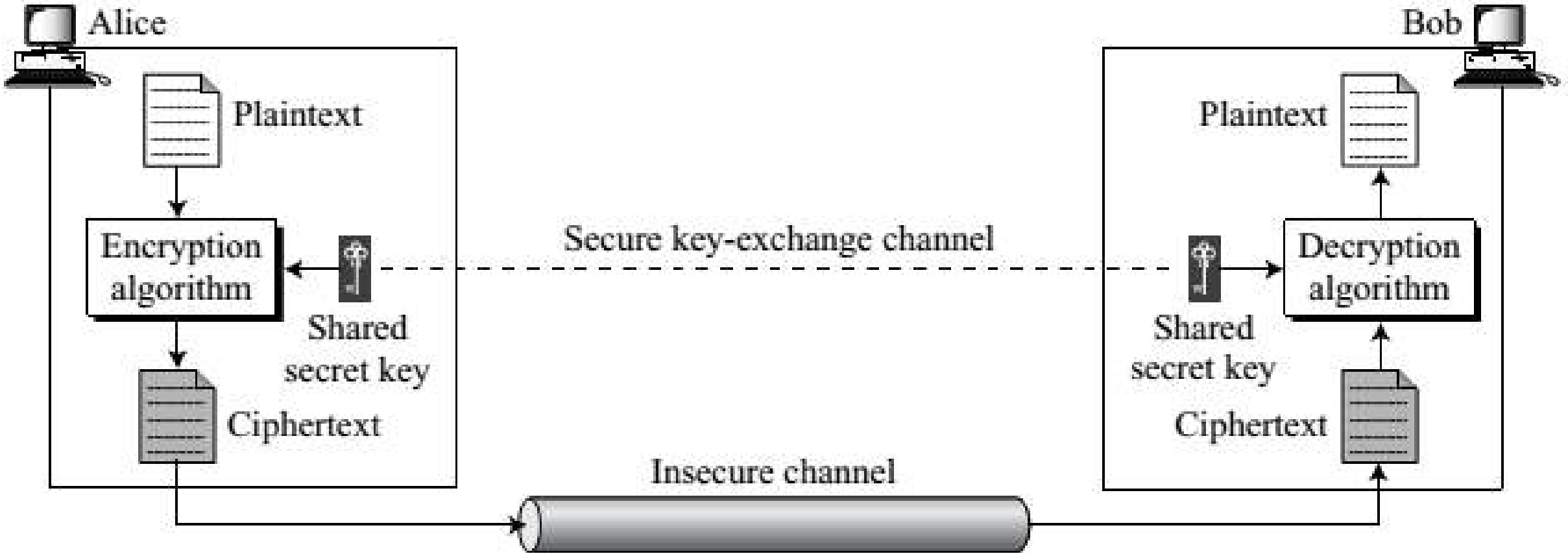
Lecture – 4b : Symmetric-Key Ciphers

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

Symmetric-key Cipher



Symmetric-key Cipher

- Symmetric-key encipherment uses a single key for both encryption and decryption.
- Encryption and decryption algorithms are inverses of each other.
- If P is the plaintext, C is the ciphertext, and K is the key, the encryption algorithm $E_k(x)$ creates the ciphertext from the plaintext; the decryption algorithm $D_k(x)$ creates the plaintext from the ciphertext.
- We assume that $E_k(x)$ and $D_k(x)$ are inverses of each other: they cancel the effect of each other if they are applied one after the other on the same input.

Encryption: $C = E_k(P)$ Decryption: $P = D_k(C)$

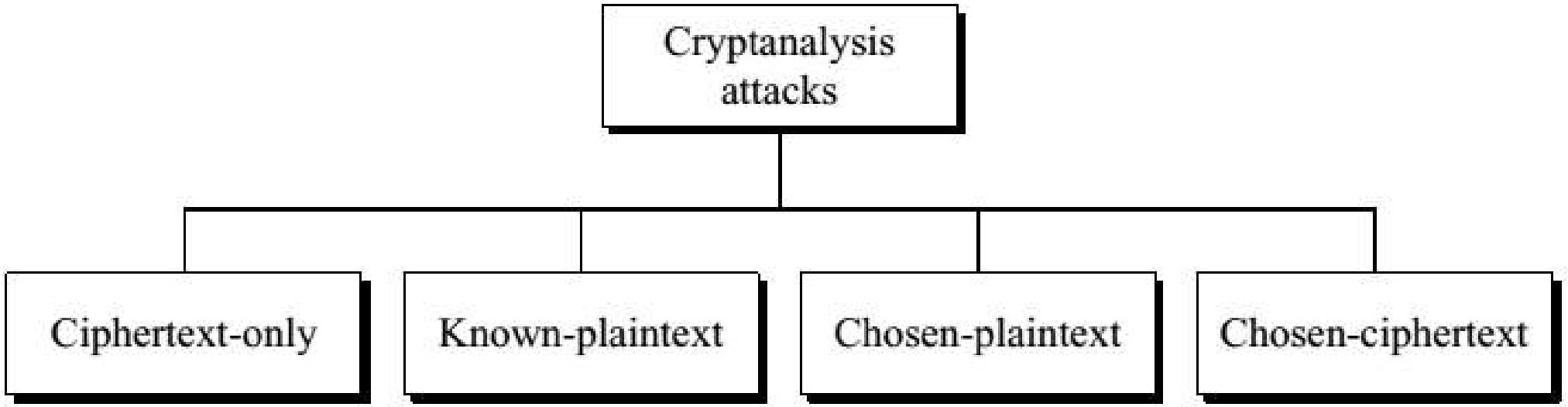
In which, $D_k(E_k(x)) = E_k(D_k(x)) = x$

Alice: $C = E_k(P)$ **Bob:** $P_1 = D_k(C) = D_k(E_k(P)) = P$

Kerckhoff's Principle

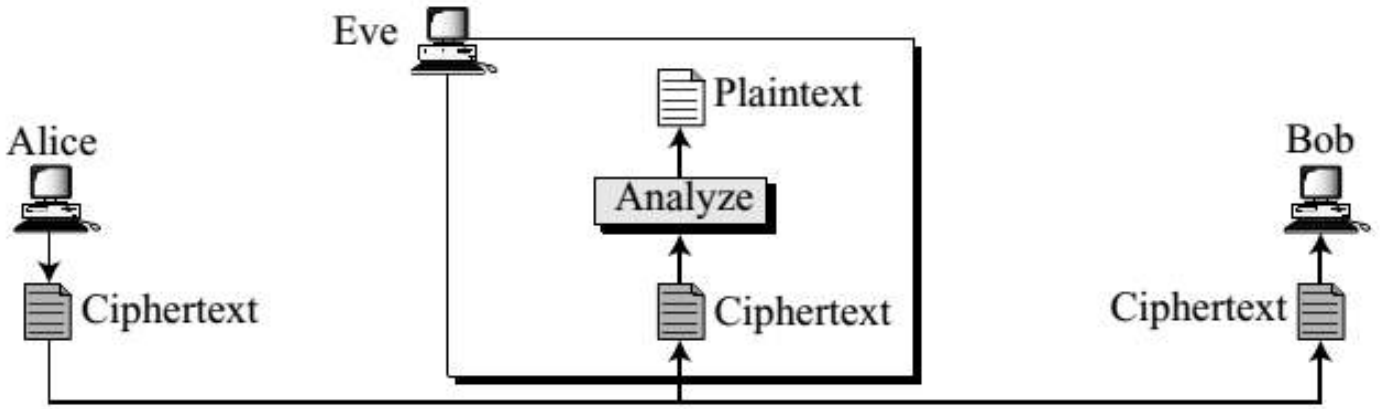
- Although it may appear that a cipher would be more secure if we hide both the encryption/decryption algorithm and the secret key, this is not recommended.
- Based on Kerckhoff's principle, one should always assume that the adversary, Eve, knows the encryption/decryption algorithm.
- The resistance of the cipher to attack must be based only on the secrecy of the key.
- In other words, guessing the key should be so difficult that there is no need to hide the encryption/decryption algorithm.
- This principle manifests itself more clearly when we study modern ciphers.
- There are only a few algorithms for modern ciphers today.
- The key domain for each algorithm, however, is so large that it makes it difficult for the adversary to find the key.

Cryptanalysis



Ciphertext-Only Attack

- In a ciphertext-only attack, Eve has access to only some ciphertext.
- She tries to find the corresponding key and the plaintext.
- The assumption is that Eve knows the algorithm and can intercept the ciphertext.
- The ciphertext-only attack is the most probable one because Eve needs only the ciphertext for this attack.
- To thwart the decryption of a message by an adversary, a cipher must be very resisting to this type of attack.

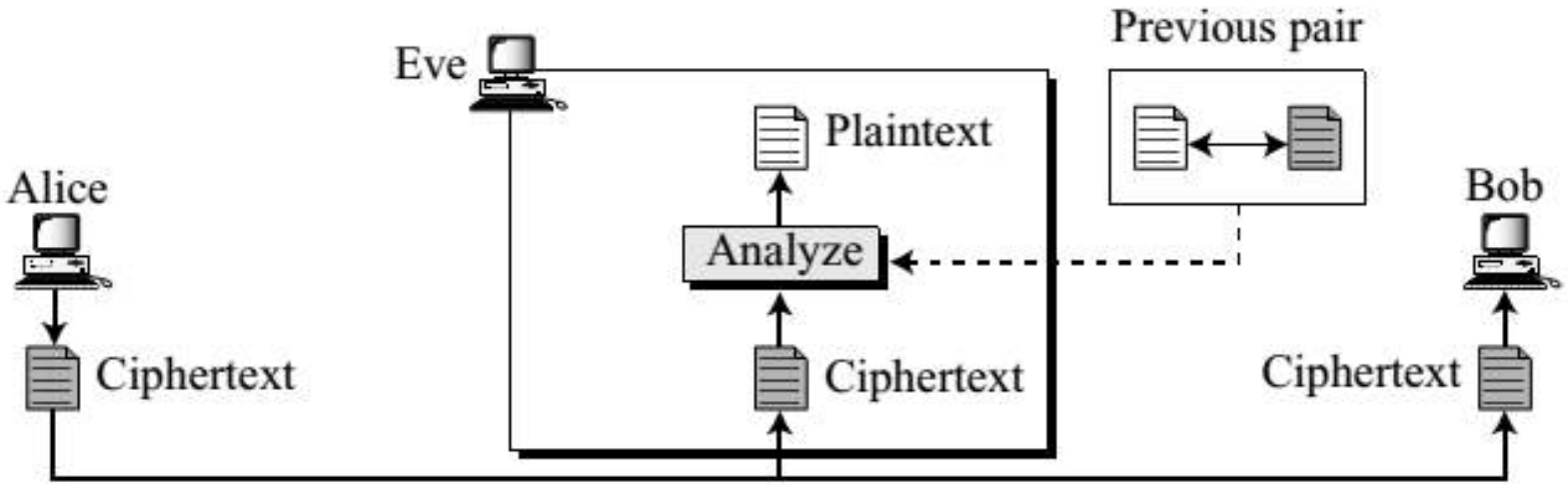


Ciphertext-Only Attack

- Brute-Force Attack
  - In the brute-force method or exhaustive-key-search method, Eve tries to use all possible keys.
  - We assume that Eve knows the algorithm and knows the key domain (the list of all possible keys).
  - Using the intercepted cipher, Eve decrypts the ciphertext with every possible key until the plaintext makes sense.
  - Using brute-force attack was a difficult task in the past; it is easier today using a computer.
  - To prevent this type of attack, the number of possible keys must be very large.
- Statistical Attack
  - The cryptanalyst can benefit from some inherent characteristics of the plaintext language to launch a statistical attack.
  - For example, we know that the letter E is the most frequently used letter in English text.
  - The cryptanalyst finds the mostly-used character in the ciphertext and assumes that the corresponding plaintext character is E.
  - After finding a few pairs, the analyst can find the key and use it to decrypt the message.
  - To prevent this type of attack, the cipher should hide the characteristics of the language.
- Pattern Attack
  - Some ciphers may hide the characteristics of the language, but may create some patterns in the ciphertext.
  - A cryptanalyst may use a pattern attack to break the cipher.
  - Therefore, it is important to use ciphers that make the ciphertext look as random as possible.

Known-Plaintext Attack

- In a known-plaintext attack, Eve has access to some plaintext/ciphertext pairs in addition to the intercepted ciphertext that she wants to break.

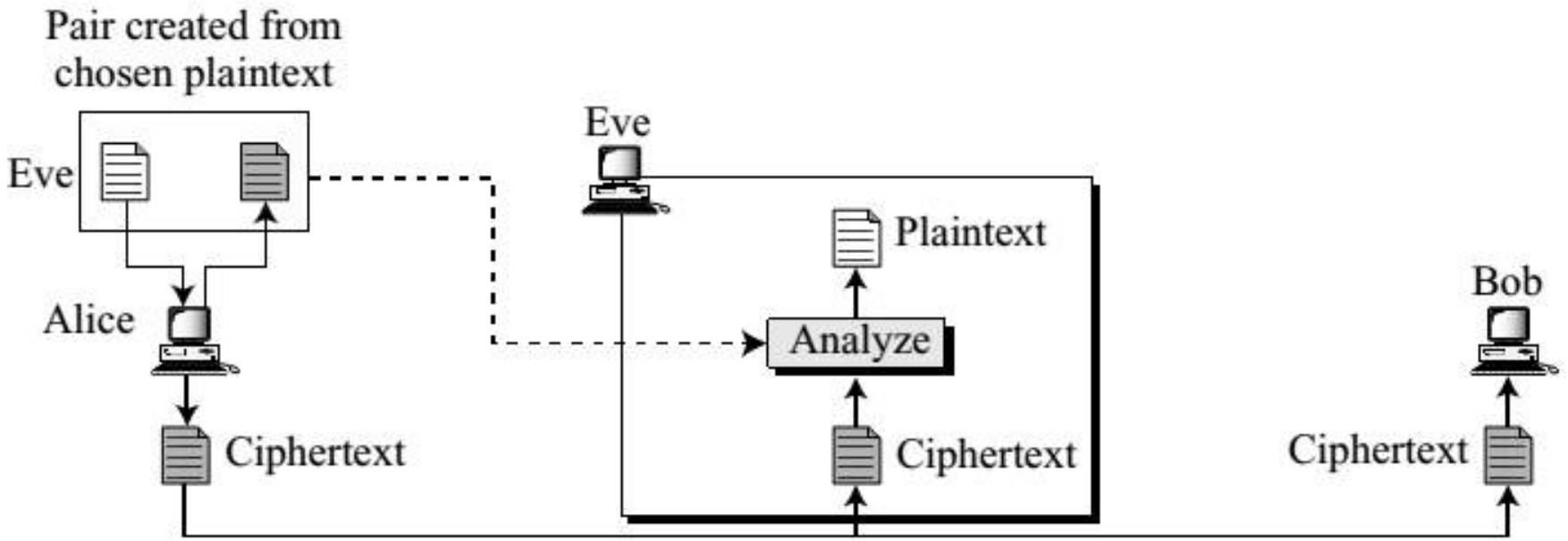


Known-Plaintext Attack

- The plaintext/ciphertext pairs have been collected earlier.
- For example, Alice has sent a secret message to Bob, but she has later made the contents of the message public.
- Eve has kept both the ciphertext and the plaintext to use them to break the next secret message from Alice to Bob, assuming that Alice has not changed her key.
- Eve uses the relationship between the previous pair to analyze the current ciphertext.
- The same methods used in a ciphertext-only attack can be applied here.
- This attack is easier to implement because Eve has more information to use for analysis.
- However, it is less likely to happen because Alice may have changed her key or may have not disclosed the contents of any previous messages.

Chosen-Plaintext Attack

- The chosen-plaintext attack is similar to the known-plaintext attack, but the plaintext/ciphertext pairs have been chosen by the attacker herself.

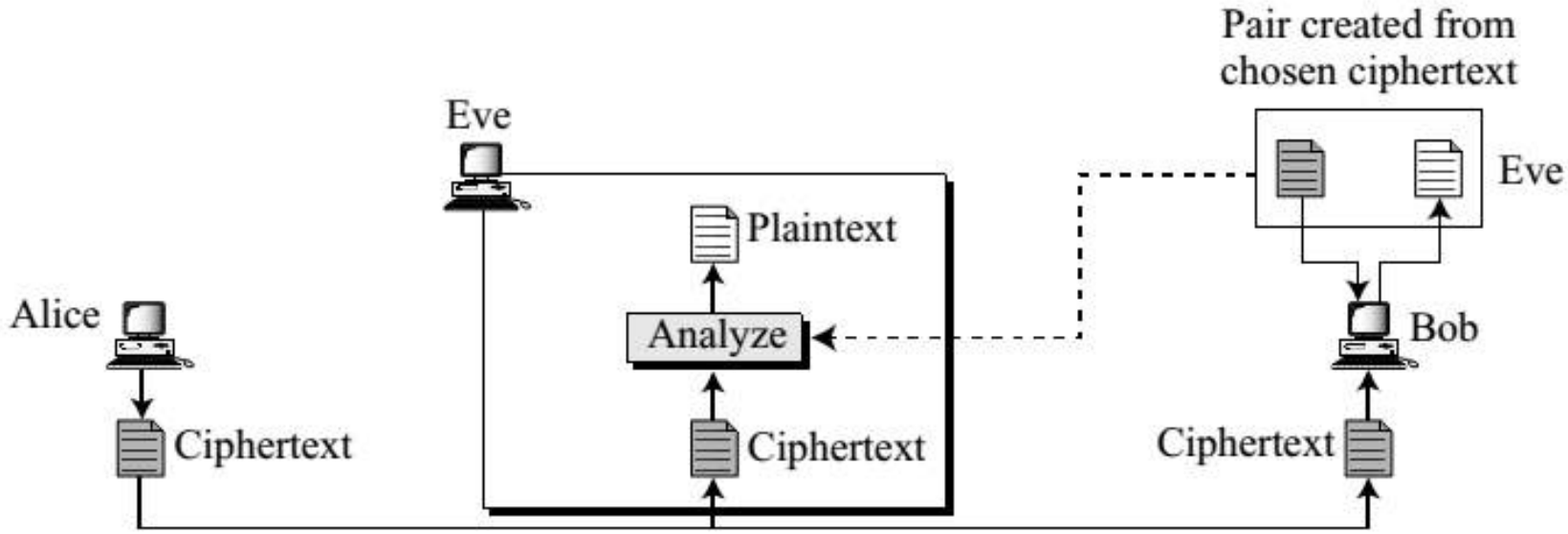


Chosen-Plaintext Attack

- This can happen, for example, if Eve has access to Alice's computer.
- She can choose some plaintext and intercept the created ciphertext.
- Of course, she does not have the key because the key is normally embedded in the software used by the sender.
- This type of attack is much easier to implement, but it is much less likely to happen.

Chosen-Ciphertext Attack

- The chosen-ciphertext attack is similar to the chosen-plaintext attack, except that Eve chooses some ciphertext and decrypts it to form a ciphertext/plaintext pair.
- This can happen if Eve has access to Bob's computer.



Categories of Traditional Ciphers

- Substitution ciphers and Transposition ciphers.
- In a substitution cipher, we replace one symbol in the ciphertext with another symbol.
- In a transposition cipher, we reorder the position of symbols in the plaintext.

SUBSTITUTION CIPHERS

- A substitution cipher replaces one symbol with another.
- Monoalphabetic Ciphers
  - In monoalphabetic substitution, the relationship between a symbol in the plaintext to a symbol in the ciphertext is always one-to-one.
  -

Plaintext: hello **Ciphertext:** KHOOR

Plaintext: hello **Ciphertext:** ABNZF

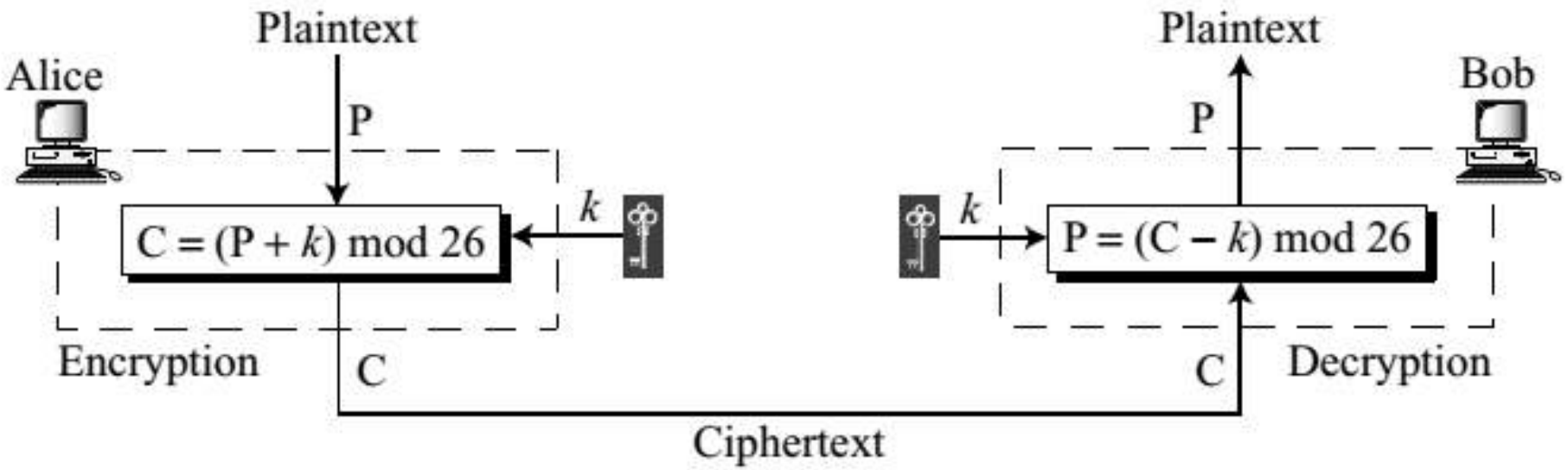
Additive Cipher

- This cipher is sometimes called a shift cipher and sometimes a Caesar cipher, but the term additive cipher better reveals its mathematical nature.
- When the cipher is additive, the plaintext, ciphertext, and key are integers in Z_{26}.

Plaintext →	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Ciphertext →	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Value →	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Representation of plaintext and ciphertext characters in Z_{26}

Additive Cipher



$$P_1 = (C - k) \bmod 26 = (P + k - k) \text{ mode } 26 = P$$

Example #1:
Use the additive cipher with key = 15 to encrypt the message “hello”.
And decrypt to verify using the same key

Plaintext: h → 07	Encryption: (07 + 15) mod 26	Ciphertext: 22 → W
Plaintext: e → 04	Encryption: (04 + 15) mod 26	Ciphertext: 19 → T
Plaintext: l → 11	Encryption: (11 + 15) mod 26	Ciphertext: 00 → A
Plaintext: l → 11	Encryption: (11 + 15) mod 26	Ciphertext: 00 → A
Plaintext: o → 14	Encryption: (14 + 15) mod 26	Ciphertext: 03 → D

Ciphertext: W → 22	Decryption: (22 – 15) mod 26	Plaintext: 07 → h
Ciphertext: T → 19	Decryption: (19 – 15) mod 26	Plaintext: 04 → e
Ciphertext: A → 00	Decryption: (00 – 15) mod 26	Plaintext: 11 → l
Ciphertext: A → 00	Decryption: (00 – 15) mod 26	Plaintext: 11 → l
Ciphertext: D → 03	Decryption: (03 – 15) mod 26	Plaintext: 14 → o

- Additive ciphers are vulnerable to ciphertext-only attacks using exhaustive key searches (brute-force attacks).
- The key domain of the additive cipher is very small;there are only 26 keys.
- However, one of the keys, zero, is useless (the ciphertext is the same as the plaintext).
- This leaves only 25 possible keys.
- Eve can easily launch a bruteforce attack on the ciphertext.

s

Example #2

Eve has intercepted the ciphertext “UVACLYFZLJBYL”. Show how she can use a brute-force attack to break the cipher.

Ciphertext: UVACLYFZLJBYL

K = 1	→	**Plaintext:** tuzbkxeykiaxk
K = 2	→	**Plaintext:** styajwdxjhzwj
K = 3	→	**Plaintext:** rsxzivcwigyvi
K = 4	→	**Plaintext:** qrwyhubvhfxuh
K = 5	→	**Plaintext:** pqvxgtaugewtg
K = 6	→	**Plaintext:** opuwfsztfdvsf
K = 7	→	**Plaintext:** notverysecure

Cryptanalysis

- Additive ciphers are also subject to statistical attacks. This is especially true if the adversary has a long ciphertext.
- The adversary can use the frequency of occurrence of characters for a particular language.

Letter	*Frequency*	*Letter*	*Frequency*	*Letter*	*Frequency*	*Letter*	*Frequency*
E	12.7	H	6.1	W	2.3	K	0.08
T	9.1	R	6.0	F	2.2	J	0.02
A	8.2	D	4.3	G	2.0	Q	0.01
O	7.5	L	4.0	Y	2.0	X	0.01
I	7.0	C	2.8	P	1.9	Z	0.01
N	6.7	U	2.8	B	1.5		
S	6.3	M	2.4	V	1.0		

Cryptanalysis

Digram	TH, HE, IN, ER, AN, RE, ED, ON, ES, ST, EN, AT, TO, NT, HA, ND, OU, EA, NG, AS, OR, TI, IS, ET, IT, AR, TE, SE, HI, OF
Trigram	THE, ING, AND, HER, ERE, ENT, THA, NTH, WAS, ETH, FOR, DTH

The most common two-letter groups (digrams) and three-letter groups (trigrams) for the English text are shown in Table

Example #3

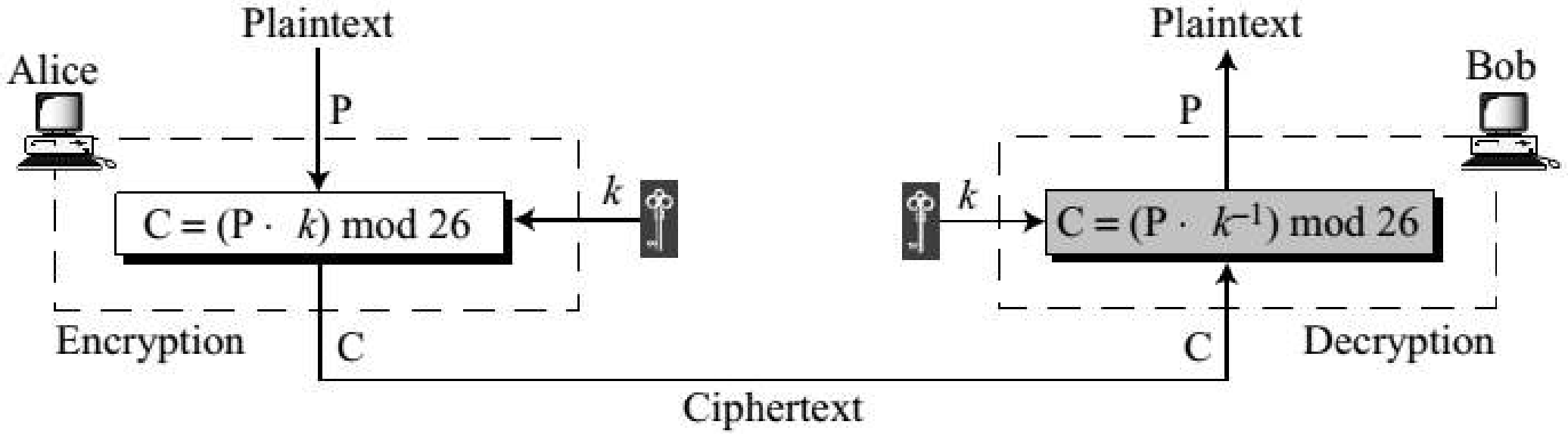
- Eve has intercepted the following ciphertext. Using a statistical attack, find the plaintext.

```
XLILSYWIMWRSAJSVWEPIJSVJSYVQMPPMSRHSPPEVWMXMWASVX-LQSVILY-
VVCFIJSVIXLIWIPPIVVIGIMZIWQSVISJJIVW
```

  - When Eve tabulates the frequency of letters in this ciphertext, she gets: I =14, V =13, S =12, and so on.
  - The most common character is I with 14 occurrences.
  - This shows that character I in the ciphertext probably corresponds to the character e in plaintext. This means key = 4. Eve deciphers the text to get

```
the house is now for sale for four million dollars it is worth more hurry before the seller
receives more offers
```

Multiplicative Ciphers



In a multiplicative cipher, the plaintext and ciphertext are integers in Z26; The key is an integer in Z26*.

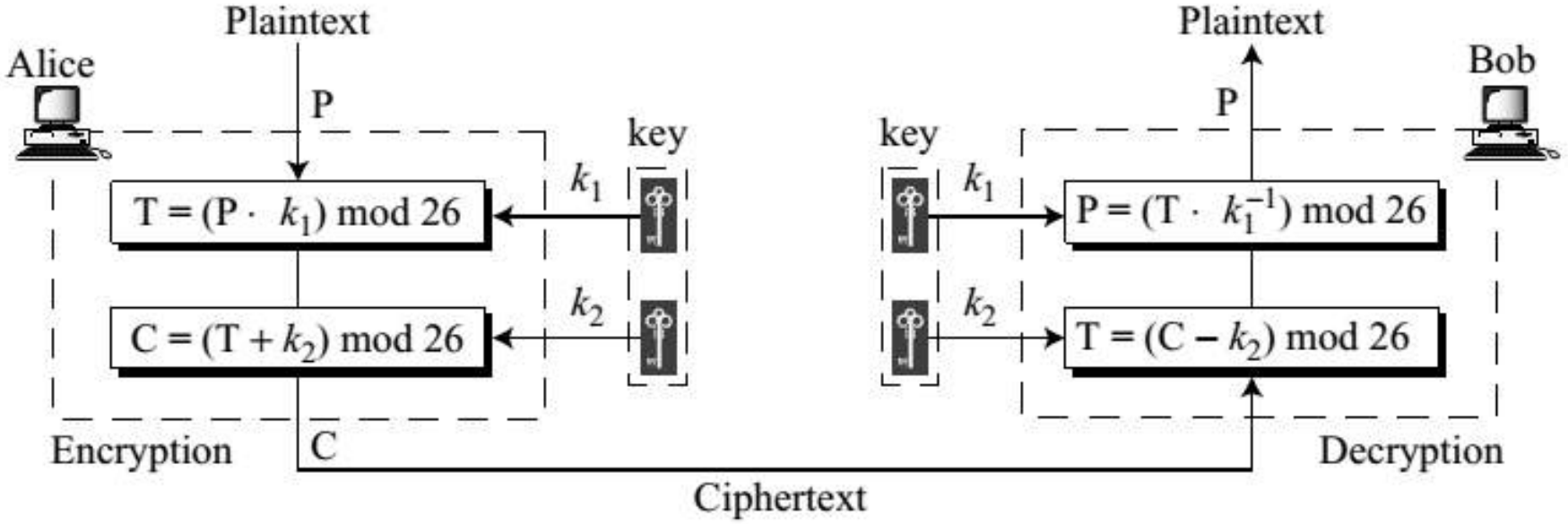
Multiplicative Ciphers

- What is the key domain for any multiplicative cipher?
  - The key needs to be in Z_{26}^*. This set has only 12 members: 1, 3, 5, 7, 9, 11, 15, 17, 19, 21,23, 25.
- We use a multiplicative cipher to encrypt the message “hello” with a key of 7.

Plaintext: h $\rightarrow$ 07	Encryption: $(07 \times 07) \bmod 26$	ciphertext: 23 $\rightarrow$ X
Plaintext: e $\rightarrow$ 04	Encryption: $(04 \times 07) \bmod 26$	ciphertext: 02 $\rightarrow$ C
Plaintext: l $\rightarrow$ 11	Encryption: $(11 \times 07) \bmod 26$	ciphertext: 25 $\rightarrow$ Z
Plaintext: l $\rightarrow$ 11	Encryption: $(11 \times 07) \bmod 26$	ciphertext: 25 $\rightarrow$ Z
Plaintext: o $\rightarrow$ 14	Encryption: $(14 \times 07) \bmod 26$	ciphertext: 20 $\rightarrow$ U

Affine Cipher

- Combine the additive and multiplicative ciphers to get what is called the affine cipher-a combination of both ciphers with a pair of keys.
- The first key is used with the multiplicative cipher; the second key is used with the additive cipher.



Affine Cipher

$$C = (P \times k_1 + k_2) \bmod 26 \qquad P = ((C - k_2) \times k_1^{-1}) \bmod 26$$

where k_1^{-1} is the multiplicative inverse of k_1 and $-k_2$ is the additive inverse of k_2

Use an affine cipher to encrypt the message “hello” with the key pair (7, 2).

P: h → 07	Encryption: $(07 \times 7 + 2) \bmod 26$	C: 25 → Z
P: e → 04	Encryption: $(04 \times 7 + 2) \bmod 26$	C: 04 → E
P: l → 11	Encryption: $(11 \times 7 + 2) \bmod 26$	C: 01 → B
P: l → 11	Encryption: $(11 \times 7 + 2) \bmod 26$	C: 01 → B
P: o → 14	Encryption: $(14 \times 7 + 2) \bmod 26$	C: 22 → W

Affine Cipher

- Use the affine cipher to decrypt the message “ZEBBW” with the key pair (7, 2) in modulus 26.
- Add the additive inverse of −2 ≡ 24 (mod 26) to the received ciphertext.
- Then multiply the result by the multiplicative inverse of $7^{-1} \equiv 15 \pmod{26}$ to find the plaintext characters.
- Because 2 has an additive inverse in Z_{26} and 7 has a multiplicative inverse in Z_{26}^*

C: Z → 25	Decryption: $((25 - 2) \times 7^{-1}) \bmod 26$	P:07 → h
C: E → 04	Decryption: $((04 - 2) \times 7^{-1}) \bmod 26$	P:04 → e
C: B → 01	Decryption: $((01 - 2) \times 7^{-1}) \bmod 26$	P:11 → l
C: B → 01	Decryption: $((01 - 2) \times 7^{-1}) \bmod 26$	P:11 → l
C: W → 22	Decryption: $((22 - 2) \times 7^{-1}) \bmod 26$	P:14 → o

Cryptanalysis of Affine Cipher

PWUFFOGWCHFDWIWEJOUUNJORSMDWRHVCMWJUPVCCG

Although the brute-force and statistical method of ciphertext-only attack can be used, let us try a chosen-plaintext attack.

Eve also very briefly obtains access to Alice's computer and has only enough time to type a two-letter plaintext: "et".

She then tries to encrypt the short plaintext using two different algorithms, because she is not sure which one is the affine cipher.

Algorithm 1:	Plaintext: et	ciphertext: → WC
Algorithm 2:	Plaintext: et	ciphertext: → WF

Cryptanalysis of Affine Cipher

Eve knows that if the first algorithm is affine, she can construct the following two equations based on the first data set.

e → W	04 → 22	$(04 \times k_1 + k_2) \equiv 22 \pmod{26}$
t → C	19 → 02	$(19 \times k_1 + k_2) \equiv 02 \pmod{26}$

These two congruence equations can be solved and the values of k_1 and k_2 can be found.

$$\begin{bmatrix} k_1 \\ k_2 \end{bmatrix} = \begin{bmatrix} 4 & 1 \\ 19 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 22 \\ 2 \end{bmatrix} = \begin{bmatrix} 19 & 7 \\ 3 & 24 \end{bmatrix} \begin{bmatrix} 22 \\ 2 \end{bmatrix} = \begin{bmatrix} 16 \\ 10 \end{bmatrix} \longrightarrow k_1 = 16 \quad k_2 = 10$$

However, this answer is not acceptable because $k_1 = 16$ cannot be the first part of the key. Its value, 16, does not have a multiplicative inverse in Z_{26}^*.

Cryptanalysis of Affine Cipher

Eve now tries the result of the second set of data

e → W	04 → 22	$(04 \times k_1 + k_2) \equiv 22 \pmod{26}$
t → F	19 → 05	$(19 \times k_1 + k_2) \equiv 05 \pmod{26}$

The square matrix and its inverse are the same. Now she has $k_1 = 11$ and $k_2 = 4$.
This pair is acceptable because k_1 has a multiplicative inverse in $Z_{26}*$.
She tries the pair of keys (19, 22), which are the inverse of the pair (11, 4), to decipher the message. The plaintext is

best time of the year is spring when flowers bloom

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 5 : Symmetric-Key Ciphers – Part-2

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

Monoalphabetic Substitution Cipher

- In this method, it is to create a mapping between each plaintext character and the corresponding ciphertext character.
- Alice and Bob can agree on a table showing the mapping for each character.

Plaintext →	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Ciphertext →	N	O	A	T	R	B	E	C	F	U	X	D	Q	G	Y	L	K	H	V	I	J	M	P	Z	S	W

An example key for monoalphabetic substitution cipher

this message is easy to encrypt but hard to find the key

ICFVQRVVNEFVRNVSIYRGAHSLIOJICNHTIYBFGTICRXRS

Cryptanalysis

- The size of the key space for the monoalphabetic substitution cipher is 26! .
- This makes a brute-force attack extremely difficult for Eve even if she is using a powerful computer.
- However, she can use statistical attack based on the frequency of characters.
- The cipher does not change the frequency of characters.
- The monoalphabetic ciphers do not change the frequency of characters in the ciphertext, which makes the ciphers vulnerable to statistical attack.

Polyalphabetic Ciphers

- In polyalphabetic substitution, each occurrence of a character may have a different substitute.
- The relationship between a character in the plaintext to a character in the ciphertext is one-to-many.
- Polyalphabetic ciphers have the advantage of hiding the letter frequency of the underlying language.
- Eve cannot use single-letter frequency statistic to break the ciphertext.

Autokey Cipher

- In this cipher, the key is a stream of subkeys, in which each subkey is used to encrypt the corresponding character in the plaintext.
- The first subkey is a predetermined value secretly agreed upon by Alice and Bob.
- The second subkey is the value of the first plaintext character (between 0 and 25).
- The third subkey is the value of the second plaintext. And so on.

$$P = P_1P_2P_3 \ldots \qquad C = C_1C_2C_3\ldots \qquad k = (k_1, P_1, P_2, \ldots)$$

Encryption: $C_i = (P_i + k_i) \bmod 26$ Decryption: $P_i = (C_i - k_i) \bmod 26$

Autokey Cipher

- However, it is still as vulnerable to the brute-force attack as the additive cipher.

Plaintext:	a	t	t	a	c	k	i	s	t	o	d	a	y
P's Values:	00	19	19	00	02	10	08	18	19	14	03	00	24
Key stream:	*12*	*00*	*19*	*19*	*00*	*02*	*10*	*08*	*18*	*19*	*14*	*03*	*00*
C's Values:	12	19	12	19	02	12	18	00	11	7	17	03	24
Ciphertext:	**M**	**T**	**M**	**T**	**C**	**M**	**S**	**A**	**L**	**H**	**R**	**D**	**Y**

However, it is still as vulnerable to the brute-force attack as the additive cipher.

Playfair Cipher

- Playfair cipher used by the British army during World War I.
- The secret key in this cipher is made of 25 alphabet letters arranged in a 5 × 5 matrix (letters I and J are considered the same when encrypting).
- Different arrangements of the letters in the matrix can create many different secret keys.
- The letters dropped in the matrix diagonally starting from the top right-hand corner.
- Before encryption,
  - if the two letters in a pair are the same, a bogus letter is inserted to separate them.
  - After inserting bogus letters, if the number of characters in the plaintext is odd, one extra bogus character is added at the end to make the number of characters even.

Secret Key =

L	G	D	B	A
Q	M	H	E	C
U	R	N	I/J	F
X	V	S	O	K
Z	Y	W	T	P

Playfair Cipher

- The cipher uses three rules for encryption:

  a. If the two letters in a pair are located in the same row of the secret key, the corresponding encrypted character for each letter is the next letter to the right in the same row (with wrapping to the beginning of the row if the plaintext letter is the last character in the row).

  b. If the two letters in a pair are located in the same column of the secret key, the corresponding encrypted character for each letter is the letter beneath it in the same column (with wrapping to the beginning of the column if the plaintext letter is the last character in the column).

  c. If the two letters in a pair are not in the same row or column of the secret, the corresponding encrypted character for each letter is a letter that is in its own row but in the same column as the other letter.

Playfair Cipher

$P = P_1P_2P_3 \ldots$ $C = C_1C_2C_3 \ldots$ $k = [(k_1, k_2), (k_3, k_4), \ldots]$

Encryption: $C_i = k_i$ Decryption: $P_i = k_i$

Let us encrypt the plaintext "hello" using the key in Figure

Secret Key =

L	G	D	B	A
Q	M	H	E	C
U	R	N	I/J	F
X	V	S	O	K
Z	Y	W	T	P

When we group the letters in two-character pairs, we get "he, ll, o".
We need to insert an x between the two l's (els), giving "he, lx, lo".

he → EC lx → QZ lo → BX

Plaintext: hello Ciphertext: ECQZBX

Cryptanalysis of a Playfair Cipher

- Obviously a brute-force attack on a Playfair cipher is very difficult.
  - The size of the key domain is 25!
- However, the frequencies of diagrams are preserved (to some extent because of filler insertion), so a cryptanalyst can use a ciphertext-only attack based on the digram frequency test to find the key.

One-Time Pad

- One of the goals of cryptography is perfect secrecy.
- A study by Shannon has shown that perfect secrecy can be achieved if each plaintext symbol is encrypted with a key randomly chosen from a key domain.
- For example, an additive cipher can be easily broken because the same key is used to encrypt every character.
- However, even this simple cipher can become a perfect cipher if the key that is used to encrypt each character is chosen randomly from the key domain (00, 01, 02, …, 25) - that is, if the first character is encrypted using the key 04, the second character is encrypted using the key 02, the third character is encrypted using the key 21; and so on.
- This idea is used in a cipher called one-time pad, invented by Vernam.
- In this cipher, the key has the same length as the plaintext and is chosen completely in random.

One-Time Pad

- A one-time pad is a perfect cipher, but it is almost impossible to implement commercially.
- If the key must be newly generated each time, how can Alice tell Bob the new key each time she has a message to send?
- However, there are some occasions when a one-time pad can be used.
- For example, if the president of a country needs to send a completely secret message to the president of another country, she can send a trusted envoy with the random key before sending the message.

TRANSPOSITION CIPHERS

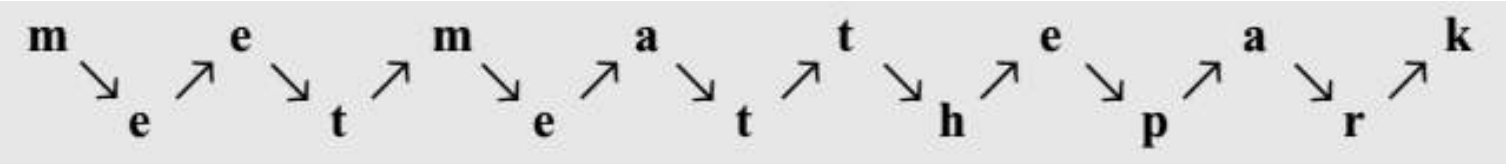
- A transposition cipher does not substitute one symbol for another, instead it changes the location of the symbols.
- A symbol in the first position of the plaintext may appear in the tenth position of the ciphertext.
- A symbol in the eighth position in the plaintext may appear in the first position of the ciphertext.
- In other words, a transposition cipher reorders (transposes) the symbols.

Keyless Transposition Ciphers

- Simple transposition ciphers, which were used in the past, are keyless.
- There are two methods for permutation of characters.
- In the first method, the text is written into a table column by column and then transmitted row by row.
- In the second method, the text is written into the table row by row and then transmitted column by column.

Keyless Transposition Ciphers
Rail fence cipher

- In this cipher, the plaintext is arranged in two lines as a zigzag pattern (which means column by column);
- The ciphertext is created reading the pattern row by row.
- For example, to send the message “Meet me at the park” to Bob, Alice writes



- She then creates the ciphertext “MEMATEAKETETHPR” by sending the first row followed by the second row.
- Bob receives the ciphertext and divides it in half (in this case the second half has one less character).
- The first half forms the first row; the second half, the second row. Bob reads the result in zigzag.

Keyless Transposition Ciphers
Transposition cipher

- Alice and Bob can agree on the number of columns and use the second method.
- Alice writes the same plaintext, row by row, in a table of four columns.
- She then creates the ciphertext "MMTAEEHREAEKTTP" by transmitting the characters column by column.
- Bob receives the ciphertext and follows the reverse process.
- He writes the received message, column by column, and reads it row by row as the plaintext.

m	e	e	t
m	e	a	t
t	h	e	p
a	r	k	

Keyless Transposition Ciphers
Transposition cipher

- The following figure shows the permutation of each character in the plaintext into the ciphertext based on the positions.

01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
01	05	09	13	02	06	10	13	03	07	11	15	04	08	12

m	**e**	**e**	**t**
m	**e**	**a**	**t**
t	**h**	**e**	**p**
a	**r**	**k**	

- The second character in the plaintext has moved to the fifth position in the ciphertext;
- The third character has moved to the ninth position; and so on.
- Although the characters are permuted, there is a pattern in the permutation: (01, 05, 09, 13), (02, 06, 10, 13), (03, 07, 11, 15), and (08,12).
- In each section, the difference between the two adjacent numbers is 4.

Keyed Transposition Ciphers

- The keyless ciphers permute the characters by using writing plaintext in one way (row by row, for example) and reading it in another way (column by column, for example).
- The permutation is done on the whole plaintext to create the whole ciphertext.
- Another method is to divide the plaintext into groups of predetermined size, called blocks, and then use a key to permute the characters in each block separately.

Keyed Transposition Ciphers

- Alice needs to send the message “Enemy attacks tonight” to Bob.
- Alice and Bob have agreed to divide the text into groups of five characters and then permute the characters in each group.
- The following shows the grouping after adding a bogus character at the end to make the last group the same size as the others.

e	n	e	m	y		a	t	t	a	c		k	s	t	o	n		i	g	h	t	z

- The key used for encryption and decryption is a permutation key, which shows how the character are permuted.
- For this message, assume that Alice and Bob used the following key

Encryption ↓

3	1	4	5	2
1	2	3	4	5

↑ Decryption

Keyed Transposition Ciphers

- The third character in the plaintext block becomes the first character in the ciphertext block;
- The first character in the plaintext block becomes the second character in the ciphertext block; and so on.
- The permutation yields

E E M Y N T A A C T T K O N S H I T Z G

- Alice sends the ciphertext “EEMYNTAACTTKONSHITZG” to Bob.
- Bob divides the ciphertext into 5-character groups and, using the key in the reverse order, finds the plaintext.

STREAM AND BLOCK CIPHERS

- Symmetric ciphers into two broad categories: stream ciphers and block ciphers.
- Stream Ciphers
  - In a stream cipher, encryption and decryption are done one symbol (such as a character or a bit) at a time.
  - We have a plaintext stream, a ciphertext stream, and a key stream.
  - Call the plaintext stream P, the ciphertext stream C, and the key stream K.

$$P = P_1P_2P_3, \ldots \qquad C = C_1C_2C_3, \ldots \qquad K = (k_1, k_2, k_3, \ldots)$$

$$C_1 = E_{k1}(P_1) \qquad C_2 = E_{k2}(P_2) \qquad C_3 = E_{k3}(P_3) \ldots$$

STREAM AND BLOCK CIPHERS

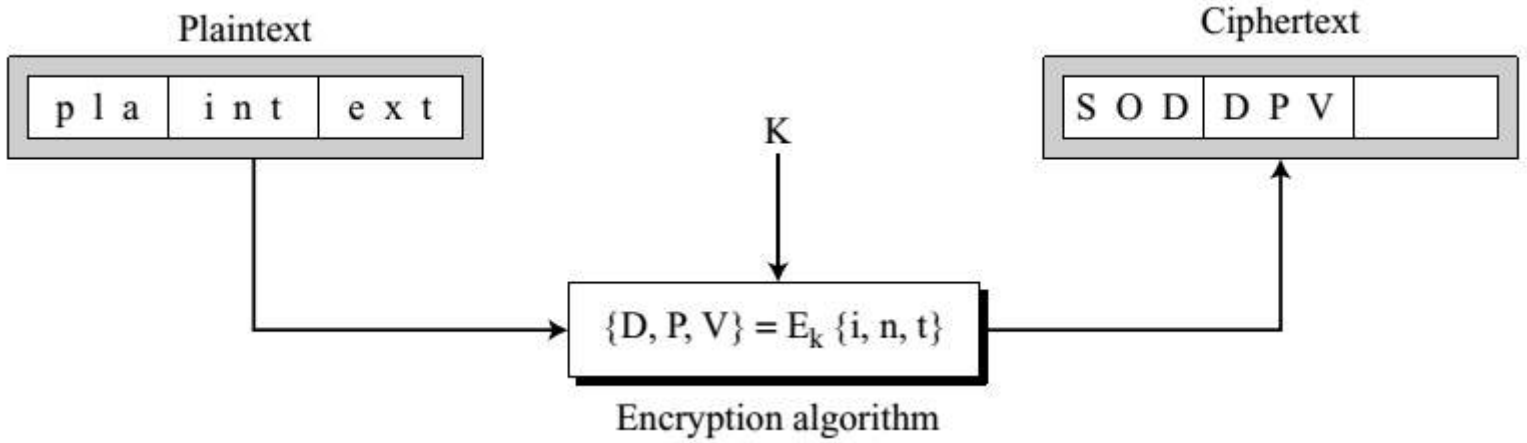
- Symmetric ciphers into two broad categories: stream ciphers and block ciphers.
- Stream Ciphers
  - In a stream cipher, encryption and decryption are done one symbol (such as a character or a bit) at a time.
  - We have a plaintext stream, a ciphertext stream, and a key stream.
  - Call the plaintext stream P, the ciphertext stream C, and the key stream K.

$$P = P_1P_2P_3, \ldots \qquad C = C_1C_2C_3, \ldots \qquad K = (k_1, k_2, k_3, \ldots)$$

$$C_1 = E_{k1}(P_1) \qquad C_2 = E_{k2}(P_2) \qquad C_3 = E_{k3}(P_3) \ldots$$

STREAM AND BLOCK CIPHERS

- Block Ciphers
  - In a block cipher, a group of plaintext symbols of size m (m > 1) are encrypted together creating a group of ciphertext of the same size.
  - Based on the definition, in a block cipher, a single key is used to encrypt the whole block even if the key is made of multiple values.
  - In a block cipher, a ciphertext block depends on the whole plaintext block.



STREAM AND BLOCK CIPHERS

- Stream ciphers

- Additive ciphers
- Monoalphabetic substitution ciphers
- Vigenere ciphers

- Block ciphers

- Playfair ciphers
- Polyalphabetic cipher
- DES and AES

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 6: Primes and Congruence

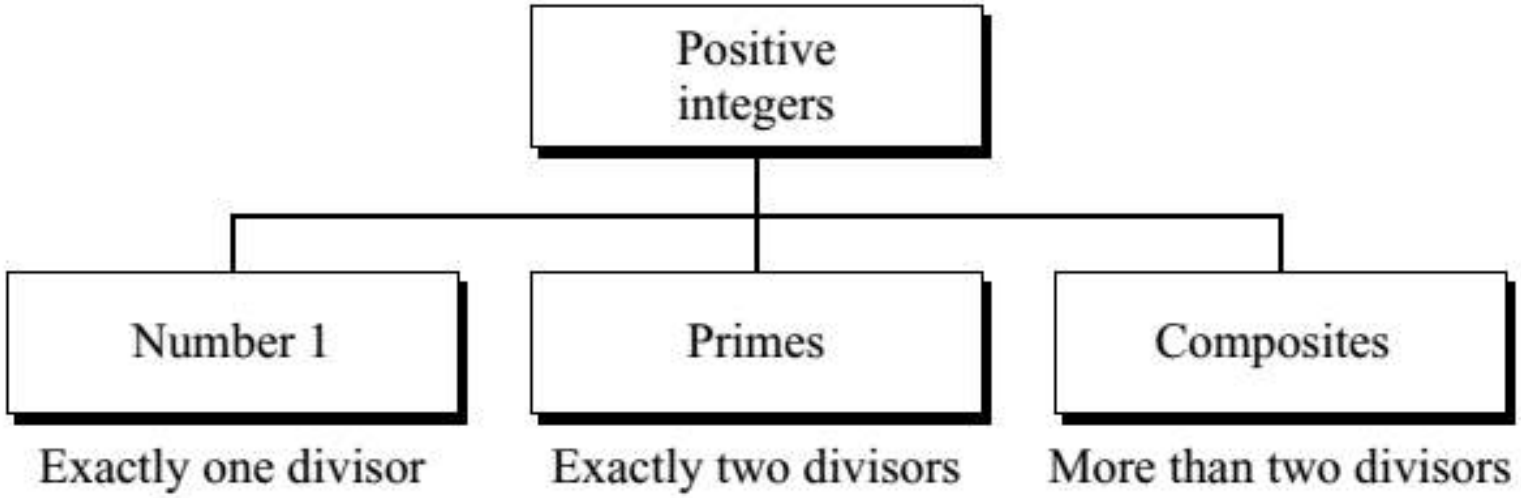
Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

PRIMES

- Asymmetric-key cryptography uses primes extensively.
- The topic of primes is a large part of any book on number theory.
- The positive integers can be divided into three groups: the number 1, primes, and composites.



PRIMES

- A positive integer is a prime if and only if it is exactly divisible by two integers, 1 and itself.
- A composite is a positive integer with more than two divisors.
- Two positive integers, a and b, are relatively prime, or coprime, if gcd (a, b) = 1.
- If p is a prime, then all integers 1 to p − 1 are relatively prime to p.
- Is there a finite number of primes or is the list infinite?
  - There is an infinite number of primes.
- A function called π(n) is defined that finds the number of primes smaller than or equal to n.

$$\pi(1) = 0 \quad \pi(2) = 1 \quad \pi(3) = 2 \quad \pi(10) = 4 \quad \pi(20) = 8 \quad \pi(50) = 15 \quad \pi(100) = 25$$

PRIMES

- But if n is very large, how can we calculate π(n)?
- The answer is that we can only use approximation.

$$[n / (\ln n)] \quad < \quad \pi(n) \quad < \quad [n/(\ln n - 1.08366)]$$

- Gauss discovered the upper limit; Lagrange discovered the lower limit.
- Find the number of primes less than 1,000,000.
- The approximation gives the range 72,383 to 78,543. The actual number of primes is 78,498.

Checking for Primeness

- Given a number n, how can we determine if n is a prime?
- The answer is that we need to see if the number is divisible by all primes less than √n.
- Is 97 a prime?
  - The floor of √97= 9.
  - The primes less than 9 are 2, 3, 5, and 7.
  - We need to see if 97 is divisible by any of these numbers.
  - It is not, so 97 is a prime.
- Is 301 a prime?
  - The floor of √301= 17.
  - We need to check 2, 3, 5, 7, 11, 13, and 17.
  - The numbers 2, 3, and 5 do not divide 301, but 7 does.
  - Therefore 301 is not a prime.

Euler's Phi-Function

- Euler's phi-function, $\phi(n)$, which is sometimes called the Euler's totient function plays a very important role in cryptography.
- The function finds the number of integers that are both smaller than n and relatively prime to n.
- The following helps to find the value of$\phi(n)$

1. $\phi(1) = 0$.
2. $\phi(p) = p - 1$ if p is a prime.
3. $\phi(m \times n) = \phi(m) \times \phi(n)$ if m and n are relatively prime.
4. $\phi(p^e) = p^e - p^{e-1}$ if p is a prime.

Euler's Phi-Function

- We can combine the above four rules to find the value of $\phi(n)$. For example, if n can be factored as

$$n = p_1^{e1} \times p_2^{e2} \times \ldots \times p_k^{ek}$$

- Then we combine the third and the fourth rule to find

$$\phi(n) = (p_1^{e_1} - p_1^{e_1-1}) \times (p_2^{e_2} - p_2^{e_2-1}) \times \cdots \times (p_k^{e_k} - p_k^{e_k-1})$$

The difficulty of finding $\phi(n)$ depends on the difficulty of finding the factorization of n.

Euler's Phi-Function

- Example 1
  - What is the value of $\phi(13)$?
- Solution
  - Because 13 is a prime, $\phi(13) = (13 - 1) = 12$.
- Example 2
  - What is the value of $\phi(10)$?
- Solution
  - We can use the third rule: $\phi(10) = \phi(2) \times \phi(5) = 1 \times 4 = 4$, because 2 and 5 are primes.

Euler's Phi-Function

- Example 3
  - What is the value of $\phi(240)$?
- Solution
  - We can write $240 = 2^4 \times 3^1 \times 5^1$. Then

$$\phi(240) = (2^4 - 2^3) \times (3^1 - 3^0) \times (5^1 - 5^0) = 64$$

- Example 4
  - Can we say that $\phi(49) = \phi(7) \times \phi(7) = 6 \times 6 = 36$????

Euler's Phi-Function

- Example 3
  - What is the value of $\phi(240)$?
- Solution
  - We can write $240 = 2^4 \times 3^1 \times 5^1$. Then

    $$\phi(240) = (2^4 - 2^3) \times (3^1 - 3^0) \times (5^1 - 5^0) = 64$$
- Example 4
  - Can we say that $\phi(49) = \phi(7) \times \phi(7) = 6 \times 6 = 36$???
- Solution
  - No. The third rule applies when *m* and *n* are relatively prime. Here $49 = 7^2$. We need to use the fourth rule: $\phi(49) = 7^2 - 7^1 = 42$.

Euler's Phi-Function

- Example 5
  - What is the number of elements in Z_{14}^*?
- Solution
  - The answer is $\phi(14) = \phi(7) \times \phi(2) = 6 \times 1 = 6$. The members are 1, 3, 5, 9, 11, and 13.

Interesting point: If $n > 2$, the value of $\phi(n)$ is even.

Fermat's Little Theorem

- First Version
  - If p is a prime and a is an integer such that p does not divide a,

$$a^{p-1} \equiv 1 \bmod p$$

- Second Version
  - Removes the condition on a
  - If p is prime and a is an integer,

$$a^{p} \equiv a \bmod p$$

Fermat's Little Theorem

- Example 1
  - Find the result of $6^{10} \bmod 11$.
- Solution
  - We have $6^{10} \bmod 11 = 1$. This is the first version of Fermat's little theorem where $p = 11$.
- Example 2
  - Find the result of $3^{12} \bmod 11$.
- Solution

Fermat's Little Theorem

- Example 1
  - Find the result of $6^{10} \bmod 11$.
- Solution
  - We have $6^{10} \bmod 11 = 1$. This is the first version of Fermat's little theorem where $p = 11$.
- Example 2
  - Find the result of $3^{12} \bmod 11$.
- Solution
  - Here the exponent (12) and the modulus (11) are not the same. With substitution this can be solved using Fermat's little theorem.

$$3^{12} \bmod 11 = (3^{11} \times 3) \bmod 11 = (3^{11} \bmod 11)\ (3 \bmod 11) = (3 \times 3) \bmod 11 = 9$$

Fermat's Little Theorem

- Multiplicative Inverses

$$a^{-1} \bmod p = a^{p-2} \bmod p$$

- The answers to multiplicative inverses modulo a prime can be found without using the extended Euclidean algorithm:

a. $8^{-1} \bmod 17 = 8^{17-2} \bmod 17 = 8^{15} \bmod 17 = 15 \bmod 17$

b. $5^{-1} \bmod 23 = 5^{23-2} \bmod 23 = 5^{21} \bmod 23 = 14 \bmod 23$

c. $60^{-1} \bmod 101 = 60^{101-2} \bmod 101 = 60^{99} \bmod 101 = 32 \bmod 101$

d. $22^{-1} \bmod 211 = 22^{211-2} \bmod 211 = 22^{209} \bmod 211 = 48 \bmod 211$

Euler's Theorem

- The modulus in the Fermat theorem is a prime, the modulus in Euler's theorem is an integer.
  - First Version
    - If a and n are coprime,

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

  - Second Version
    - Removes the condition that a and n should be coprime

$$a^{k \times \phi(n) + 1} \equiv a \pmod{n}$$

The second version of Euler's theorem is used in the RSA cryptosystem

Euler's Theorem

- Example 1
  - Find the result of $6^{24} \bmod 35$.
- Solution
  - We have $6^{24} \bmod 35 = 6^{\phi(35)} \bmod 35 = 1$.
- Example 2
  - Find the result of $20^{62} \bmod 77$???

Euler's Theorem

- Example 1
  - Find the result of $6^{24} \bmod 35$.
- Solution
  - We have $6^{24} \bmod 35 = 6^{\phi(35)} \bmod 35 = 1$.
- Example 2
  - Find the result of $20^{62} \bmod 77$.
- Solution

If we let $k = 1$ on the second version, we have

$$20^{62} \bmod 77 = (20 \bmod 77)\,(20^{\phi(77)+1} \bmod 77) \bmod 77$$
$$= (20)(20) \bmod 77 = 15.$$

Euler's Theorem

- Multiplicative Inverses
  - Euler's theorem can be used to find multiplicative inverses modulo a composite.

$$a^{-1} \bmod n = a^{\phi(n)-1} \bmod n$$

Euler's Theorem

- Example
  - The answers to multiplicative inverses modulo a composite can be found without using the extended Euclidean algorithm if we know the factorization of the composite:

a. $8^{-1} \bmod 77 = 8^{\phi(77)-1} \bmod 77 = 8^{59} \bmod 77 = 29 \bmod 77$

b. $7^{-1} \bmod 15 = 7^{\phi(15)-1} \bmod 15 = 7^{7} \bmod 15 = 13 \bmod 15$

c. $60^{-1} \bmod 187 = 60^{\phi(187)-1} \bmod 187 = 60^{159} \bmod 187 = 53 \bmod 187$

d. $71^{-1} \bmod 100 = 71^{\phi(100)-1} \bmod 100 = 71^{39} \bmod 100 = 31 \bmod 100$

Generating Primes

Mersenne Primes

$$M_p = 2^p - 1$$

$M_2 = 2^2 - 1 = 3$
$M_3 = 2^3 - 1 = 7$
$M_5 = 2^5 - 1 = 31$
$M_7 = 2^7 - 1 = 127$
$M_{11} = 2^{11} - 1 = 2047$ **Not a prime ($2047 = 23 \times 89$)**
$M_{13} = 2^{13} - 1 = 8191$
$M_{17} = 2^{17} - 1 = 131071$

A number in the form $M_p = 2^p - 1$ is called a Mersenne number and may or may not be a prime.

Generating Primes

Fermat Primes $F_n = 2^{2^n} + 1$

$F_0 = 3$
$F_1 = 5$
$F_2 = 17$
$F_3 = 257$
$F_4 = 65537$
$F_5 = 4294967297 = 641 \times 6700417$ **Not a prime**

No number greater than F4 has been proven to be a prime.
As a matter of fact many numbers up to F24 have been proven to be composite numbers.

Primality Testing

- Finding an algorithm to correctly and efficiently test a very large integer and output a prime or a composite has always been a challenge in number theory.
- Two types
  - Deterministic Algorithms
  - Probabilistic Algorithms

Primality Testing

```
Divisibility_Test (n)                    // n is the number to test for primality
{
  r ← 2
  while (r < √n )
  {
   if (r | n) return "a composite"
   r ← r + 1
  }
  return "a prime"
}
```

The bit-operation complexity of the divisibility test is $O(2^{n/2})$ (exponential)

Deterministic Algorithms

```
Divisibility_Test (n)                    // n is the number to test for primality
{
  r ← 2
  while (r < √n )
  {
   if (r | n) return "a composite"
   r ← r + 1
  }
  return "a prime"
}
```

The bit-operation complexity of the divisibility test is $O(2^{n\sqrt{2}})$ (exponential)

Deterministic Algorithms

- Example
  - Assume n has 200 bits. What is the number of bit operations needed to run the divisibility-test algorithm?
- Solution
  - The bit-operation complexity of this algorithm is $2^{n_b/2}$. This means that the algorithm needs 2^{100} bit operations. On a computer capable of doing 2^{30} bit operations per second, the algorithm needs 2^{70} seconds to do the testing !!!!!

Deterministic Algorithms

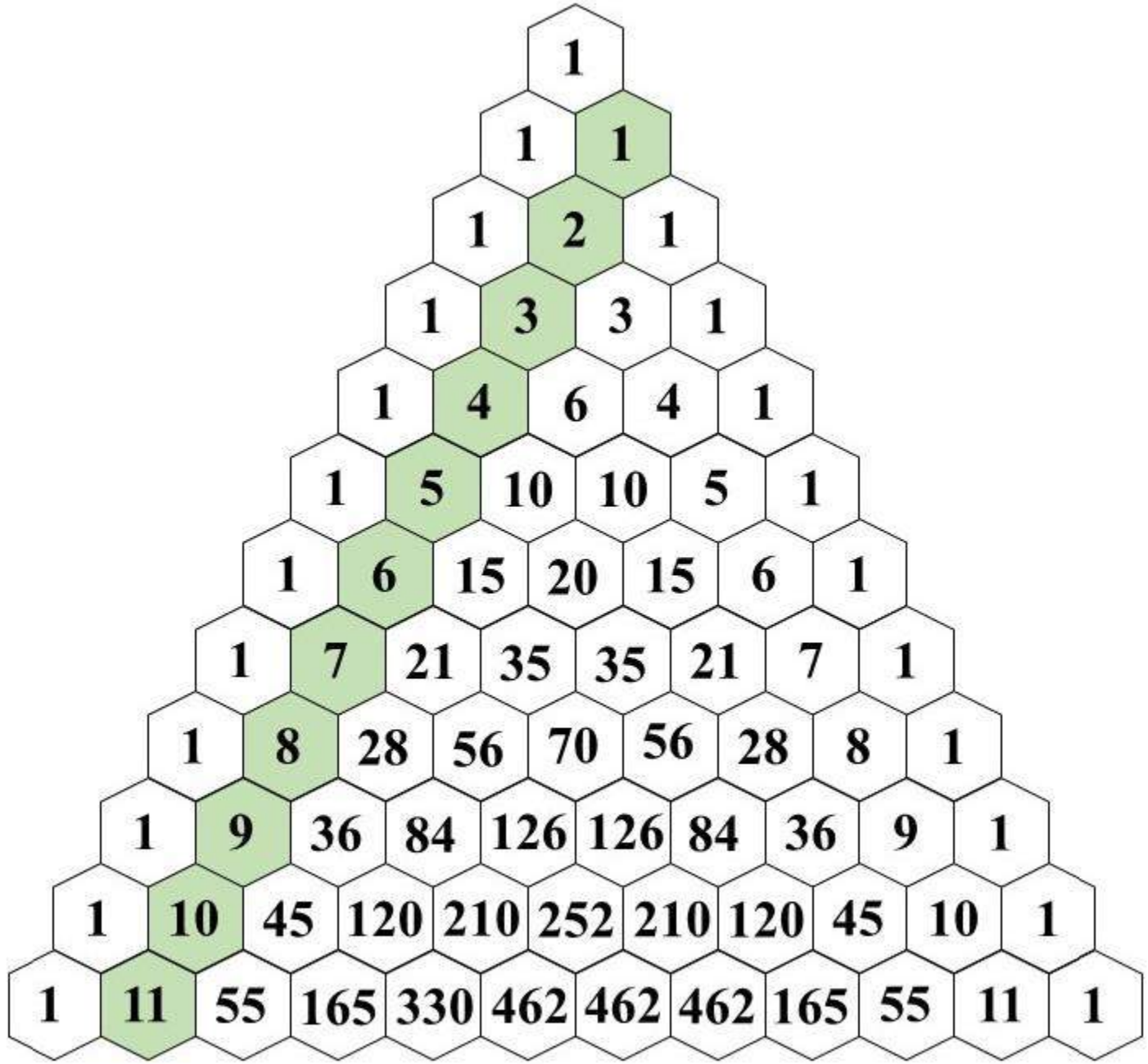
AKS Algorithm

$$(x - a)^p \equiv (x^p - a) \bmod p.$$

$$O((\log_2 n_b)^{12})$$

Tutorial

- Example
  - Assume n has 200 bits. What is the number of bit operations needed to run the AKS algorithm?
- Solution
  - This algorithm needs only $(\log_2 200)^{12}$ = 39,547,615,483 bit operations. On a computer capable of doing 1 billion bit operations per second, the algorithm needs only 40 seconds.


1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
1 10 45 120 210 252 210 120 45 10 1
1 11 55 165 330 462 462 462 165 55 11 1

Probabilistic Algorithms

Fermat Test

If n is a prime, then $a^{n-1} \equiv 1 \bmod n$.

If n is a prime, $a^{n-1} \equiv 1 \bmod n$
If n is a composite, it is possible that $a^{n-1} \equiv 1 \bmod n$

Probabilistic Algorithms

- Example
  - Does the number 561 pass the Fermat test?
- Solution
  - Use base 2

$$2^{561-1} = 1 \bmod 561$$

  - The number passes the Fermat test, but it is not a prime, because 561 = 33 × 17.

Probabilistic Algorithms

Square Root Test

If n is a prime, $\sqrt{1} \bmod n = \pm 1$.
If n is a composite, $\sqrt{1} \bmod n = \pm 1$ and possibly other values.

- Example
  - What are the square roots of 1 mod n if n is 7 (a prime)?
- Solution
  - The only square roots are 1 and −1. We can see that

$$1^2 = 1 \bmod 7 \qquad (-1)^2 = 1 \bmod 7$$
$$2^2 = 4 \bmod 7 \qquad (-2)^2 = 4 \bmod 7$$
$$3^2 = 2 \bmod 7 \qquad (-3)^2 = 2 \bmod 7$$

  - Note that we don't have to test 4, 5 and 6 because $4 = -3 \bmod 7$, $5 = -2 \bmod 7$ and $6 = -1 \bmod 7$.

Probabilistic Algorithms

Square Root Test

- Example
  - What are the square roots of 1 mod n if n is 8 (a composite)?
- Solution
  - There are four solutions: 1, 3, 5, and 7 (which is −1). We can see that

$$1^2 = 1 \bmod 8 \qquad (-1)^2 = 1 \bmod 8$$
$$3^2 = 1 \bmod 8 \qquad 5^2 = 1 \bmod 8$$

Probabilistic Algorithms

Square Root Test

- Example
  - What are the square roots of 1 mod n if n is 17 (a prime)?
- Solution
  - There are only two solutions: 1 and −1

$1^2 = 1 \bmod 17$	$(-1)^2 = 1 \bmod 17$
$2^2 = 4 \bmod 17$	$(-2)^2 = 4 \bmod 17$
$3^2 = 9 \bmod 17$	$(-3)^2 = 9 \bmod 17$
$4^2 = 16 \bmod 17$	$(-4)^2 = 16 \bmod 17$
$5^2 = 8 \bmod 17$	$(-5)^2 = 8 \bmod 17$
$6^2 = 2 \bmod 17$	$(-6)^2 = 2 \bmod 17$
$(7)^2 = 15 \bmod 17$	$(-7)^2 = 15 \bmod 17$
$(8)^2 = 13 \bmod 17$	$(-8)^2 = 13 \bmod 17$

Probabilistic Algorithms

Square Root Test

- Example
  - What are the square roots of 1 mod n if n is 22 (a composite)??????

    - Solution
      - Surprisingly, there are only two solutions, +1 and −1, although 22 is a composite.

$$1^2 = 1 \bmod 22$$
$$(-1)^2 = 1 \bmod 22$$

Probabilistic Algorithms

- Miller-Rabin Test

$$n - 1 = m \times 2^k$$

$$a^{n-1} = a^{m \times 2^k} = [a^m]^{2^k} = [a^m]^{2^{2^{\cdot^{\cdot^{\cdot^{2}}}}}} \quad \text{(k times)}$$

The Miller-Rabin test needs from step 0 to step k − 1.

Probabilistic Algorithms

```
Miller_Rabin_Test (n, a)                    // n is the number; a is the base.
{
    Find m and k such that n − 1 = m × 2^k
    T ← a^m mod n
    if (T = ± 1)   return "a prime"
    for (i ← 1 to k − 1)                    // k − 1 is the maximum number of steps.
    {
        T ← T^2 mod n
        if (T = +1) return "a composite"
        if (T = −1) return "a prime"
    }
    return "a composite"
}
```

Probabilistic Algorithms

- Example
  - Does the number 561 pass the Miller-Rabin test?
- Solution
  - Using base 2, let $561 - 1 = 35 \times 2^4$, which means m = 35, k = 4, and a = 2.

Initialization:	$T = 2^{35} \bmod 561 = 263 \bmod 561$	
$k = 1$:	$T = 263^2 \bmod 561 = 166 \bmod 561$	
$k = 2$:	$T = 166^2 \bmod 561 = 67 \bmod 561$	
$k = 3$:	$T = 67^2 \bmod 561 = +1 \bmod 561$	$\rightarrow$ **a composite**

Probabilistic Algorithms

- Example
  - We already know that 27 is not a prime. Let us apply the Miller-Rabin test.
- Solution
  - With base 2, let $27 - 1 = 13 \times 2^1$, which means that $m = 13$, $k = 1$, and $a = 2$. The initialization step: $T = 2^{13} \bmod 27 = 11 \bmod 27$. However, because the algorithm enters the loop only once, it returns a composite.

Probabilistic Algorithms

- Example
  - We know that 61 is a prime, let us see if it passes the Miller-Rabin test.
- Solution
  - We use base 2.

$61 - 1 = 15 \times 2^2 \rightarrow m = 15 \quad k = 2 \quad a = 2$

Initialization: $T = 2^{15} \bmod 61 = 11 \bmod 61$

$k = 1 \qquad T = 11^2 \bmod 61 = -1 \bmod 61 \rightarrow$ **a prime**

Recommended Primality test

- Combination of the divisibility test and the Miller-Rabin test.
- Example
  - The number 4033 is a composite (37 × 109). Does it pass the recommended primality test?
- Solution
  1. Perform the divisibility tests first. The numbers 2, 3, 5, 7, 11, 17, and 23 are not divisors of 4033.
  2. Perform the Miller-Rabin test with a base of 2, $4033 - 1 = 63 \times 64$, which means m is 63 and k is 6.

Initialization: $T \equiv 2^{63} \pmod{4033} \equiv 3521 \pmod{4033}$

$k = 1$ $\quad T \equiv T^2 \equiv 3521^2 \pmod{4033} \equiv -1 \pmod{4033}$ $\rightarrow$ **Passes**

Recommended Primality test

But we are not satisfied. We continue with another base, 3.

Initialization: $T \equiv 3^{63} \pmod{4033} \equiv 3551 \pmod{4033}$

$k = 1$	$T \equiv T^2 \equiv 3551^2 \pmod{4033} \equiv 2443 \pmod{4033}$
$k = 2$	$T \equiv T^2 \equiv 2443^2 \pmod{4033} \equiv 3442 \pmod{4033}$
$k = 3$	$T \equiv T^2 \equiv 3442^2 \pmod{4033} \equiv 2443 \pmod{4033}$
$k = 4$	$T \equiv T^2 \equiv 2443^2 \pmod{4033} \equiv 3442 \pmod{4033}$
$k = 5$	$T \equiv T^2 \equiv 3442^2 \pmod{4033} \equiv 2443 \pmod{4033}$ **→Failed (composite)**

FACTORIZATION

Fundamental Theorem of Arithmetic

$$n = p_1^{e1} \times p_2^{e2} \times \cdots \times p_k^{ek}$$

- Greatest Common Divisor

$$a = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_k^{a_k} \qquad b = p_1^{b_1} \times p_2^{b_2} \times \cdots \times p_k^{b_k}$$

$$\gcd(a, b) = p_1^{\min(a_1, b_1)} \times p_2^{\min(a_2, b_2)} \times \cdots \times p_k^{\min(a_k, b_k)}$$

- Least Common Multiplier

$$a = p_1^{a1} \times p_2^{a2} \times \cdots \times p_k^{ak} \qquad b = p_1^{b1} \times p_2^{b2} \times \cdots \times p_k^{bk}$$

$$\text{lcm}(a, b) = p_1^{\max(a_1, b_1)} \times p_2^{\max(a_2, b_2)} \times \cdots \times p_k^{\max(a_k, b_k)}$$

$$\text{lcm}(a, b) \times \gcd(a, b) = a \times b$$

Factorization methods

- Trial Division Method

```
Trial_Division_Factorization (n)                    // n is the number to be factored
{
    a ← 2
    while (a ≤ √n )
    {
        while (n mod a = 0)
        {
            output a                                // Factors are output one by one
            n = n / a
        }
        a ← a + 1
    }
    if (n > 1) output n                             // n has no more factors
}
```

FACTORIZATION

- Example
  - Use the trial division algorithm to find the factors of 1233.
- Solution
  - We run a program based on the algorithm and get the following result.

$$1233 = 3^2 \times 137$$

FACTORIZATION

- Example
  - Use the trial division algorithm to find the factors of 1523357784
- Solution
  - We run a program based on the algorithm and get the following result.

$$1523357784 = 2^3 \times 3^2 \times 13 \times 37 \times 43987$$

Fermat Method

$n = x^2 - y^2 = a \times b$ with a = (x + y) and b = (x − y)

```
Feramat_Factorization (n)          // n is the number to be factored
{
    x ← √n                         // smallest integer greater than √n
    while (x < n)
    {
    w ← x² − n
    if (w is perfect square)  y ← √w;  a ← x+y;  b ← x−y;  return a and b
    x ← x + 1
    }
}
```

FACTORIZATION

- More methods
  - Pollard p-1
  - Pollard rho
  - Number Field Sieve
  - Quadratic Sieve

Tutorial

CHINESE REMAINDER THEOREM

Used to solve a set of congruent equations with one variable but different moduli, which are relatively prime

$$
\begin{aligned}
x &\equiv a_1 \pmod{m_1} \\
x &\equiv a_2 \pmod{m_2} \\
&\cdots \\
x &\equiv a_k \pmod{m_k}
\end{aligned}
$$

CHINESE REMAINDER THEOREM

Used to solve a set of congruent equations with one variable but different moduli, which are relatively prime

$$
\begin{aligned}
x &\equiv a_1 \pmod{m_1} \\
x &\equiv a_2 \pmod{m_2} \\
&\cdots \\
x &\equiv a_k \pmod{m_k}
\end{aligned}
$$

CHINESE REMAINDER THEOREM

- Example
  - The following is an example of a set of equations with different moduli:

$$x \equiv 2 \pmod 3$$
$$x \equiv 3 \pmod 5$$
$$x \equiv 2 \pmod 7$$

  - The solution to this set of equations is given in the next section; for the moment, note that the answer to this set of equations is x = 23. This value satisfies all equations: 23 ≡ 2 (mod 3), 23 ≡ 3 (mod 5), and 23 ≡ 2 (mod 7).

CHINESE REMAINDER THEOREM

- Solution To Chinese Remainder Theorem
  - Find $M = m_1 \times m_2 \times \ldots \times m_k$. This is the common modulus.
  - Find $M_1 = M/m_1$, $M_2 = M/m_2$, …, $M_k = M/m_k$.
  - Find the multiplicative inverse of M_1, M_2, …, M_k using the corresponding moduli (m_1, m_2, …, m_k). Call the inverses M_1^{-1}, M_2^{-1}, …, M_k^{-1}.
  - The solution to the simultaneous equations is

$$x = (a_1 \times M_1 \times M_1^{-1} + a_2 \times M_2 \times M_2^{-1} + \cdots + a_k \times M_k \times M_k^{-1}) \bmod M$$

CHINESE REMAINDER THEOREM

- Example
  - Find the solution to the simultaneous equations:

$$x \equiv 2 \pmod 3$$
$$x \equiv 3 \pmod 5$$
$$x \equiv 2 \pmod 7$$

- Solution: We follow the four steps.

1. $M = 3 \times 5 \times 7 = 105$

2. $M_1 = 105 / 3 = 35$, $M_2 = 105 / 5 = 21$, $M_3 = 105 / 7 = 15$

3. The inverses are $M_1^{-1} = 2$, $M_2^{-1} = 1$, $M_3^{-1} = 1$

4. $x = (2 \times 35 \times 2 + 3 \times 21 \times 1 + 2 \times 15 \times 1) \bmod 105 = 23 \bmod 105$

CHINESE REMAINDER THEOREM

- Example
  - Find an integer that has a remainder of 3 when divided by 7 and 13, but is divisible by 12.
- Solution ????

CHINESE REMAINDER THEOREM

- Example
  - Find an integer that has a remainder of 3 when divided by 7 and 13, but is divisible by 12.
- Solution
  - This is a CRT problem. We can form three equations and solve them to find the value of x.

$$x = 3 \bmod 7$$
$$x = 3 \bmod 13$$
$$x = 0 \bmod 12$$

  - If we follow the four steps, we find x = 276. We can check that
    276 = 3 mod 7, 276 = 3 mod 13 and 276 is divisible by 12 (the quotient is 23 and the remainder is zero).

CHINESE REMAINDER THEOREM

- Assume we need to calculate z = x + y where x = 123 and y = 334, but our system accepts only numbers less than 100. These numbers can be represented as follows:

$$\begin{array}{ll} x \equiv 24 \pmod{99} & y \equiv 37 \pmod{99} \\ x \equiv 25 \pmod{98} & y \equiv 40 \pmod{98} \\ x \equiv 26 \pmod{97} & y \equiv 43 \pmod{97} \end{array}$$

- Adding each congruence in x with the corresponding congruence in y gives

$$\begin{array}{lll} x + y \equiv 61 \pmod{99} & \rightarrow & z \equiv 61 \pmod{99} \\ x + y \equiv 65 \pmod{98} & \rightarrow & z \equiv 65 \pmod{98} \\ x + y \equiv 69 \pmod{97} & \rightarrow & z \equiv 69 \pmod{97} \end{array}$$

- Now three equations can be solved using the Chinese remainder theorem to find z. One of the acceptable answers is z = 457.

CHINESE REMAINDER THEOREM

Secret Sharing scheme in cryptography aims to distribute and later recover secret S among n parties. Secret S is distributed in form of shares which are generated from secret. Without cooperation of k no. of parties, the secret cannot be reconstructed from shares directly. Consider the following example:

Say our secret is S. The shares for n=4 no. of parties are generated taking modulus 11,13,17 and 19. They are respectively 1,12,2 and 3 and given by following equations:

$S \equiv 1 \bmod 11,$

$S \equiv 12 \bmod 13,$

$S \equiv 2 \bmod 17,$

$S \equiv 3 \bmod 19.$

Now, from four possible sets of k=3 shares (as k shares are necessary to reconstruct the secret), consider one possible set {1, 12, 2} and recover the secret S from it.

CHINESE REMAINDER THEOREM

Solution: The problem can be solved by Chinese remainder theorem.

For the set {1,12,2}, the equations available are,

$S \equiv 1 \bmod 11$,

$S \equiv 12 \bmod 13$,

$S \equiv 2 \bmod 17$,

Now solving this equation using CRT, M=11 *13*17 = 2431,

M1 = 2431/11=221,

M2 = 2431/13=187,

M3=2431/17=143

$M1^{-1}$, $M2^{-1}$ and $M3^{-1}$ can be calculated using Extended Euclidean Algorithm.

$M1^{-1} = 1$

$M2^{-1} = 8$

$M3^{-1} = 5$

Now, secret S= ((1*221*1) + (12*187*8) + (2*143*5)) mod 2431

$S = 155 \bmod 2431$

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 7:

ALGEBRAIC STRUCTURES - PART II : Rings and Fields
Exponentiation and Logarithms

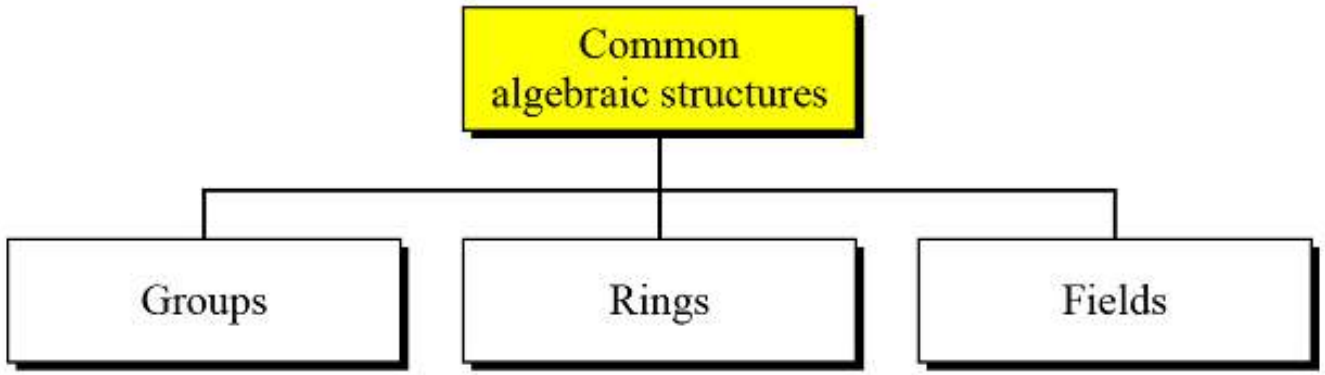
Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

ALGEBRAIC STRUCTURES

- Cryptography requires sets of integers and specific operations that are defined for those sets.
- The combination of the set and the operations that are applied to the elements of the set is called an algebraic structure.
- Three common algebraic structures:
  - Groups
  - Rings, and
  - Fields.



Ring

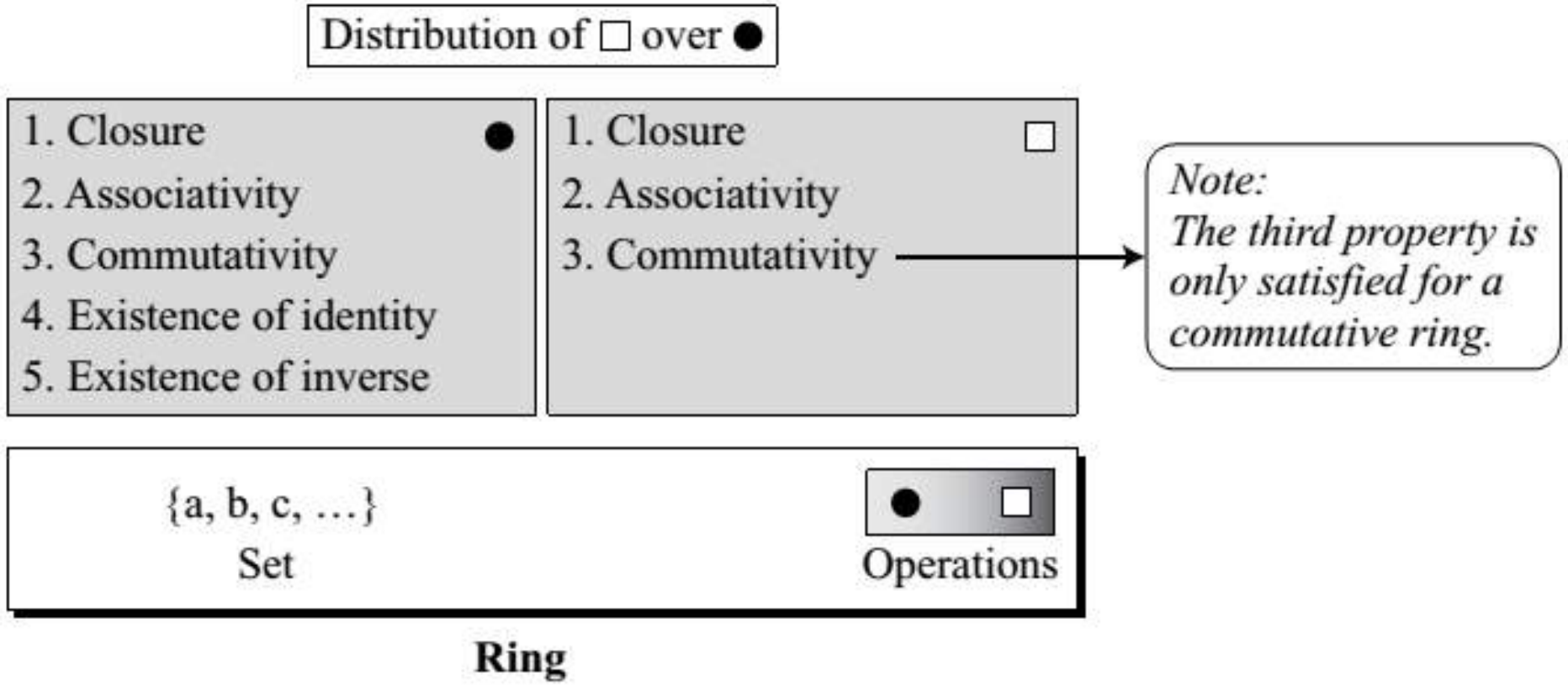
- A ring, R = <{…}, •, ■ >, is an algebraic structure with two operations.
- First operation must satisfy all five properties
- Second operation must satisfy only the first two
- In addition, second operation must be distributed over first

i.e. for all a, b, and c elements of R, we have,

a■ (b • c) = (a ■ b) • (a ■ c) and

(a • b) ■ c = (a■c) • (a■ c)

Ring


Distribution of □ over ●
1. Closure ●
2. Associativity
3. Commutativity
4. Existence of identity
5. Existence of inverse
1. Closure □
2. Associativity
3. Commutativity
Note:
The third property is only satisfied for a commutative ring.
{a, b, c, …}
Set
● □
Operations
Ring

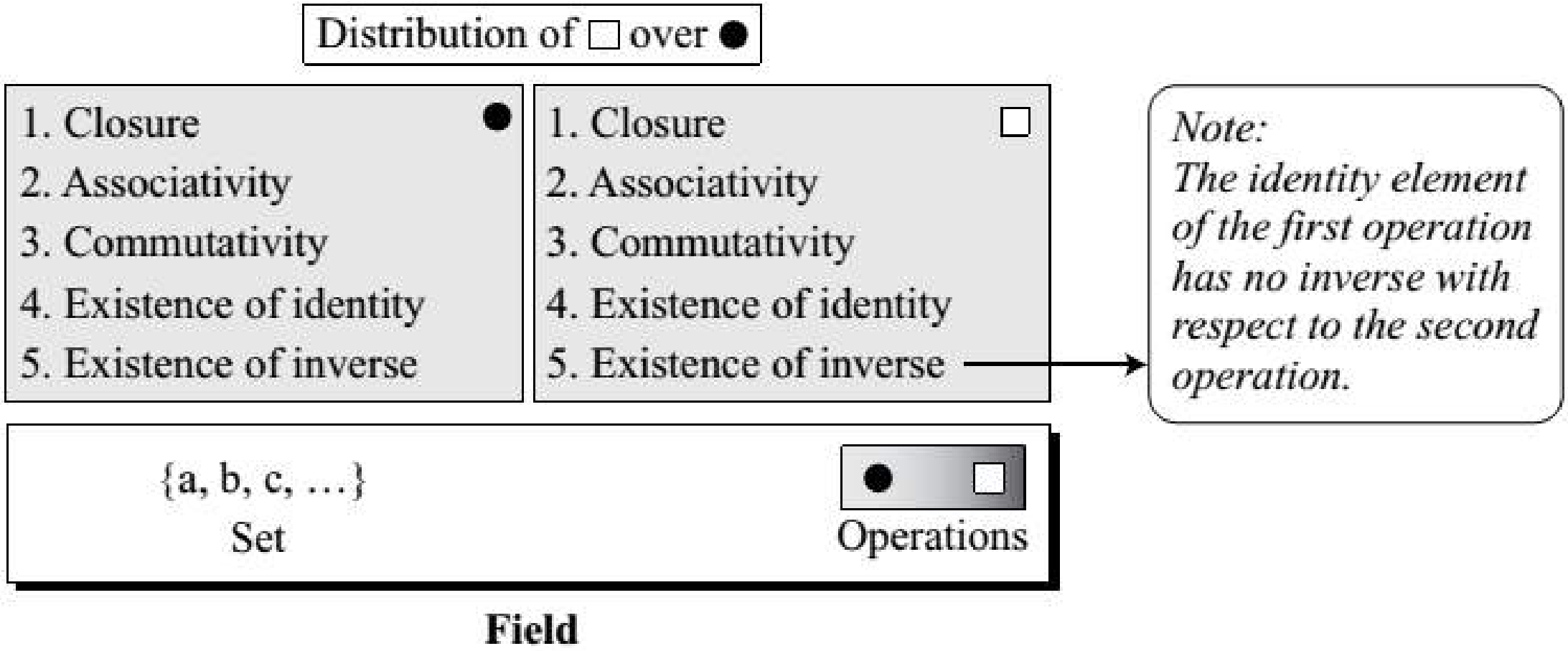
Ring

- The set Z with two operations, addition and multiplication, is a commutative ring.
- We show it by R = <Z, +, ×>.
- Addition satisfies all of the five properties;
- Multiplication satisfies only three properties.
- For example,
  - 5 × (3 + 2) = (5 × 3) +(5 × 2) = 25.
  - Although, we can perform addition and subtraction on this set, we can perform only multiplication, but not division.
  - Division is not allowed in this structure because it yields an element out of the set.
  - The result of dividing 12 by 5 is 2.4, which is not in the set.

Field

- A field, denoted by F = <{…}, •, ▫ > is a commutative ring in which the second operation satisfies all five properties defined for the first operation except that the identity of the first operation has no inverse.
- Application
  - A field is a structure that supports two pairs of operations that we have used in mathematics: addition/subtraction and multiplication/division.
  - There is one exception: division by zero is not allowed.

Field


Distribution of □ over ●
1. Closure
2. Associativity
3. Commutativity
4. Existence of identity
5. Existence of inverse
1. Closure
2. Associativity
3. Commutativity
4. Existence of identity
5. Existence of inverse
Note:
The identity element of the first operation has no inverse with respect to the second operation.
{a, b, c, …}
Set
Operations
Field

Fields

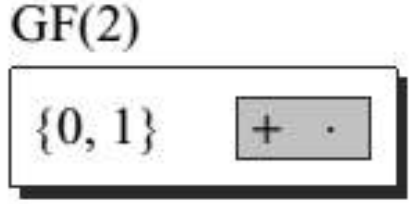
- Finite Fields
  - Galois showed that for a field to be finite, the number of elements should be p^n, where p is a prime and n is a positive integer.

A Galois field, GF(p^n), is a finite field with p^n elements.

- GF(p) Fields
  - When n = 1, we have GF(p) field.
  - This field can be the set Z_p, {0, 1, …, p − 1}, with two arithmetic operations.

Fields

- A very common field in this category is GF(2) with the set {0, 1} and two operations, addition and multiplication.



+	0	1
0	0	1
1	1	0

Addition

·	0	1
0	0	0
1	0	1

Multiplication

a	0	1
$-a$	1	0

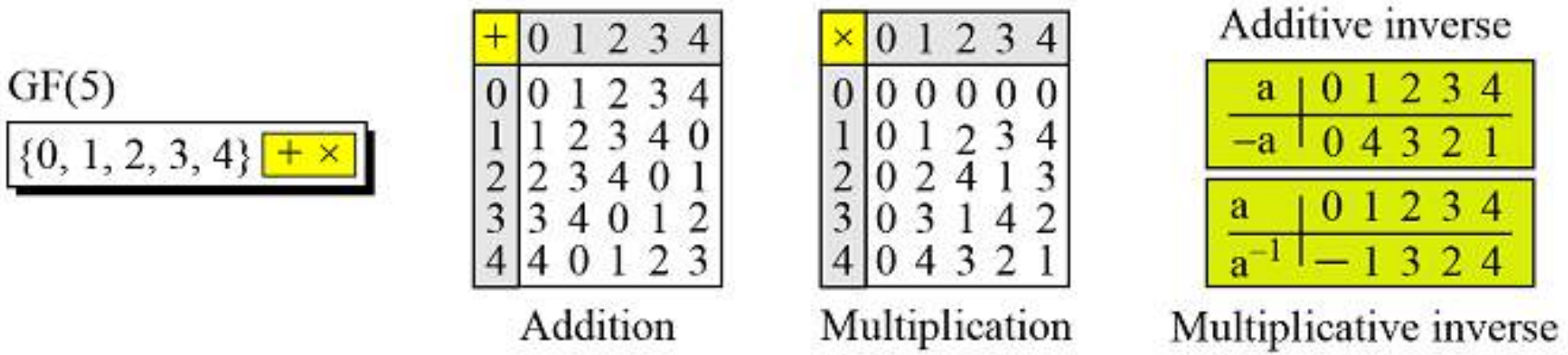
a	0	1
a^{-1}	—	1

Inverses

Addition/subtraction in GF(2) is the same as the XOR operation; multiplication/division is the same as the AND operation.

Fields

- We can define GF(5) on the set Z_5 (5 is a prime) with addition and multiplication operators.



+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

Addition

×	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

Multiplication

Additive inverse

a	0	1	2	3	4
−a	0	4	3	2	1

a	0	1	2	3	4
a^{-1}	—	1	3	2	4

Multiplicative inverse

GF(5) field

Summary

Algebraic Structure	*Supported Typical Operations*	*Supported Typical Sets of Integers*
Group	$(+ \ -)$ or $(\times \ \div)$	$\mathbf{Z}_n$ or $\mathbf{Z}_n^*$
Ring	$(+ \ -)$ and $(\times)$	$\mathbf{Z}$
Field	$(+ \ -)$ and $(\times \ \div)$	$\mathbf{Z}_p$

GF(2^n) FIELDS

- In cryptography, we often need to use four operations (addition, subtraction, multiplication and division).
- In other words, we need to use fields.
- However, when we work with computers, the positive integers are stored in the computers as n-bit words in which n is usually 8, 16, 32 and so on.
- Range of integers is 0 to $2^n - 1$
- Hence, the modulus is 2^n.
- So we have two choices if we want to use a field!!!!

GF(2^n) FIELDS

- We can use GF(p) with the set Z_p, where p is the largest prime number less than 2^n.
- Although this scheme works, it is inefficient because we cannot use the integers from p to $2^n - 1$.
- For example,
  - if n = 4, the largest prime less than 2^4 is 13. This means that we cannot use integers 13, 14, and 15.
  - If n = 8, the largest prime less than 2^8 is 251, so we cannot use 251, 252, 253, 254, and 255.
- We can work in GF(2^n) and uses a set of 2^n elements.
- The elements in this set are n-bit words.
- For example,
  - if n = 3, the set is {000, 001, 010, 011, 100, 101, 110, 111}
- 2^n is not prime. So, we need to define a set of n-bit words and two new operations that satisfies the properties defined for a field.

GF(2^n) FIELDS

- Let us define a GF(2^2) field in which the set has four 2-bit words: {00, 01, 10, 11}.
- We can redefine addition and multiplication for this field in such a way that all properties of these operations are satisfied.

Addition

⊕	00	01	10	11
00	00	01	10	11
01	01	00	11	10
10	10	11	00	01
11	11	10	01	00

Identity: 00

Multiplication

⊗	00	01	10	11
00	00	00	00	00
01	00	01	10	11
10	00	10	11	01
11	00	11	01	10

Identity: 01

An example of a GF(2^2) field

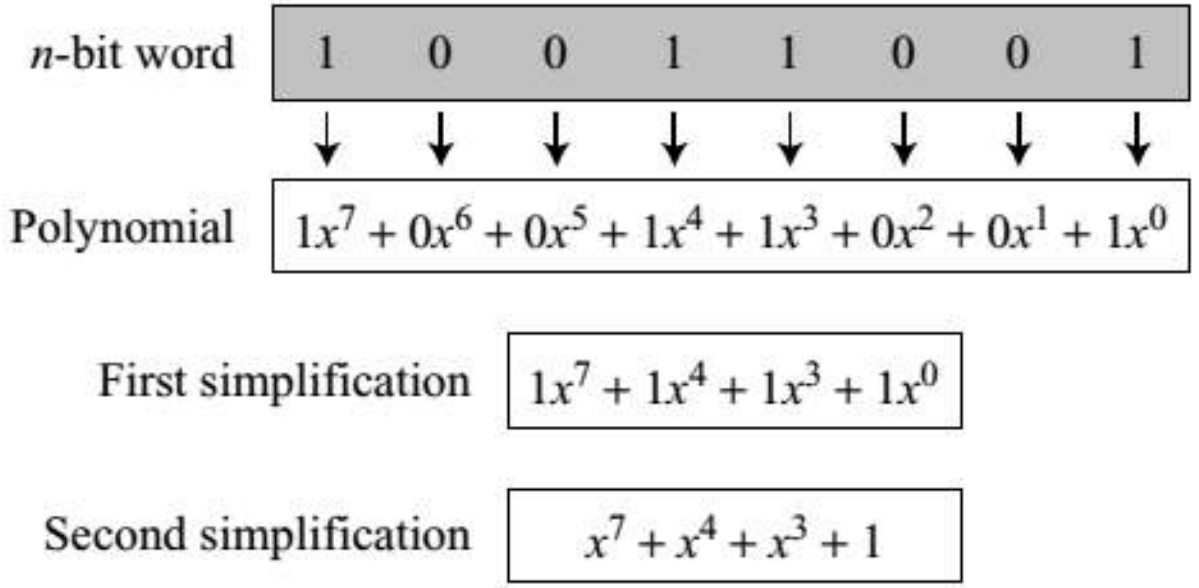
Polynomials

- A polynomial of degree $n - 1$ is an expression of the form

$$f(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1x^1 + a_0x^0$$

where x^i is called the ith term and a_i is called coefficient of the ith term.

- We can represent the 8-bit word (10011001) using a polynomial.



Polynomials

- Find the 8-bit word related to the polynomial $x^5 + x^2 + x$, we first supply the omitted terms.
- Since $n = 8$, it means the polynomial is of degree 7. The expanded polynomial is,

$$0x^7 + 0x^6 + 1x^5 + 0x^4 + 0x^3 + 1x^2 + 1x^1 + 0x^0$$

- This is related to the 8-bit word 00100110.

Polynomials Operations

- Operations on polynomials
  - Actually involves two operations
- Operation on coefficients and operation on polynomials
- Hence, need to define two fields for each
- What for coefficient??
  - Coefficients are made of 0 or 1; we can use the GF(2) field for this purpose.
- What for polynomials???
  - For the polynomials we need the field GF(2^n).

Polynomials

- Modulus
  - For the sets of polynomials in GF(2^n), a group of polynomials of degree *n* is defined as the modulus.
  - Such polynomials are referred to as irreducible polynomials.
- Irreducible polynomials.
  - Prime Polynomial: No polynomial in the set can divide this polynomial
  - Can not be factored into a polynomial with degree of less than n

Degree	*Irreducible Polynomials*
1	$(x + 1), (x)$
2	$(x^2 + x + 1)$
3	$(x^3 + x^2 + 1), (x^3 + x + 1)$
4	$(x^4 + x^3 + x^2 + x + 1), (x^4 + x^3 + 1), (x^4 + x + 1)$
5	$(x^5 + x^2 + 1), (x^5 + x^3 + x^2 + x + 1), (x^5 + x^4 + x^3 + x + 1),$ $(x^5 + x^4 + x^3 + x^2 + 1), (x^5 + x^4 + x^2 + x + 1)$

Polynomials

- Polynomial addition
  - ***Addition and subtraction operations on polynomials are the same operation***
  - Adding two polynomials of degree n − 1 always create a polynomial with degree n − 1, which means that we do not need to reduce the result using the modulus.
- Example:
  - Let us do $(x^5 + x^2 + x) \oplus (x^3 + x^2 + 1)$ in GF(2^8).
  - We use the symbol $\oplus$ to show that we mean polynomial addition. The following shows the procedure:

$$
\begin{array}{ll}
0x^7 + 0x^6 + 1x^5 + 0x^4 + 0x^3 + 1x^2 + 1x^1 + 0x^0 & \oplus \\
0x^7 + 0x^6 + 0x^5 + 0x^4 + 1x^3 + 1x^2 + 0x^1 + 1x^0 & \\
\hline
0x^7 + 0x^6 + 1x^5 + 0x^4 + 1x^3 + 0x^2 + 1x^1 + 1x^0 & \rightarrow \quad x^5 + x^3 + x + 1
\end{array}
$$

Polynomials

- Short cut method
  - Addition in GF(2) means the exclusive-or (XOR) operation.
  - So we can exclusive-or the two words, bits by bits, to get the result.
  - In the previous example, $x^5 + x^2 + x$ is 00100110 and $x^3 + x^2 + 1$ is 00001101.
  - The result is 00101011 or in polynomial notation $x^5 + x^3 + x + 1$.

Polynomials

- Multiplication
  - The coefficient multiplication is done in GF(2).
  - The multiplying x^i by x^j results in x^{i+j}.
  - The multiplication may create terms with degree more than $n - 1$, which means the result needs to be reduced using a modulus polynomial.
- For example
  - Find the result of $(x^5 + x^2 + x) \otimes (x^7 + x^4 + x^3 + x^2 + x)$ in GF(2^8) with irreducible polynomial $(x^8 + x^4 + x^3 + x + 1)$.

$$P_1 \otimes P_2 = x^5(x^7 + x^4 + x^3 + x^2 + x) + x^2(x^7 + x^4 + x^3 + x^2 + x) + x(x^7 + x^4 + x^3 + x^2 + x)$$
$$P_1 \otimes P_2 = x^{12} + x^9 + x^8 + x^7 + x^6 + x^9 + x^6 + x^5 + x^4 + x^3 + x^8 + x^5 + x^4 + x^3 + x^2$$
$$P_1 \otimes P_2 = (x^{12} + x^7 + x^2) \bmod (x^8 + x^4 + x^3 + x + 1) = x^5 + x^3 + x^2 + x + 1$$

Polynomials

- To find the final result, divide the polynomial of degree 12 by the polynomial of degree 8 (the modulus) and keep only the remainder.

$$
\begin{array}{r|l}
 & x^4 + 1 \\
\hline
x^8 + x^4 + x^3 + x + 1 & x^{12} + x^7 + x^2 \\
 & x^{12} + x^8 + x^7 + x^5 + x^4 \\
\hline
 & x^8 + x^5 + x^4 + x^2 \\
 & x^8 + x^4 + x^3 + x + 1 \\
\hline
\text{Remainder} & \boxed{x^5 + x^3 + x^2 + x + 1}
\end{array}
$$

Polynomial division
with
coefficients in GF(2)

Polynomials

- Example:
  - In GF (2^4), find the inverse of ($x^2 + 1$) modulo ($x^4 + x + 1$).
- Solution
  - The answer is ($x^3 + x + 1$)

q	r_1	r_2	r	t_1	t_2	t
$(x^2 + 1)$	$(x^4 + x + 1)$	$(x^2 + 1)$	(x)	(0)	(1)	$(x^2 + 1)$
(x)	$(x^2 + 1)$	(x)	(1)	(1)	$(x^2 + 1)$	$(x^3 + x + 1)$
(x)	(x)	(1)	(0)	$(x^2 + 1)$	$(x^3 + x + 1)$	(0)
	(1)	(0)		$(x^3 + x + 1)$	(0)	

Polynomials

- Example:
  - In GF(2^8), find the inverse of (x^5) modulo ($x^8 + x^4 + x^3 + x + 1$)..
- Solution

q	r_1	r_2	r	t_1	t_2	t
(x^3)	$(x^8 + x^4 + x^3 + x + 1)$	(x^5)	$(x^4 + x^3 + x + 1)$	(0)	(1)	(x^3)
$(x + 1)$	(x^5)	$(x^4 + x^3 + x + 1)$	$(x^3 + x^2 + 1)$	(1)	(x^3)	$(x^4 + x^3 + 1)$
(x)	$(x^4 + x^3 + x + 1)$	$(x^3 + x^2 + 1)$	(1)	(x^3)	$(x^4 + x^3 + 1)$	$(x^5 + x^4 + x^3 + x)$
$(x^3 + x^2 + 1)$	$(x^3 + x^2 + 1)$	(1)	(0)	$(x^4 + x^3 + 1)$	$(x^5 + x^4 + x^3 + x)$	(0)
	(1)	(0)		$(x^5 + x^4 + x^3 + x)$	(0)	

Polynomials

- A better algorithm: Obtain the result by repeatedly multiplying a reduced polynomial by *x*.
- For example, instead of finding the result of ($x^2 \otimes$ P2), the program finds the result of ($x \otimes (x \otimes$ P2)).

- Example:
  - Find the result of multiplying P1 = ($x^5 + x^2 + x$) by P_2 = ($x^7 + x^4 + x^3 + x^2 + x$) in GF($2^8$) with irreducible polynomial ($x^8 + x^4 + x^3 + x + 1$)
- Solution
  - We first find the partial result of multiplying x^0, x^1, x^2, x^3, x^4, and x^5 by P_2.
  - Note that although only three terms are needed, the product of $x^m \otimes$ P2 for *m* from 0 to 5 because each calculation depends on the previous result

Polynomials

Powers	*Operation*	*New Result*	*Reduction*
$x^0 \otimes P_2$		$x^7 + x^4 + x^3 + x^2 + x$	No
$x^1 \otimes P_2$	$\boldsymbol{x} \otimes (x^7 + x^4 + x^3 + x^2 + x)$	$x^5 + x^2 + x + 1$	**Yes**
$x^2 \otimes P_2$	$\boldsymbol{x} \otimes (x^5 + x^2 + x + 1)$	$x^6 + x^3 + x^2 + x$	No
$x^3 \otimes P_2$	$\boldsymbol{x} \otimes (x^6 + x^3 + x^2 + x)$	$x^7 + x^4 + x^3 + x^2$	No
$x^4 \otimes P_2$	$\boldsymbol{x} \otimes (x^7 + x^4 + x^3 + x^2)$	$x^5 + x + 1$	**Yes**
$x^5 \otimes P_2$	$\boldsymbol{x} \otimes (x^5 + x + 1)$	$x^6 + x^2 + x$	No
$\mathbf{P_1 \times P_2} = (x^6 + x^2 + x) + (x^6 + x^3 + x^2 + x) + (x^5 + x^2 + x + 1) = \boldsymbol{x^5 + x^3 + x^2 + x + 1}$			

EXPONENTIATION AND LOGARITHM

Exponentiation: $y = a^x$ $\rightarrow$ **Logarithm:** $x = \log_a y$

Exponentiation

- Fast Exponentiation
  – The idea behind the square-and-multiply method

$$x = x_{n_b-1} \times 2^{k-1} + x_{n_b-2} \times 2^{k-2} + \cdots + x_2 \times 2^2 + x_1 \times 2^1 + x_0 \times 2^0$$

$y = a^{x_{n_b-1} \times 2^{n_b-1} + x_{n_b-2} \times 2^{n_b-2} + \cdots + x_1 \times 2^1 + x_0 \times 2^0}$ in which x_i is 0 or 1

$$y = \left(a^{2^{n_b-1}} \text{ or } 1\right) \times \left(a^{2^{n_b-2}} \text{ or } 1\right) \times \cdots \times \left(a^2 \text{ or } 1\right) \times \left(a \text{ or } 1\right)$$

Example:

$$y = a^9 = a^{1001_2} = a^8 \times 1 \times 1 \times a$$

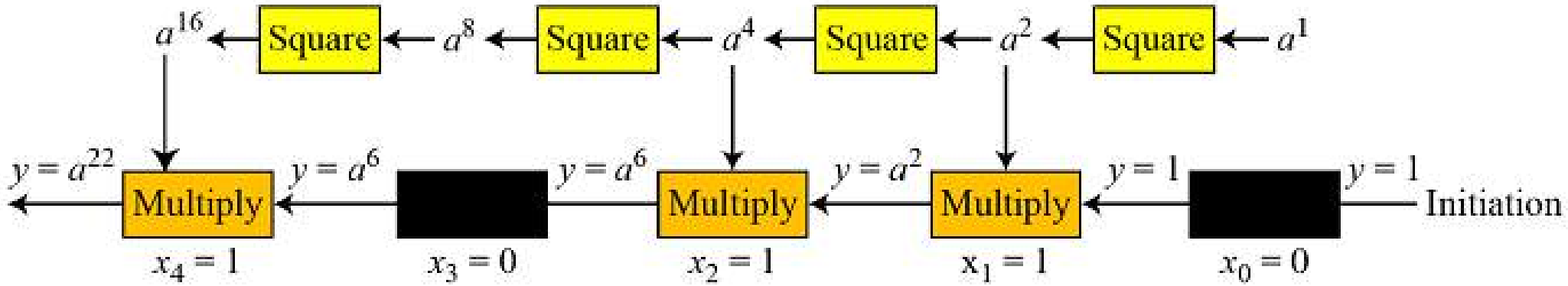
Exponentiation

```
Square_and_Multiply (a, x, n)
{
    y ← 1
    for (i ← 0 to n_b − 1)                    // n_b is the number of bits in x
    {
        if (x_i = 1)    y ← a × y  mod n      // multiply only if the bit is 1

        a ← a^2 mod n                         // squaring is not needed in the last iteration
    }
    return y
}
```

Exponentiation

- The process for calculating $y = a^x$
- In this case, $x = 22 = (10110)_2$ in binary.



Exponentiation

Calculation of $17^{22} \bmod 21$

i	x_i	*Multiplication (Initialization: $y = 1$)*		*Squaring (Initialization: $a = 17$)*
0	0		$\rightarrow$	$a = 17^2 \bmod 21 = 16$
1	1	$y = 1 \times 16 \bmod 21 = 16$	$\rightarrow$	$a = 16^2 \bmod 21 = 4$
2	1	$y = 16 \times 4 \bmod 21 = 1$	$\rightarrow$	$a = 4^2 \bmod 21 = 16$
3	0		$\rightarrow$	$a = 16^2 \bmod 21 = 4$
4	1	$y = 1 \times 4 \bmod 21 = 4$	$\rightarrow$	

Logarithm

- In cryptography we need to discuss modular logarithm

Exhaustive search for modular logarithm

```
Modular_Logarithm (a, y, n)
{
    for (x = 1 to n – 1)                          // k is the number of bits in x
    {
        if (y ≡ a^x mod n) return x
    }
    return failure
}
```

Logarithm

- Order of the Group.
- Example:
  - What is the order of group G = $<Z_{21}*, x>$?
    - $|G| = \phi(21) = \phi(3) \times \phi(7) = 2 \times 6 = 12$. There are 12 elements in this group: 1, 2, 4, 5, 8, 10, 11, 13, 16, 17, 19, and 20. All are relatively prime with 21.

Logarithm

- The order of an element, a, is the smallest integer i such that $a^i \equiv e \pmod{n}$.

The order of an element divides the order of the group (Lagrange theorem).

- Example:
  - Find the order of all elements in G = $<Z_{10}^*, \times>$.
  - This group has only $\phi(10) = 4$ elements: 1, 3, 7, 9.

a. $1^1 \equiv 1 \bmod (10) \rightarrow \text{ord}(1) = 1.$

b. $3^1 \equiv 3 \bmod (10); 3^2 \equiv 9 \bmod (10); 3^4 \equiv 1 \bmod (10) \rightarrow \text{ord}(3) = 4.$

c. $7^1 \equiv 7 \bmod (10); 7^2 \equiv 9 \bmod (10); 7^4 \equiv 1 \bmod (10) \rightarrow \text{ord}(7) = 4.$

d. $9^1 \equiv 9 \bmod (10); 9^2 \equiv 1 \bmod (10) \rightarrow \text{ord}(9) = 2.$

Logarithm

- Primitive roots
  - In the group G = $<Z_n^*, x>$, when the order of an element is the same as $\phi(n)$, that element is called the primitive root of the group.
  - Example
    - There are no primitive roots in G = $<Z_8^*, x>$ because no element has the order equal to $\phi(8) = 4$.

Logarithm

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$
$a = 1$	**x: 1**	x: 1	x: 1	x: 1	x: 1	x: 1	x: 1
$a = 3$	x: 3	**x: 1**	x: 3	x: 1	x: 3	x: 1	x: 3
$a = 5$	x: 5	**x: 1**	x: 5	x: 1	x: 5	x: 1	x: 5
$a = 7$	x: 7	**x: 1**	x: 7	x: 1	x: 7	x: 1	x: 7

The first time when x is 1, the value of i gives us the order of the element (double-sided boxes). The orders of elements are ord(1) = 1, ord(3) = 2, ord(5) = 2, and ord(7) = 2.

Logarithm

- Example
  - the result of $a^i \equiv x \pmod 7$ for the group $G = \langle Z_7^*, \times \rangle$. In this group, $\phi(7) = 6$.

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$
$a = 1$	x: 1	x: 1	x: 1	x: 1	x: 1	x: 1
$a = 2$	x: 2	x: 4	x: 1	x: 2	x: 4	x: 1
Primitive root → $a = 3$	x: 3	x: 2	x: 6	x: 4	x: 5	x: 1
$a = 4$	x: 4	x: 2	x: 1	x: 4	x: 2	x: 1
Primitive root → $a = 5$	x: 5	x: 4	x: 6	x: 2	x: 3	x: 1
$a = 6$	x: 6	x: 1	x: 6	x: 1	x: 6	x: 1

The orders of elements are ord(1) = 1, ord(2) = 3, ord(3) = 6, ord(4) = 3, ord(5) = 6, and ord(6) = 2.

Therefore, this group has only two primitive roots: 3 and 5.

Logarithm

The group $G = \langle Z_n^*, \times \rangle$ has primitive roots only if n is 2, 4, p^t, or $2p^t$.

If the group $G = \langle Z_n^*, \times \rangle$ has any primitive root, the number of primitive roots is $\phi(\phi(n))$.

The group $G = \langle Z_n^*, \times \rangle$ is a cyclic group if it has primitive roots.
The group $G = \langle Z_p^*, \times \rangle$ is always cyclic.

The idea of Discrete Logarithm

Properties of $G = \langle Z_p^*, \times \rangle$:

1. Its elements include all integers from 1 to $p - 1$.
2. It always has primitive roots.
3. It is cyclic. The elements can be created using g^x where x is an integer from 1 to $\phi(n) = p - 1$.

4. The primitive roots can be thought as the base of logarithm. If the group has k primitive roots, calculations can be done in k different bases. Given $x = \log_g y$ for any element y in the set, there is another element x that is the log of y in base g. This type of logarithm is called **discrete logarithm**. A discrete logarithm is designated by several different symbols in the literature, but we will use the notation L_g to show that the base is g (the modulus is understood).

Solution to Modular Logarithm Using Discrete Logs

Tabulation of Discrete Logarithms

Discrete logarithm for $\mathbf{G} = \langle \mathbf{Z}_7^*, \times \rangle$

y	1	2	3	4	5	6
$x = L_3\, y$	6	2	1	4	5	3
$x = L_5\, y$	6	4	5	2	1	3

- Find x in each of the following cases:

  a. $4 \equiv 3^x \pmod 7$

  b. $6 \equiv 5^x \pmod 7$

- Solution
  - Use the tabulation of the discrete logarithm

    a. $4 \equiv 3^x \bmod 7 \rightarrow x = L_3 4 \bmod 7 = 4 \bmod 7$

    b. $6 \equiv 5^x \bmod 7 \rightarrow x = L_5 6 \bmod 7 = 3 \bmod 7$

Logarithm

Using Properties of Discrete Logarithms

Traditional Logarithm	*Discrete Logarithms*
$\log_a 1 = 0$	$L_g 1 \equiv 0 \pmod{\phi(n)}$
$\log_a (x \times y) = \log_a x + \log_a y$	$L_g(x \times y) \equiv (L_g x + L_g y) \pmod{\phi(n)}$
$\log_a x^k = k \times \log_a x$	$L_g x^k \equiv k \times L_g x \pmod{\phi(n)}$

The discrete logarithm problem has the same complexity as the factorization problem.

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 8 :
ASYMMETRIC-KEY CRYPTOGRAPHY

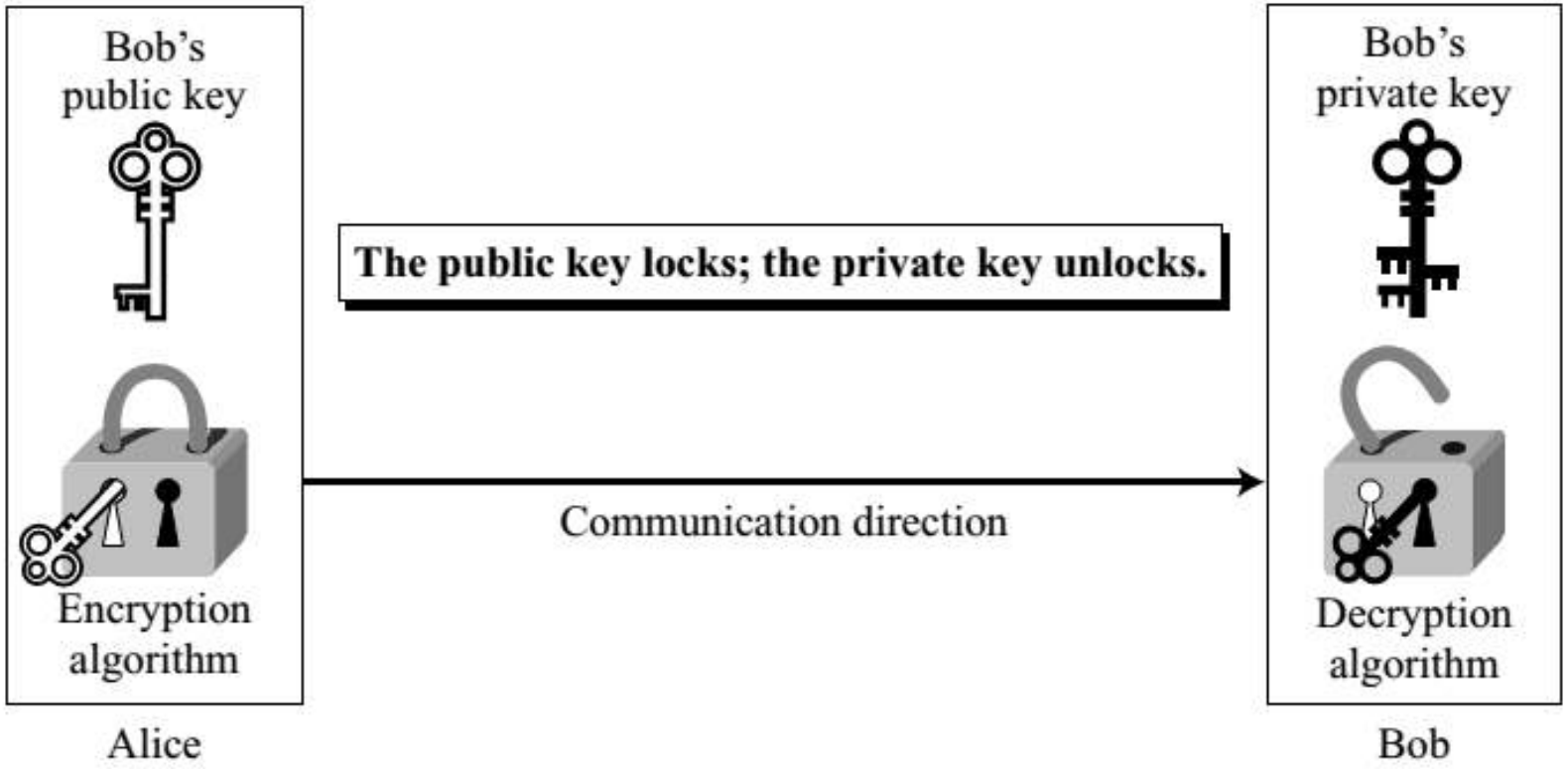
Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

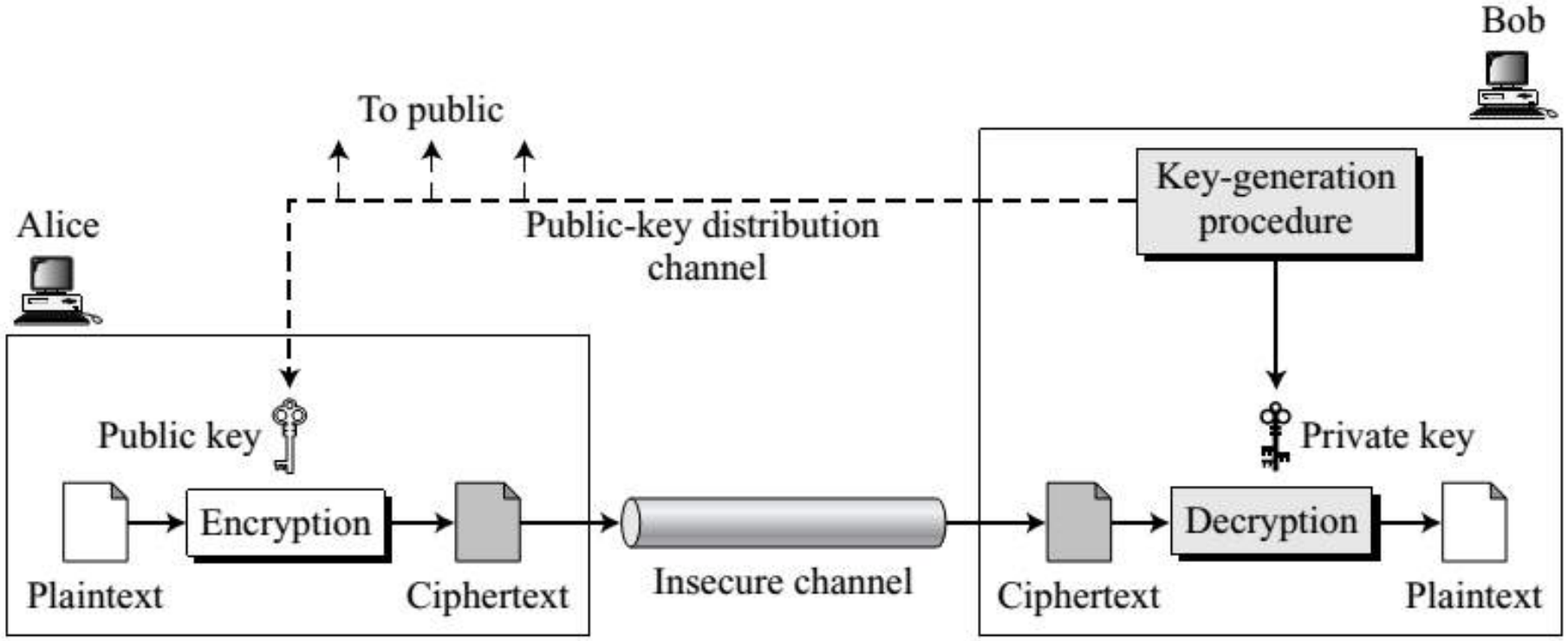
Asymmetric-Key Cryptography

- Locking and unlocking in asymmetric-key cryptosystem



Asymmetric-Key Cryptography

- General idea of asymmetric-key cryptosystem



Asymmetric-Key Cryptography

- Encryption/Decryption
  - The ciphertext can be thought of as $C = f(K_{public}, P)$;
  - The plaintext can be thought of as $P = g(K_{private}, C)$.
  - The function f is used only for encryption;
  - The function g is used only for decryption.
- Need for Both
  - Asymmetric-key cryptography is much slower than symmetric-key cryptography
  - Asymmetric-key cryptography is still needed for authentication, digital signatures, and secret-key exchanges.

Trapdoor One-Way Function

- The main idea behind asymmetric-key cryptography is the concept of the trapdoor oneway function.
-

One-Way Function

A **one-way function (OWF)** is a function that satisfies the following two properties:

1. f is easy to compute. In other words, given x, $y = f(x)$ can be easily computed.
2. f^{-1} is difficult to compute. In other words, given y, it is computationally infeasible to calculate $x = f^{-1}(y)$.

Trapdoor One-Way Function

A **trapdoor one-way function (TOWF)** is a one-way function with a third property:

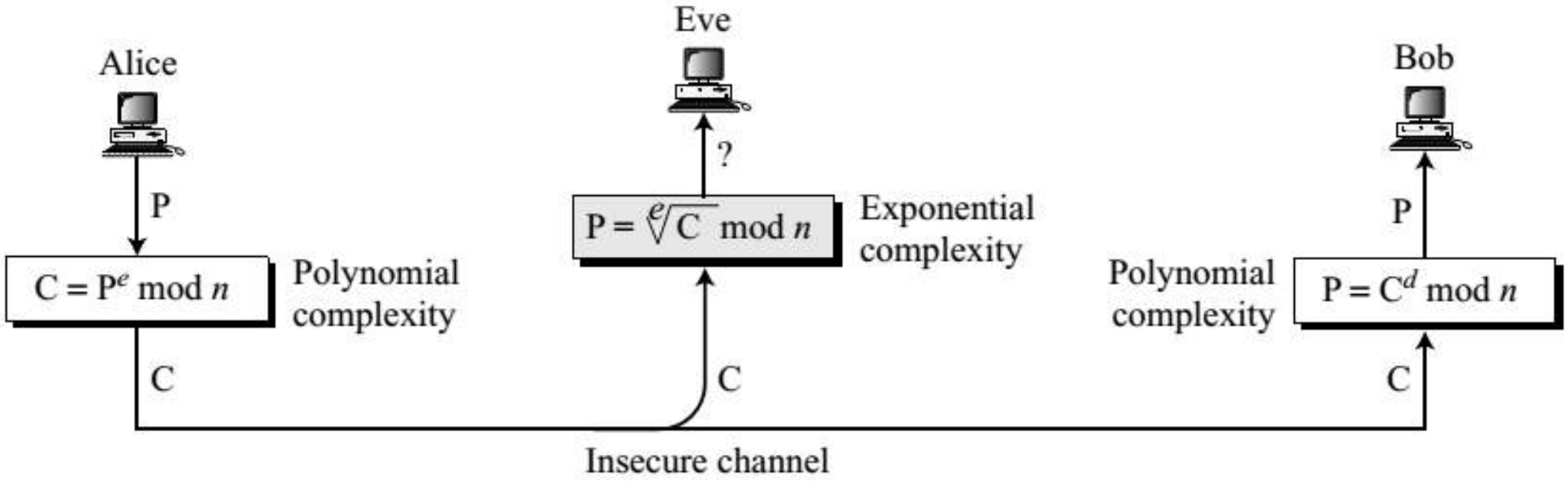
3. Given y and a **trapdoor** (secret), x can be computed easily.

Trapdoor One-Way Function

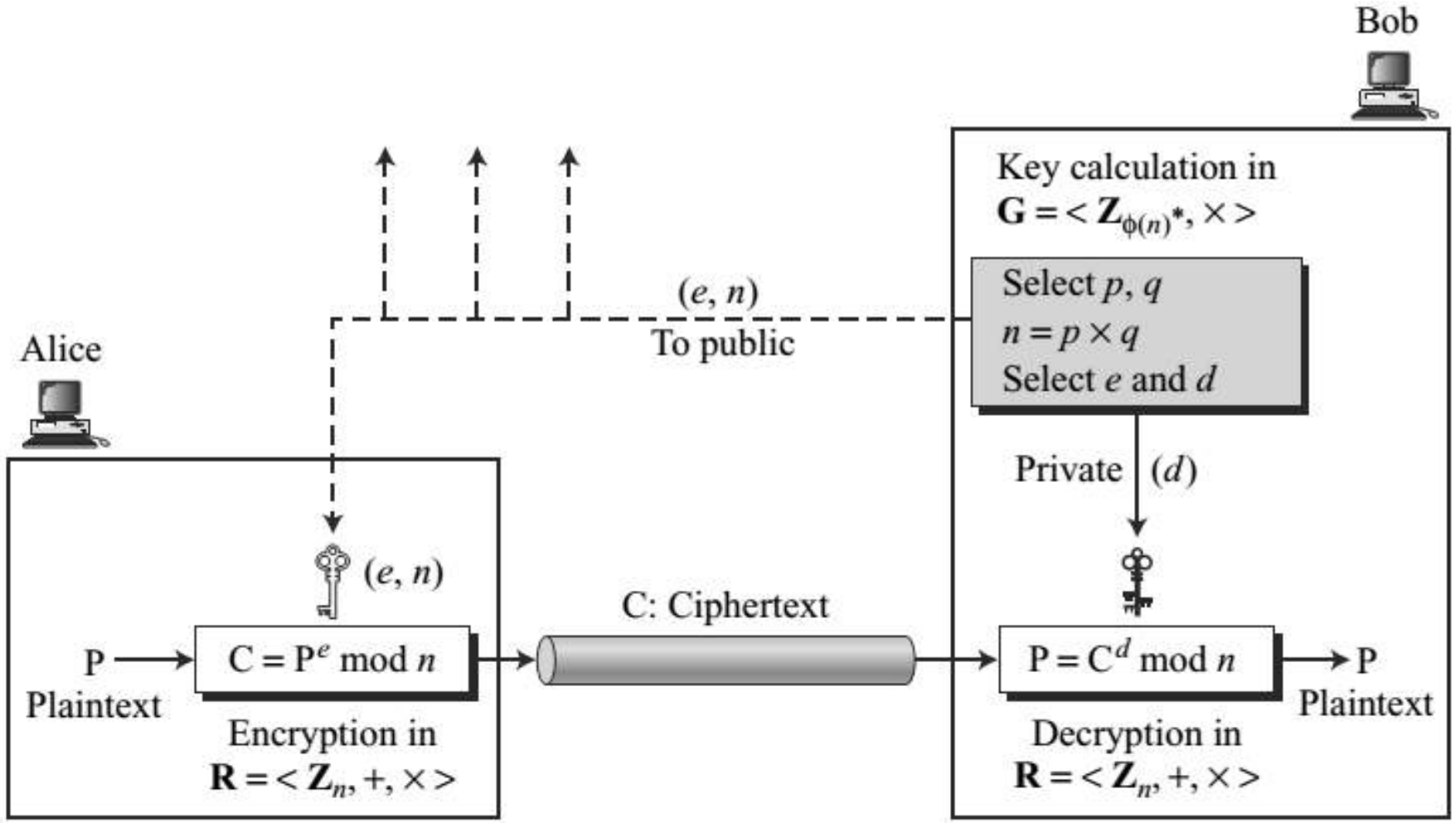
- When n is large, $n = p \times q$ is a one-way function.
- In this function x is a tuple (p, q) of two primes and y is n.
- Given p and q, it is always easy to calculate n; given n, it is very difficult to compute p and q.
- This is the factorization problem.
- There is not a polynomial time solution to the f^{-1} function in this case.
- When n is large, the function $y = x^k \bmod n$ is a trapdoor one-way function.
- Given x, k, and n, it is easy to calculate y using the fast exponential algorithm
- Given y, k, and n, it is very difficult to calculate x.
- However, if we know the trapdoor, k' such that $k \times k' = 1 \bmod \varphi(n)$, we can use $x = y^{k'} \bmod n$ to find x.

RSA CRYPTOSYSTEM

- The most common public-key algorithm is the RSA cryptosystem, named for its inventors (Rivest, Shamir, and Adleman).



Encryption, decryption, and key generation in RSA



RSA Key Generation

RSA_Key_Generation

{

Select two large primes p and q such that $p \neq q$.

$n \leftarrow p \times q$

$\phi(n) \leftarrow (p-1) \times (q-1)$

Select e such that $1 < e < \phi(n)$ and e is coprime to $\phi(n)$

$d \leftarrow e^{-1} \bmod \phi(n)$ // d is inverse of e modulo $\phi(n)$

Public_key $\leftarrow$ (e, n) // To be announced publicly

Private_key $\leftarrow d$ // To be kept secret

return Public_key and Private_key

}

In RSA, the tuple (e, n) is the public key; the integer d is the private key.

Encryption and Decryption

```
RSA_Encryption (P, e, n)                    // P is the plaintext in Z_n and P < n
{
    C ← Fast_Exponentiation (P, e, n)       // Calculation of (P^e mod n)
    return C
}

RSA_Decryption (C, d, n)                    //C is the ciphertext in Z_n
{
    P ← Fast_Exponentiation (C, d, n)       // Calculation of (C^d mod n)
    return P
}
```

Some Trivial Examples

Bob chooses 7 and 11 as p and q and calculates $n = 7 \times 11 = 77$. The value of $\phi(n) = (7 - 1)(11 - 1)$ or 60. Now he chooses two exponents, e and d, from $\mathbf{Z}_{60}$*. If he chooses e to be 13, then d is 37. Note that $e \times d \bmod 60 = 1$ (they are inverses of each other). Now imagine that Alice wants to send the plaintext 5 to Bob. She uses the public exponent 13 to encrypt 5.

Plaintext: 5	$C = 5^{13} = 26 \bmod 77$	Ciphertext: 26

Bob receives the ciphertext 26 and uses the private key 37 to decipher the ciphertext:

Ciphertext: 26	$P = 26^{37} = 5 \bmod 77$	Plaintext: 5

The plaintext 5 sent by Alice is received as plaintext 5 by Bob.

Some Trivial Examples

#Class Exercise

Now assume that another person, John, wants to send a message to Bob. John can use the same public key announced by Bob (probably on his website), 13; John's plaintext is 63. John calculates the following:

Some Trivial Examples

Now assume that another person, John, wants to send a message to Bob. John can use the same public key announced by Bob (probably on his website), 13; John's plaintext is 63. John calculates the following:

Plaintext: 63	$C = 63^{13} = 28 \bmod 77$	Ciphertext: 28

Bob receives the ciphertext 28 and uses his private key 37 to decipher the ciphertext:

Ciphertext: 28	$P = 28^{37} = 63 \bmod 77$	Plaintext: 63

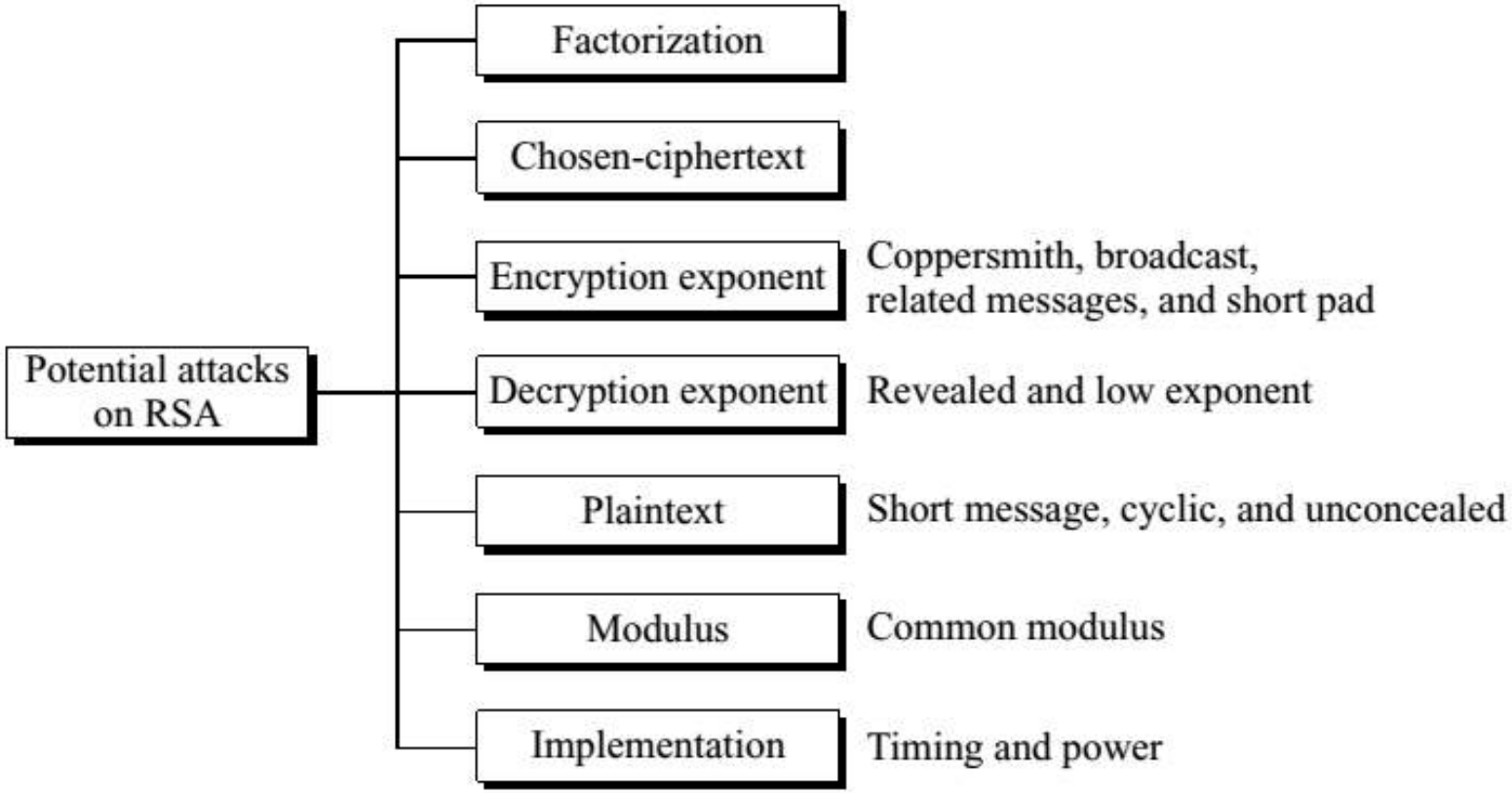
RSA

RSA uses two algebraic structures:
a public ring $R = \langle Z_n, +, \times \rangle$ and a private group $G = \langle Z_{\phi(n)}^*, \times \rangle$.

To be secure, the recommended size for each prime, p or q, is 512 bits (almost 154 decimal digits). This makes the size of n, the modulus, 1024 bits (309 digits).

Attacks on RSA

- No devastating attacks on RSA have been yet discovered.



Tutorial

Recommendations

1. The number of bits for n should be at least 1024. This means that n should be around 2^{1024}, or 309 decimal digits.
2. The two primes p and q must each be at least 512 bits. This means that p and q should be around 2^{512} or 154 decimal digits.
3. The values of p and q should not be very close to each other.
4. Both p − 1 and q − 1 should have at least one large prime factor.
5. The ratio p/q should not be close to a rational number with a small numerator or denominator.
6. The modulus n must not be shared.
7. The value of e should be $2^{16} + 1$ or an integer close to this value.
8. If the private key d is leaked, Bob must immediately change n as well as both e and d. It has been proven that knowledge of n and one pair (e, d) can lead to the discovery of other pairs of the same modulus.
9. Messages must be padded using OAEP (Tutorial)

Recommendations

1. The number of bits for n should be at least 1024. This means that n should be around 2^{1024}, or 309 decimal digits.
2. The two primes p and q must each be at least 512 bits. This means that p and q should be around 2^{512} or 154 decimal digits.
3. The values of p and q should not be very close to each other.
4. Both p − 1 and q − 1 should have at least one large prime factor.
5. The ratio p/q should not be close to a rational number with a small numerator or denominator.
6. The modulus n must not be shared.
7. The value of e should be $2^{16} + 1$ or an integer close to this value.
8. If the private key d is leaked, Bob must immediately change n as well as both e and d. It has been proven that knowledge of n and one pair (e, d) can lead to the discovery of other pairs of the same modulus.
9. Messages must be padded using OAEP (Tutorial)

Why modulus n must not be shared?

- The common modulus attack can be launched if a community uses a common modulus, n.
- For example, people in a community might let a trusted party select p and q, calculate n and $\phi(n)$, and create a pair of exponents (ei, di) for each entity.
- Now assume Alice needs to send a message to Bob. The ciphertext to Bob is $C = P^{eB} \bmod n$. Bob uses his private exponent, dB, to decrypt his message, $P = C^{dB} \bmod n$.
- The problem is that Eve can also decrypt the message if she is a member of the community and has been assigned a pair of exponents (e_E and d_E), as we learned in the section “Low Decryption Exponent Attack”.
- Using her own exponents (e_E and d_E), Eve can launch a probabilistic attack to factor n and find Bob’s dB.
- To thwart this type of attack, the modulus must not be shared.
- Each entity needs to calculate her or his own modulus.

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 9 : Elgamal and ECC
ASYMMETRIC-KEY CRYPTOGRAPHY

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

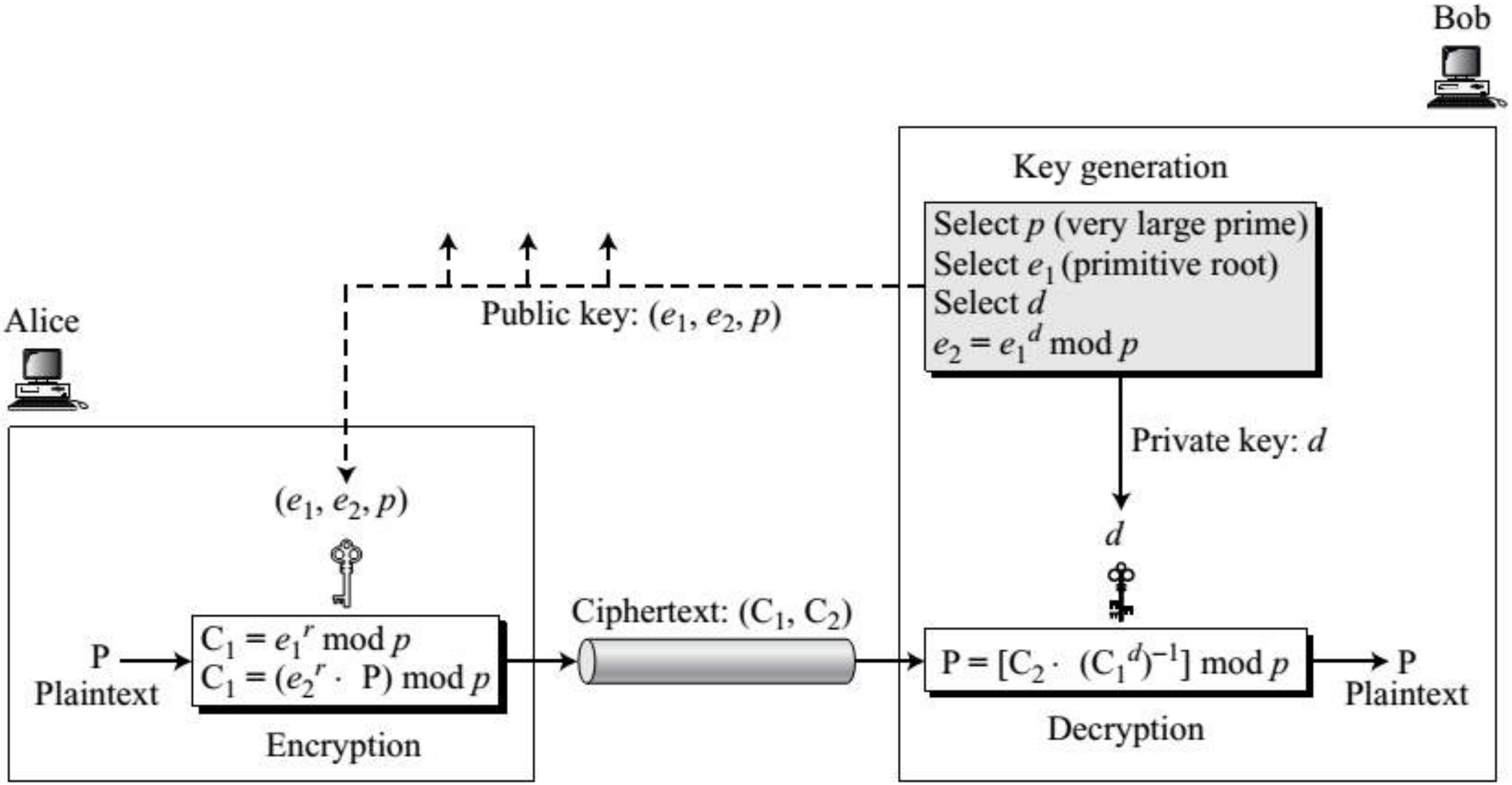
Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

ELGAMAL CRYPTOSYSTEM

- If p is a very large prime,
- e_1 is a primitive root in the group G = <Zp*, × > and
- r is an integer, then $e_2 = e_1^r \bmod p$ is easy to compute using the fast exponential algorithm (square-and-multiply method),
- but given e_2, e_1, and p, it is infeasible to calculate $r = \log_{e1} e2 \bmod p$ (discrete logarithm problem).

ELGAMAL CRYPTOSYSTEM



ELGAMAL CRYPTOSYSTEM: Key Generation

ElGamal_Key_Generation

{

Select a large prime p

Select d to be a member of the group $\mathbf{G} = < \mathbf{Z}_p^*, \times >$ such that $1 \leq d \leq p - 2$

Select e_1 to be a primitive root in the group $\mathbf{G} = < \mathbf{Z}_p^*, \times >$

$e_2 \leftarrow e_1^{\,d} \bmod p$

Public_key $\leftarrow$ (e_1, e_2, p) // To be announced publicly

Private_key $\leftarrow d$ // To be kept secret

return Public_key and Private_key

}

ELGAMAL CRYPTOSYSTEM Encryption and Decryption

ElGamal_Encryption $(e_1, e_2, p, \mathrm{P})$ // P is the plaintext

{

Select a random integer r in the group $\mathbf{G} = < \mathbf{Z}_p^*, \times >$

$\mathrm{C}_1 \leftarrow e_1^{\,r} \bmod p$

$\mathrm{C}_2 \leftarrow (\mathrm{P} \times e_2^{\,r}) \bmod p$ // C_1 and C_2 are the ciphertexts

return C_1 and C_2

}

ElGamal_Decryption $(d, p, \mathrm{C}_1, \mathrm{C}_2)$ // C_1 and C_2 are the ciphertexts

{

$\mathrm{P} \leftarrow [\mathrm{C}_2 \, (\mathrm{C}_1^{\,d})^{-1}] \bmod p$ // P is the plaintext

return P

}

ELGAMAL CRYPTOSYSTEM

- The ElGamal decryption expression $C_2 \times (C_1{}^{d})^{-1}$ can be verified to be P
- Proof

$$[C_2 \times (C_1{}^{d})^{-1}] \bmod p = [(e_2{}^{r} \times P) \times (e_1{}^{rd})^{-1}] \bmod p = (e_1{}^{dr}) \times P \times (e_1{}^{rd})^{-1} = P$$

ELGAMAL CRYPTOSYSTEM: Example

- Bob chooses 11 as p. He then chooses $e_1 = 2$. Note that 2 is a primitive root in Z_{11}^*. Bob then chooses d = 3 and calculates $e_2 = e_1^d = 8$. So the public keys are (2, 8, 11) and the private key is 3. Alice chooses r = 4 and calculates C_1 and C_2 for the plaintext 7.

Plaintext: 7
$C_1 = e_1^r \bmod 11 = 16 \bmod 11 = 5 \bmod 11$
$C_2 = (P \times e_2^r) \bmod 11 = (7 \times 4096) \bmod 11 = 6 \bmod 11$
Ciphertext: (5, 6)

Bob receives the ciphertexts (5 and 6) and calculates the plaintext.

$[C_2 \times (C_1^d)^{-1}] \bmod 11 = 6 \times (5^3)^{-1} \bmod 11 = 6 \times 3 \bmod 11 = 7 \bmod 11$
Plaintext: 7

Security of ElGamal

- Low-Modulus Attacks
  - If the value of p is not large enough, Eve can use some efficient algorithms to solve the discrete logarithm problem to find d or r.
  - If p is small, Eve can easily find $d = \log_{e1} e_2 \bmod p$ and store it to decrypt any message sent to Bob.
  - This can be done once and used as long as Bob uses the same keys.
  - It is recommended that p be at least 1024 bits (300 decimal digits).

Security of ElGamal

- Known-Plaintext Attack
  - If Alice uses the same random exponent r, to encrypt two plaintexts P and P′, Eve discovers P′ if she knows P.
  - Assume that $C_2 = P \times (e_2^r) \bmod p$ and $C'_2 = P' \times (e_2^r) \bmod p$.
  - Eve finds P′ using the following steps

1. $(e_2^r) = C_2 \times P^{-1} \bmod p$
2. $P' = C'_2 \times (e_2^r)^{-1} \bmod p$

It is recommended that Alice use a fresh value of r to thwart the known-plaintext attacks

ELLIPTIC CURVE CRYPTOSYSTEMS

- Although RSA and ElGamal are secure asymmetric-key cryptosystems, their security comes with a price, their large keys.
- Researchers have looked for alternatives that give the same level of security with smaller key sizes.
- One of these promising alternatives is the elliptic curve cryptosystem (ECC).
- The system is based on the theory of elliptic curves.

ELLIPTIC CURVE CRYPTOSYSTEMS

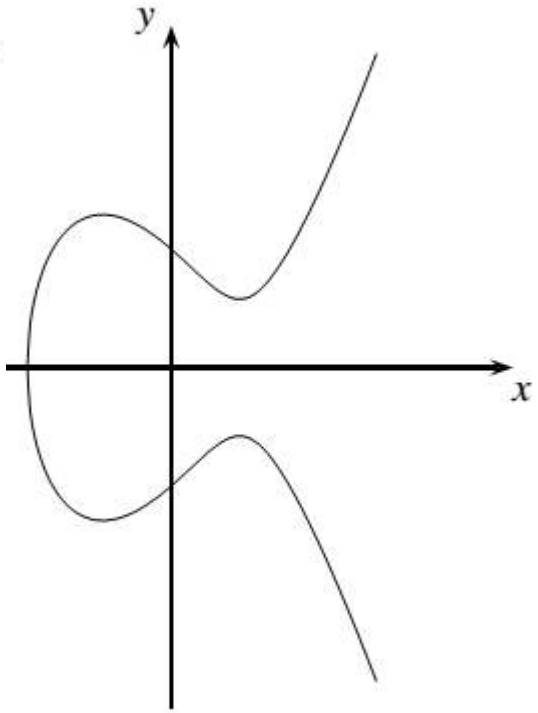
The elliptic curve *over* $\mathbb{Z}_p$, $p > 3$, *is the set of all pairs* $(x, y) \in \mathbb{Z}_p$ *which fulfill*

$$y^2 \equiv x^3 + a \cdot x + b \bmod p$$

together with an imaginary point of infinity $\mathcal{O}$, *where*

$$a, b \in \mathbb{Z}_p$$

and the condition $4 \cdot a^3 + 27 \cdot b^2 \neq 0 \bmod p$.



$4.a^3 + 27.b^2 = 0$ => Singular Ellipic Curve => No three distinct roots

$y^2 = x^3 - 3x + 3$ over $\mathbb{R}$

Finding Points on the Curve

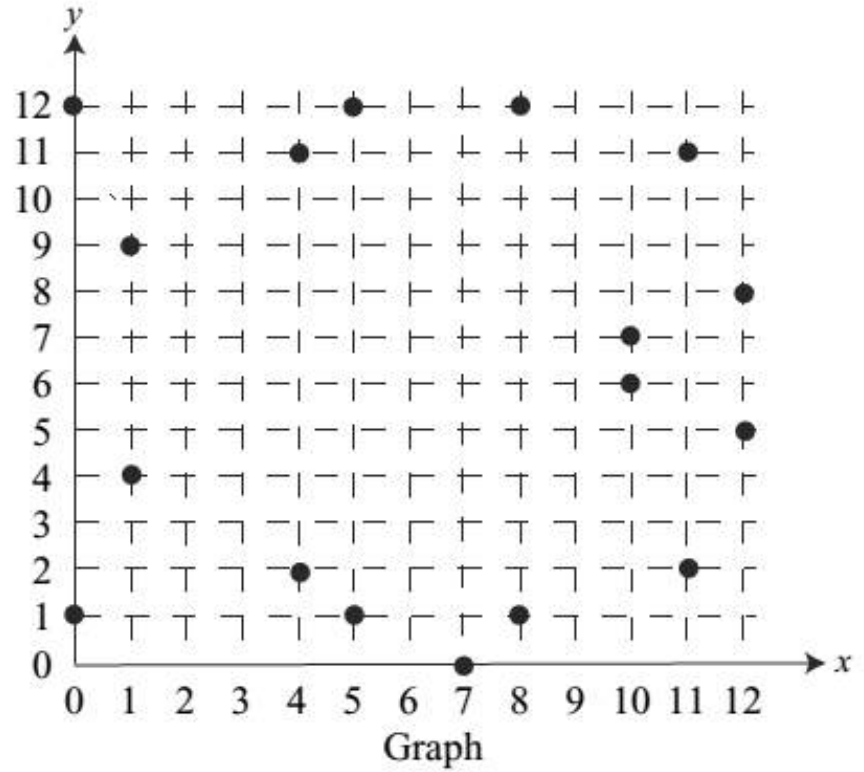
```
ellipticCurve_points (p, a, b)                          // p is the modulus
{
    x ← 0
    while (x < p)
    {
        w ← (x^3 + ax + b) mod p                        // w is y^2
        if (w is a perfect square in Z_p) output (x, √w) (x, − √w)
        x ← x + 1
    {
}
```

Finding Points on the Curve

- Define an elliptic curve $E_{13}(1, 1)$. The equation is $y^2 = x^3 + x + 1$ and the calculation is done modulo 13. Points on the curve can be found as shown in Figure

(0, 1)	(0, 12)
(1, 4)	(1, 9)
(4, 2)	(4, 11)
(5, 1)	(5, 12)
(7, 0)	(7, 0)
(8, 1)	(8, 12)
(10, 6)	(10, 7)
(11, 2)	(11, 11)
(12, 5)	(12, 8)

Points



Graph

Finding Points on the Curve

- Some values of y^2 do not have a square root in modulo 13 arithmetic. These are not points on this elliptic curve. For example, the points with x = 2, x = 3, x = 6, and x = 9 are not on the curve.
- Each point defined for the curve has an inverse. The inverses are listed as pairs. Note that (7, 0) is the inverse of itself.
- Note that for a pair of inverse points, the y values are additive inverses of each other in Z_p. For example, 4 and 9 are additive inverses in Z_{13}. So we can say that if 4 is y, then 9 is −y.
- The inverses are on the same vertical lines.

Group Operations on Elliptic Curves

- The group operation with the addition symbol2 “+”.
- “Addition” means that given two points and their coordinates, say $P = (x1, y1)$ and $Q = (x2, y2)$, we have to compute the coordinates of a third point R such that:

$$P + Q = R$$

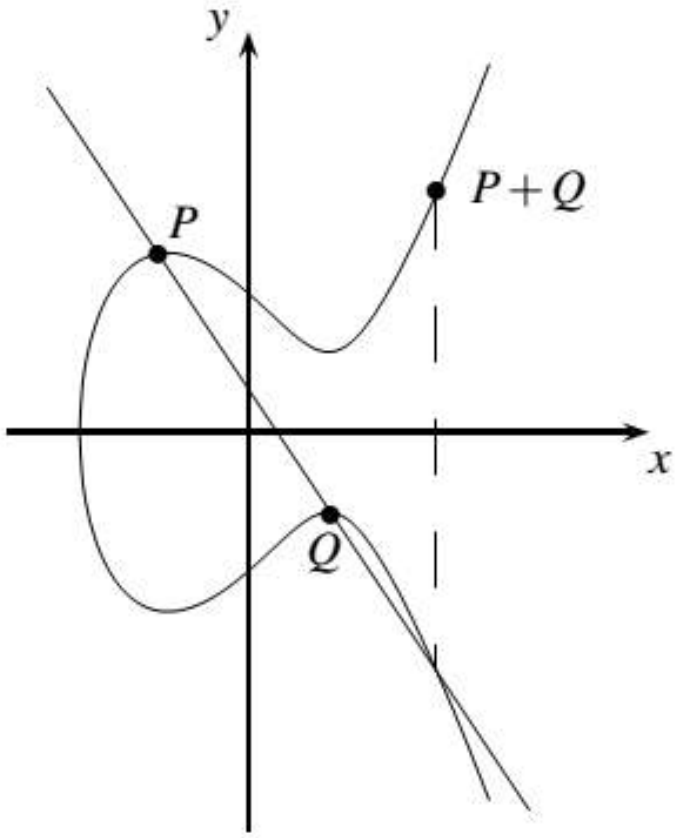
$$(x_1, y_1) + (x_2, y_2) = (x_3, y_3)$$

- **Point Addition P+ Q**
- **Point Doubling P+ P**

$$P_1 \oplus P_2 = \begin{cases} \mathcal{O}_E, & \text{if } x_1 = x_2 \ \& \ y_1 = -y_2 \\ (x_3, y_3), & \text{otherwise.} \end{cases}$$

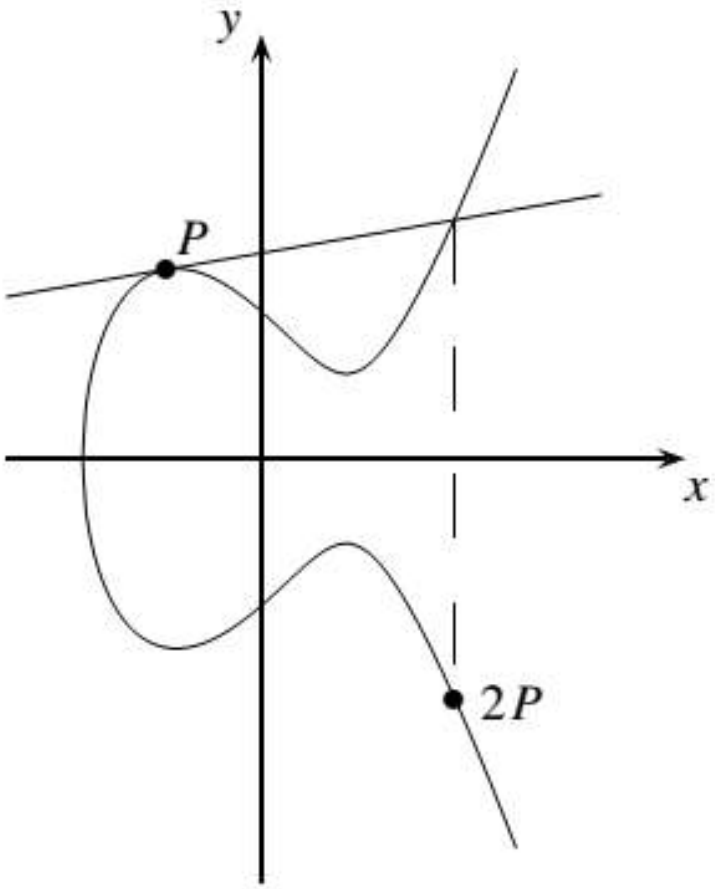
Group Operations on Elliptic Curves

- **Point Addition P+ Q**
- This is the case where we compute $R = P + Q$ and $P \neq Q$.
- The construction works as follows:
  - Draw a line through *P* and *Q* and obtain a third point of intersection between the elliptic curve and the line.
  - Mirror this third intersection point along the *x*-axis.
  - This mirrored point is, by definition, the point *R*.
  - Figure shows the point addition on an elliptic curve over the real numbers.



Group Operations on Elliptic Curves

- **Point Doubling P+ P**
- This is the case where we compute *P*+ *Q* but *P* = *Q*.
- Hence, we can write *R* = *P* + *P* = 2*P*.
- We need a slightly different construction here.
- We mirror the point of the second intersection along the *x*-axis.
- This mirrored point is the result *R* of the doubling.
- Figure shows the doubling of a point on an elliptic curve over the real numbers.



Group Operations on Elliptic Curves

Elliptic Curve Point Addition and Point Doubling

$$x_3 = s^2 - x_1 - x_2 \bmod p$$
$$y_3 = s(x_1 - x_3) - y_1 \bmod p$$

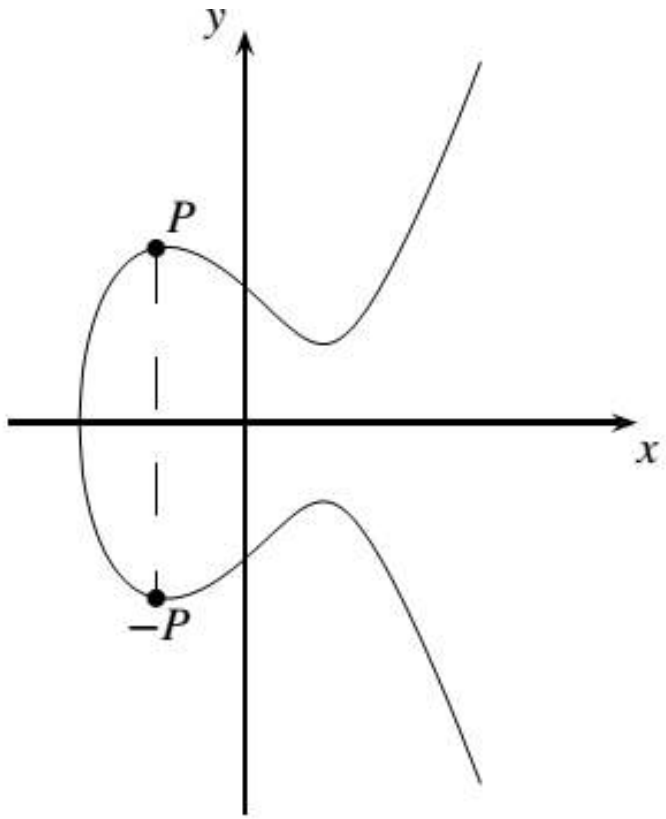
where

$$s = \begin{cases} \frac{y_2 - y_1}{x_2 - x_1} \bmod p \text{ ; if } P \neq Q \text{ (point addition)} \\ \frac{3x_1^2 + a}{2y_1} \bmod p \text{ ; if } P = Q \text{ (point doubling)} \end{cases}$$

s is the slope of the line

Group Operations on Elliptic Curves

- Given P, how do we find $-P$?
- If we apply the tangent-and-chord method from above, it turns out that the inverse of the point $P = (x_p, y_p)$ is the point $-P = (x_p, -y_p)$, i.e., the point that is reflected along the x-axis.
- We simply take the negative of its y coordinate.
- In the case of elliptic curves over a prime field $GF(p)$, this is easily achieved since $-y_p \equiv p - y_p \bmod p$, hence, $-P = (x_p, p - y_p)$.



Group Operations on Elliptic Curves

- Example: We consider a curve over the small field Z_{17}:

$$E : y^2 \equiv x^3 + 2x + 2 \bmod 17.$$

$$y^2 \equiv x^3 + 2 \cdot x + 2 \bmod 17$$
$$3^2 \equiv 6^3 + 2 \cdot 6 + 2 \bmod 17$$
$$9 = 230 \equiv 9 \bmod 17$$

We want to double the point $P = (5, 1)$.

$$2P = P + P = (5,1) + (5,1) = (x_3, y_3)$$

$$s = \frac{3x_1^2 + a}{2y_1} = (2 \cdot 1)^{-1}(3 \cdot 5^2 + 2) = 2^{-1} \cdot 9 \equiv 9 \cdot 9 \equiv 13 \bmod 17$$

$$x_3 = s^2 - x_1 - x_2 = 13^2 - 5 - 5 = 159 \equiv 6 \bmod 17$$

$$y_3 = s(x_1 - x_3) - y_1 = 13(5 - 6) - 1 = -14 \equiv 3 \bmod 17$$

$$2P = (5,1) + (5,1) = (6,3)$$

Multiplying a Point by a Constant

- In arithmetic, multiplying a number by a constant k means adding the number to itself k times.
- The situation here is the same. Multiplying a point P on an elliptic curve by a constant k means adding the point P to itself k times.
- For example, in E_{13} (1, 1),
  - if the point (1, 4) is multiplied by 4, the result is the point (5, 1).
  - If the point (8, 1) is multiplied by 3, the result is the point (10, 7).

Discrete Logarithm Problem with Elliptic Curves

Hasse's theorem

Given an elliptic curve E modulo p, the number of points on the curve is denoted by #E and is bounded by:

$$p+1-2\sqrt{p} \leq \#E \leq p+1+2\sqrt{p}.$$

Elliptic Curved Discrete Logarithm Problem (ECDLP)

Given is an elliptic curve E. We consider a primitive element P and another element T. The DL problem is finding the integer d, where $1 \leq d \leq \#E$*, such that:*

$$\underbrace{P+P+\cdots+P}_{d \text{ times}} = dP = T.$$

Elliptic Curve Cryptography Simulating ElGamal

- Generating Public and Private Keys
  1. Bob chooses E(a, b) with an elliptic curve over GF(p) or GF(2^n). (#Tutorial)
  2. Bob chooses a point on the curve, $e_1(x_1, y_1)$.
  3. Bob chooses an integer d.
  4. Bob calculates $e_2(x_2, y_2) = d \times e_1(x_1, y_1)$. Note that multiplication here means multiple addition of points as defined before.
  5. Bob announces E(a, b), $e_1(x_1, y_1)$, and $e_2(x_2, y_2)$ as his public key; he keeps d as his private key.

Elliptic Curve Cryptography Simulating ElGamal

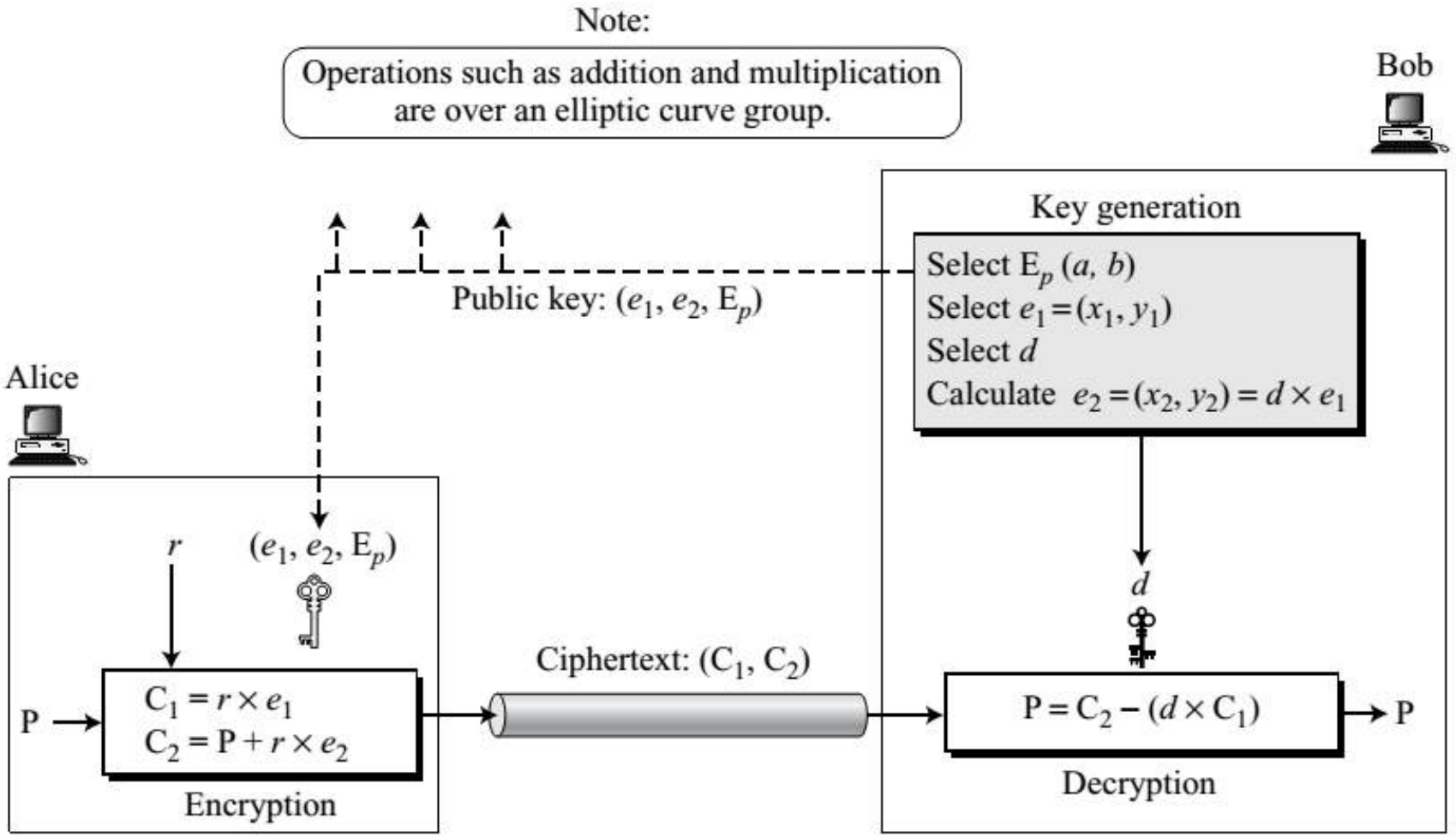
- Encryption
  - Alice selects P, a point on the curve, as her plaintext, P.
  - She then calculates a pair of points on the text as ciphertexts:

$$C_1 = r \times e_1 \qquad C_2 = P + r \times e_2$$

- Decryption
  - Bob, after receiving C_1 and C_2, calculates P, the plaintext using the following formula.
  - $P = C_2 - (d \times C_1)$ The minus sign here means adding with the inverse.

$$P + r \times e_2 - (d \times r \times e_1) = P + (r \times d \times e_1) - (r \times d \times e_1) = P + \boldsymbol{O} = P$$

Elliptic Curve Cryptography Simulating ElGamal



Elliptic Curve Cryptography Simulating ElGamal

- Example

Here is a very trivial example of encipherment using an elliptic curve over **GF**(p).

1. Bob selects $E_{67}(2, 3)$ as the elliptic curve over **GF**(p).
2. Bob selects $e_1 = (2, 22)$ and $d = 4$.
3. Bob calculates $e_2 = (13, 45)$, where $e_2 = d \times e_1$.
4. Bob publicly announces the tuple (E, e_1, e_2).
5. Alice wants to send the plaintext $P = (24, 26)$ to Bob. She selects $r = 2$.
6. Alice finds the point $C_1 = (35, 1)$, where $C_1 = r \times e_1$.
7. Alice finds the point $C_2 = (21, 44)$, where $C_2 = P + r \times e_2$.
8. Bob receives C_1 and C_2. He uses $2 \times C_1$ (35, 1) to get (23, 25).
9. Bob inverts the point (23, 25) to get the point (23, 42).
10. Bob adds (23, 42) with $C_2 = (21, 44)$ to get the original plaintext $P = (24, 26)$.

Security of ECC

- To decrypt the message, Eve needs to find the value of r or d.

a. If Eve knows r, she can use $P = C_2 - (r \times e_2)$ to find the point P related to the plaintext. But to find r, Eve needs to solve the equation $C_1 = r \times e_1$. This means, given two points on the curve, C_1 and e_1, Eve must find the multiplier that creates C_1 starting from e_1. This is referred to as the **elliptic curve logarithm problem,** and the only method available to solve it is the Polard rho algorithm, which is infeasible if r is large, and p in GF(p) or n in GF(2^n) is large.

b. If Eve knows d, she can use $P = C_2 - (d \times C_1)$ to find the point P related to the plaintext. Because $e_2 = d \times e_1$, this is the same type of problem. Eve knows the value of e_1 and e_2; she needs to find the multiplier d.

Modulus Size

- For the same level of security (computational effort), the modulus, n, can be smaller in ECC than in RSA.
- For example, ECC over the GF(2^n) with n of 160 bits can provide the same level of security as RSA with n of 1024 bits.

Class Exercise #1

- ECC over the GF(2^n) with n of 160 bits can provide the same level of security as RSA with n of 1024 bits. Finding points on the curve is non-trivial and on the elliptic curve E : $y^2 = x^3-2x -3$, there is a point P (3, 2) over Z/7Z. Compute 10P.

$$P_1 \oplus P_2 = \begin{cases} \mathcal{O}_E, & \text{if } x_1 = x_2 \ \& \ y_1 = -y_2 \\ (x_3, y_3), & \text{otherwise.} \end{cases}$$

$$\begin{aligned}
2P &= P \oplus P = (3,2) \oplus (3,2) = (2,6), \\
3P &= P \oplus 2P = (3,2) \oplus (2,6) = (4,2), \\
4P &= P \oplus 3P = (3,2) \oplus (4,2) = (0,5), \\
5P &= P \oplus 4P = (3,2) \oplus (0,5) = (5,0), \\
6P &= P \oplus 5P = (3,2) \oplus (5,0) = (0,2), \\
7P &= P \oplus 6P = (3,2) \oplus (0,2) = (4,5), \\
8P &= P \oplus 7P = (3,2) \oplus (4,5) = (2,1), \\
9P &= P \oplus 8P = (3,2) \oplus (2,1) = (3,5), \\
10P &= P \oplus 9P = (3,2) \oplus (3,5) = \mathcal{O}_E.
\end{aligned}$$

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 10 : Message Integrity and Message Authentication

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

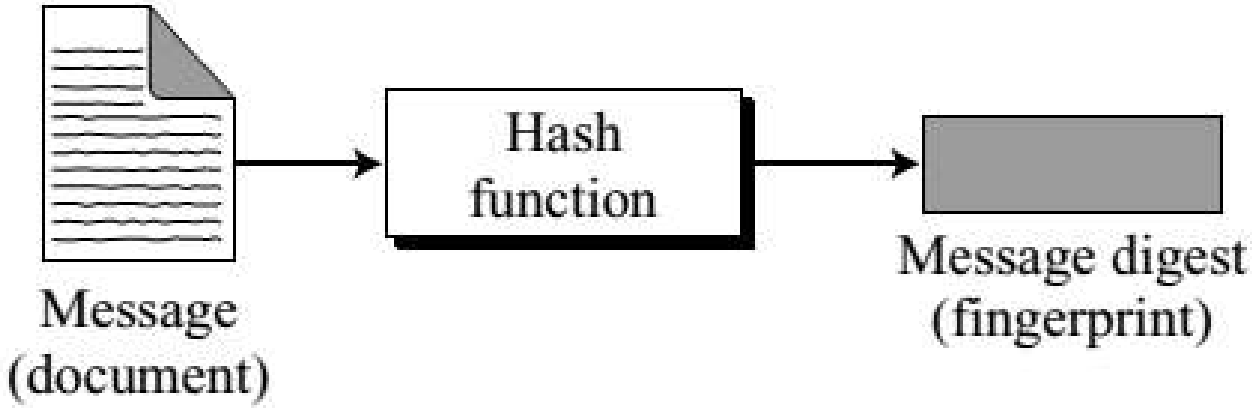
Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

MESSAGE INTEGRITY

- Alice may write a will to distribute her estate upon her death.
- The will does not need to be encrypted. After her death, anyone can examine the will.
- The integrity of the will, however, needs to be preserved.
- Alice does not want the contents of the will to be changed.
- Document and Fingerprint
  - If Alice needs to be sure that the contents of her document will not be changed, she can put her fingerprint at the bottom of the document.
- Eve cannot modify the contents of this document or create a false document because she cannot forge Alice's fingerprint.

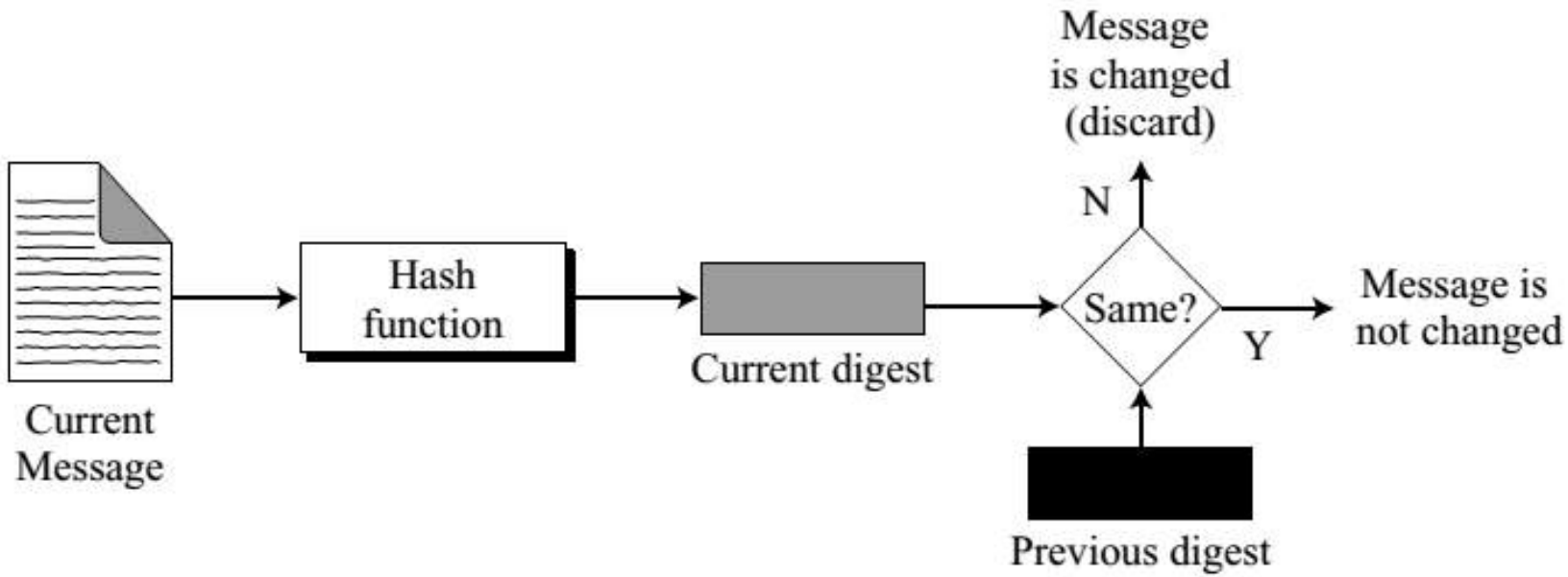
Message and Message Digest

- The electronic equivalent of the document and fingerprint pair is the message and digest pair.
- To preserve the integrity of a message, the message is passed through an algorithm called a cryptographic hash function.
- The function creates a compressed image of the message that can be used like a fingerprint.



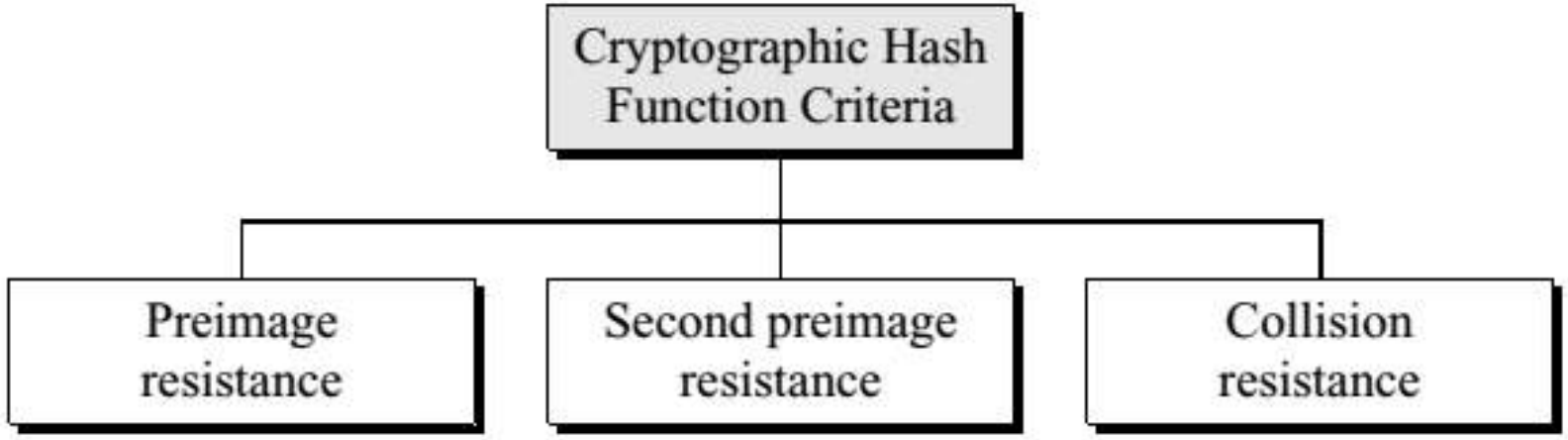
Checking Integrity

- To check the integrity of a message, or document, we run the cryptographic hash function again and compare the new message digest with the previous one.
- If both are the same, we are sure that the original message has not been changed



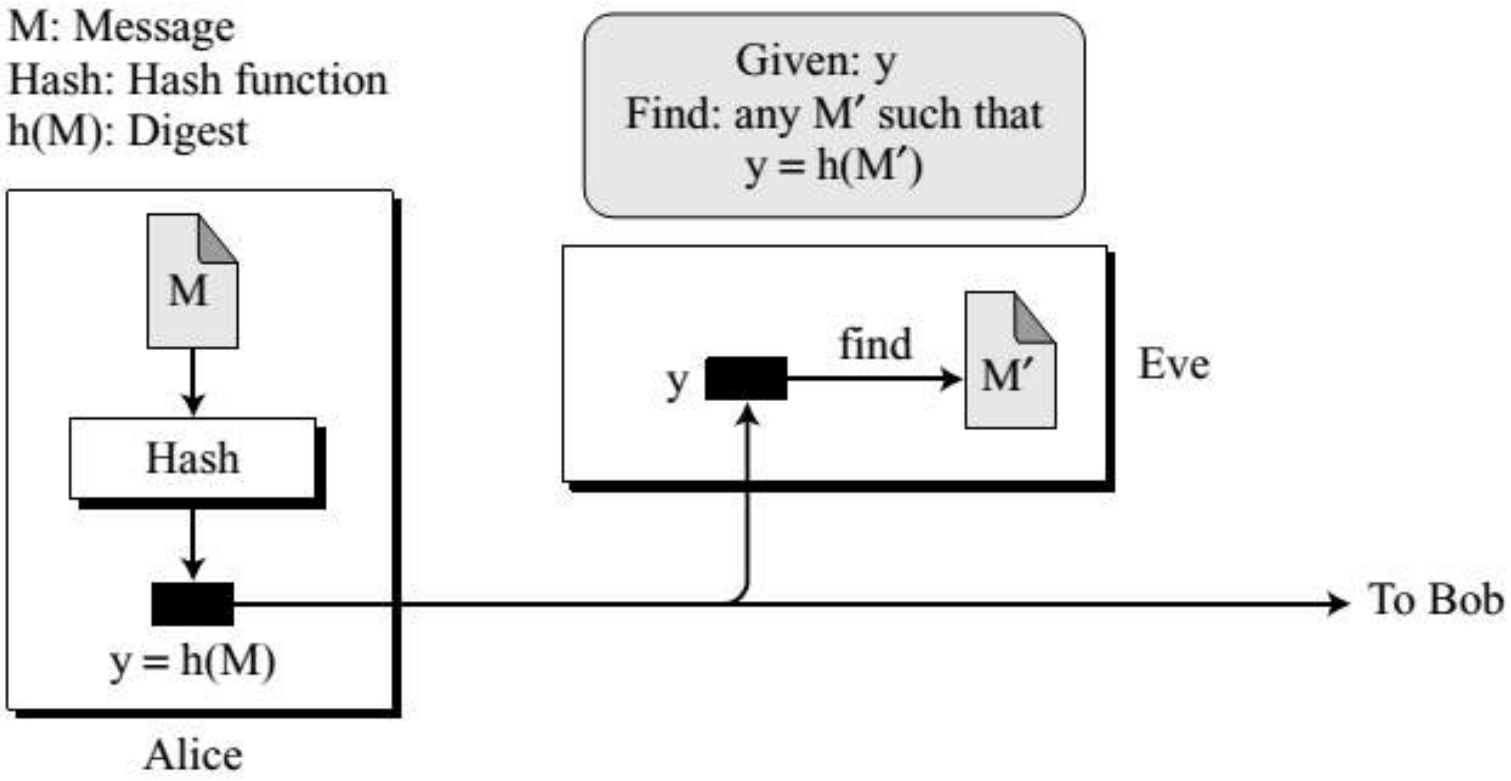
Cryptographic Hash Function Criteria

- A cryptographic hash function must satisfy three criteria:
  - preimage resistance,
  - second preimage resistance, and
  - collision resistance,



Preimage Resistance

- A cryptographic hash function must be preimage resistant.
- Given a hash function h and y = h(M), it must be extremely difficult for Eve to find any message, M′, such that y = h(M′).



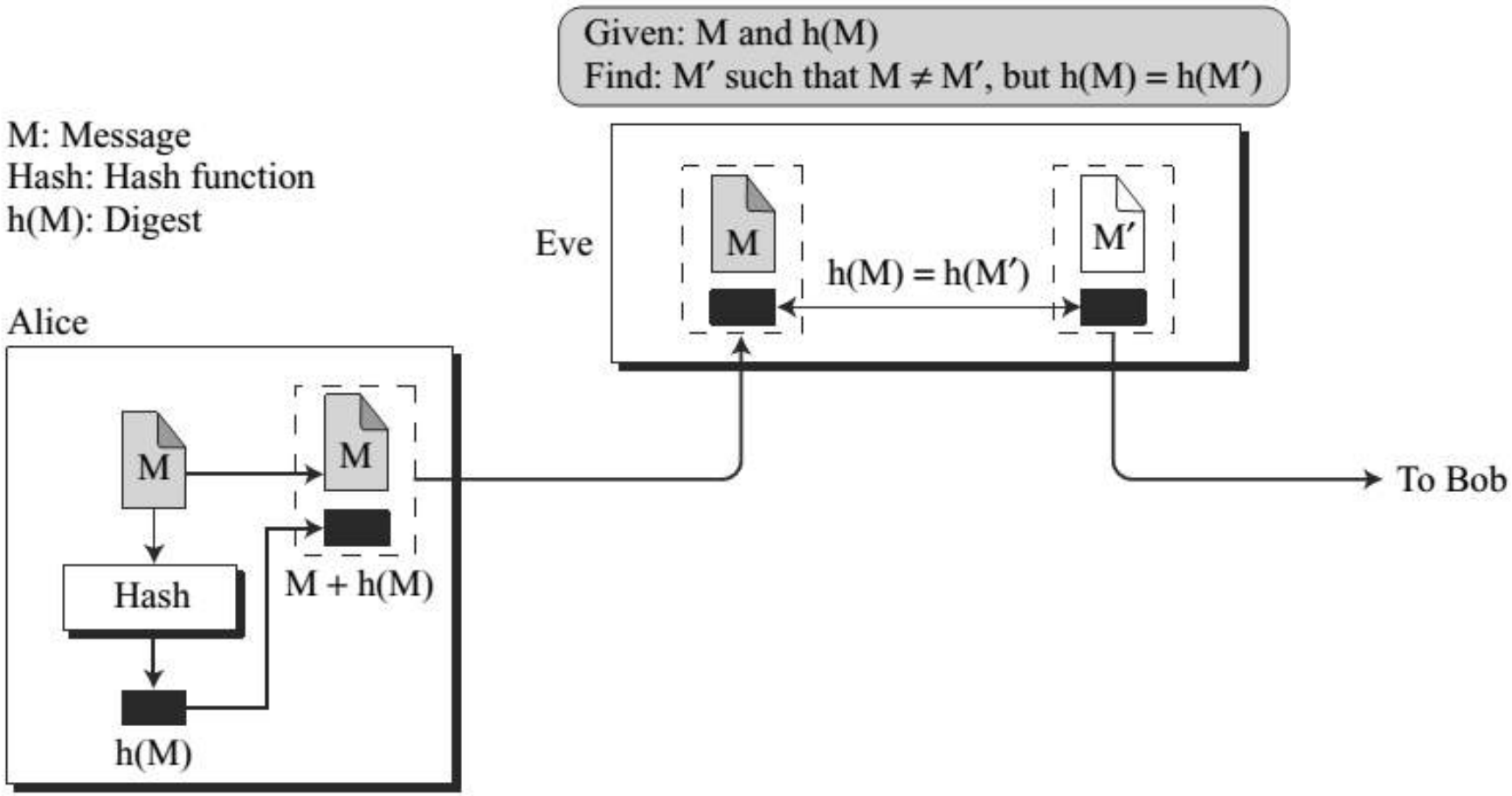
Preimage Resistance

- If the hash function is not preimage resistant, Eve can intercept the digest h(M) and create a message M’.
- Eve can then send M’ to Bob pretending it is M.
- Can we use a conventional lossless compression method such as StuffIt as a cryptographic hash function?
  - We cannot. A lossless compression method creates a compressed message that is reversible. You can uncompress the compressed message to get the original one.
- Can we use a checksum function as a cryptographic hash function?
  - We cannot. A checksum function is not preimage resistant, Eve may find several messages whose checksum matches the given one.

Second Preimage Resistance

- The second criterion, second preimage resistance, ensures that a message cannot easily be forged.
- If Alice creates a message and a digest and sends both to Bob, this criterion ensures that Eve cannot easily create another message that hashes to the exact same digest.
- In other words, given a specific message and its digest, it is impossible (or at least very difficult) to create another message with the same digest.
- Eve intercepts (has access to) a message M and its digest h(M). She creates another message M’≠ M, but h(M) = h(M’). Eve sends the M’ and h(M’) to Bob. Eve has forged the message.

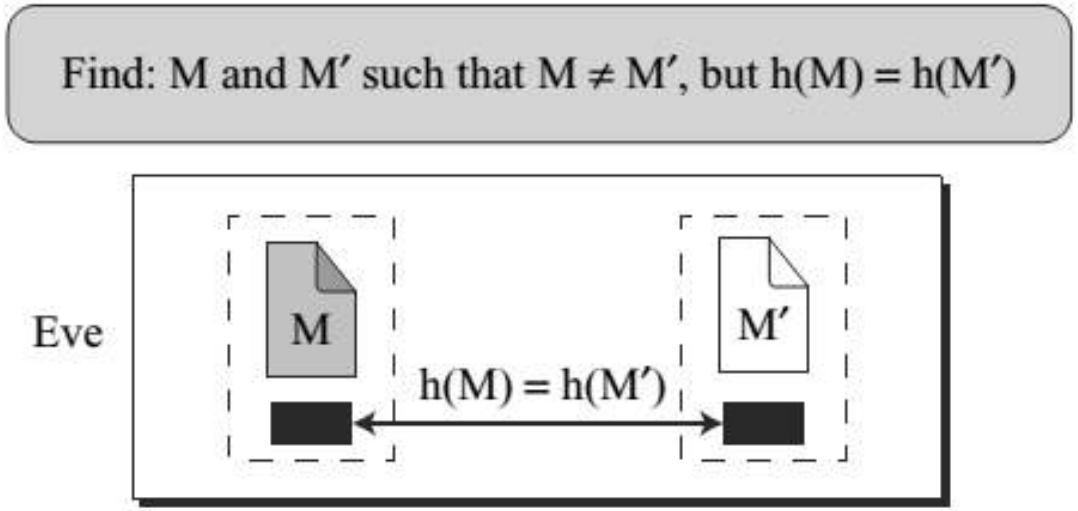
Second Preimage Resistance



Collision Resistance

- Collision resistance, ensures that Eve cannot find two messages that hash to the same digest.
- The adversary can create two messages (out of scratch) and hashed to the same digest

M: Message
Hash: Hash function
h(M): Digest

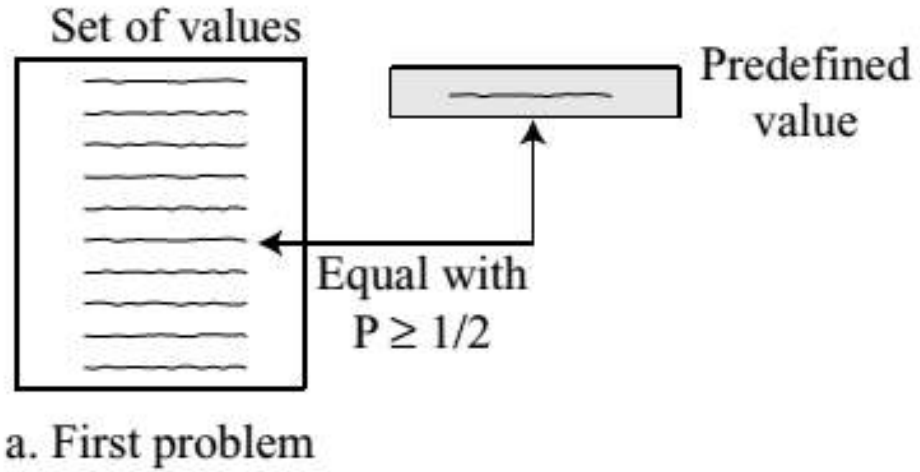


Birthday Problems

- Four different birthday problems are usually encountered in the probability courses.
- The third problem, sometimes referred to as birthday paradox, is the most common one in the literature.
- Description of Problems
  - Below the birthday problems are described in terms that can be applied to the security of hash functions.
  - Note that the term likely in all cases means with the probability $P \geq 1/2$.

Problem 1

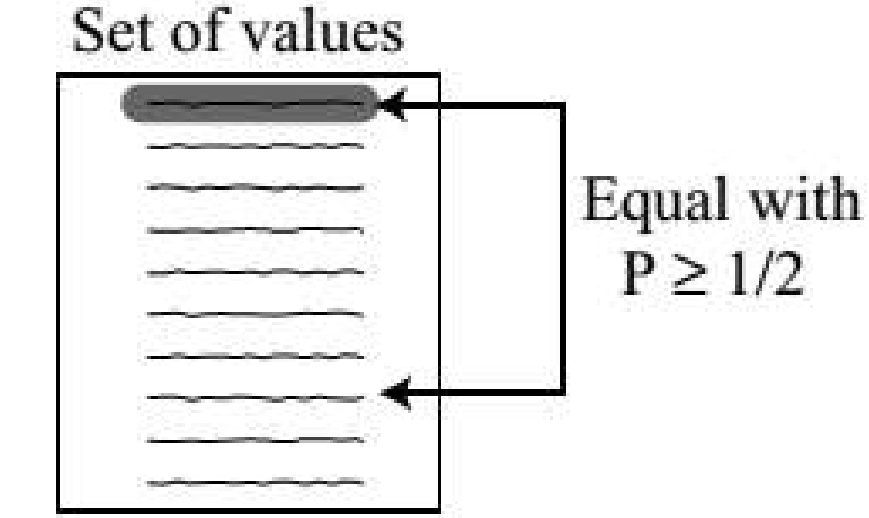
- What is the minimum number, k, of students in a classroom such that it is likely that at least one student has a predefined birthday?
- This problem can be generalized as follows.
  - We have a uniformly distributed random variable with N possible values (between 0 and N − 1).
  - What is the minimum number of instances, k, such that it is likely that at least one instance is equal to a predefined value?



a. First problem

Problem 2

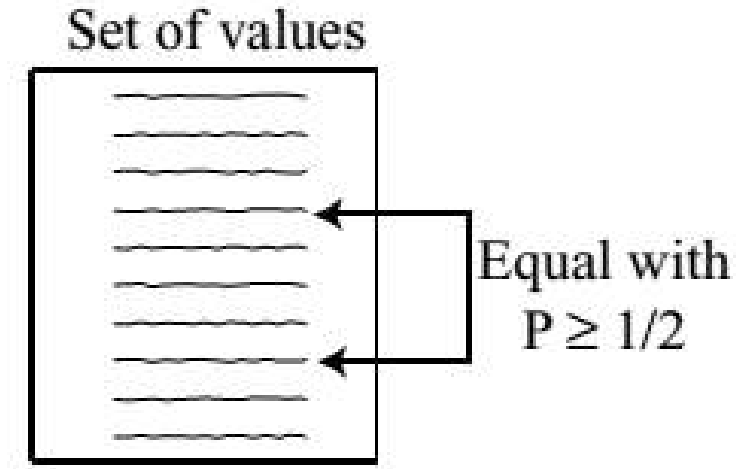
- What is the minimum number, k, of students in a classroom such that it is likely that at least one student has the same birthday as the student selected by the professor?
- This problem can be generalized as follows.
  - We have a uniformly distributed random variable with N possible values (between 0 and N − 1).
  - What is the minimum number of instances, k, such that it is likely that at least one instance is equal to the selected one?



b. Second problem

Problem 3

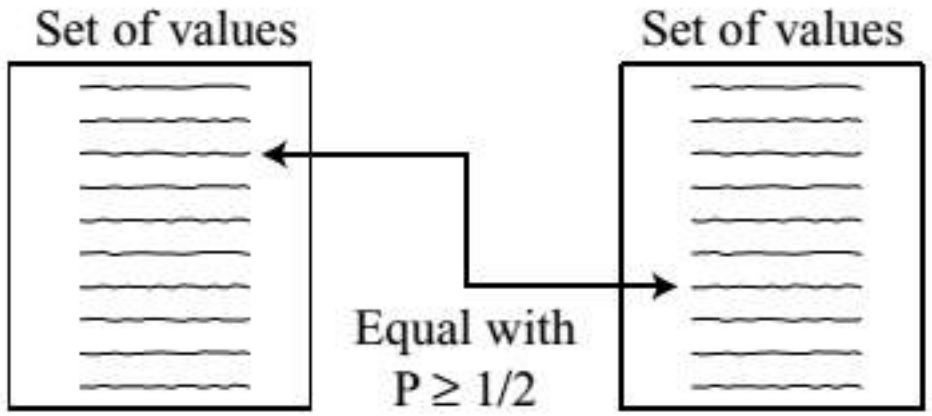
- What is the minimum number, k, of students in a classroom such that it is likely that at least two students have the same birthday?
- This problem can be generalized as follows.
  - We have a uniformly distributed random variable with N possible values (between 0 and N − 1).
  - What is the minimum number of instances, k, such that it is likely that at least two instances are equal?



c. Third problem

Problem 4

- We have two classes, each with k students. What is the minimum value of k so that it is likely that at least one student from the first classroom has the same birthday as a student from the second classroom?
- This problem can be generalized as follows.
  - We have a uniformly distributed random variable with N possible values (between 0 and N − 1).
  - We generate two sets of random values each with k instances.
  - What is the minimum number of, k, such that it is likely that at least one instance from the first set is equal to one instance in the second set?



d. Fourth problem

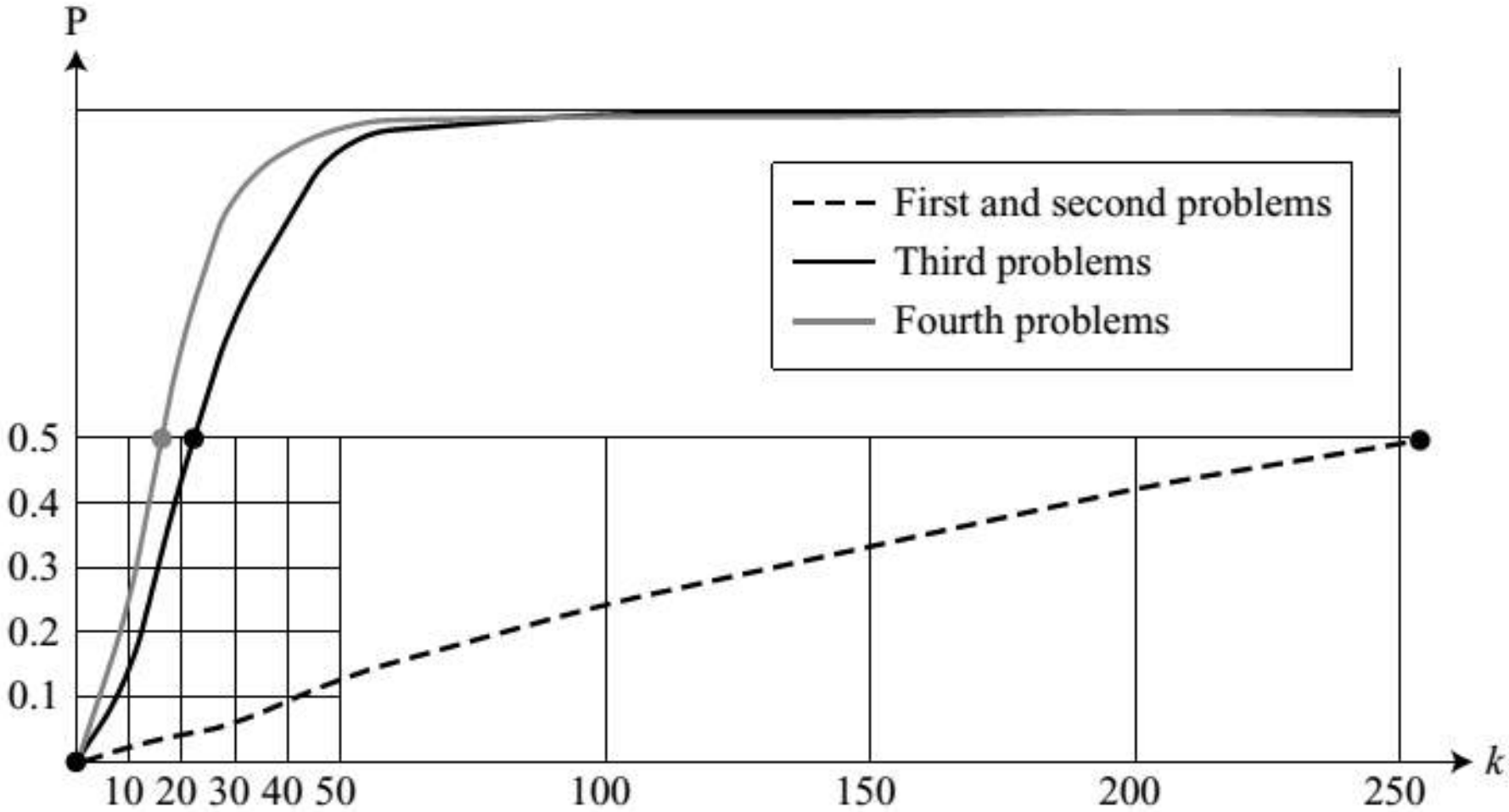
Summary of Solutions

Problem	*Probability*	*General value for k*	*Value of k with P = 1/2*	*Number of students (N = 365)*
1	$P \approx 1 - e^{-k/N}$	$k \approx \ln[1/(1-P)] \times N$	$k \approx 0.69 \times N$	253
2	$P \approx 1 - e^{-(k-1)/N}$	$k \approx \ln[1/(1-P)] \times N + 1$	$k \approx 0.69 \times N + 1$	254
3	$P \approx 1 - e^{-k(k-1)/2N}$	$k \approx \{2 \ln [1/(1-P)]\}^{1/2} \times N^{1/2}$	$k \approx 1.18 \times N^{1/2}$	23
4	$P \approx 1 - e^{-k^2/2N}$	$k \approx \{\ln [1/(1-P)]\}^{1/2} \times N^{1/2}$	$k \approx 0.83 \times N^{1/2}$	16

Comparison

- The value of k in problems 1 or 2 is proportional to N; the value of k in problems 3 or 4 is proportional to $N^{1/2}$.
- The first two problems are related to preimage and second preimage attacks; the third and the fourth problems are related to the collision attack.
- The comparison shows it is much more difficult to launch a preimage or second preimage attack than to launch a collision attack.
- For the first and second problem only one graph is shown (probabilities are very close). The graphs for the second and the third problems are more distinct.

Comparison


P
0.5
0.4
0.3
0.2
0.1
10 20 30 40 50
100
150
200
250
k
First and second problems
Third problems
Fourth problems

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 11 : Message Integrity and Message Authentication-Part-2

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

RANDOM ORACLE MODEL

- A function based on this model behaves as follows:

1. When a new message of any length is given, the oracle creates and gives a fixed length message digest that is a random string of 0s and 1s. The oracle records the message and the message digest.

2. When a message is given for which a digest exists, the oracle simply gives the digest in the record.

3. The digest for a new message needs to be chosen independently from all previous digests. This implies that the oracle cannot use a formula or an algorithm to calculate the digest.

Example

- Assume an oracle with a table and a fair coin. The table has two columns.
- The left column shows the messages whose digests have been issued by the oracle.
- The second column lists the digests created for those messages.
- We assume that the digest is always 16 bits regardless of the size of the message.

Message	*Message Digest*
4523AB1352CDEF45126	13AB
723BAE38F2AB3457AC	02CA
AB45CD1048765412AAAB6662BE	A38B

Example

- The message and the message digest are listed in hexadecimal. The oracle has already created three digests.
- Assume two events occur
- The message AB1234CD8765BDAD is given for digest calculation.
- The oracle checks its table. This message is not in the table, so the oracle flips its coin 16 times.
- Assume that result is HHTHHHTTHTHHTTTH, in which the letter H represents heads and the letter T represents tails.
- The oracle interprets H as a 1-bit and T as a 0-bit and gives 1101110010110001 in binary, or DCB1 in hexadecimal, as the message digest for this message and adds the note of the message and the digest in the table.

Example

Message	*Message Digest*
4523AB1352CDEF45126	13AB
723BAE38F2AB3457AC	02CA
AB1234CD8765BDAD	DCB1
AB45CD1048765412AAAB6662BE	A38B

Example

- The message 4523AB1352CDEF45126 is given for digest calculation.
- The oracle checks its table and finds that there is a digest for this message in the table (first row).
- The oracle simply gives the corresponding digest (13AB).
- The oracle example cannot use a formula or algorithm to create the digest for a message.
- For example, imagine the oracle uses the formula h(M) = M mod n.
- Now suppose that the oracle has already given h(M1) and h(M2).
- If a new message is presented as M3 = M1 + M2, the oracle does not have to calculate the h(M3).
- The new digest is just [h(M1) + h(M2)] mod n since
  h(M3) = (M1 + M2) mod n = M1 mod n + M2 mod n = [h(M1) + h(M2)] mod n
- This violates the third requirement that each digest must be randomly chosen based on the message given to the oracle.

Attacks on Random Oracle Model

- Suppose that the hash function creates digests of n bits.
- Then the digest can be thought of as a random variable uniformly distributed between 0 and N − 1 in which $N = 2^n$.
- In other words, there are 2^n possible values for the digest; each time the oracle randomly selects one of these values for a message.
- Note that this does not mean that the selection is exhaustive; some values may never be selected, but some may be selected several times.
- We assume that the hash function algorithm is public and Eve knows the size of the digest, n.

Preimage Attack

- Eve has intercepted a digest D = h(M); she wants to find any message M′ such that D = h(M′).
- Eve can create a list of k messages and run Algorithm

```
Preimage_Attack (D)
{
   for (i = 1 to k)
   {
        create (M [i])
        T ← h(M [i])                    // T is a temporary digest
        if (T = D) return M [i]
   }
   return failure
}
```

Preimage Attack

- The algorithm can find a message for which D is the digest or it may fail.
- What is the probability of success of this algorithm? Obviously, it depends on the size of list, k, chosen by Eve.
- To find the probability, we use the first birthday problem.
- The digest created by the program defines the outcomes of a random variable.
- The probability of success is $P \approx 1 - e^{-k/N}$.
- If Eve needs to be at least 50 percent successful, what should be the size of k?: $k \approx 0.69 \times N$, or $k \approx 0.69 \times 2^n$.
- In other words, for Eve to be successful more than 50 percent of the time, she needs to create a list of digest that is proportional to 2^n

Preimage Attack

- A cryptographic hash function uses a digest of 64 bits. How many digests does Eve need to create to find the original message with the probability more than 0.5
- The number of digests to be created is $k \approx 0.69 \times 2^n \approx 0.69 \times 2^{64}$. This is a large number. Even if Eve can create 2^{30} (almost one billion) messages per second, it takes 0.69×2^{34} seconds or more than 500 years. This means that a message digest of size 64 bits is secure with respect to preimage attack.

Second Preimage Attack

- Eve has intercepted a digest D = h(M) and the corresponding message M; she wants to find another message M′ so that h(M′) = D.
- Eve can create a list of k − 1 messages and run Algorithm

```
Second_Preimage_Attack (D, M)
{
   for (i = 1 to k – 1)
   {
        create (M [i])
        T ← h (M [i])                    // T is a temporary digest
        if (T = D) return M [i]
   }
   return failure
}
```

Second Preimage Attack

- The algorithm can find a second message for which D is also the digest or it may fail.
- What is the probability of success of this algorithm? Obviously, it depends on the size of list, k, chosen by Eve.
- To find the probability, we use the second birthday problem.
- The digest created by the program defines the outcomes of a random variable.
- The probability of success is $P \approx 1 - e^{-(k-1)/N}$.
- If Eve needs to be at least 50 percent successful, what should be the size of k? :$k \approx 0.69 \times N + 1$ or $k \approx 0.69 \times 2^n + 1$.
- In other words, for Eve to be successful more than 50 percent of the time, she needs to create a list of digest that is proportional to 2^n

Collision Attack

- Eve needs to find two messages, M and M’; such that h(M) = h(M’). Eve can create a list of k messages and run Algorithm

```
Collision_Attack
{
   for (i = 1 to k )
   {
        create (M[i])
        D[i] ← h (M[i])                    // D [i] is a list of created digests
        for (j = 1 to i – 1)
        {
           if (D[i] = D[j]) return (M[i] and M[j])
        }
   }
   return failure
}
```

Collision Attack

- The algorithm can find two messages with the same digest.
- What is the probability of success of this algorithm? Obviously, it depends on the size of list, k, chosen by Eve.
- To find the probability, we use the third birthday problem.
- The digest created by program defines the outcomes of a random variable.
- The probability of success is $P \approx 1 - e^{-k(k-1)/2N}$.
- If Eve needs to be at least fifty percent successful, what should be the size of k? : $k \approx 1.18 \times N^{1/2}$, or $k \approx 1.18 \times 2^{n/2}$.
- In other words, for Eve to be successful more than 50 percent of the time, she needs to create a list of digests that is proportional to $2^{n/2}$.

Example

- A cryptographic hash function uses a digest of 64 bits. How many digests does Eve need to create to find two messages with the same digest with the probability more than 0.5?
- The number of digests to be created is $k \approx 1.18 \times 2^{n/2} \approx 1.18 \times 2^{32}$. If Eve can test 2^{20} (almost one million) messages per second, it takes 1.18×2^{12} seconds, or less than two hours. This means that a message digest of size 64 bits is not secure against the collision attack.

Alternate Collision Attack

- The previous collision attack may not be useful for Eve.
- The adversary needs to create two messages, one real and one bogus, that hash to the same value.
- Each message should be meaningful.
- The previous algorithm does not provide this type of collision.
- The solution is to create two meaningful messages, but add redundancies or modifications to the message to change the contents of the message without changing the meaning of each.
- For example, a number of messages can be made from the first message by adding spaces, or changing the words, or adding some redundant words, and so on.
- The second message can also create a number of messages.
- Let us call the original message M and the bogus message M’.

Alternate Collision Attack

- Eve creates k different variants of M (M1, M2, …, Mk) and k different variants of M′ (M′1, M′2, … , M′k).
- Eve then uses below Algorithm to launch the attack.

```
Alternate_Collision_Attack (M [k], M′[k])
{
   for (i = 1 to k )
   {
      D[i] ← h (M[i])
      D′[i] ← h (M′[i])
      if (D [i] = D′[j]) return (M[i], M′[j])
   }
   return failure
}
```

Alternate Collision Attack

- What is the probability of success of this algorithm? Obviously, it depends on the size of the list, k, chosen by Eve.
- To find the probability, we use the fourth birthday problem.
- The two digest lists created by program defines the two outcomes of a random variable.
- The probability of success is $P \approx 1 - e^{-k^2/N}$.
- If Eve needs to be at least 50 percent successful, what should be the size of k? : $k \approx 0.83 \times N^{1/2}$ or $k \approx 0.83 \times 2^{n/2}$.
- In other words, for Eve to be successful more than 50% of the time, she needs to create a list of digests that is proportional to $2^{n/2}$.

Summary of Attacks

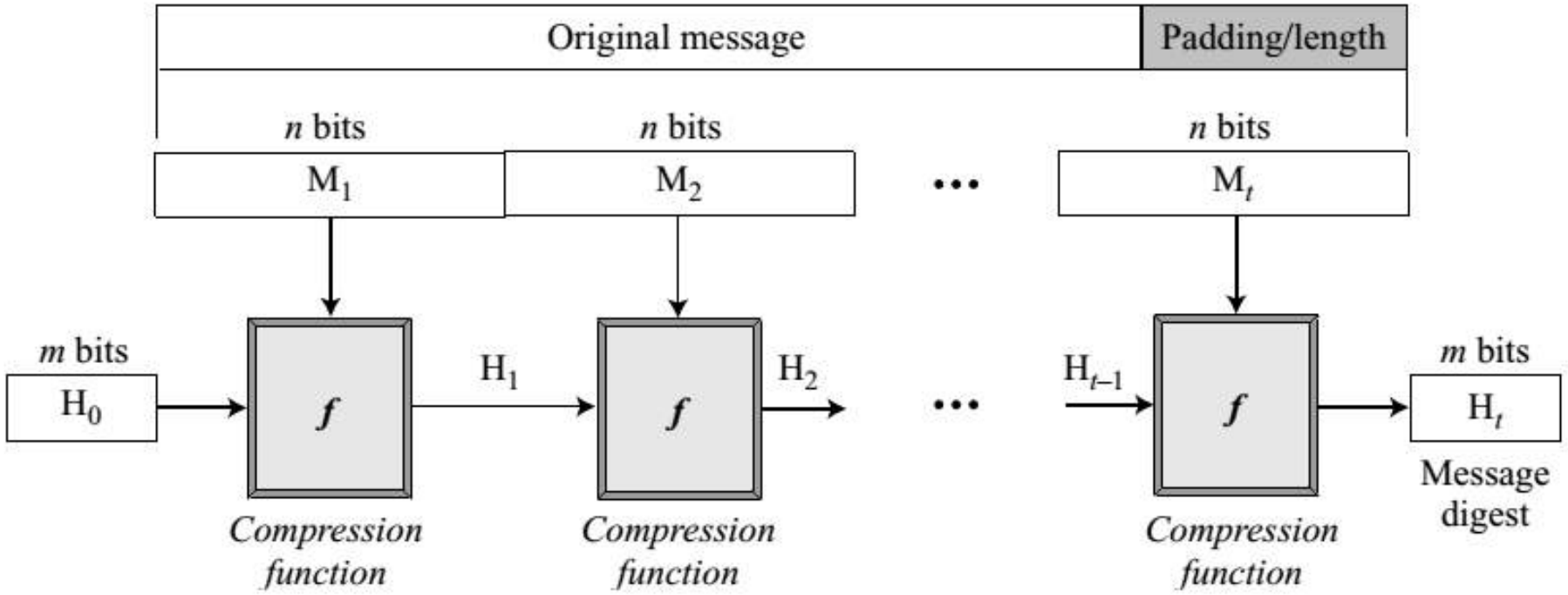
Attack	*Value of k with P=1/2*	*Order*
Preimage	$k \approx 0.69 \times 2^n$	2^n
Second preimage	$k \approx 0.69 \times 2^n + 1$	2^n
Collision	$k \approx 1.18 \times 2^{n/2}$	$2^{n/2}$
Alternate collision	$k \approx 0.83 \times 2^{n/2}$	$2^{n/2}$

Cryptographic Hash Functions

- All cryptographic hash functions need to create a fixed-size digest out of a variable-size message.
- Creating such a function is best accomplished using iteration.
- Instead of using a hash function with variable-size input, a function with fixed-size input is created and is used a necessary number of times.
- The fixed-size input function is referred to as a compression function.
- It compresses an n-bit string to create an m-bit string where n is normally greater than m.
- The scheme is referred to as an iterated cryptographic hash function.

Merkle-Damgard Scheme

- The Merkle-Damgard scheme is an iterated hash function that is collision resistant if the compression function is collision resistant.



Merkle-Damgard Scheme

1. The message length and padding are appended to the message to create an augmented message that can be evenly divided into blocks of n bits, where n is the size of the block to be processed by the compression function.
2. The message is then considered as t blocks, each of n bits. We call each block M_1, M_2,…, M_t. We call the digest created at t iterations H_1, H_2,…, H_t.
3. Before starting the iteration, the digest H_0 is set to a fixed value, normally called IV (initial value or initial vector).
4. The compression function at each iteration operates on H_{i-1} and M_i to create a new H_i. In other words, we have $H_i = f(H_{i-1}, M_i)$, where f is the compression function.
5. H_t is the cryptographic hash function of the original message, that is, h(M).

Two Groups of Compression Functions

- In the first approach, the compression function is made from scratch:
  - Message Digest (MD): Several versions (MD2, MD4, and MD5)
  - The last version, MD5, is a strengthened version of MD4 that divides the message into blocks of 512 bits and creates a 128-bit digest.
  - It turned out that a message digest of size 128 bits is too small to resist collision attack.

- In the second approach, a symmetric-key block cipher serves as a compression function.
  - Secure Hash Algorithm (SHA)
  - Also referred to as Secure Hash Standard (SHS) four new versions: SHA-224, SHA-256, SHA-384, and SHA-512.

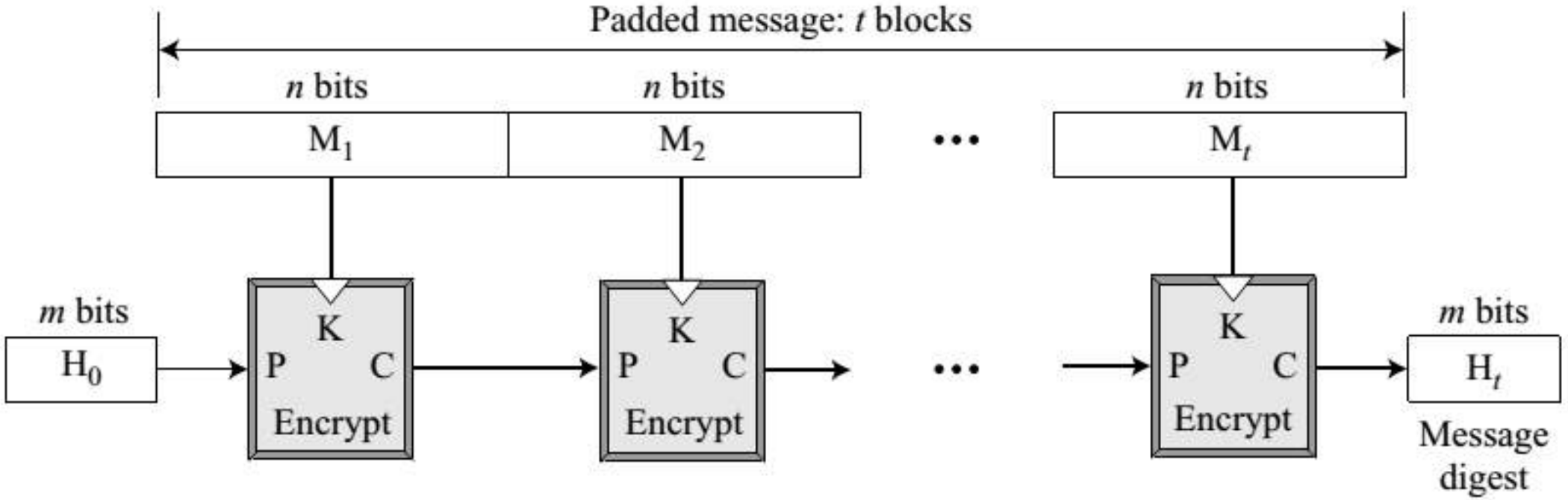
Characteristics of Secure Hash Algorithms (SHAs)

Characteristics	*SHA-1*	*SHA-224*	*SHA-256*	*SHA-384*	*SHA-512*
Maximum Message size	$2^{64} - 1$	$2^{64} - 1$	$2^{64} - 1$	$2^{128} - 1$	$2^{128} - 1$
Block size	512	512	512	1024	1024
Message digest size	160	224	256	384	512
Number of rounds	80	64	64	80	80
Word size	32	32	32	64	64

Rabin Scheme

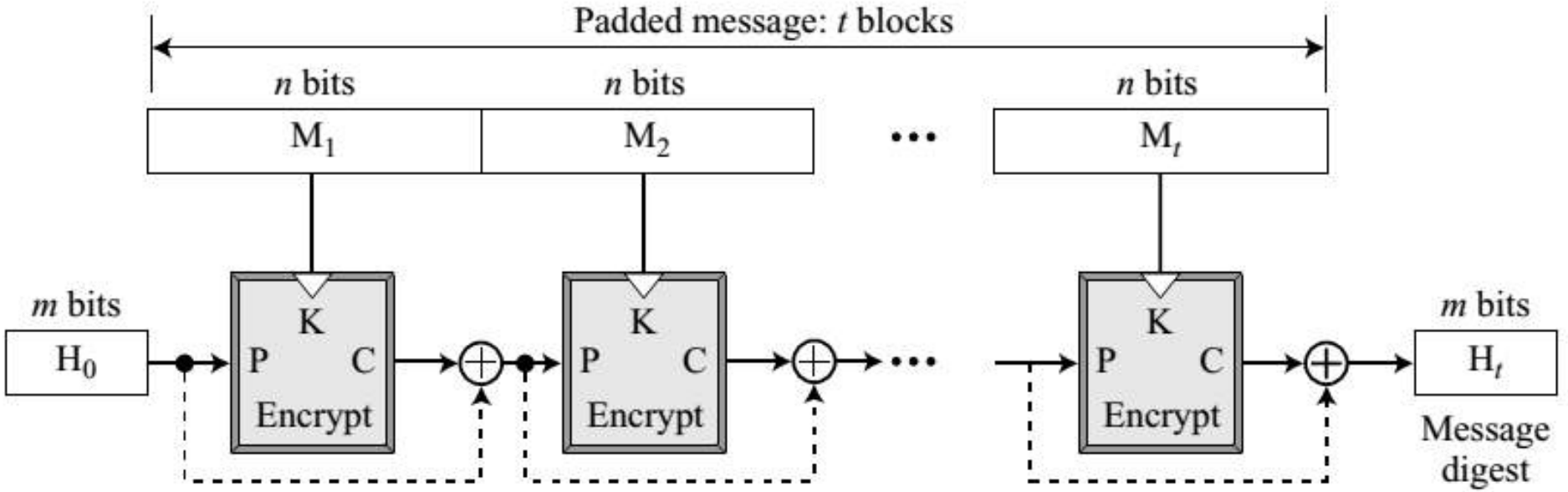
- Rabin scheme is based on the Merkle-Damgard scheme.
- The compression function is replaced by any encrypting cipher.
- The message block is used as the key; the previously created digest is used as the plaintext.
- The ciphertext is the new message digest.
- Note that the size of the digest is the size of data block cipher in the underlying cryptosystem.
- For example, if DES is used as the block cipher, the size of the digest is only 64 bits.
- Although the scheme is very simple, it is subject to a meet-in-the-middle attack

Rabin Scheme



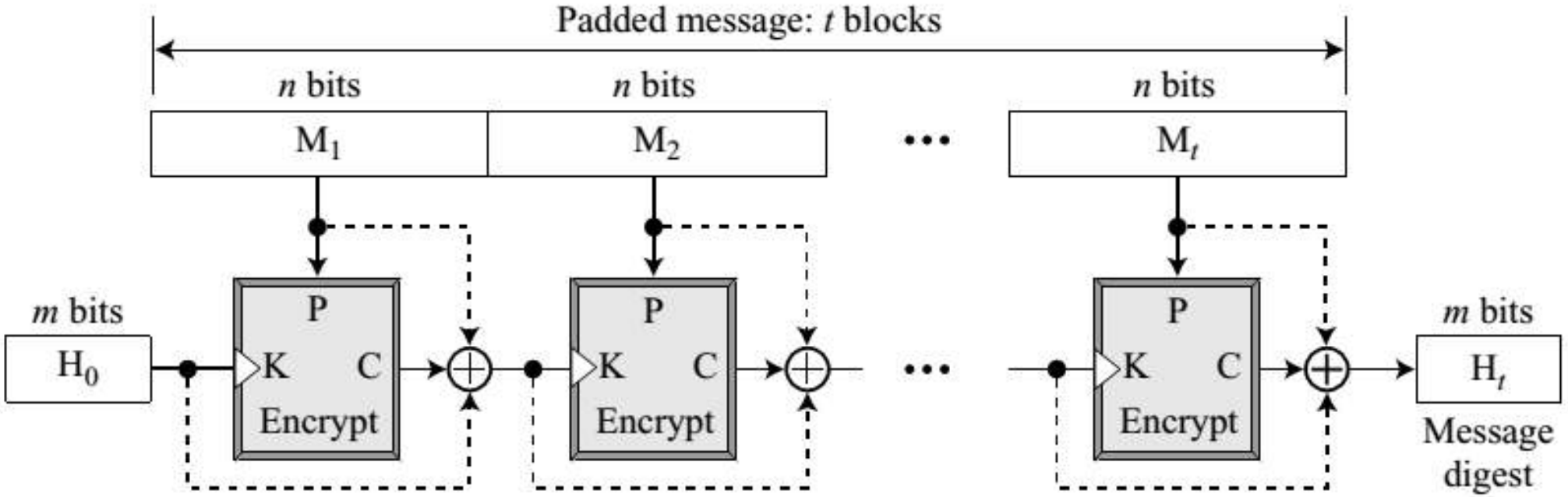
Davies-Meyer Scheme

- The Davies-Meyer scheme is basically the same as the Rabin scheme except that it uses forward feed to protect against meet-in-the-middle attack.



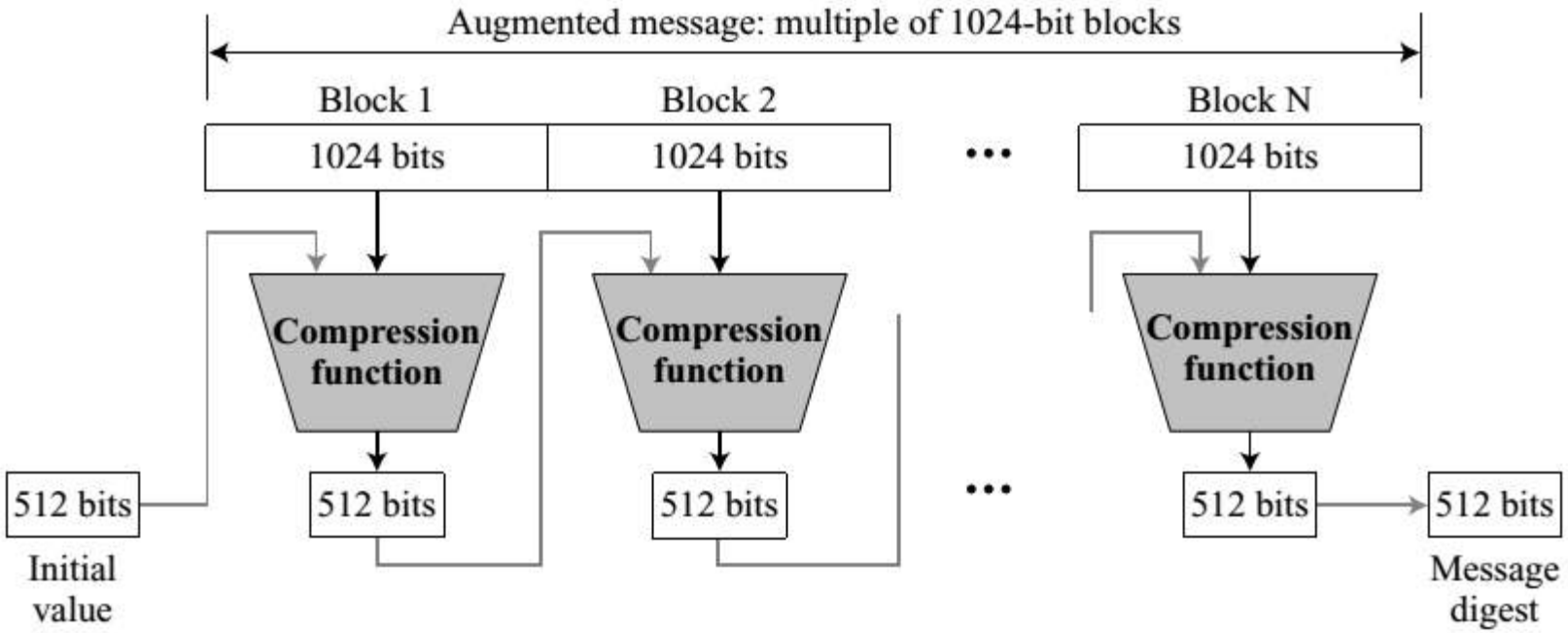
Miyaguchi-Preneel Scheme

- To make the algorithm stronger against attack, the plaintext, the cipher key, and the ciphertext are all exclusive-ored together to create the new digest.



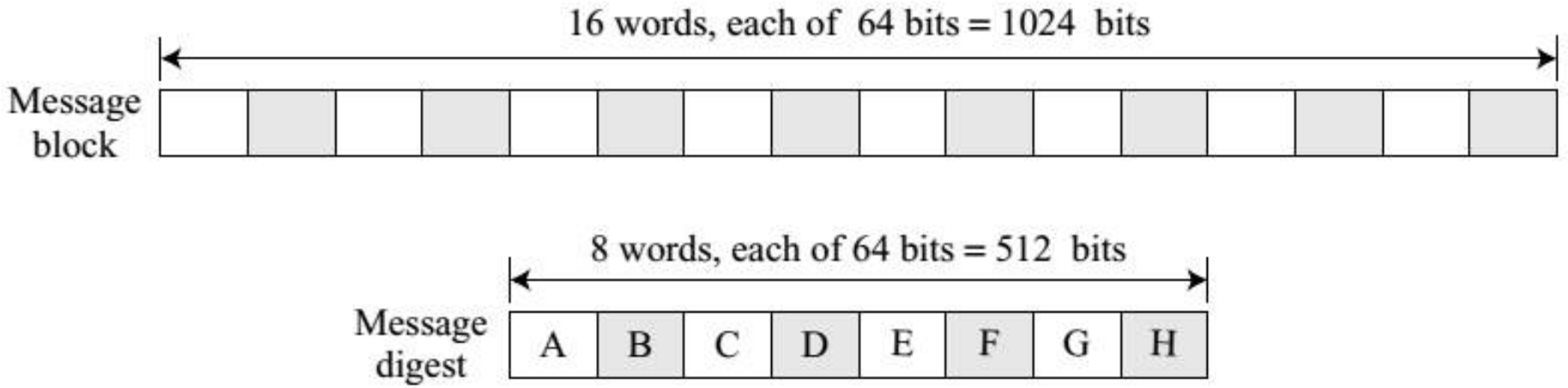
SHA-512

- SHA-512 creates a digest of 512 bits from a multiple-block message.
- Each block is 1024 bits in length.



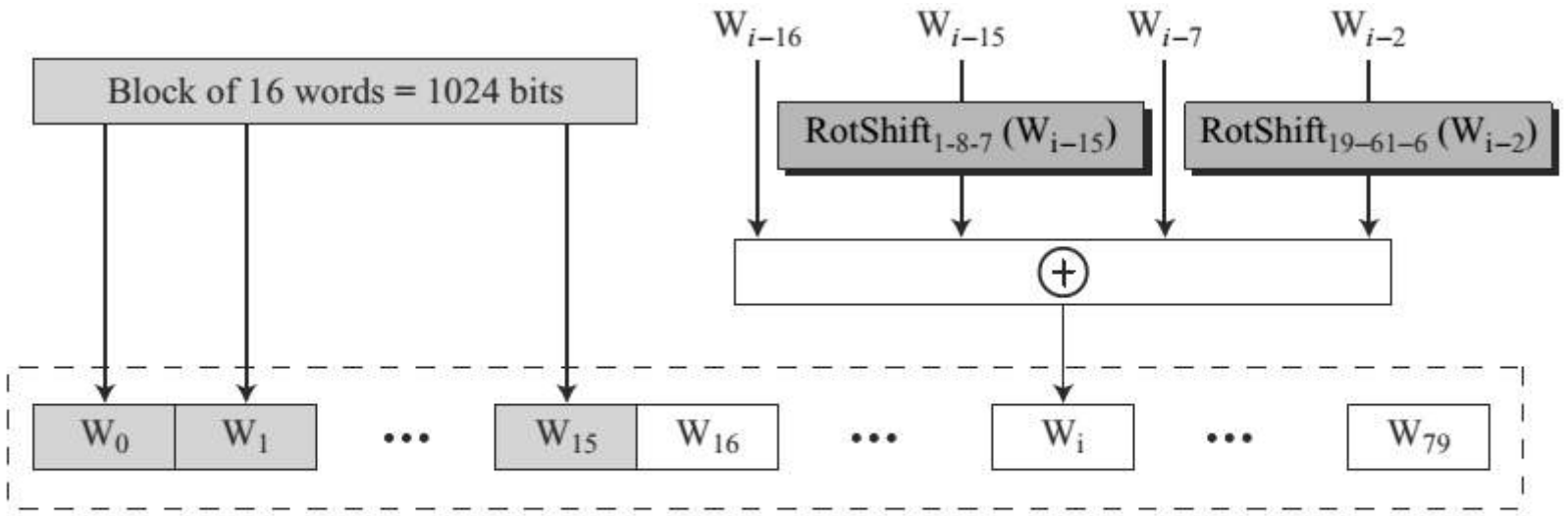
SHA-512

- SHA-512 operates on words; it is word oriented. A word is defined as 64 bits.
- The message digest is also made of 64-bit words, but the message digest is only eight words and the words are named A, B,C, D, E, F, G, and H.



SHA-512: Word Expansion

- A block is made of 1024 bits, or sixteen 64-bit words.
- 16-word block needs to be expanded to 80 words, from W0 to W79



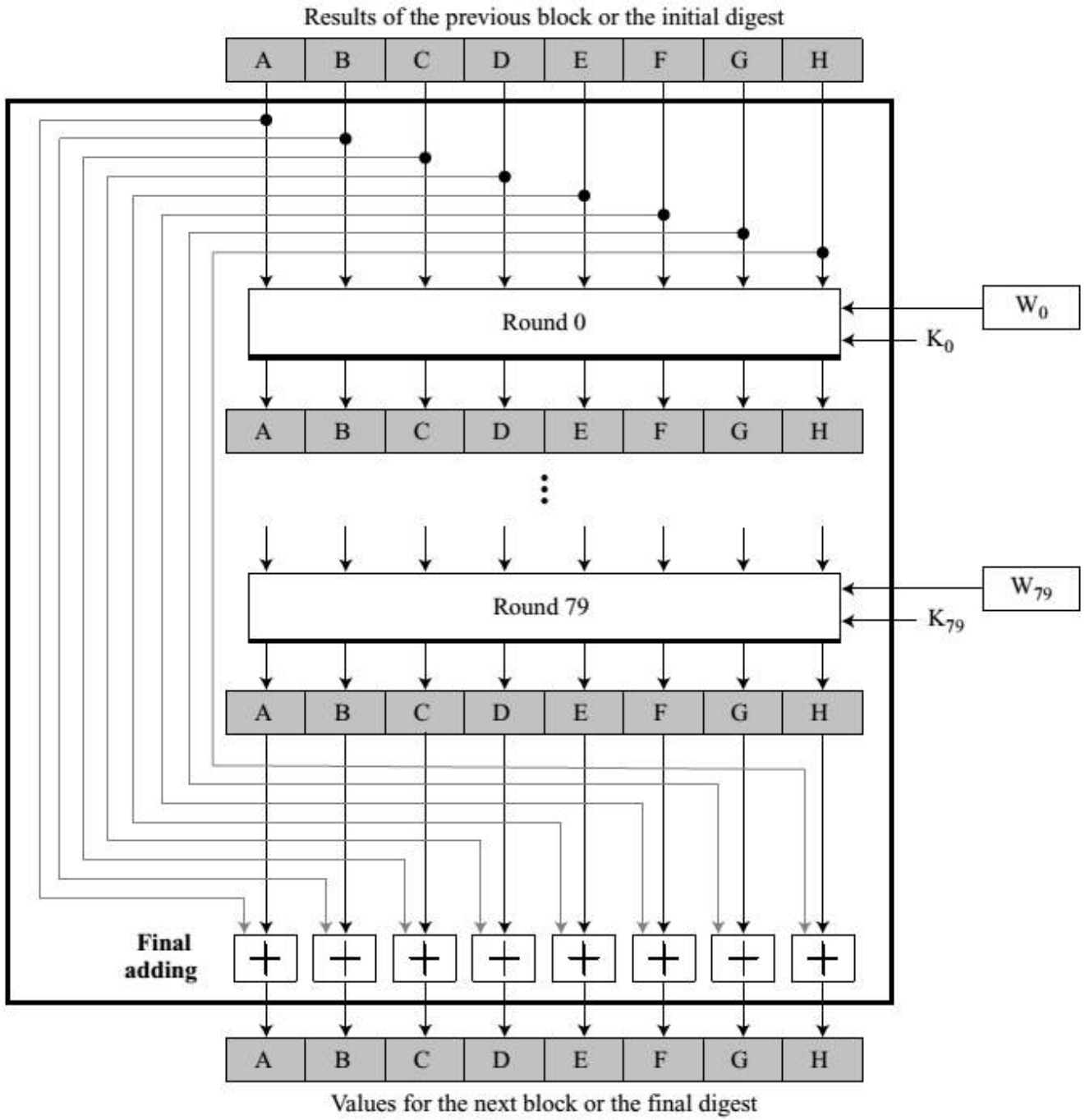
$RotShift_{1\text{-}m\text{-}n}(x)$: $\boldsymbol{RotR_l(x)} \oplus \boldsymbol{RotR_m(x)} \oplus \boldsymbol{ShL_n(x)}$

$\boldsymbol{RotR_i(x)}$: Right-rotation of the argument x by i bits
$\boldsymbol{ShL_i(x)}$: Shift-left of the argument x by i bits and padding the left by 0's.

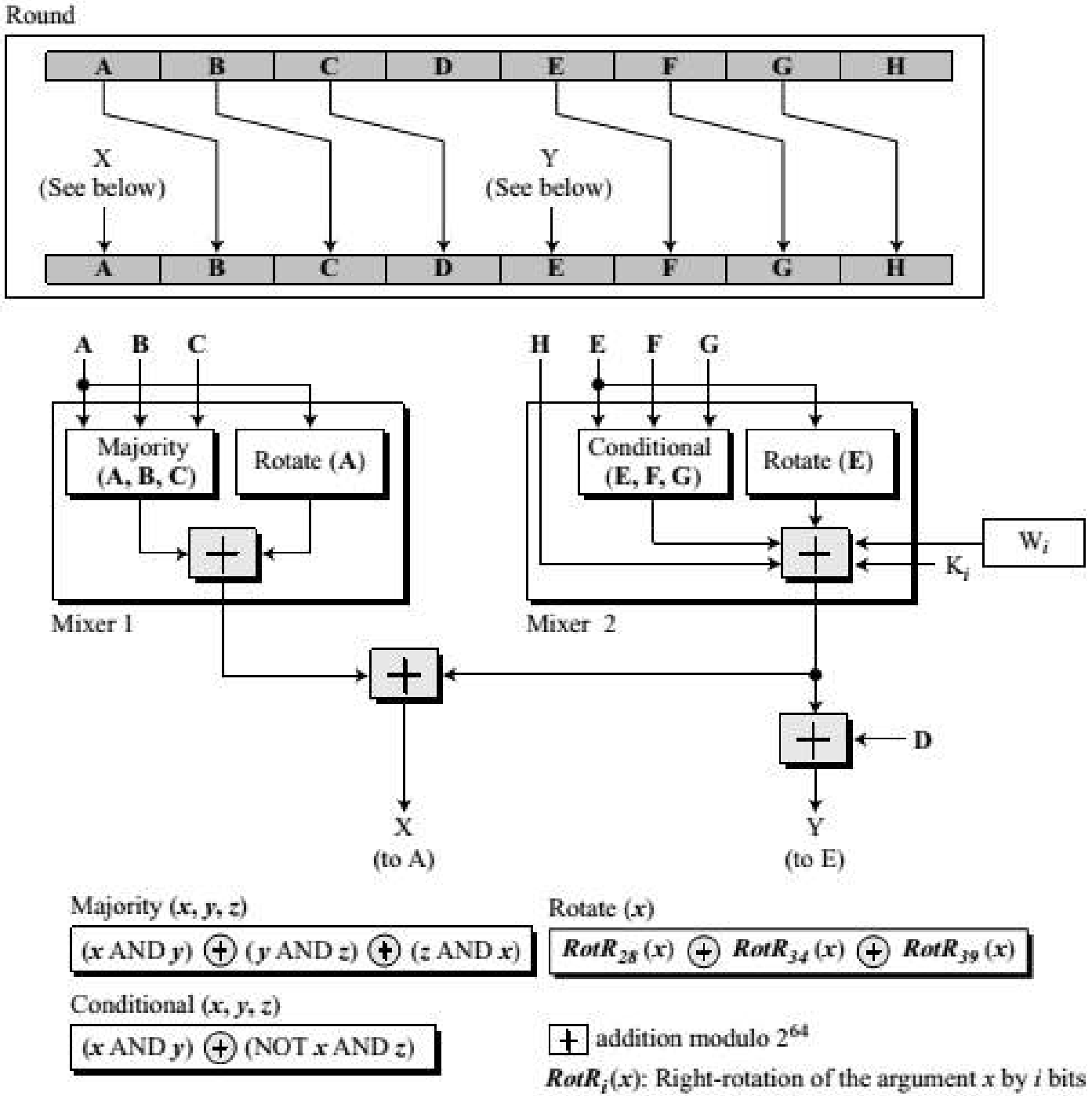
SHA-512: Compression Function

- SHA-512 creates a 512-bit (eight 64-bit words) message digest from a multiple-block message where each block is 1024 bits.
- The processing of each block of data in SHA-512 involves 80 rounds.
- In each round, the contents of eight previous buffers, one word from the expanded block (Wi), and one 64-bit constant (Ki) are mixed together and then operated on to create a new set of eight buffers.
- At the beginning of processing, the values of the eight buffers are saved into eight temporary variables.
- At the end of the processing (after step 79), these values are added to the values created from step 79. We call this last operation the final adding

SHA-512: Compression Function



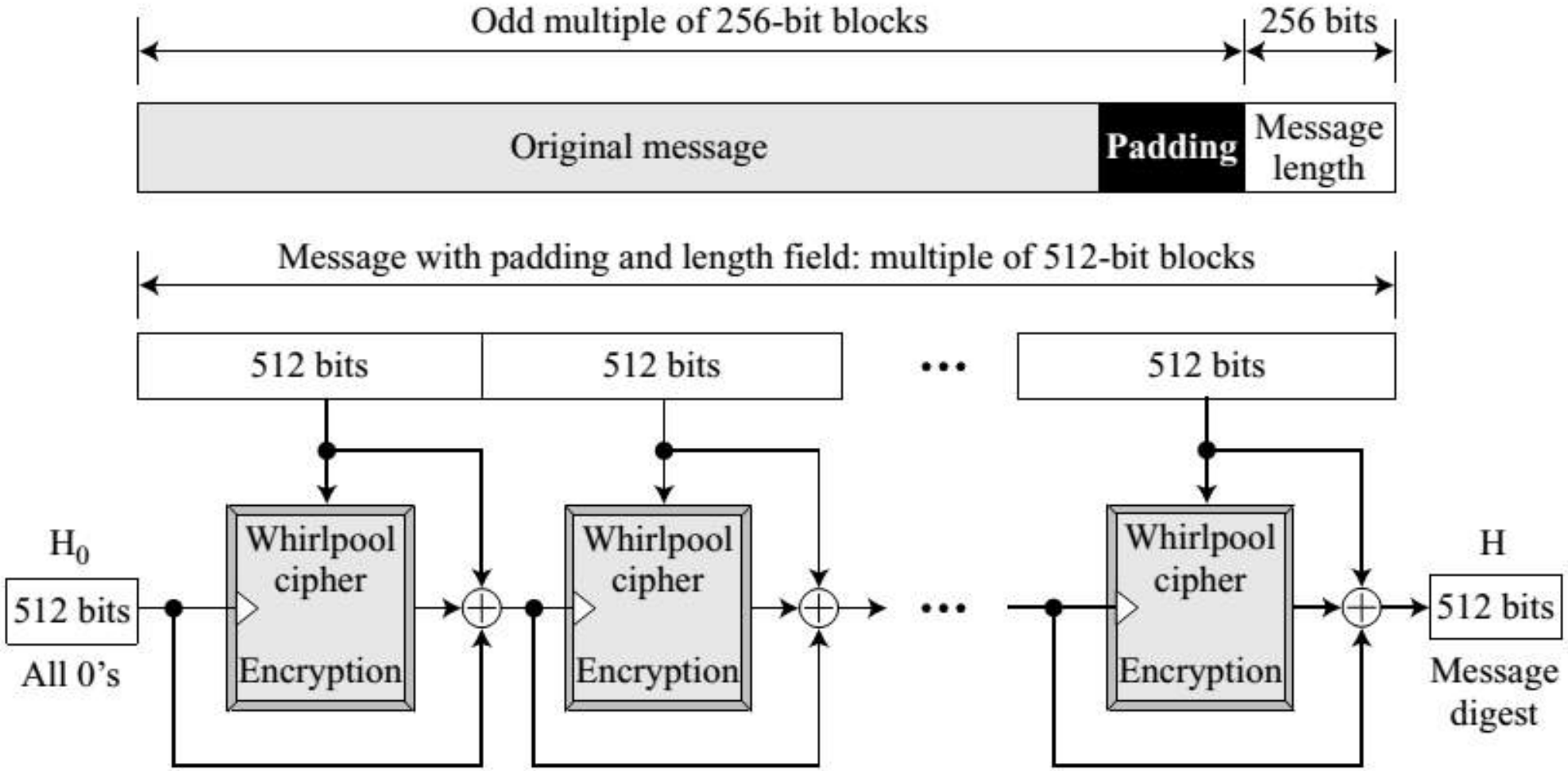
Structure of Each Round



WHIRLPOOL

- Whirlpool is the New European Schemes for Signatures, Integrity, and Encryption (NESSIE).
- Whirlpool is an iterated cryptographic hash function, based on the Miyaguchi-Preneel scheme, that uses a symmetric-key block cipher in place of the compression function.
- The block cipher is a modified AES cipher that has been tailored for this purpose.

WHIRLPOOL


Odd multiple of 256-bit blocks
256 bits
Original message
Padding
Message length
Message with padding and length field: multiple of 512-bit blocks
512 bits
512 bits
512 bits
H_0
512 bits
All 0's
Whirlpool cipher
Encryption
Whirlpool cipher
Encryption
Whirlpool cipher
Encryption
H
512 bits
Message digest

Assignment #3

- Draw a creative diagram/picture of the concepts you learned from assignment 1 and assignment 2.
  - (You can use a poster presentation if you like).
  - It shouldn't have been copied from internet or from any other sources.

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 12 : Message Authentication and Digital Signatures

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

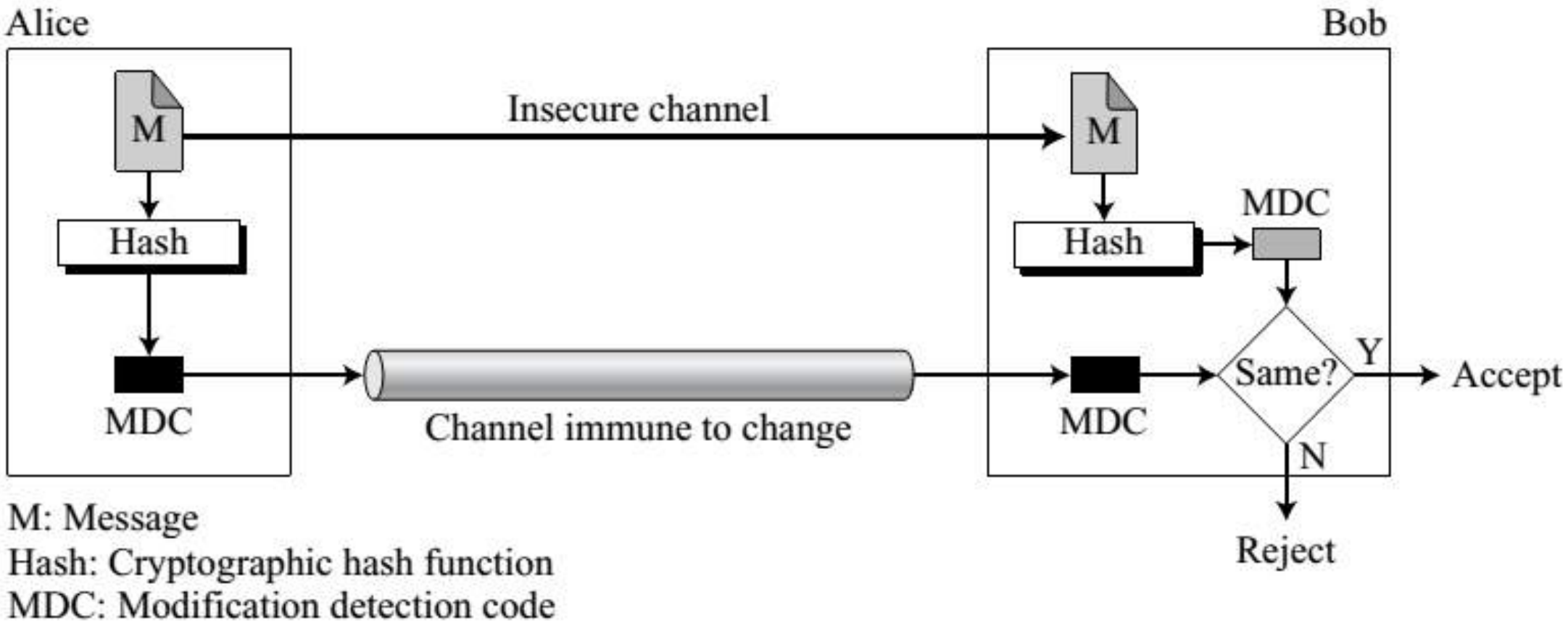
Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

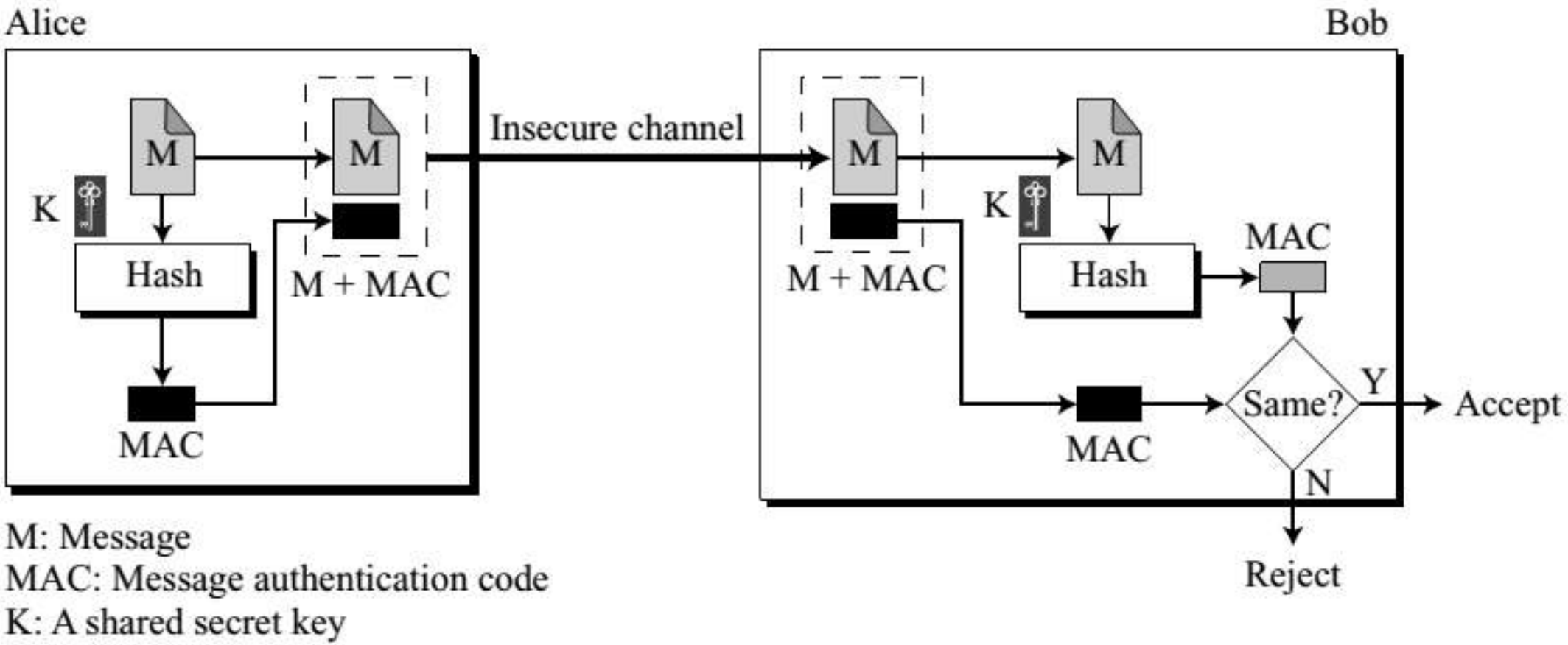
MESSAGE AUTHENTICATION

- A message digest guarantees the integrity of a message.
  - It guarantees that the message has not been changed.
- A message digest, however, does not authenticate the sender of the message.
  - When Alice sends a message to Bob, Bob needs to know if the message is coming from Alice.
- To provide message authentication, Alice needs to provide proof that it is Alice sending the message and not an impostor.
- A message digest per se cannot provide such a proof.
- The digest created by a cryptographic hash function is normally called a modification detection code (MDC).
  - The code can detect any modification in the message.
- What we need for message authentication (data origin authentication) is a message authentication code (MAC).

Modification detection code (MDC)



Message Authentication Code (MAC)



Message Authentication Code (MAC)

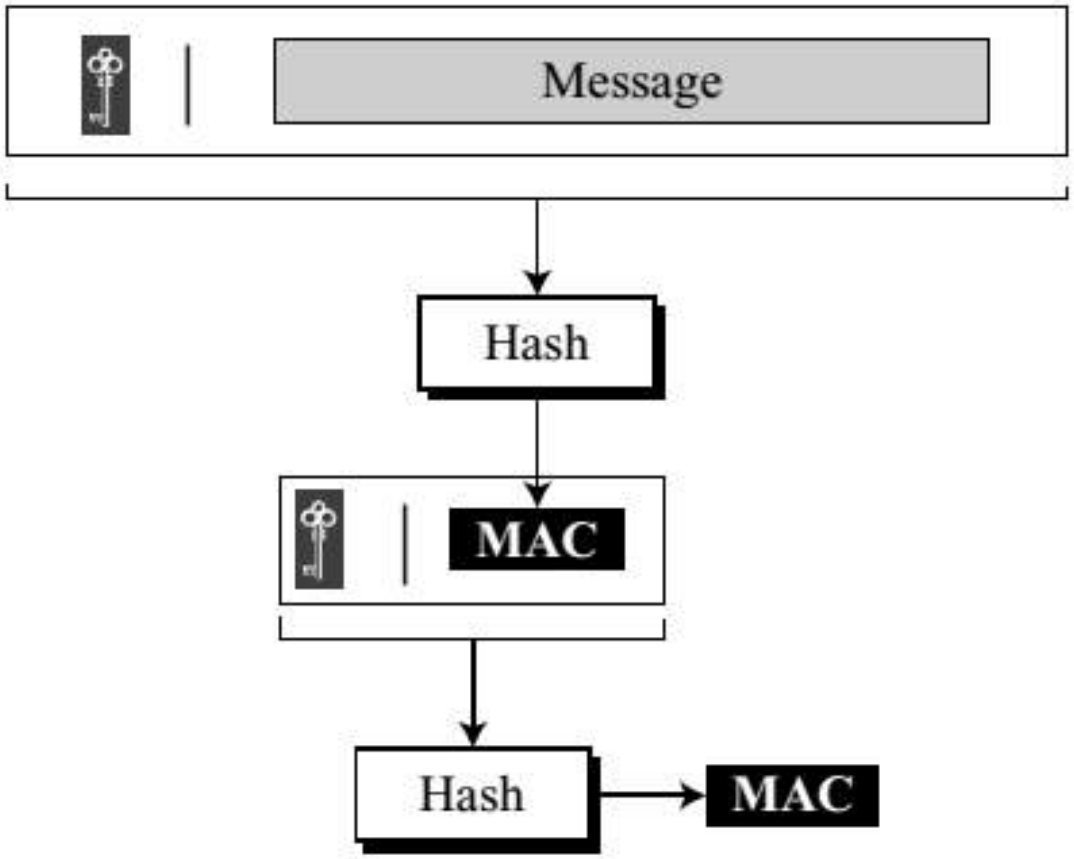
- Alice uses a hash function to create a MAC from the concatenation of the key and the message, h (K|M).
- She sends the message and the MAC to Bob over the insecure channel.
- Bob separates the message from the MAC.
- He then makes a new MAC from the concatenation of the message and the secret key.
- Bob then compares the newly created MAC with the one received.
- If the two MACs match, the message is authentic and has not been modified by an adversary.
- The MAC we have described is referred to as a prefix MAC because the secret key is appended to the beginning of the message.
- We can have a postfix MAC, in which the key is appended to the end of the message.
- We can combine the prefix and postfix MAC, with the same key or two different keys.

Security of a MAC

- Suppose Eve has intercepted the message M and the digest h(K| M). How can Eve forge a message without knowing the secret key? There are three possible cases:
- If the size of the key allows exhaustive search, Eve may prepend all possible keys at the beginning of the message and make a digest of the (K| M) to find the digest equal to the one intercepted. She then knows the key and can successfully replace the message with a forged message of her choosing.
- The size of the key is normally very large in a MAC, but Eve can use another tool: the preimage attack. She uses the algorithm until she finds X such that h(X) is equal to the MAC she has intercepted. She now can find the key and successfully replace the message with a forged one. Because the size of the key is normally very large for exhaustive search, Eve can only attack the MAC using the preimage algorithm.
- Given some pairs of messages and their MACs, Eve can manipulate them to come up with a new message and its MAC.

The security of a MAC depends on the security of the underlying hash algorithm

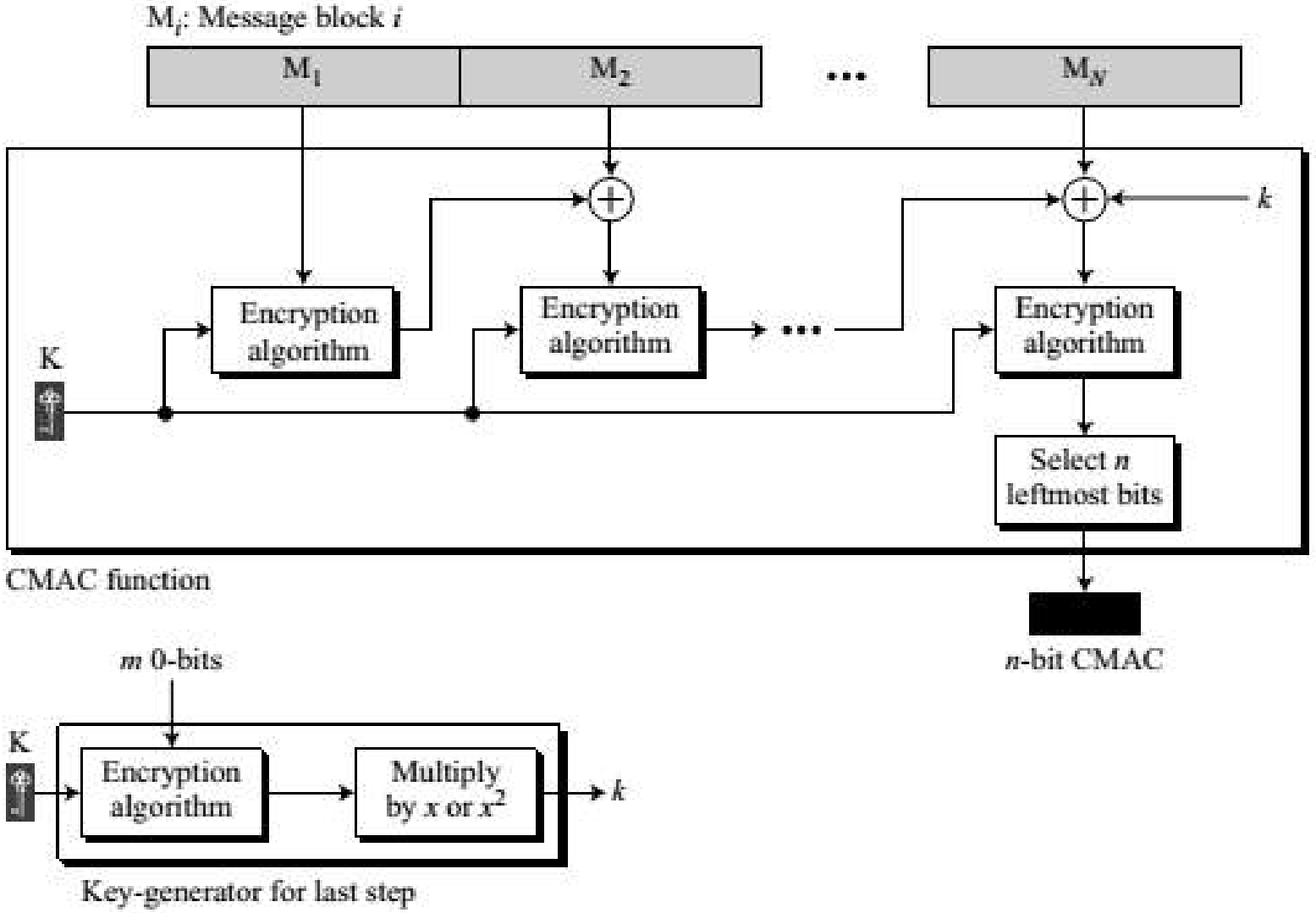
Nested MAC (OR) HMAC (Hashed MAC)



CMAC, or CBCMAC

CMAC, or CBCMAC is also called Data Authentication Algorithm.

The method is similar to the cipher block chaining (CBC) mode



Digital Signature

Digital Signature

- A person signs a document to show that it originated from her or was approved by her.
- The signature is proof to the recipient that the document comes from the correct entity.
- When Alice sends a message to Bob, Bob needs to check the authenticity of the sender; he needs to be sure that the message comes from Alice and not Eve.
- Bob can ask Alice to sign the message electronically.
- In other words, an electronic signature can prove the authenticity of Alice as the sender of the message.
- We refer to this type of signature as a digital signature.

Digital Signature: Inclusion

- A conventional signature is included in the document; it is part of the document.
- When we write a check, the signature is on the check; it is not a separate document.
- But when we sign a document digitally, we send the signature as a separate document.
- The sender sends two documents: the message and the signature.
- The recipient receives both documents and verifies that the signature belongs to the supposed sender.
- If this is proven, the message is kept; otherwise, it is rejected.

Digital Signature: Verification Method

- For a conventional signature, when the recipient receives a document, she compares the signature on the document with the signature on file.
- If they are the same, the document is authentic.
- The recipient needs to have a copy of this signature on file for comparison.
- For a digital signature, the recipient receives the message and the signature.
- A copy of the signature is not stored anywhere.
- The recipient needs to apply a verification technique to the combination of the message and the signature to verify the authenticity.

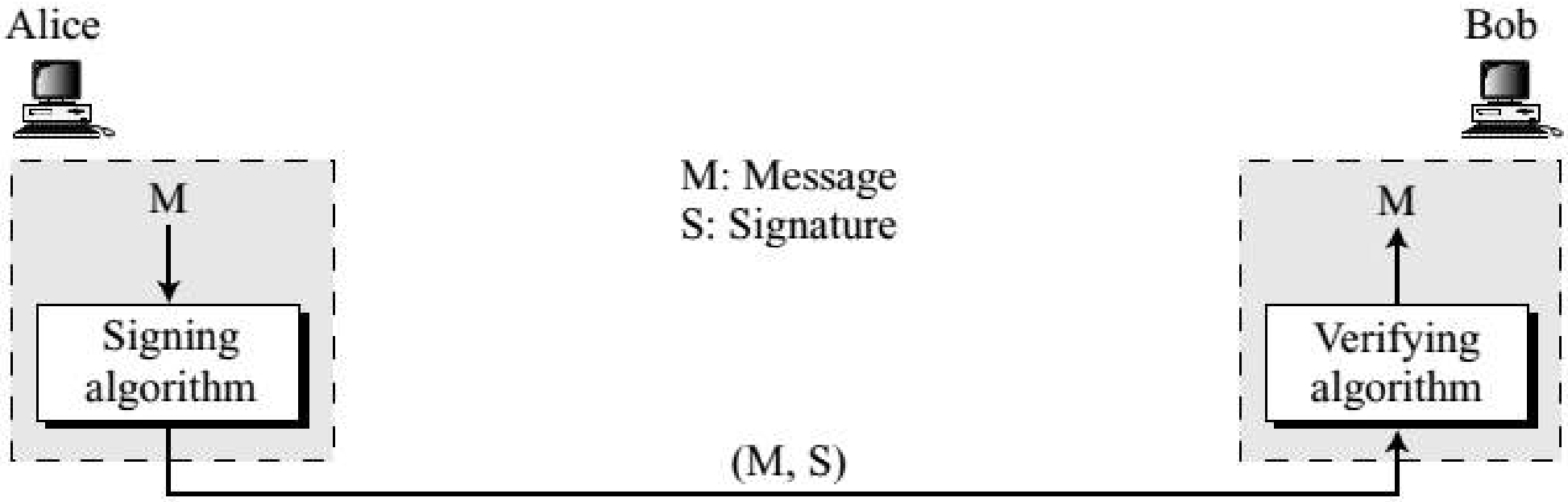
Digital Signature: Relationship

- For a conventional signature, there is normally a one-to-many relationship between a signature and documents.
- A person uses the same signature to sign many documents.
- For a digital signature, there is a one-to-one relationship between a signature and a message.
- Each message has its own signature. The signature of one message cannot be used in another message.
- If Bob receives two messages, one after another, from Alice, he cannot use the signature of the first message to verify the second.
- Each message needs a new signature.

Digital Signature: Duplicity

- In conventional signature, a copy of the signed document can be distinguished from the original one on file.
- In digital signature, there is no such distinction unless there is a factor of time (such as a timestamp) on the document.
- For example,
  - Suppose Alice sends a document instructing Bob to pay Eve.
  - If Eve intercepts the document and the signature, she can replay it later to get money again from Bob.

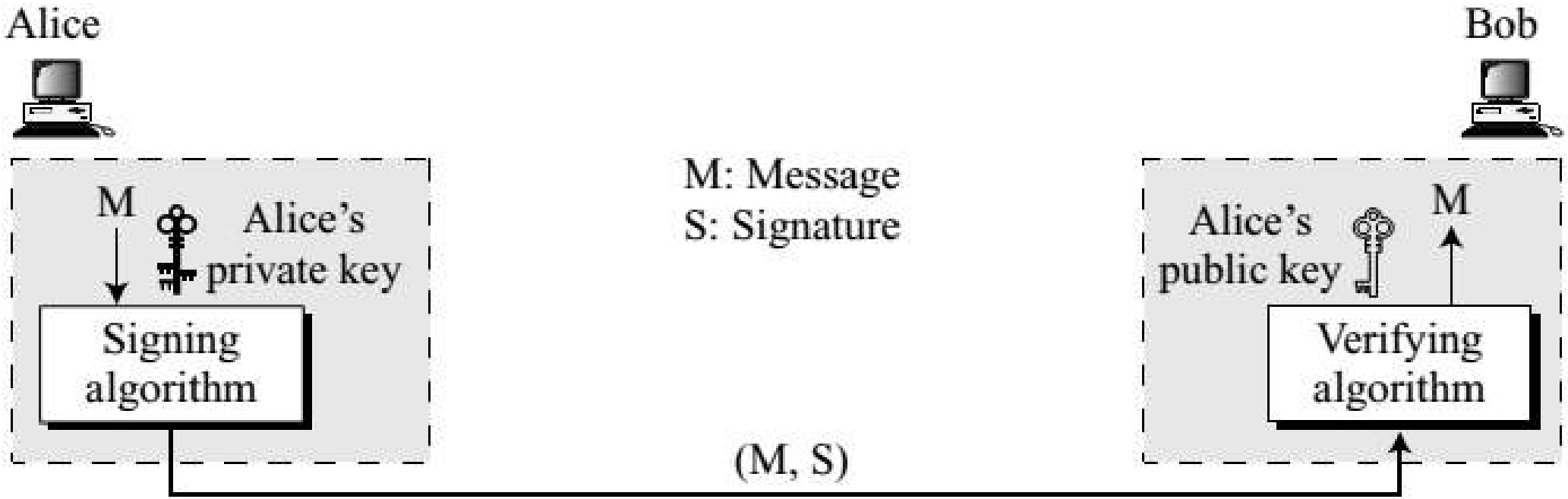
Digital Signature Process



Need for Keys

- A conventional signature is like a private “key” belonging to the signer of the document. The signer uses it to sign documents; no one else has this signature.
- The copy of the signature is on file like a public key; anyone can use it to verify a document, to compare it to the original signature.
- In a digital signature, the signer uses her private key, applied to a signing algorithm, to sign the document.
- The verifier, on the other hand, uses the public key of the signer, applied to the verifying algorithm, to verify the document.
- Note that when a document is signed, anyone, including Bob, can verify it because everyone has access to Alice’s public key.
- Alice must not use her public key to sign the document because then anyone could forge her signature.

Adding key to the digital signature process



Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 13 : Digital Signatures Schemes

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

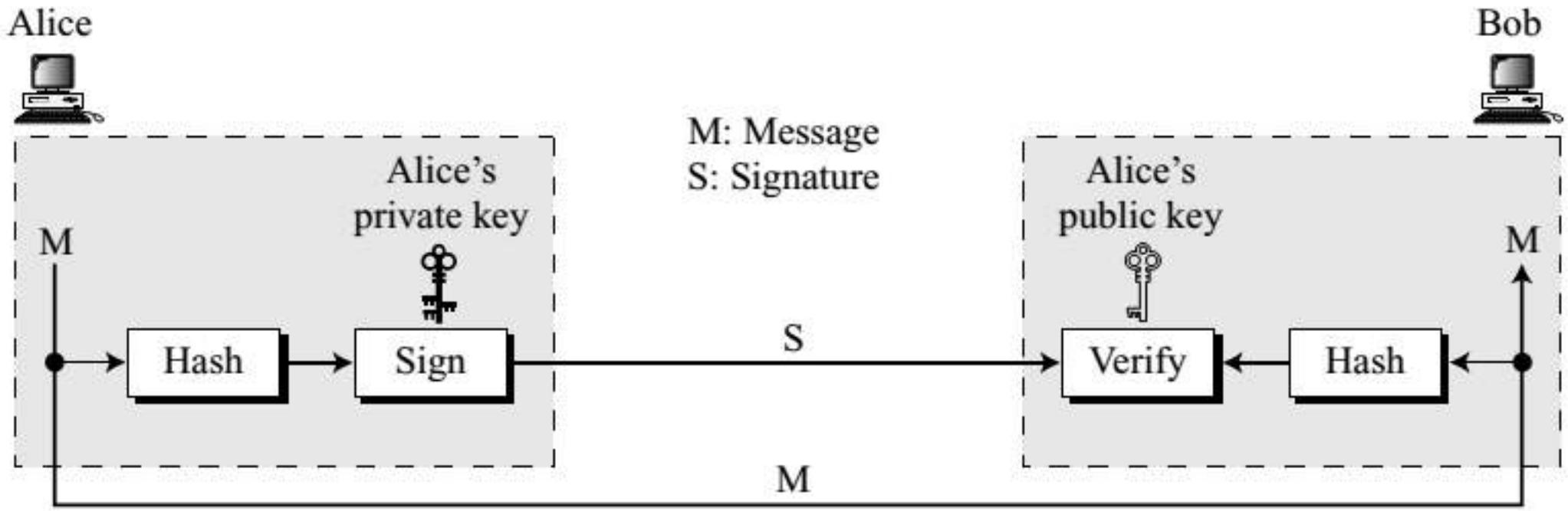
Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

Digital Signature

- A digital signature needs a public-key system. The signer signs with her private key; the verifier verifies with the signer’s public key.

- A cryptosystem uses the private and public keys of the receiver: a digital signature uses the private and public keys of the sender.

Signing the Digest



Digital Signature SERVICES

- Several security services were discussed including
  - message confidentiality,
  - message authentication,
  - message integrity, and
  - nonrepudiation.

- A digital signature can directly provide the last three;

- For message confidentiality we still need encryption/decryption.

Digital Signature : Message Authentication

- A secure digital signature scheme, like a secure conventional signature (one that cannot be easily copied) can provide message authentication (also referred to as data-origin authentication).
- Bob can verify that the message is sent by Alice because Alice’s public key is used in verification.
- Alice’s public key cannot verify the signature signed by Eve’s private key.
- A digital signature provides message authentication

Digital Signature : Message Integrity

- The integrity of the message is preserved even if we sign the whole message because we cannot get the same signature if the message is changed.

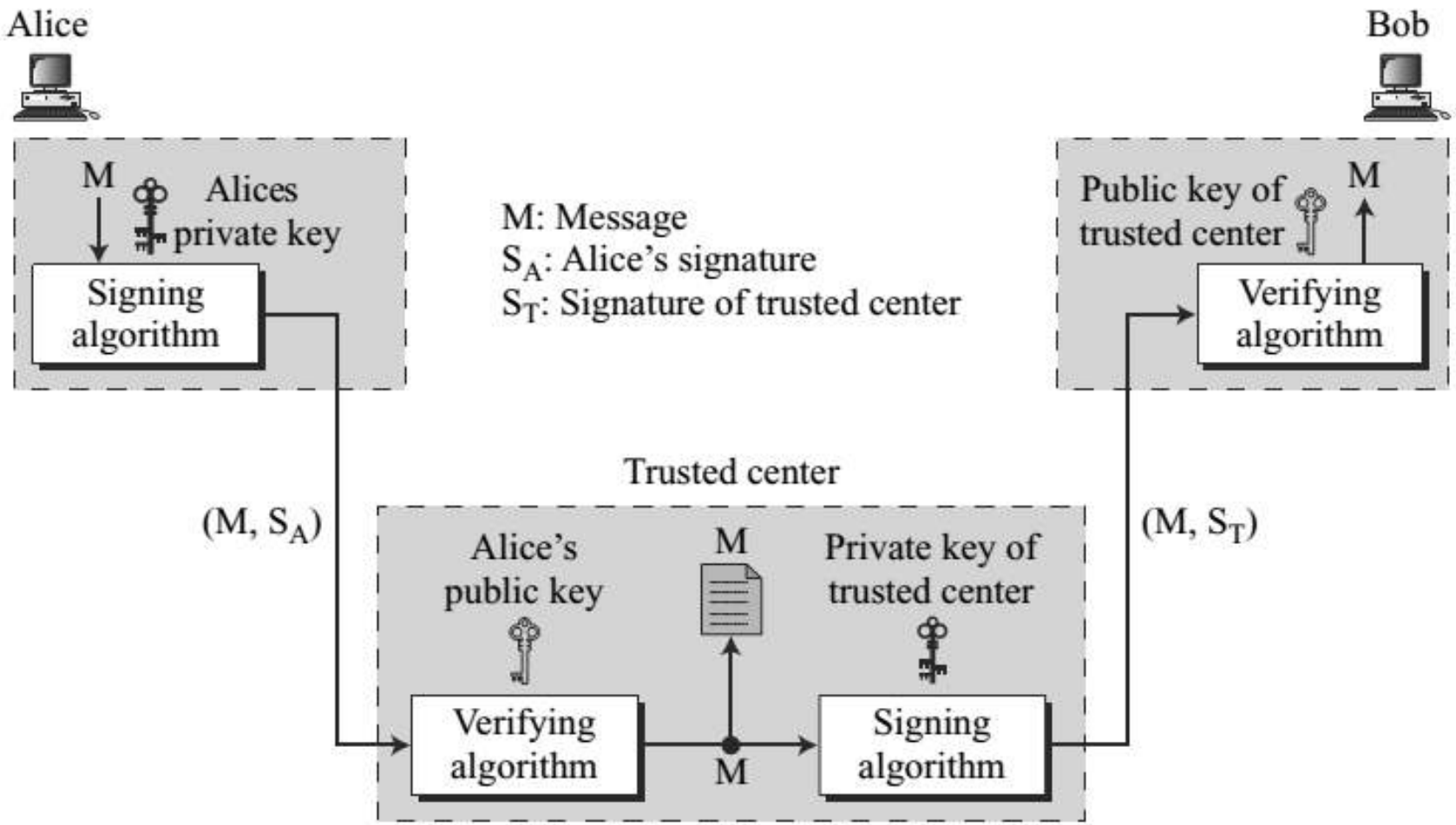
- The digital signature schemes today use a hash function in the signing and verifying algorithms that preserve the integrity of the message.

- A digital signature provides message integrity

Digital Signature : Nonrepudiation

- If Alice signs a message and then denies it, can Bob later prove that Alice actually signed it?
- For example, if Alice sends a message to a bank (Bob) and asks to transfer $10,000 from her account to Ted's account, can Alice later deny that she sent this message?
- With the scheme we have presented so far, Bob might have a problem.
- Bob must keep the signature on file and later use Alice's public key to create the original message to prove the message in the file and the newly created message are the same.
- This is not feasible because Alice may have changed her private or public key during this time; she may also claim that the file containing the signature is not authentic.
- One solution is a trusted third party.

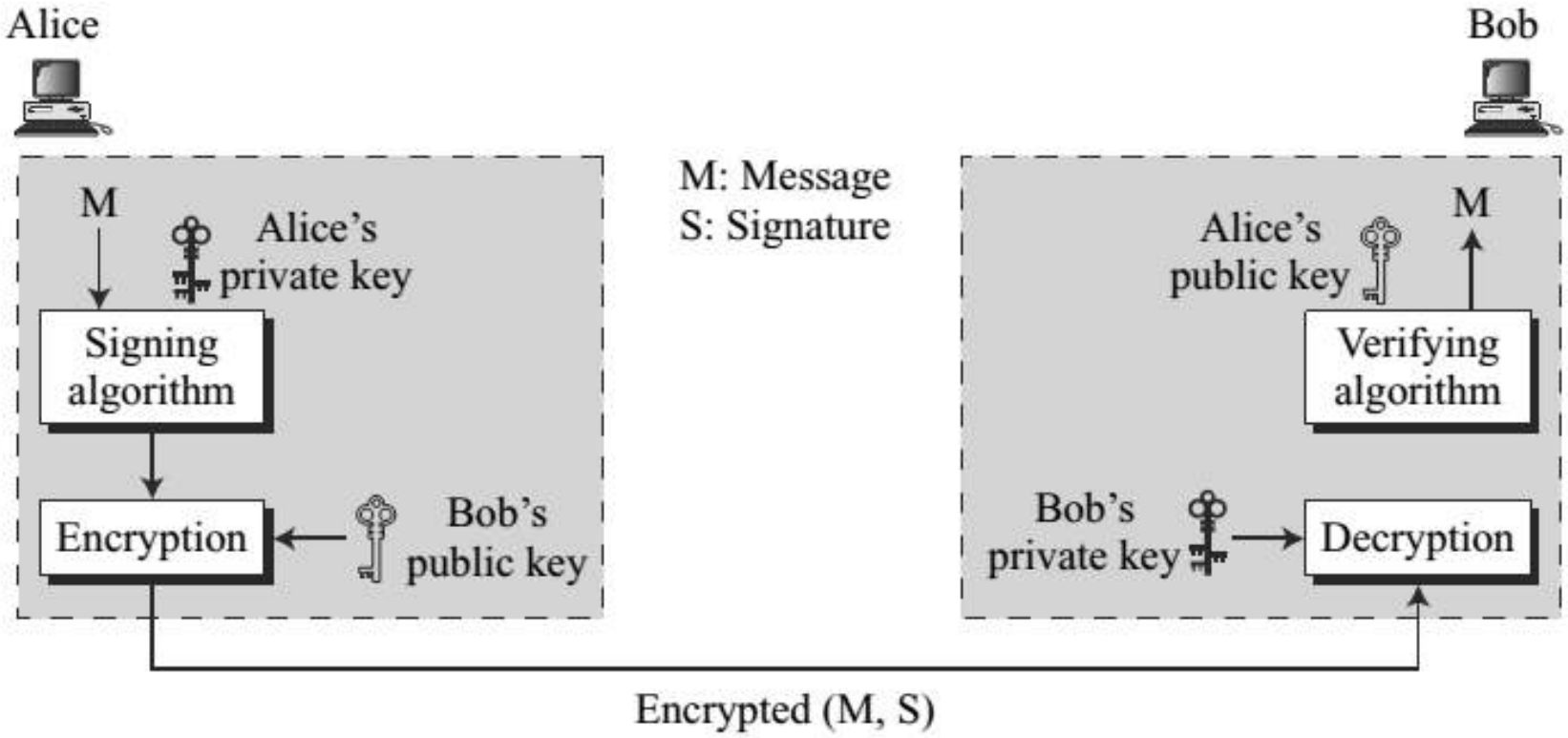
Digital Signature : Nonrepudiation



Digital Signature : Nonrepudiation

- Alice creates a signature from her message (S_A) and sends the message, her identity, Bob's identity, and the signature to the center.
- The center, after checking that Alice's public key is valid, verifies through Alice's public key that the message came from Alice.
- The center then saves a copy of the message with the sender identity, recipient identity, and a timestamp in its archive.
- The center uses its private key to create another signature (S_T) from the message.
- The center then sends the message, the new signature, Alice's identity, and Bob's identity to Bob.
- Bob verifies the message using the public key of the trusted center.
- **Nonrepudiation can be provided using a trusted party.**

Digital Signature : Confidentiality



A digital signature does not provide confidential communication.

If confidentiality is required, the message and the signature must be encrypted using either a secret-key or public-key cryptosystem.

A digital signature does not provide privacy.
If there is a need for privacy, another layer of encryption/ decryption must be applied.

ATTACKS ON DIGITAL SIGNATURE

- Key-Only Attack
  - In the key-only attack, Eve has access only to the public information released by Alice.
  - To forge a message, Eve needs to create Alice's signature to convince Bob that the message is coming from Alice.
  - This is the same as the ciphertext-only attack we discussed for encipherment.
- Known-Message Attack
  - In the known-message attack, Eve has access to one or more message-signature pairs.
  - In other words, she has access to some documents previously signed by Alice.
  - Eve tries to create another message and forge Alice's signature on it.
  - This is similar to the known-plaintext attack we discussed for encipherment.

ATTACKS ON DIGITAL SIGNATURE

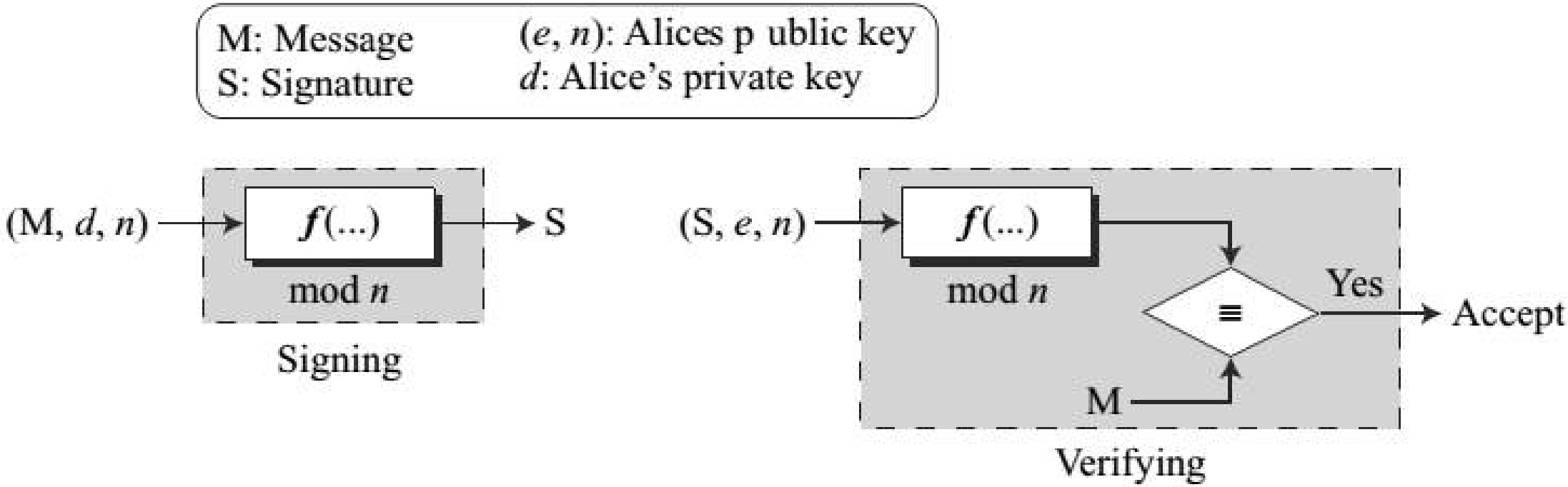
- Chosen-Message Attack
  - In the chosen-message attack, Eve somehow makes Alice sign one or more messages for her.
  - Eve now has a chosen-message/signature pair.
  - Eve later creates another message, with the content she wants, and forges Alice's signature on it.
  - This is similar to the chosen-plaintext attack we discussed for encipherment.
- If the attack is successful, the result is a forgery.
- We can have two types of forgery:
  - existential and
  - selective.

Forgery Types

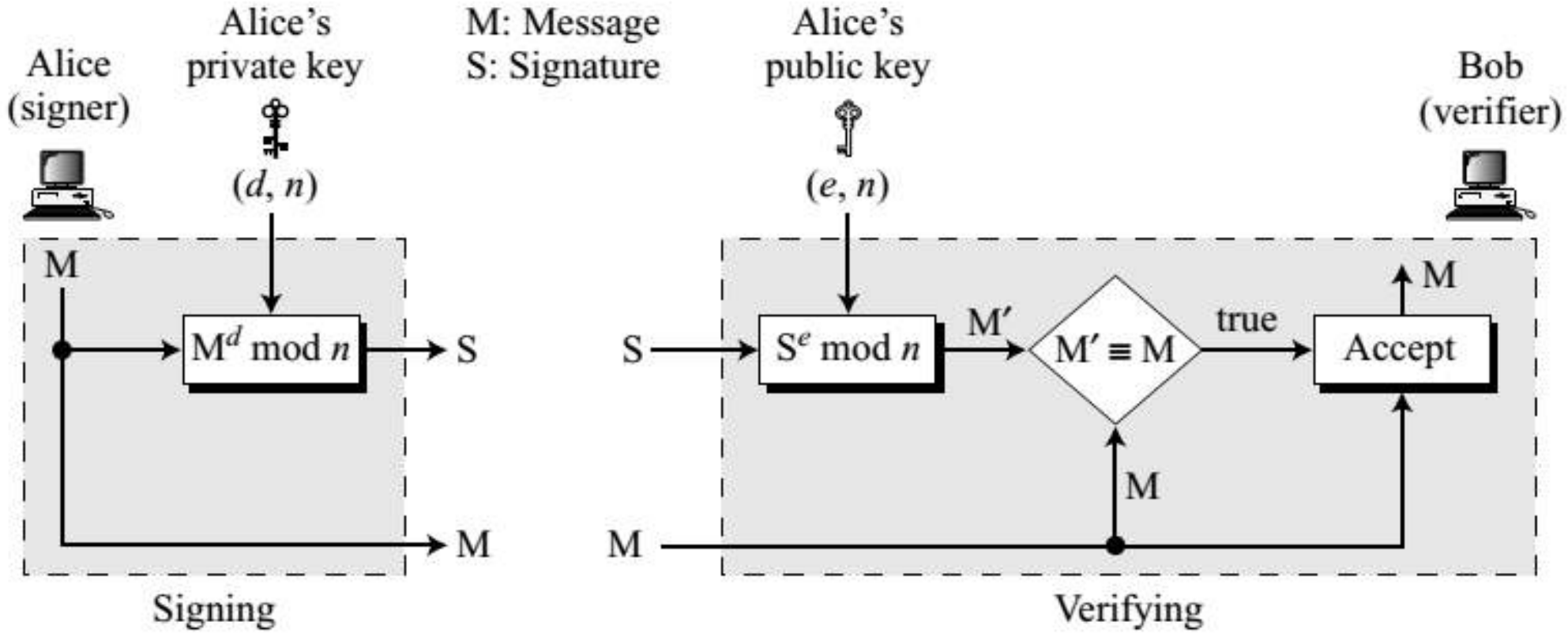
- Existential Forgery
  - In an existential forgery, Eve may be able to create a valid message-signature pair, but not one that she can really use.
  - In other words, a document has been forged, but the content is randomly calculated.
  - This type of forgery is probable, but fortunately Eve cannot benefit from it very much.
  - Her message could be syntactically or semantically unintelligible.
- Selective Forgery
  - In selective forgery, Eve may be able to forge Alice's signature on a message with the content selectively chosen by Eve.
  - Although this is beneficial to Eve, and may be very detrimental to Alice, the probability of such forgery is low, but not negligible.

DIGITAL SIGNATURE SCHEMES

RSA Digital Signature Scheme



RSA Digital Signature Scheme



$$M' \equiv M \pmod{n} \quad \rightarrow \quad S^e \equiv M \pmod{n} \quad \rightarrow \quad M^{d \times e} \equiv M \pmod{n}$$

RSA Digital Signature Scheme

- For the security of the signature, the value of p and q must be very large.
- As a trivial example, suppose that Alice chooses p = 823 and q = 953, and calculates n = 784319.
- The value of $\phi(n)$ is 782544. Now she chooses e = 313 and calculates d = 160009.
- At this point key generation is complete. Now imagine that Alice wants to send a message with the value of M = 19070 to Bob.
- She uses her private exponent, 160009, to sign the message

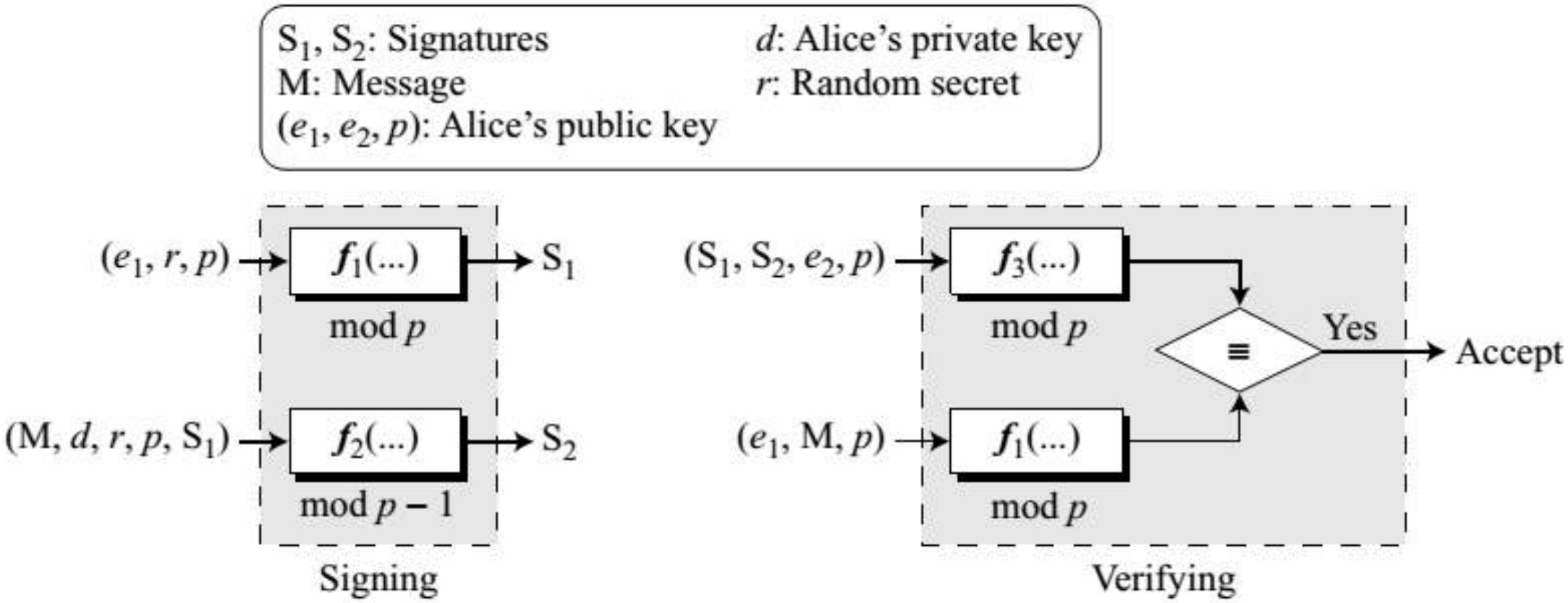
$$\text{M: } 19070 \quad \rightarrow \quad \text{S} = (19070^{160009}) \bmod 784319 = 210625 \bmod 784319$$

- Alice sends the message and the signature to Bob. Bob receives the message and the signature. He calculates

$$\text{M}' = 210625^{313} \bmod 784319 = 19070 \bmod 784319 \quad \rightarrow \quad \text{M} \equiv \text{M}' \bmod n$$

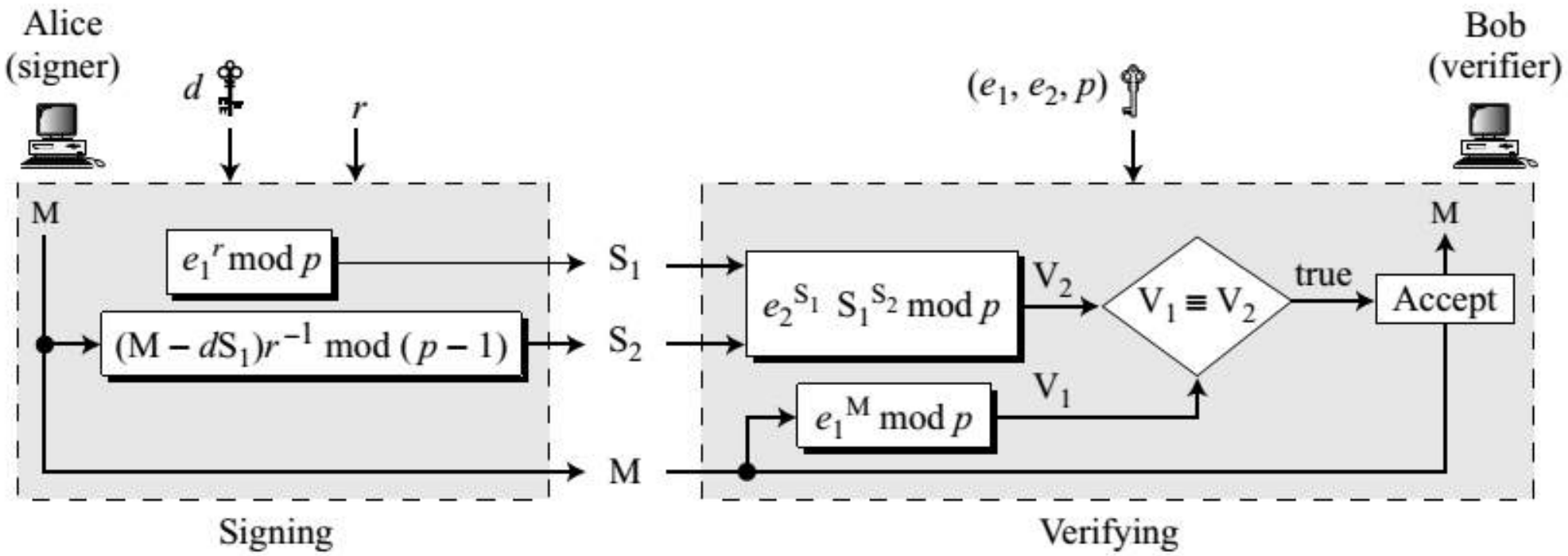
- Bob accepts the message because he has verified Alice's signature.

ElGamal Digital Signature Scheme



ElGamal Digital Signature Scheme

M: Message
S_1, S_2: Signatures
V_1, V_2: Verifications
r: Random secret
d: Alice's private key
(e_1, e_2, p): Alice's public key



ElGamal Digital Signature Scheme

$$V_1 \equiv V_2 \pmod{p} \rightarrow {e_1}^{M} \equiv {e_2}^{S_1} \times {S_1}^{S_2} \pmod{p} \equiv ({e_1}^{d})^{S_1} ({e_1}^{r})^{S_2} \pmod{p} \equiv {e_1}^{d S_1 + r S_2} \pmod{p}$$

We get: ${e_1}^{M} \equiv {e_1}^{d S_1 + r S_2} \pmod{p}$

ElGamal Digital Signature Scheme

- Here is a trivial example. Alice chooses p = 3119, e1 = 2, d = 127 and calculates e2 = 2^{127} mod 3119 = 1702. She also chooses r to be 307. She announces e1, e2, and p publicly; she keeps d secret. The following shows how Alice can sign a message.

$$M = 320$$

$$S_1 = e_1{}^r = 2^{307} = 2083 \bmod 3119$$

$$S_2 = (M - d \times S_1) \times r^{-1} = (320 - 127 \times 2083) \times 307^{-1} = 2105 \bmod 3118$$

Alice sends M, S_1, and S_2 to Bob. Bob uses the public key to calculate V_1 and V_2.

$$V_1 = e_1{}^M = 2^{320} = 3006 \bmod 3119$$

$$V_2 = d^{S_1} \times S_1{}^{S_2} = 1702^{2083} \times 2083^{2105} = 3006 \bmod 3119$$

Because V_1 and V_2 are congruent, Bob accepts the message and he assumes that the message has been signed by Alice because no one else has Alice's private key, d.

ElGamal Digital Signature Scheme

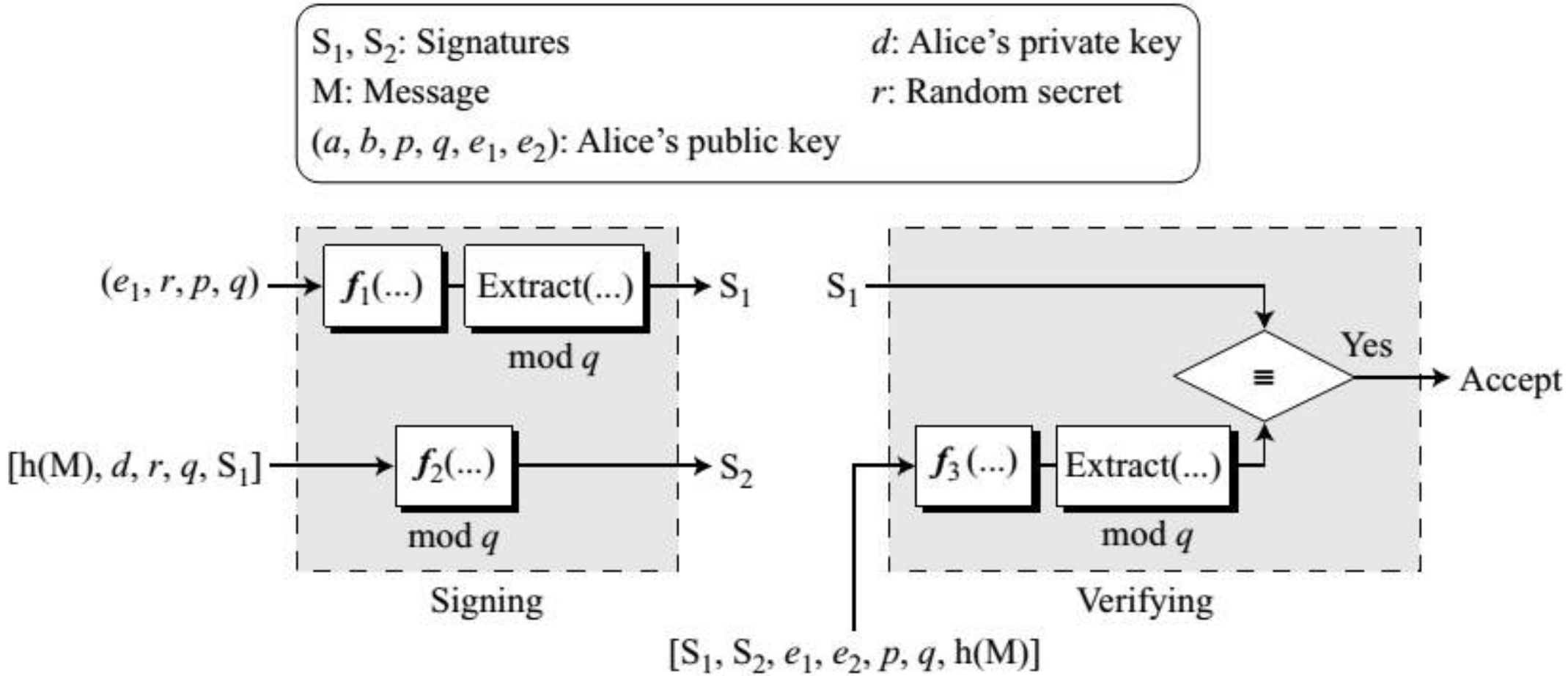
Now imagine that Alice wants to send another message, M = 3000, to Ted. She chooses a new r, 107. Alice sends M, S_1, and S_2 to Ted. Ted uses the public keys to calculate V_1 and V_2.

$$M = 3000$$
$$S_1 = e_1{}^r = 2^{107} = 2732 \bmod 3119$$
$$S_2 = (M - d \times S_1)\, r^{-1} = (3000 - 127 \times 2083) \times 107^{-1} = 2526 \bmod 3118$$

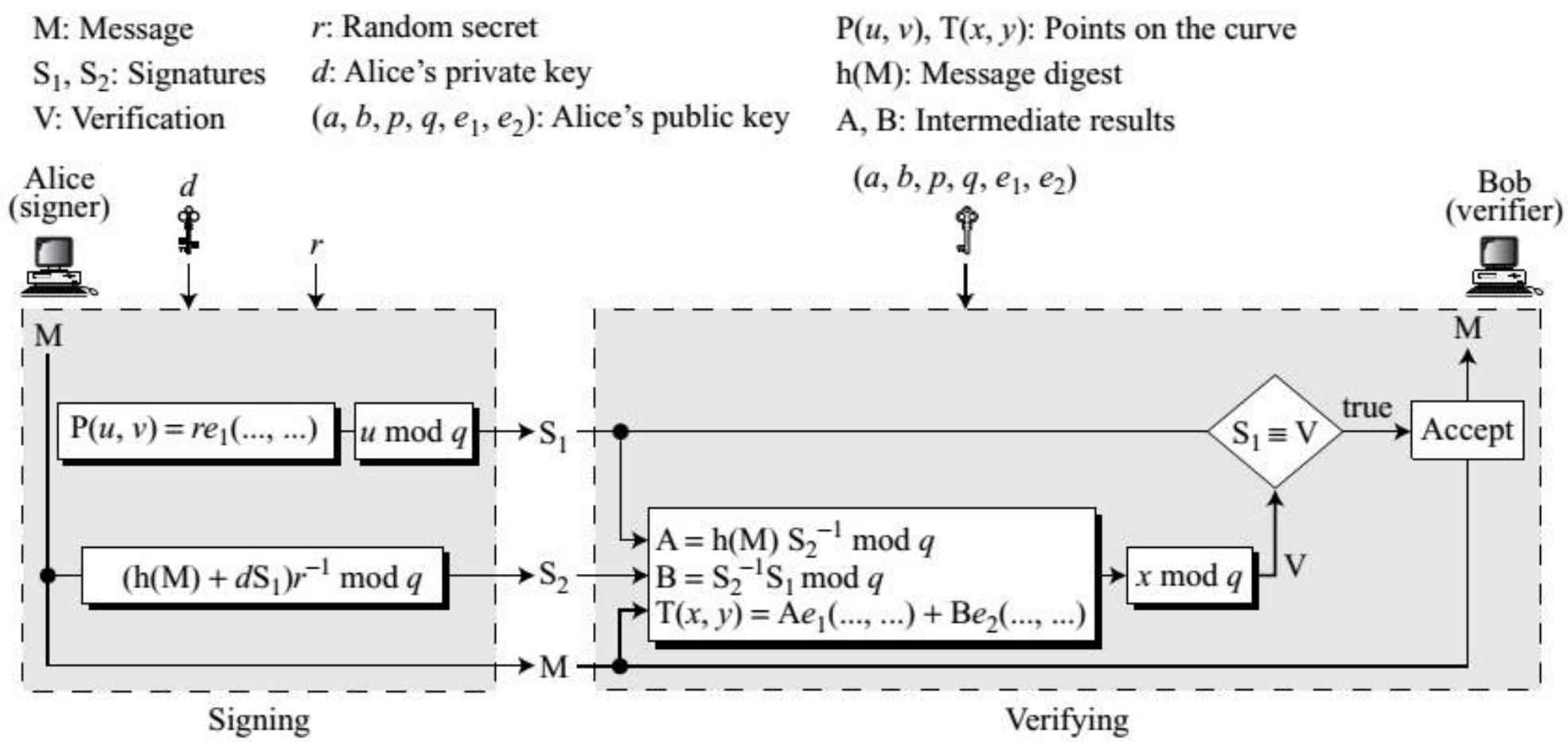
$$V_1 = e_1{}^M = 2^{3000} = 704 \bmod 3119$$
$$V_2 = d^{S_1} \times S_1{}^S = 1702^{2732} \times 2083^{2526} = 704 \bmod 3119$$

Because V_1 and V_2 are congruent, Ted accepts the message; he assumes that the message has been signed by Alice because no one else has Alice's private key, d. Note that any person can receive the message. The goal is not to hide the message, but to prove that it is sent by Alice.

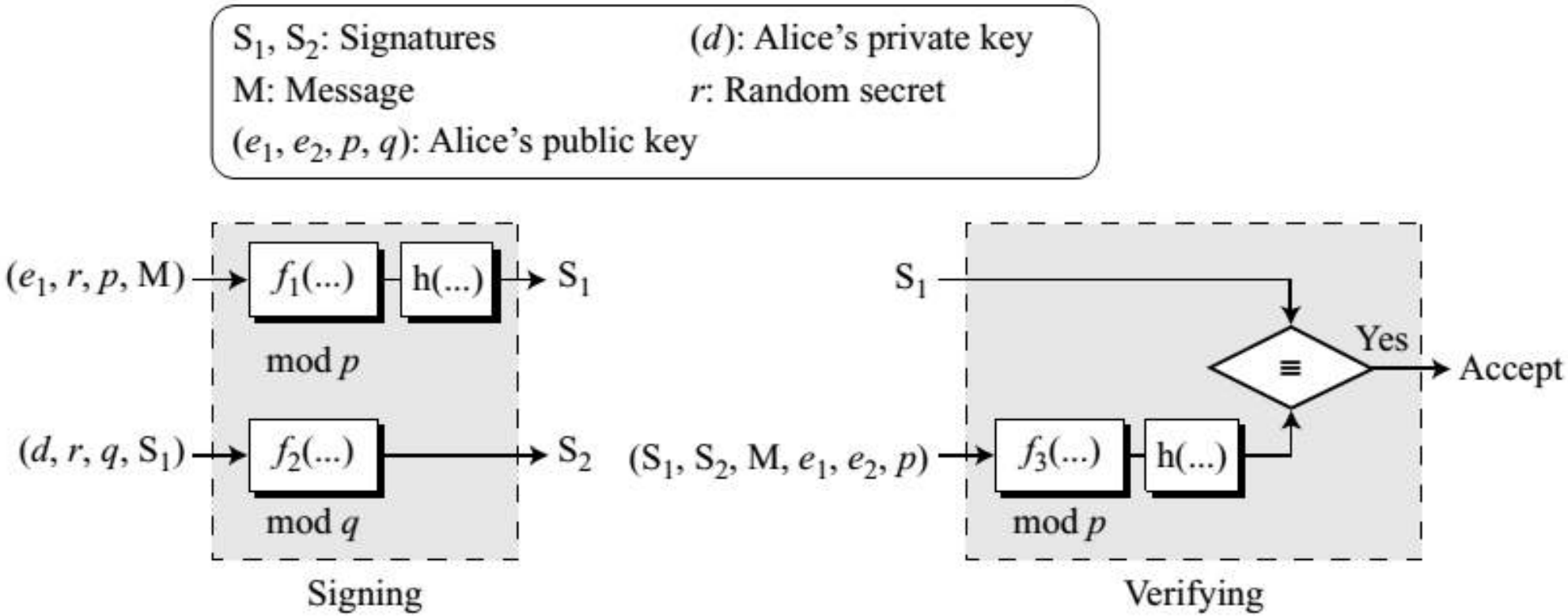
Elliptic Curve Digital Signature Scheme



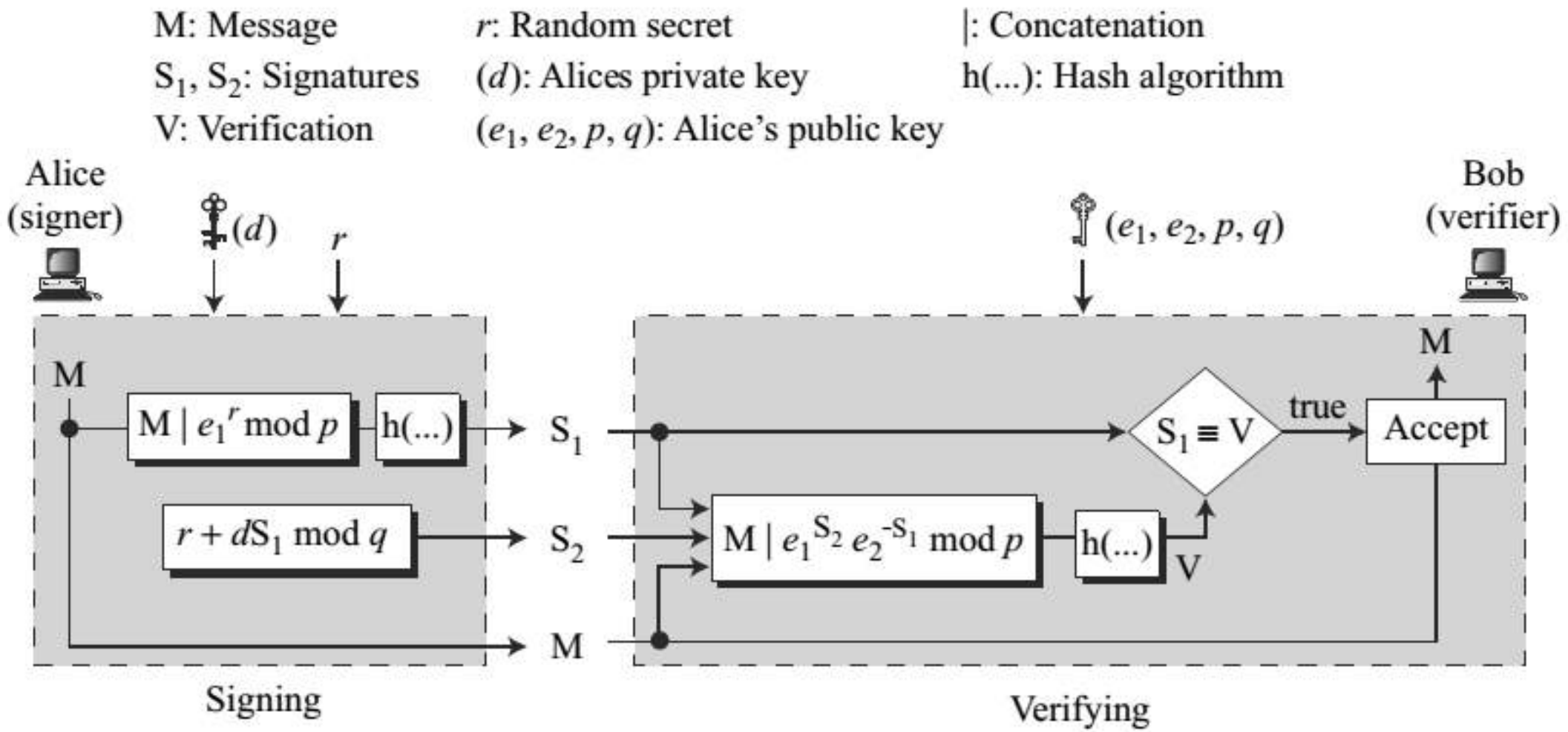
Elliptic Curve Digital Signature Scheme



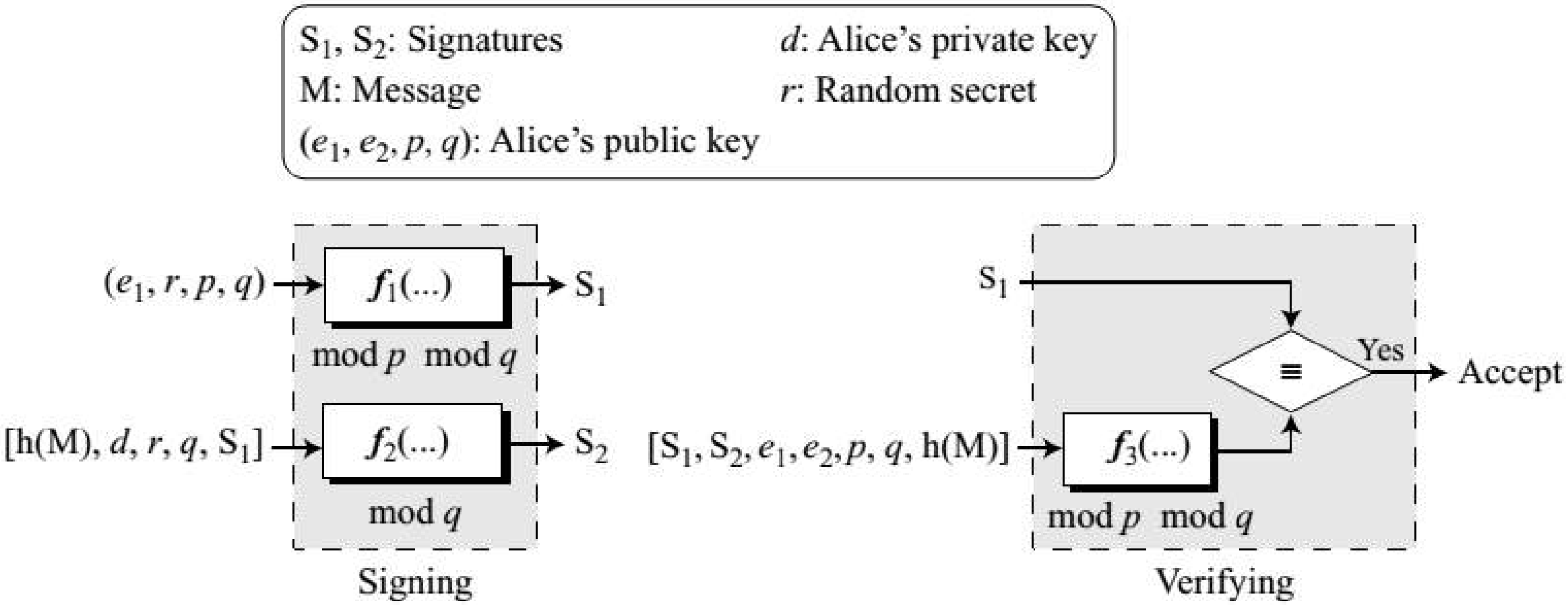
Schnorr Digital Signature Scheme



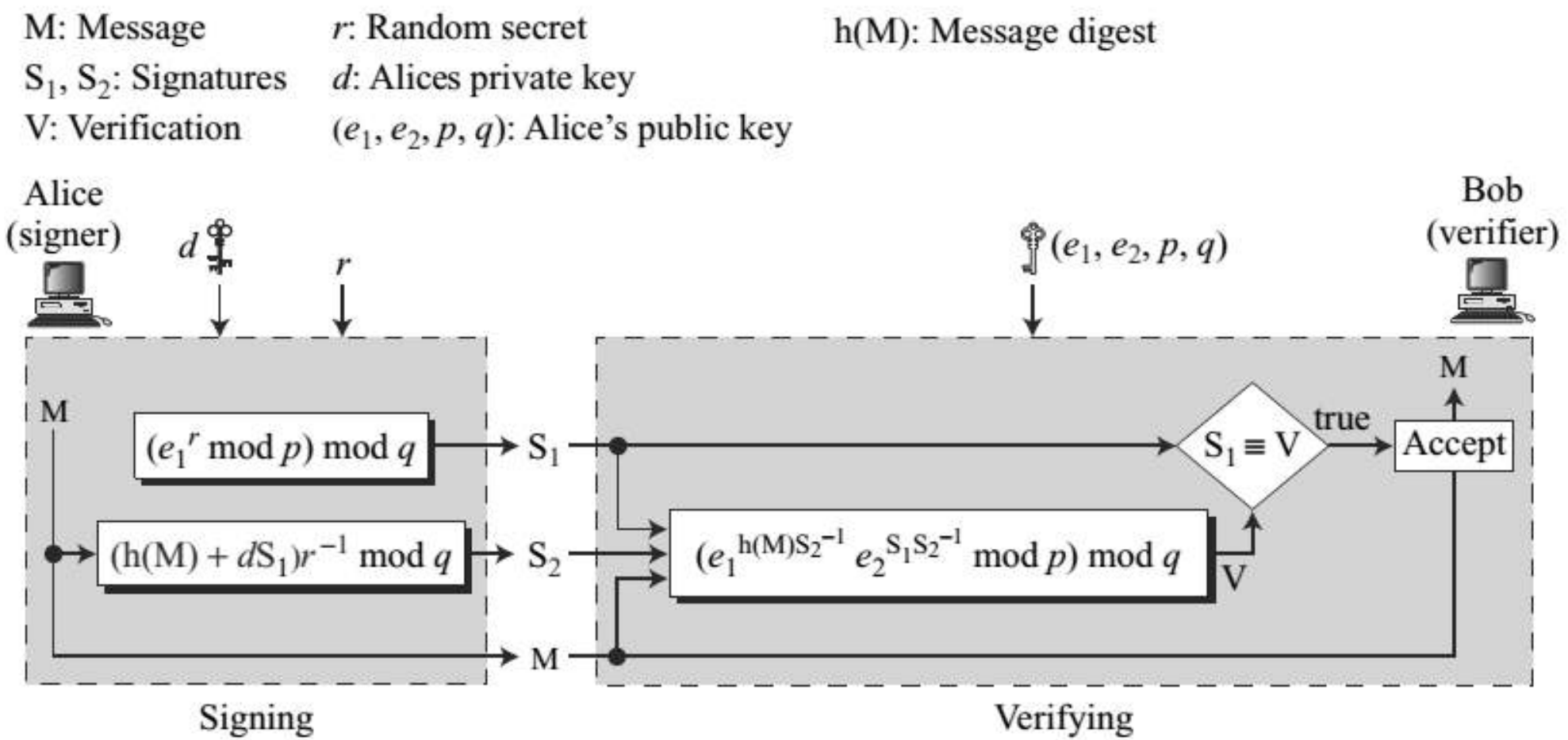
Schnorr Digital Signature Scheme



Digital Signature Standard (DSS)



Digital Signature Standard (DSS)



Tutorial

- Explore the security attacks on schemes

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 14 : Modern Symmetric-Key Ciphers

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

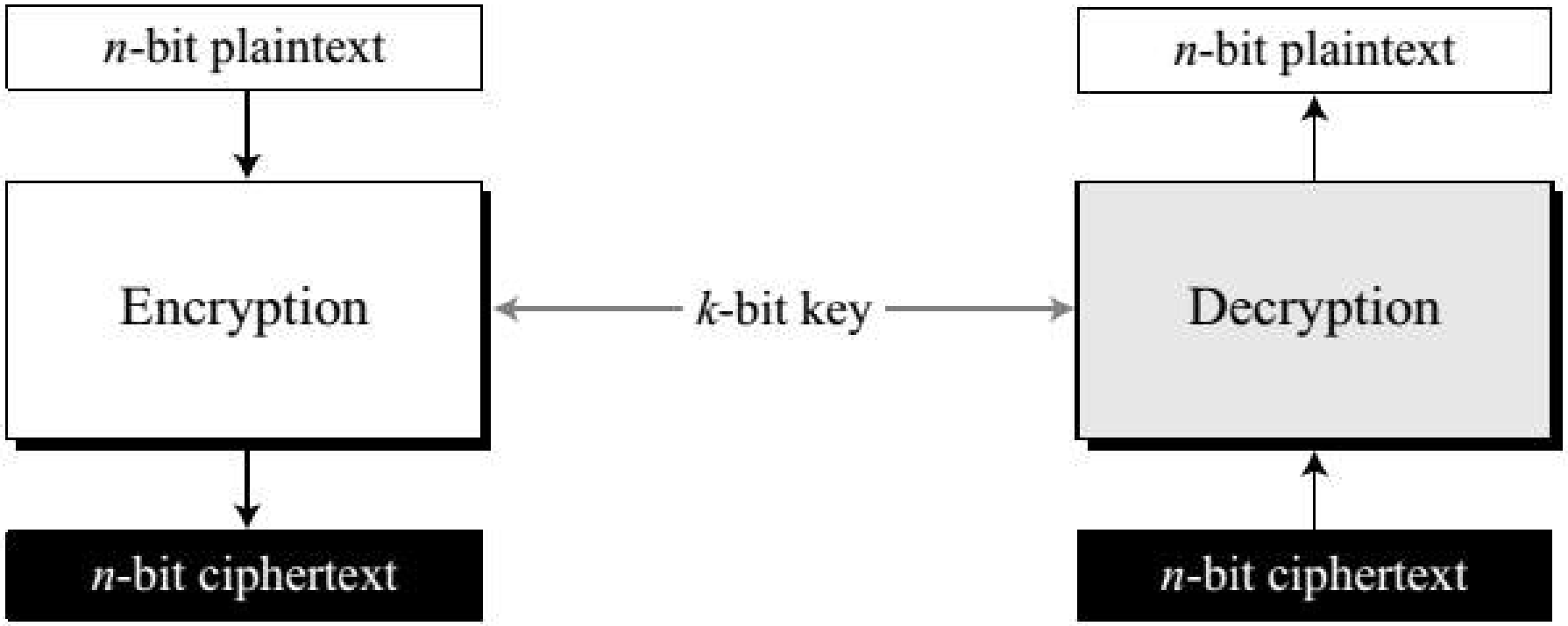
Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

Introduction

- The traditional symmetric-key ciphers that we have studied so far are character-oriented ciphers.
- With the advent of the computer, we need bit-oriented ciphers.
- This is because the information to be encrypted is not just text; it can also consist of numbers, graphics, audio, and video data.
- MODERN BLOCK CIPHERS
- MODERN STREAM CIPHERS

A modern block cipher



How many padding bits must be added to a message of 100 characters if 8-bit ASCII is used for encoding and the block cipher accepts blocks of 64 bits?

- Encoding 100 characters using 8-bit ASCII results in an 800-bit message.
- The plaintext must be divisible by 64.
- If |M| and |Pad| are the length of the message and the length of the padding,

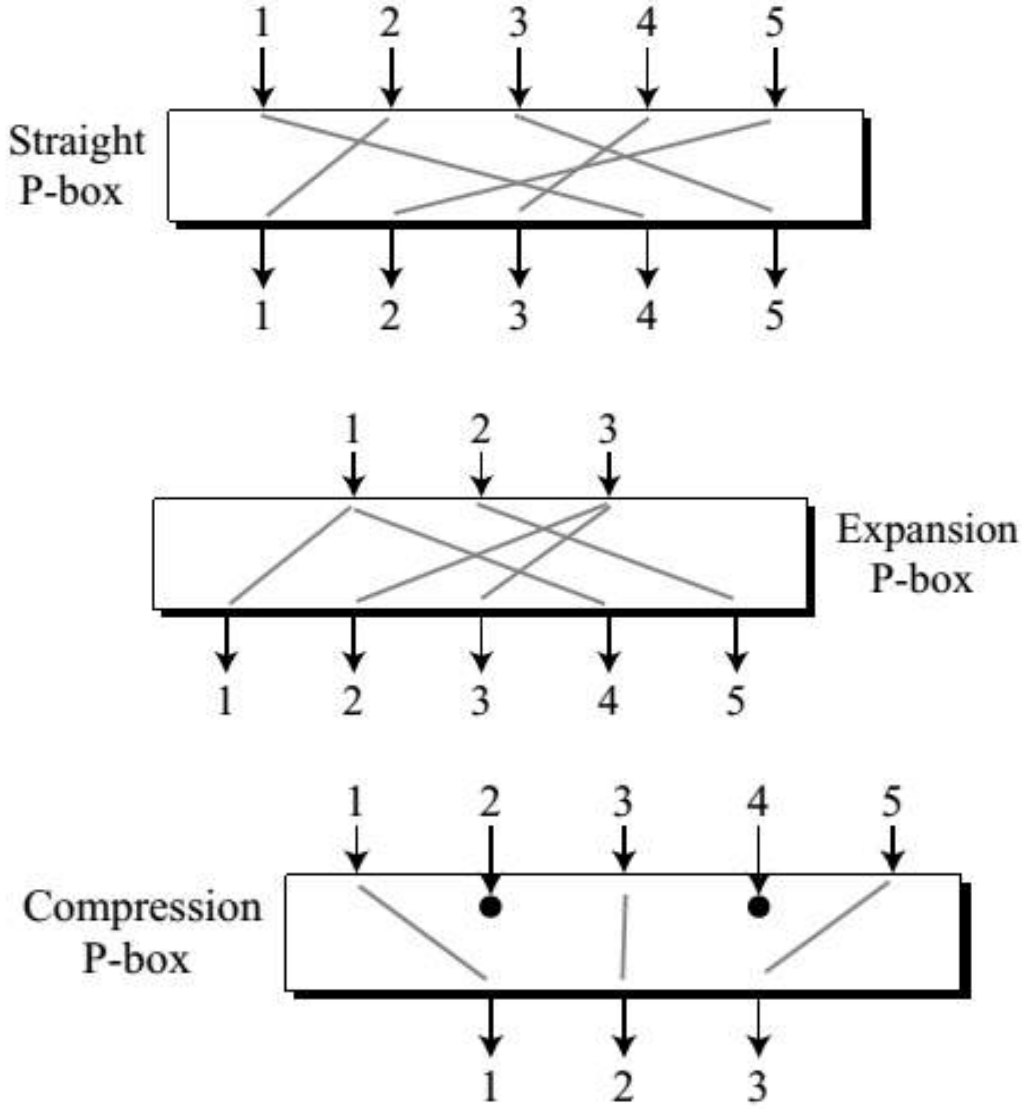
  $|M| + |Pad| = 0 \bmod 64 \rightarrow |Pad| = -800 \bmod 64 \rightarrow 32 \bmod 64$
- This means that 32 bits of padding (for example, 0's) need to be added to the message.
- The plaintext then consists of 832 bits or thirteen 64-bit blocks.
- Note that only the last block contains padding.
- The cipher uses the encryption algorithm thirteen times to create thirteen ciphertext blocks.

Components of a Modern Block Cipher

- Modern block ciphers normally are keyed substitution ciphers in which the key allows only partial mappings from the possible inputs to the possible outputs.
- However, modern block ciphers normally are not designed as a single unit.
- To provide the required properties of a modern block cipher, such as diffusion and confusion, a modern block cipher is made of a combination of transposition units (called P-boxes), substitution units (called S-boxes), and some other units.

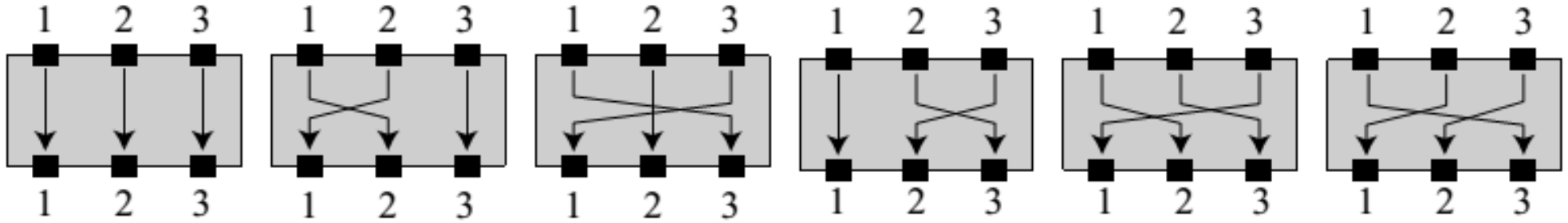
P-Boxes

- A P-box (permutation box) parallels the traditional transposition cipher for characters.
- It transposes bits.
- We can find three types of P-boxes in modern block ciphers:
  - Straight P-boxes,
    - Ex: (5 × 5 straight P-box)
  - Expansion P-boxes,
    - Ex: (3 × 5 expansion P-box) and
  - Compression P-boxes
    - (5 × 3 compression P-box)



Straight P-Boxes

- A straight P-Box with n inputs and n outputs is a permutation. There are n! possible mappings.



All 6 possible mappings of a 3 × 3 P-box.

58	50	42	34	26	18	10	02	60	52	44	36	28	20	12	04
62	54	46	38	30	22	14	06	64	56	48	40	32	24	16	08
57	49	41	33	25	17	09	01	59	51	43	35	27	19	11	03
61	53	45	37	29	21	13	05	63	55	47	39	31	23	15	07

An example of a straight permutation table when n is 64.

Design an 8 × 8 permutation table for a straight P-box that moves the two middle bits (bits 4 and 5) in the input word to the two ends (bits 1 and 8) in the output words. Relative positions of other bits should not be changed.

- We need a straight P-box with the table [4 1 2 3 6 7 8 5]. The relative positions of input bits 1, 2, 3, 6, 7, and 8 have not been changed, but the first output takes the fourth input and the eighth output takes the fifth input.

Compression P-Boxes

- A compression P-box is a P-box with n inputs and m outputs where m < n.
- Some of the inputs are blocked and do not reach the output
- We need to know that a permutation table for a compression P-box has m entries, but the content of each entry is from 1 to n with some missing values (those inputs that are blocked).
- An example of a permutation table for a 32 × 24 compression P-box.
- Note that inputs 7, 8, 9, 15, 16, 23, 24, and 25 are blocked.

01	02	03	21	22	26	27	28	29	13	14	17
18	19	20	04	05	06	10	11	12	30	31	32

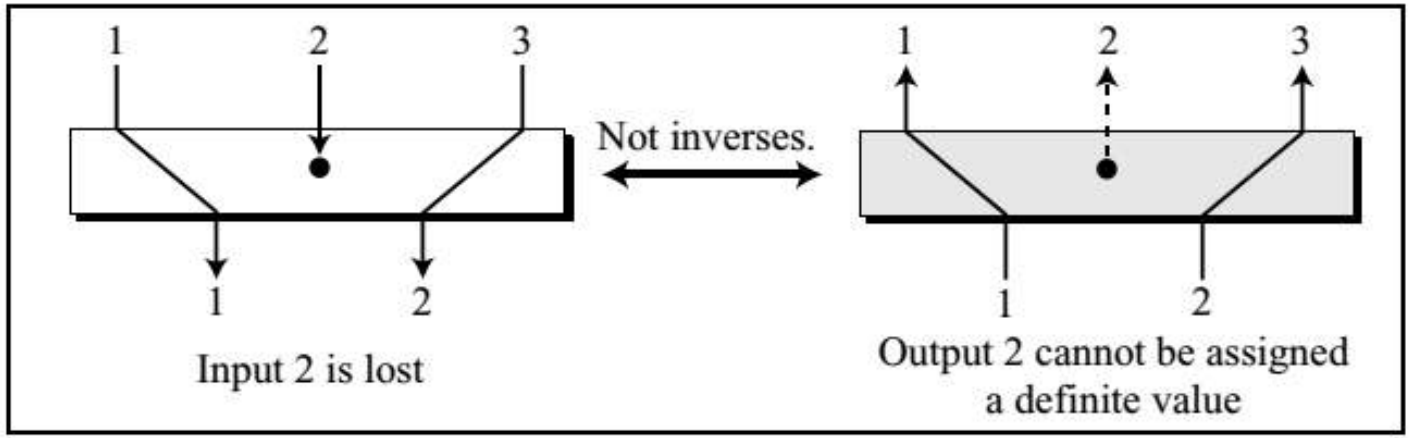
Expansion P-Boxes

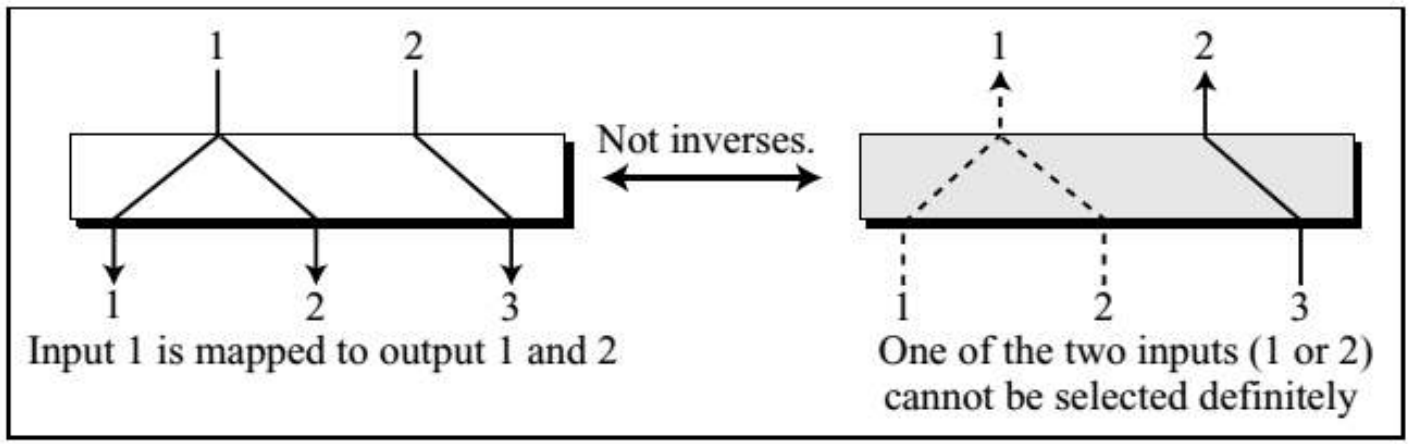
- An expansion P-box is a P-box with n inputs and m outputs where m > n.
- Some of the inputs are connected to more than one input
- The expansion P-boxes used in modern block ciphers normally are keyless, where a permutation table shows the rule for transposing bits.
- We need to know that a permutation table for an expansion P-box has m entries, but m – n of the entries are repeated (those inputs mapped to more than one output).
- An example of a permutation table for a 12 × 16 expansion P-box.
- Note that each of the inputs 1, 3, 9, and 12 is mapped to two outputs.

01	09	10	11	12	01	02	03	03	04	05	06	07	08	09	12

Invertibility

Compression P-box

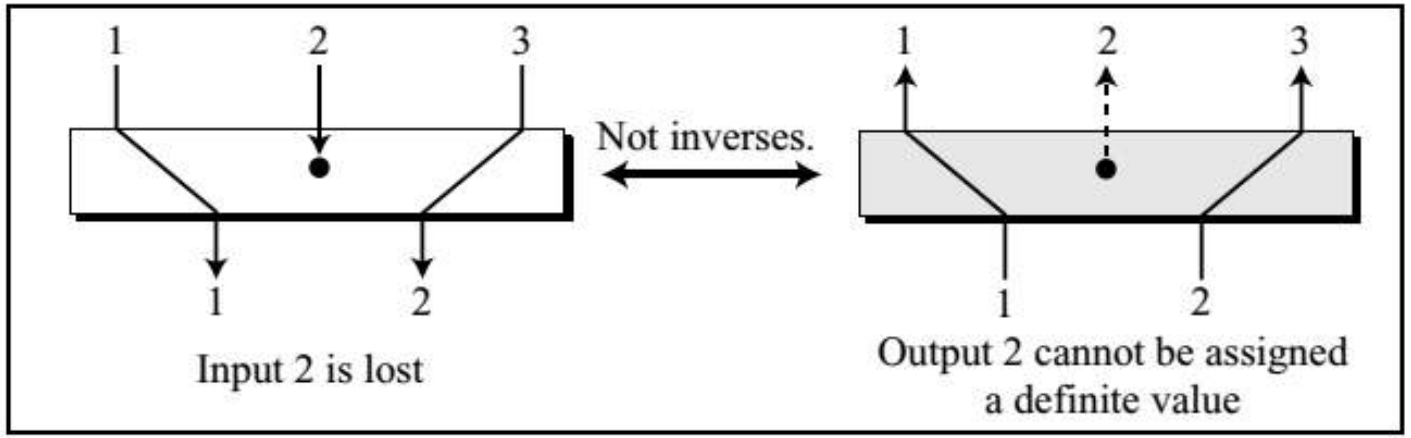


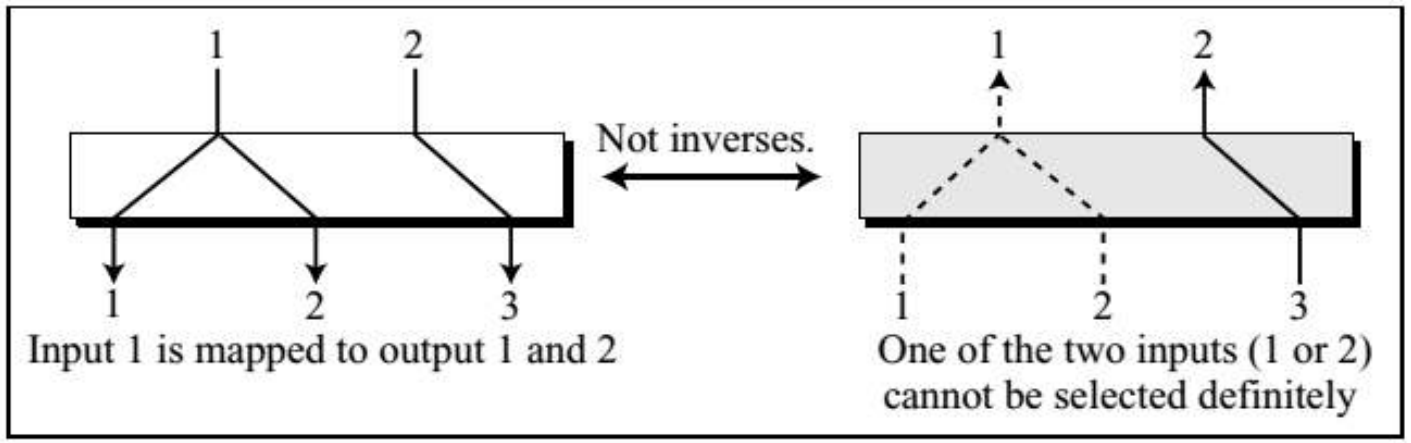


Expansion P-box

Invertibility

Compression P-box





Expansion P-box

A straight P-box is invertible, but compression and expansion P-boxes are not.

S-Boxes

- An S-box (substitution box) can be thought of as a miniature substitution cipher.
- However, an S-box can have a different number of inputs and outputs.
- In other words, the input to an S-box could be an n-bit word, but the output can be an m-bit word, where m and n are not necessarily the same.

Linear Versus Nonlinear S-Boxes

- In an S-box with n inputs and m outputs, we call the inputs x_0, x_1, …, x_n and the outputs y_1, …, y_m.
- The relationship between the inputs and the outputs can be represented as a set of equations

$$
\begin{aligned}
y_1 &= f_1(x_1, x_2, \ldots, x_n) \\
y_2 &= f_2(x_1, x_2, \ldots, x_n) \\
&\ldots \\
y_m &= f_m(x_1, x_2, \ldots, x_n)
\end{aligned}
$$

Linear Versus Nonlinear S-Boxes

- In a linear S-box, the above relations can be expressed as

$$
\begin{aligned}
y_1 &= a_{1,1}x_1 \oplus a_{1,2}x_1 \oplus \cdots \oplus a_{1,n}x_n \\
y_2 &= a_{2,1}x_1 \oplus a_{2,2}x_1 \oplus \cdots \oplus a_{2,n}x_n \\
&\cdots \\
y_m &= a_{m,1}x_1 \oplus a_{m,2}x_1 \oplus \cdots \oplus a_{m,n}x_n
\end{aligned}
$$

- In a nonlinear S-box we cannot have the above relations for every output.

Linear Versus Nonlinear S-Boxes

- In an S-box with three inputs and two outputs, we have $y_1 = x_1 \oplus x_2 \oplus x_3$ $y_2 = x_1$. Is it linear or not?
- The S-box is linear because $a_{1,1} = a_{1,2} = a_{1,3} = a_{2,1} = 1$ and $a_{2,2} = a_{2,3} = 0$.
- The relationship can be represented by matrices, as shown below

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Linear Versus Nonlinear S-Boxes

- In an S-box with three inputs and two outputs, we have

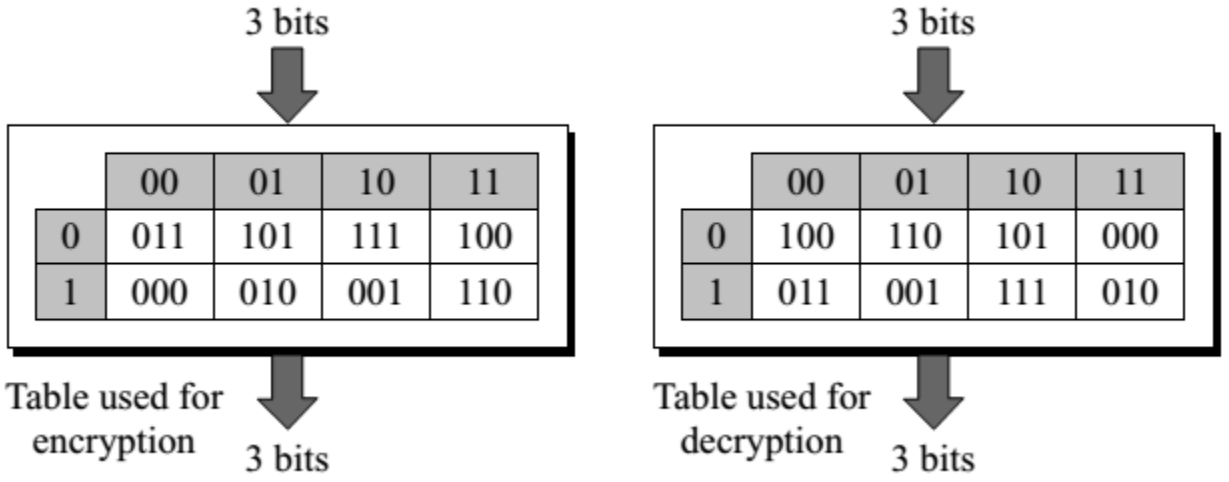
  $y_1 = (x_1)^3 + x_2$, $y_2 = (x_1)^2 + x_1x_2 + x_3$

  where multiplication and addition is in GF(2). The S-box is nonlinear because there is no linear relationship between the inputs and the outputs.

Invertibility

- S-boxes are substitution ciphers in which the relationship between input and output is defined by a table or mathematical relation.
- An S-box may or may not be invertible.
- In an invertible S-box, the number of input bits should be the same as the number of output bits.

For example, if the input to the left box is 001, the output is 101. The input 101 in the right table creates the output 001, which shows that the two tables are inverses of each other.



Exclusive-Or

- The five properties of the exclusive-or operation in the GF(2^n) field makes this operation a very interesting component for use in a block cipher.
- Closure: This property guarantees that the result of exclusive-oring two n-bit words is another n-bit word.
- Associativity: This property allows us to use more than one exclusive-or operator in any order. { $x \oplus (y \oplus z) \leftrightarrow (x \oplus y) \oplus z$ }
- Commutativity: This property allows us to swap the inputs without affecting the output. { $x \oplus y \leftrightarrow y \oplus x$ }
- Existence of identity: The identity element for the exclusive-or operation is an n-bit word that consists of all 0's, or (00… 0). This implies that exclusive-oring of a word with the identity element does not change that word. { $x \oplus (00...0) = x$ }
- Existence of inverse: In the GF(2^n) field, each word is the additive inverse of itself. This implies that exclusive-oring of a word with itself yields the identity element. {$x \oplus x = (00…0)$}

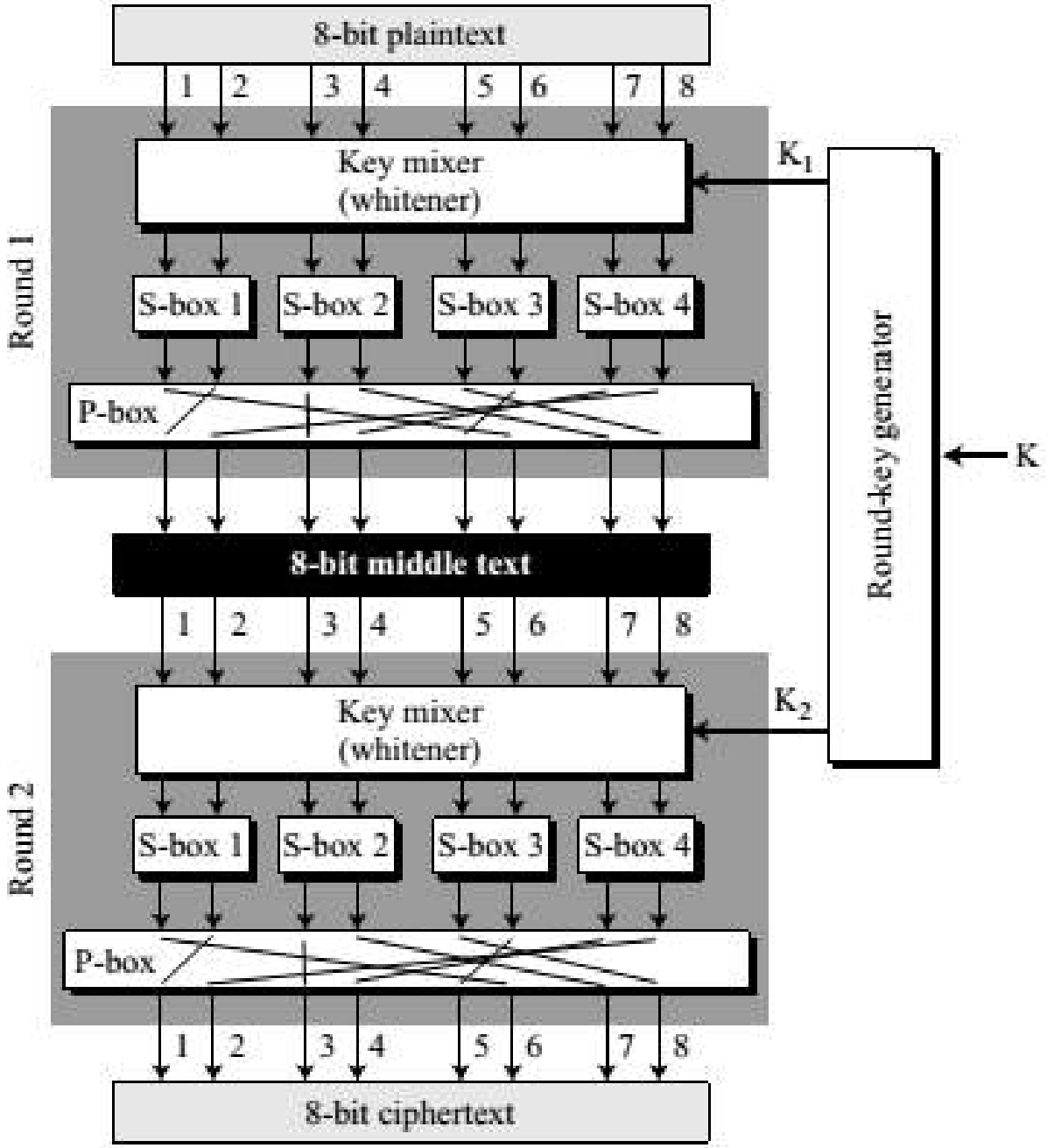
Diffusion and Confusion

- The idea of diffusion is to hide the relationship between the ciphertext and the plaintext.
- This will frustrate the adversary who uses ciphertext statistics to find the plaintext.
- Diffusion implies that each symbol (character or bit) in the ciphertext is dependent on some or all symbols in the plaintext.
- In other words, if a single symbol in the plaintext is changed, several or all symbols in the ciphertext will also be changed.

- The idea of confusion is to hide the relationship between the ciphertext and the key.
- This will frustrate the adversary who tries to use the ciphertext to find the key.
- In other words, if a single bit in the key is changed, most or all bits in the ciphertext will also be changed.

Rounds

- Diffusion and confusion can be achieved using iterated product ciphers where each iteration is a combination of S-boxes, P-boxes, and other components.
- Each iteration is referred to as a round.
- In an N-round cipher, the plaintext is encrypted N times to create the ciphertext.
- The ciphertext is decrypted N times to create the plaintext.
- We refer to the text created at the intermediate levels (between two rounds) as the middle text.



Rounds

A simple product cipher with two rounds.

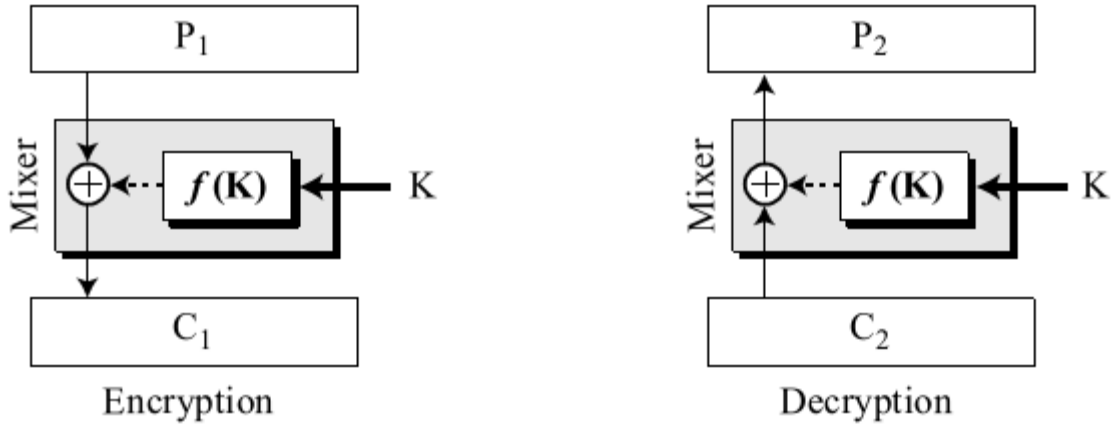
The 8-bit text is mixed with the key to whiten the text (hide the bits using the key).
This is normally done by exclusive-oring the 8-bit word with the 8-bit key.

Two Classes of Product Ciphers

- Modern block ciphers are all product ciphers, but they are divided into two classes.
- The ciphers in the first class use both invertible and noninvertible components.
  - The ciphers in this class are normally referred to as Feistel ciphers.
  - The block cipher DES is a good example of a Feistel cipher.
- The ciphers in the second class use only invertible components.
  - We refer to ciphers in this class as non-Feistel ciphers (for the lack of another name).
  - The block cipher AES is a good example of a non-Feistel cipher.

Feistel Ciphers

- A Feistel cipher can have three types of components: self-invertible, invertible, and noninvertible.
- A Feistel cipher combines all noninvertible elements in a unit and uses the same unit in the encryption and decryption algorithms.
- The effects of a noninvertible component in the encryption algorithm can be canceled in the decryption algorithm if we use an exclusive-or operation



Feistel Ciphers

- In other words, if $C_2 = C_1$ (no change in the ciphertext during transmission), then $P_2 = P_1$

Encryption: $C_1 = P_1 \oplus f(K)$

Decryption: $P_2 = C_2 \oplus f(K) = C_1 \oplus f(K) = P_1 \oplus f(K) \oplus f(K) = P_1 \oplus (00\ldots0) = P_1$

The plaintext and ciphertext are each 4 bits long and the key is 3 bits long. Assume that the function takes the first and third bits of the key, interprets these two bits as a decimal number, squares the number, and interprets the result as a 4-bit binary pattern.
Show the results of encryption and decryption if the original plaintext is 0111 and the key is 101.

The function extracts the first and second bits to get 11 in binary or 3 in decimal. The result of squaring is 9, which is 1001 in binary.

Encryption: $C = P \oplus f(K) = 0111 \oplus 1001 = 1110$

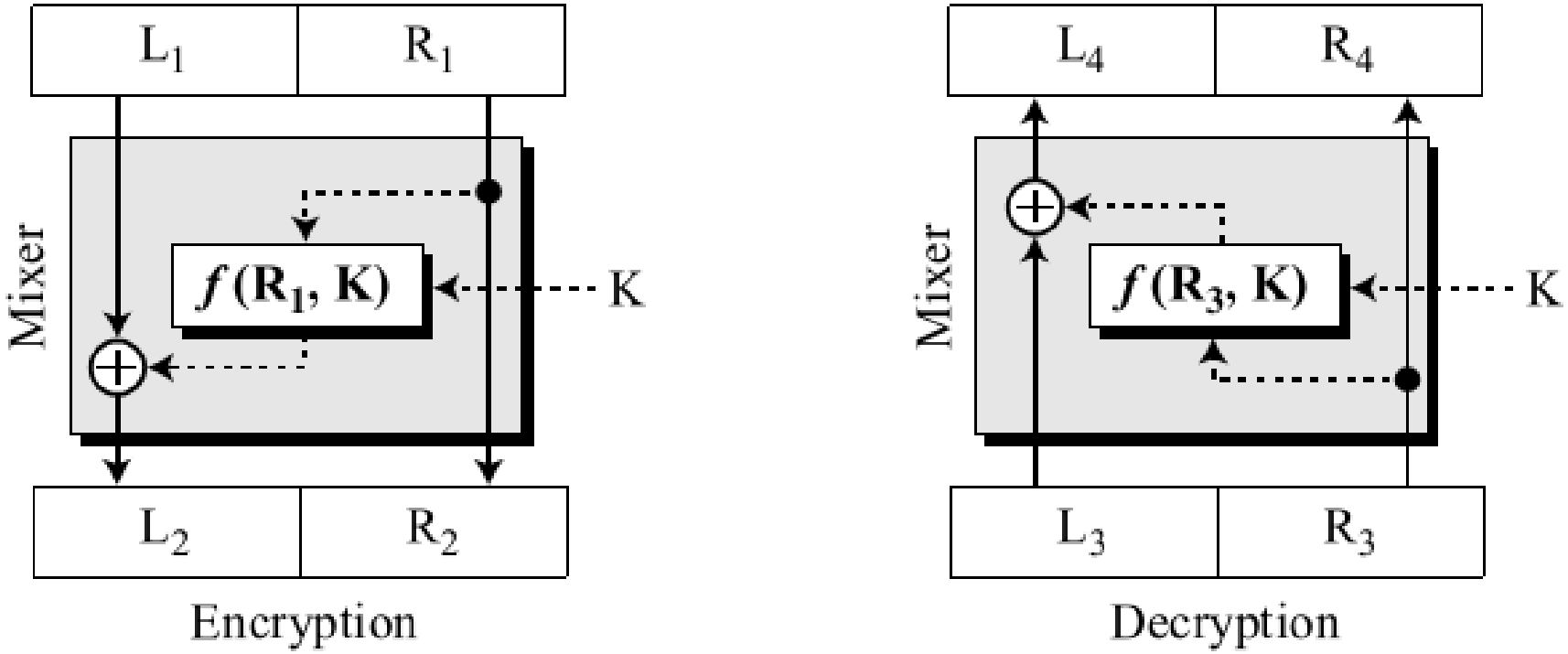
Decryption: $P = C \oplus f(K) = 1110 \oplus 1001 = 0111$ Same as the original P

The function ƒ(101) = 1001 is non-invertible, but the exclusive-or operation allows us to use the function in both encryption and decryption algorithms.

Improvement of the previous Feistel design

- The input to the function to also be part of the plaintext in the encryption and part of the ciphertext in the decryption
- The key can be used as the second input to the function.
- In this way, our function can be a complex element with some keyless elements and some keyed elements.
- To achieve this goal, divide the plaintext and the ciphertext into two equal-length blocks, left and right.
- We call the left block L and the right block R.
- Let the right block be the input to the function, and let the left block be exclusive-ored with the function output.

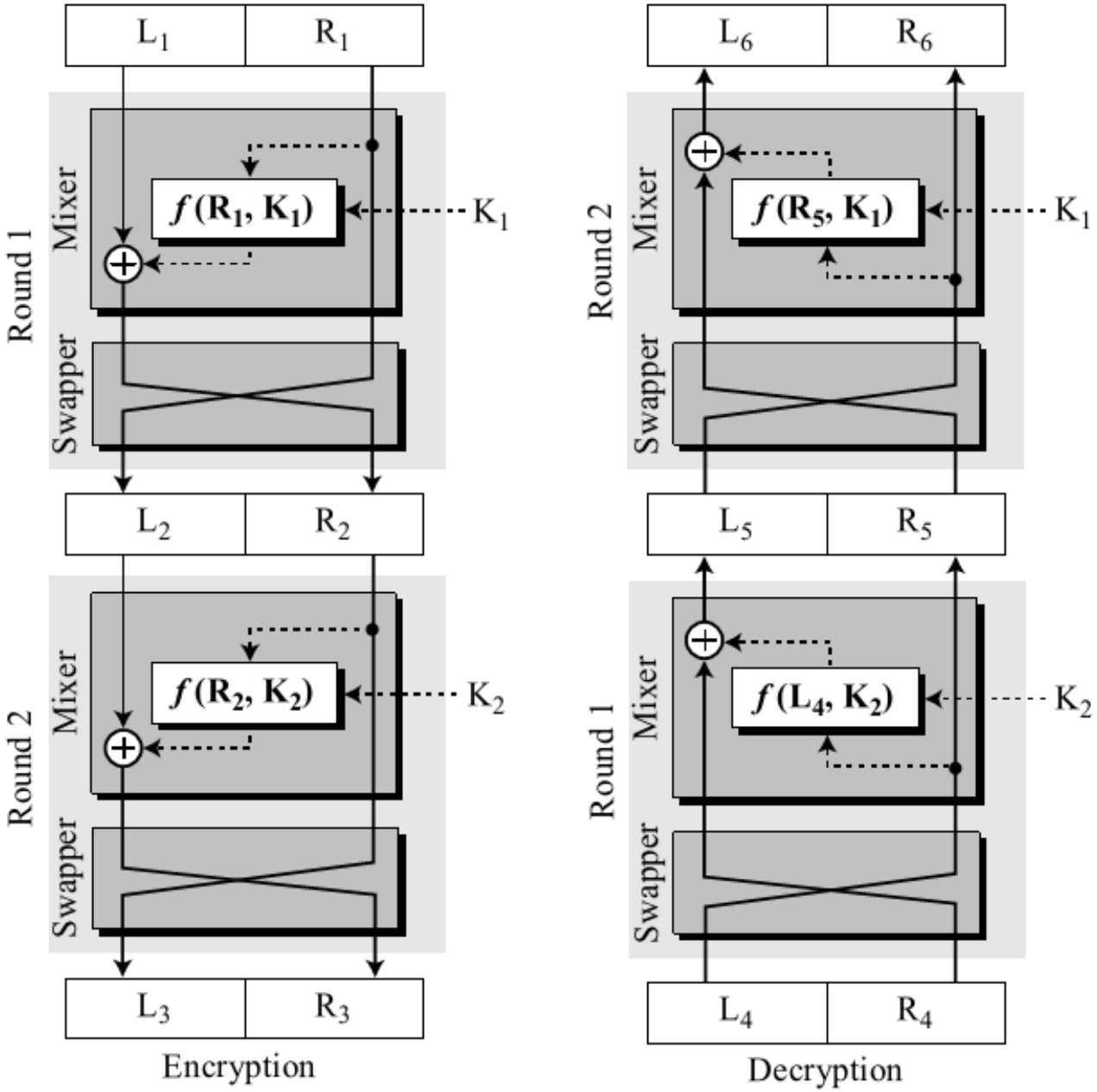
Improvement of the previous Feistel design



$$R_4 = R_3 = R_2 = R_1$$
$$L_4 = L_3 \oplus f(R_3, K) = L_2 \oplus f(R_2, K) = L_1 \oplus f(R_1, K) \oplus f(R_1, K) = L_1$$

Final Design of the Feistel design

- The preceding improvement has one flaw.
  - The right half of the plain-text never changes.
  - Eve can immediately find the right half of the plaintext by inter-cepting the ciphertext and extracting the right half of it.
- Needs more improvement.
  - First, increase the number of rounds.
  - Second,add a new element to each round: a swapper.



Final design of a Feistel cipher with two rounds

Final design of a Feistel cipher with two rounds

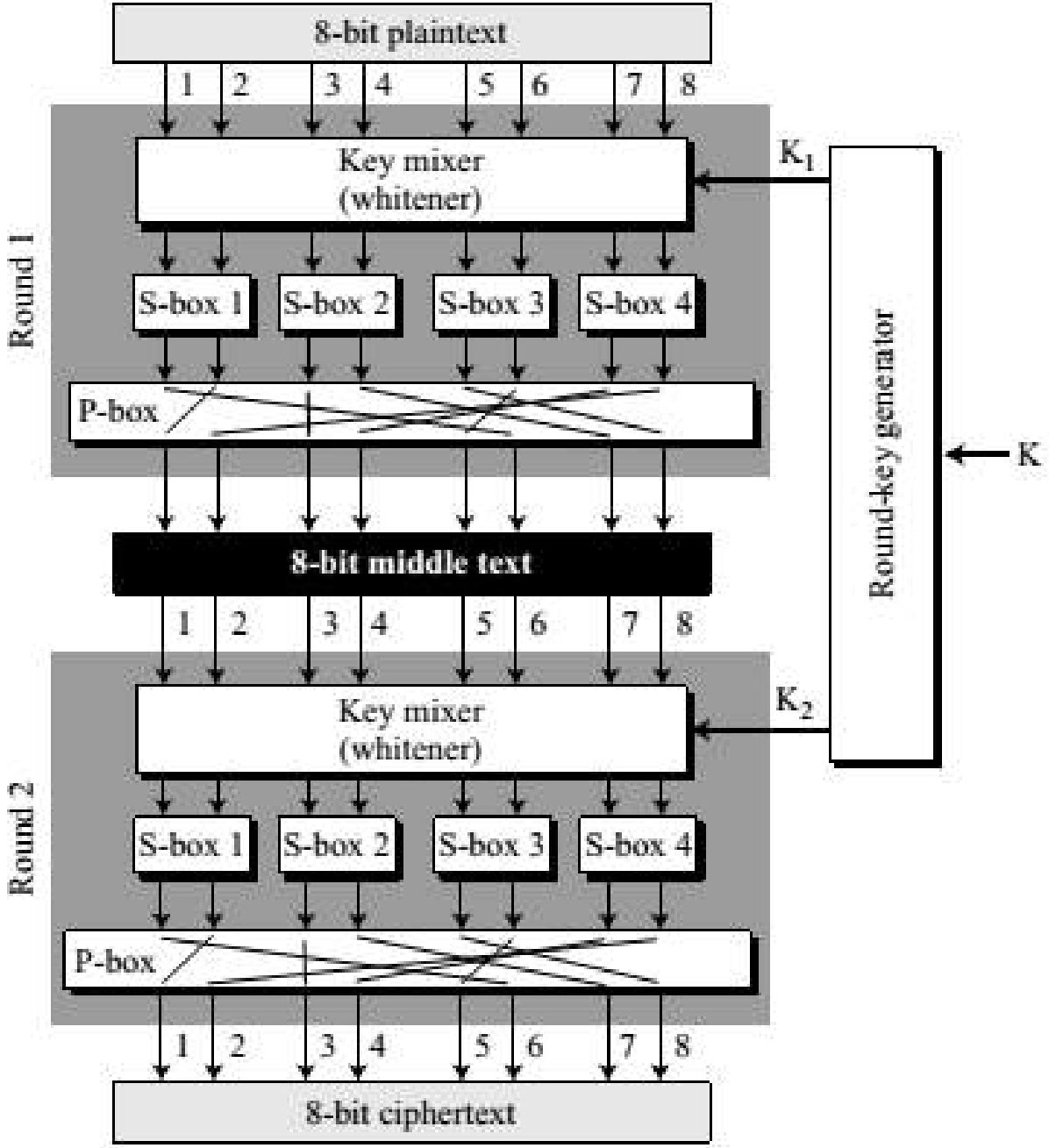
- Because the two mixers are inverses of each other, and the swappers are inverses of each other, it should be clear that the encryption and decryption ciphers are inverses of each other

$$L_5 = R_4 \oplus f(L_4, K_2) = R_3 \oplus f(R_2, K_2) = L_2 \oplus f(R_2, K_2) \oplus f(R_2, K_2) = L_2$$
$$R_5 = L_4 = L_3 = R_2$$

$$L_6 = R_5 \oplus f(L_5, K_1) = R_2 \oplus f(L_2, K_1) = L_1 \oplus f(R_1, K_1) \oplus f(R_1, K_1) = L_1$$
$$R_6 = L_5 = L_2 = R_1$$

Non-Feistel Ciphers

- A non-Feistel cipher uses only invertible components.
- A component in the encryption cipher has the corresponding component in the decryption cipher.
- For example, S-boxes need to have an equal number of inputs and outputs to be compatible.
- No compression or expansion P-boxes are allowed, because they are not invertible.
- In a non-Feistel cipher, there is no need to divide the plaintext into two halves as we saw in the Feistel ciphers.



Rounds

A simple product cipher with two rounds.

The 8-bit text is mixed with the key to whiten the text (hide the bits using the key).
This is normally done by exclusive-oring the 8-bit word with the 8-bit key.

An example of a non-Feistel cipher

Non-Feistel Ciphers

- Because the only components in each round are the exclusive-or operation (self-invertible), 2 ×2 S-boxes that can be designed to be invertible, and a straight P-box that is invertible using the appropriate permutation table.
- Because each component is invertible, it can be shown that each
- round is invertible.
- We only need to use the round keys in the reverse order.
- The encryption uses round keys K_1 and K_2.
- The decryption algorithm needs to use round keys K_2 and K_1

Attacks on Block Ciphers

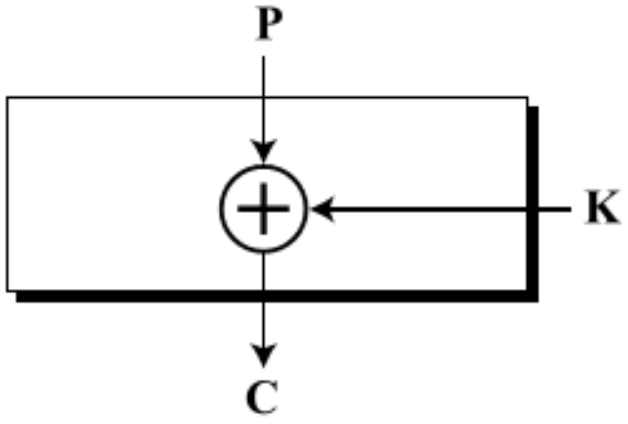
- Brute-force attack on the key is usually infeasible because the keys normally are very large.
- New attacks on block ciphers have been devised that are based on the structure of the modern block ciphers.
- These attacks use differential and linear cryptanalysis techniques.

Differential Cryptanalysis

- This is a chosen-plaintext attack.
- Eve can somehow access Alice's computer, submitting chosen plaintext and obtaining the corresponding ciphertext.
- The goal is to find Alice's cipher key.
- Some ciphers have weaknesses in their structures that can allow Eve to find a relation-ship between the plaintext differences and ciphertext differences without knowing the key.
- Differential cryptanalysis aims to map bitwise differences in inputs to differences in the output in order to reverse engineer the action of the encryption algorithm.

Differential Cryptanalysis

- Example: Assume that the cipher is made only of one exclusive-or operation
- Without knowing the value of the key, Eve can easily find the relationship between plaintext differences and ciphertext differences if by plaintext difference we mean P1 ⊕ P2 and by cipher-text difference, we mean C1⊕C2.
- The following proves that C1 ⊕C2= P1⊕ P2



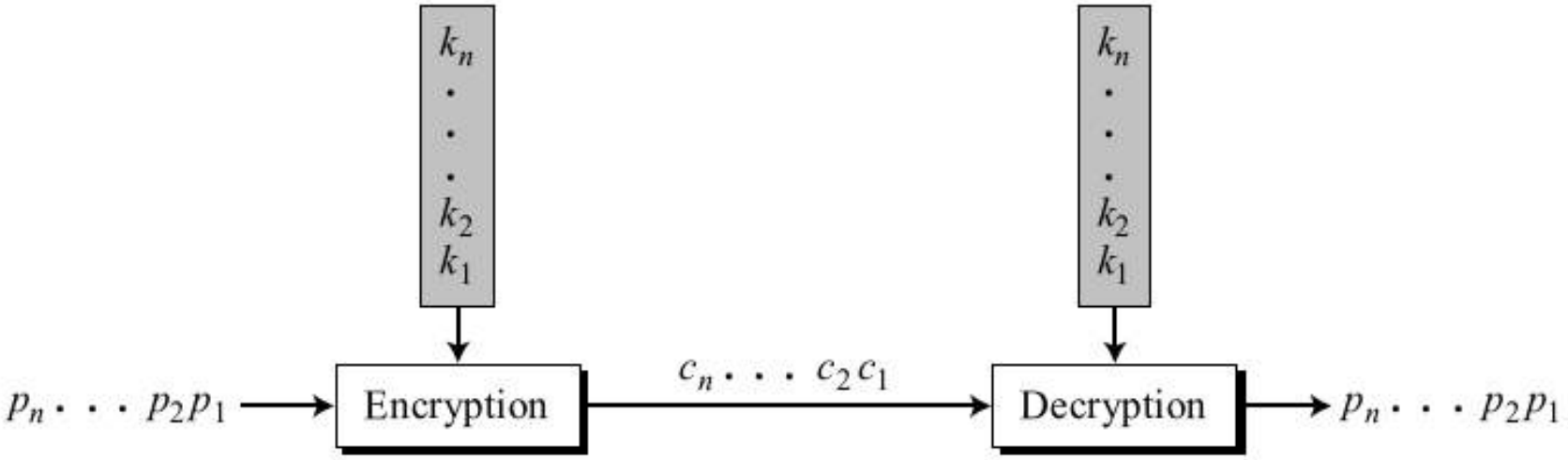
Linear Cryptanalysis

- The analysis uses known-plaintext attacks (versus the chosen-plaintext attacks in differential cryptanalysis).
- The thorough discussion of this attack is based on some probability concepts.
- Linear Approximation
  - In some modern block ciphers, it may happen that some S-boxes are not totally nonlinear.
  - They can be approximated, probabilistically, by some linear functions.
  - In general, given a cipher with plaintext and ciphertext of n bits and a key of m bits, we are looking for some equations of the form

$$(k_0 \oplus k_1 \oplus \cdots \oplus k_x) = (p_0 \oplus p_1 \oplus \cdots \oplus p_y) \oplus (c_0 \oplus c_1 \oplus \cdots \oplus c_z)$$

MODERN STREAM CIPHERS

- In a modern stream cipher,encryption and decryption are done r bits at a time.
- We have a plaintext bit stream P = $p_n \ldots p_2 p_1$, a ciphertext bit stream C = $c_n \ldots c_2 c_1$, and a key bit stream K = $k_n \ldots k_2 k_1$, in which p_i, c_i, and k_i are r-bit words.



MODERN STREAM CIPHERS

- Modern stream ciphers are divided into two broad categories: synchronous and nonsynchronous.
- In a synchronous stream cipher the key is independent of the plaintext or ciphertext.
  - One-Time Pad
- In a nonsynchronous stream cipher, the key depends on either the plaintext or ciphertext.

Course: Cryptography and Network Security
Code: CS-16104
Branch: B.Tech - CS 6th Semester

Lecture – 15 : DES and AES

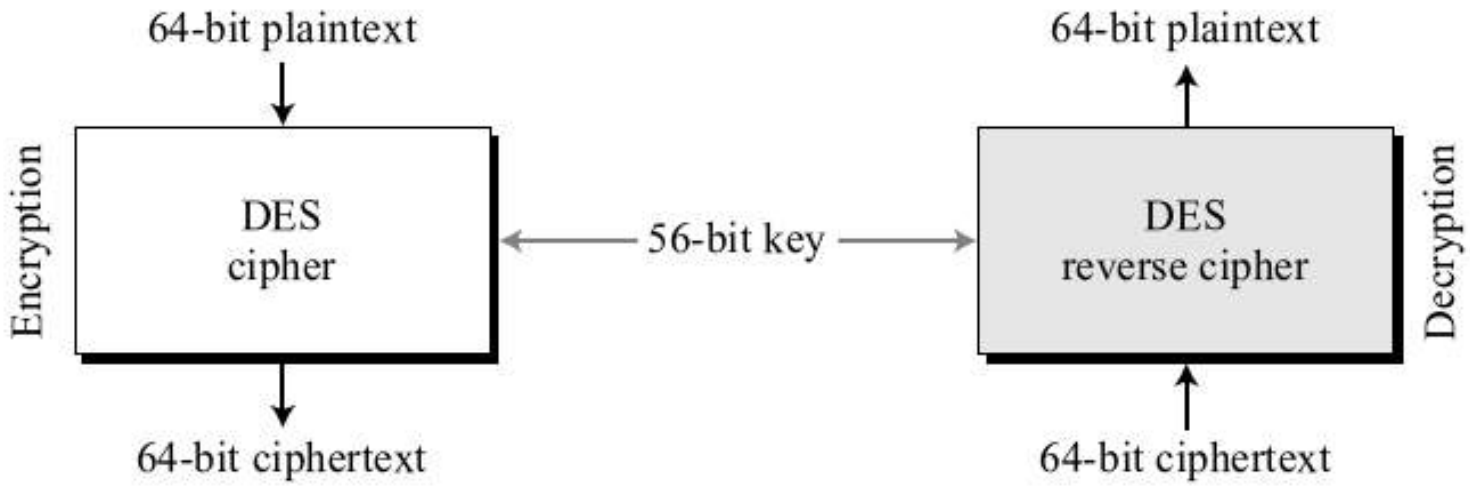
Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

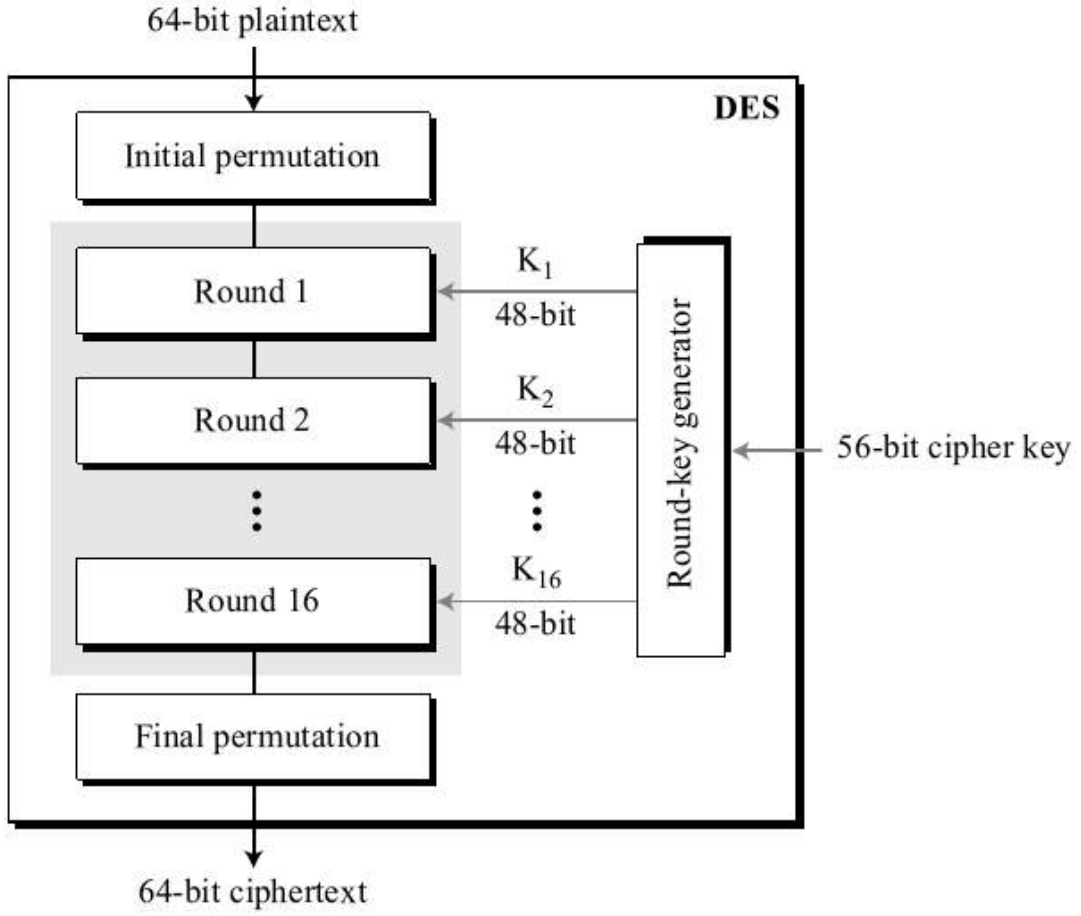
INTRODUCTION

- The **Data Encryption Standard (DES)** is a symmetric-key block cipher published by the **National Institute of Standards and Technology (NIST).**
- DES is a block cipher

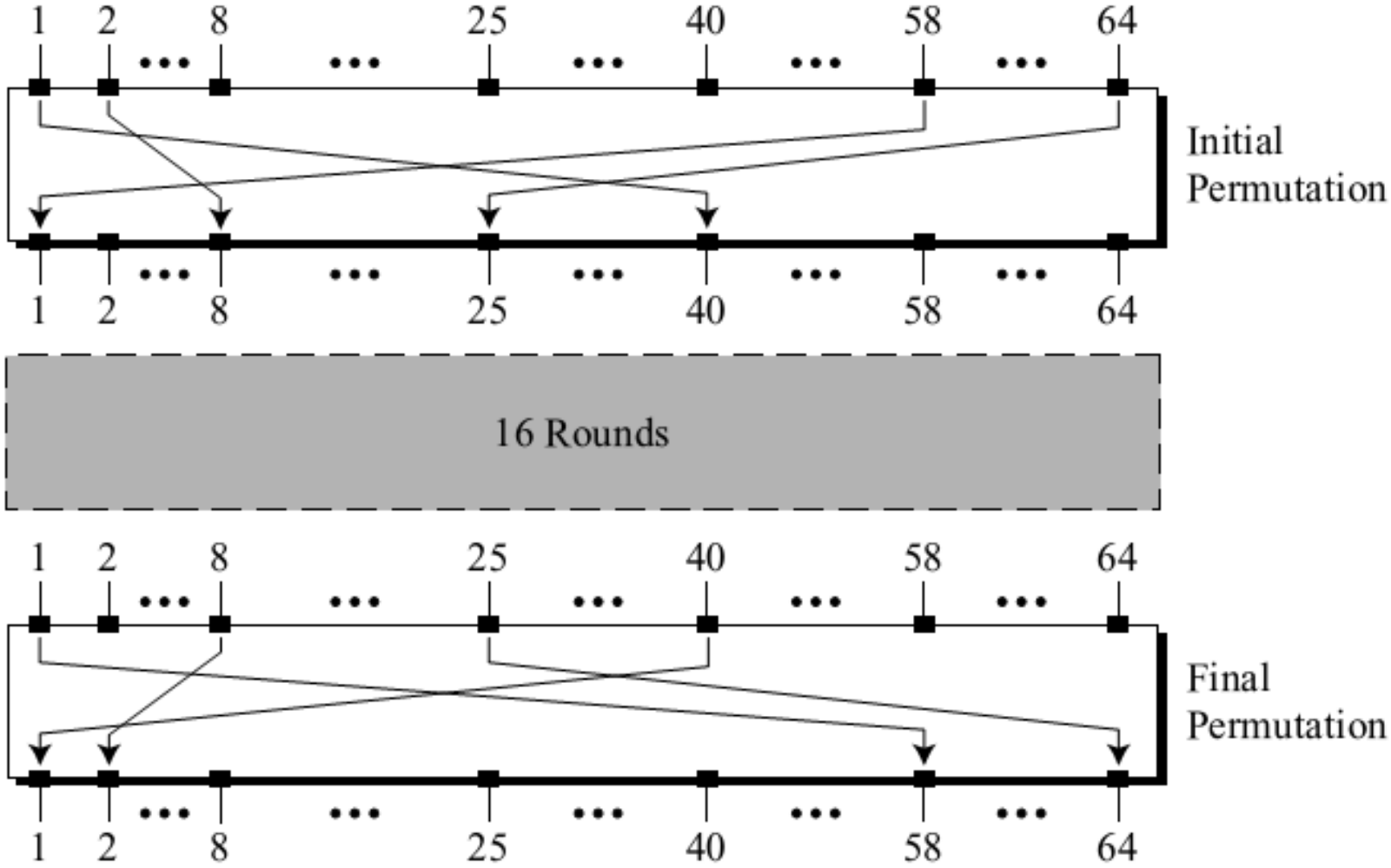


DES STRUCTURE

- The encryption process is made of two permutations (P-boxes), which we call initial and final permutations, and sixteen Feistel rounds.
- Each round uses a different 48-bit round key generated from the cipher key according to a predefined algorithm
- The initial and final permutations are straight P-boxes that are inverses of each other.
- They have no cryptography significance in DES.



Initial and Final Permutations

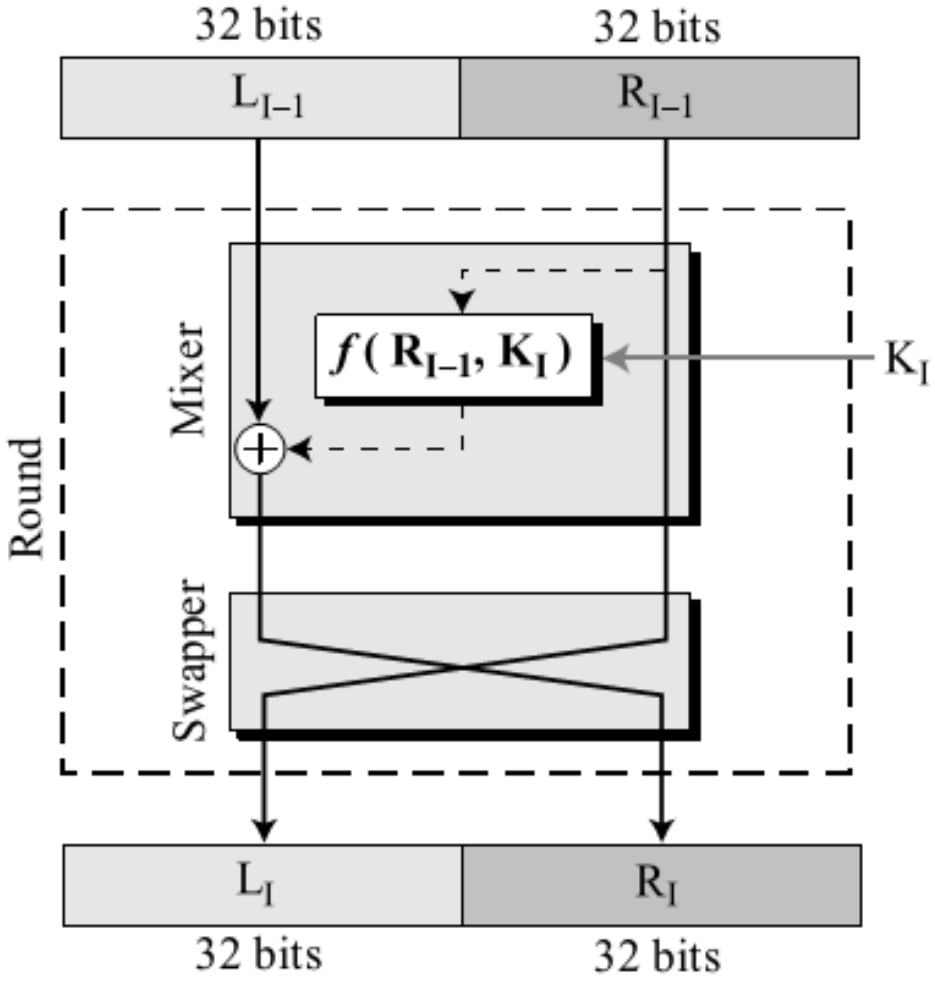


Initial and final permutation tables

Initial Permutation								*Final Permutation*							
58	50	42	34	26	18	10	02	40	08	48	16	56	24	64	32
60	52	44	36	28	20	12	04	39	07	47	15	55	23	63	31
62	54	46	38	30	22	14	06	38	06	46	14	54	22	62	30
64	56	48	40	32	24	16	08	37	05	45	13	53	21	61	29
57	49	41	33	25	17	09	01	36	04	44	12	52	20	60	28
59	51	43	35	27	19	11	03	35	03	43	11	51	19	59	27
61	53	45	37	29	21	13	05	34	02	42	10	50	18	58	26
63	55	47	39	31	23	15	07	33	01	41	09	49	17	57	25

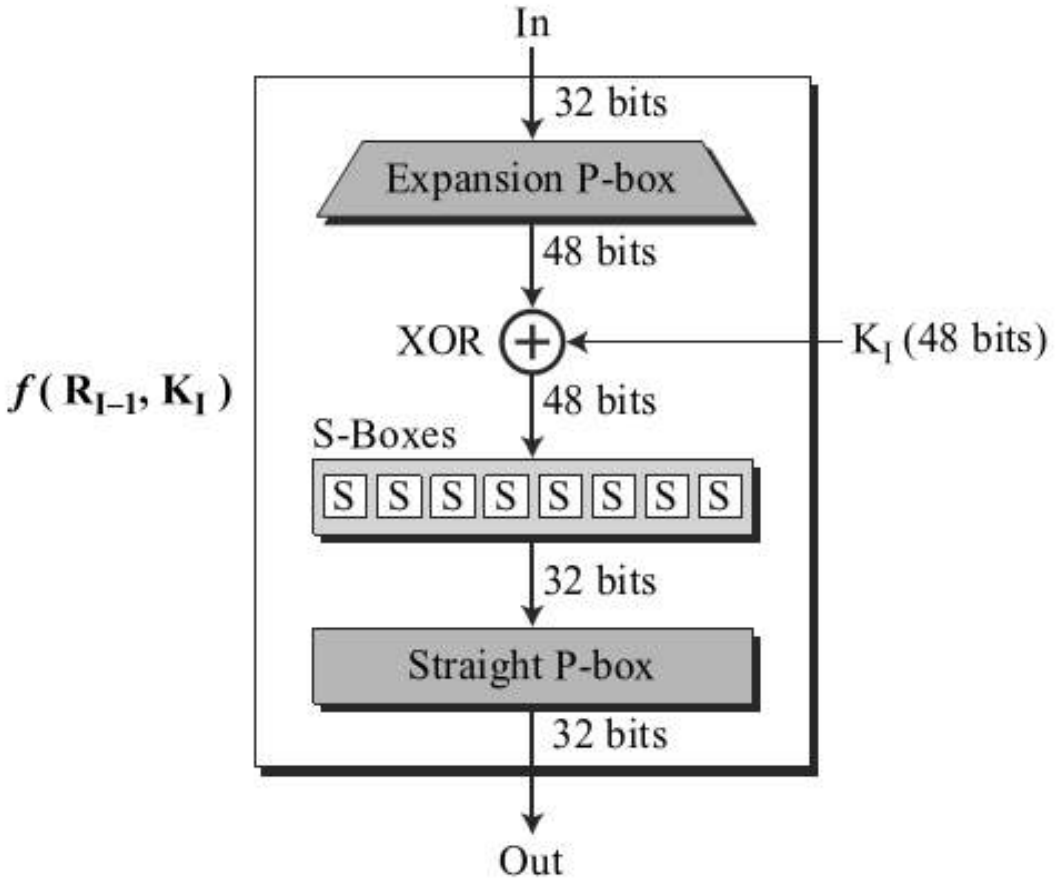
Rounds

- DES uses 16 rounds.
- Each round of DES is a Feistel cipher
- The round takes $L_{I-1}$and R_{I-1} from previous round (or the initial permutation box) and creates L_I and R_I , which go to the next round (or final permutation box).
- The mixer is invertible because of the XOR operation.



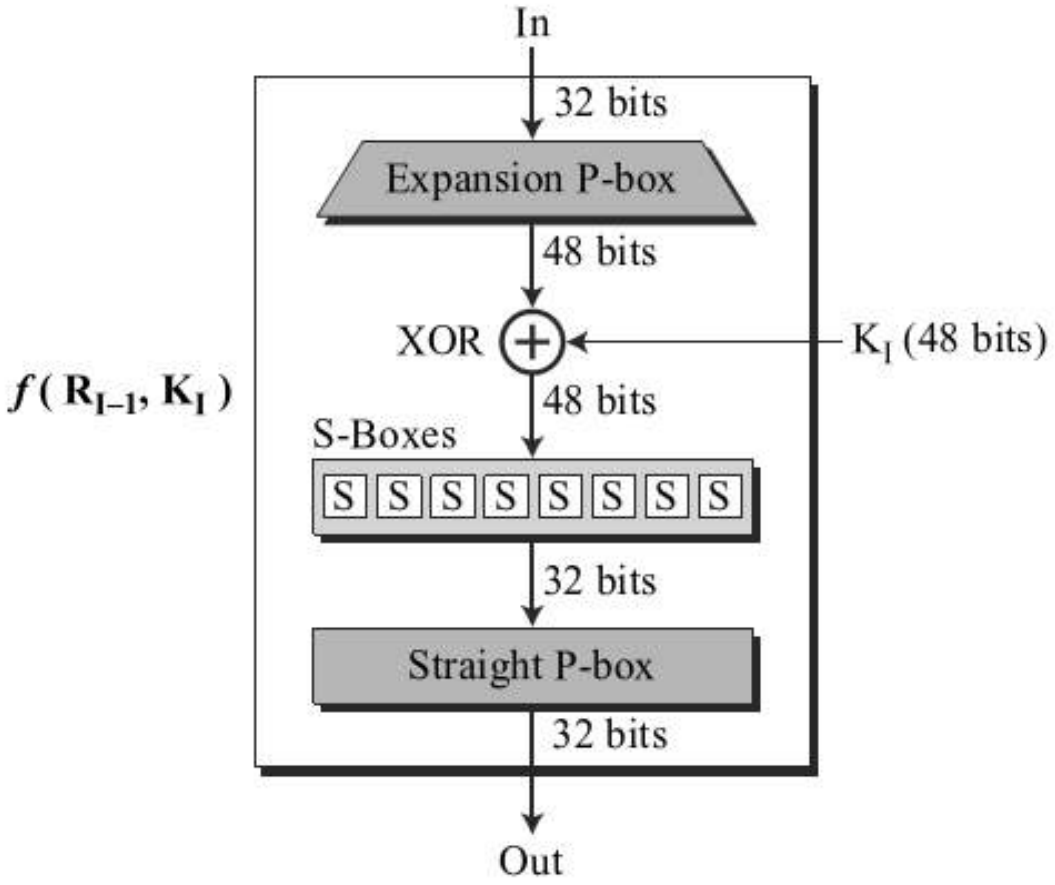
DES Function

- The heart of DES is the DES function.
- The DES function applies a 48-bit key to the rightmost 32 bits (R_{I-1}) to produce a 32-bit output.
- This function is made up of four sections:
  - an expansion P-box,
  - a whitener (that adds key),
  - a group of S-boxes, and
  - a straight P-box

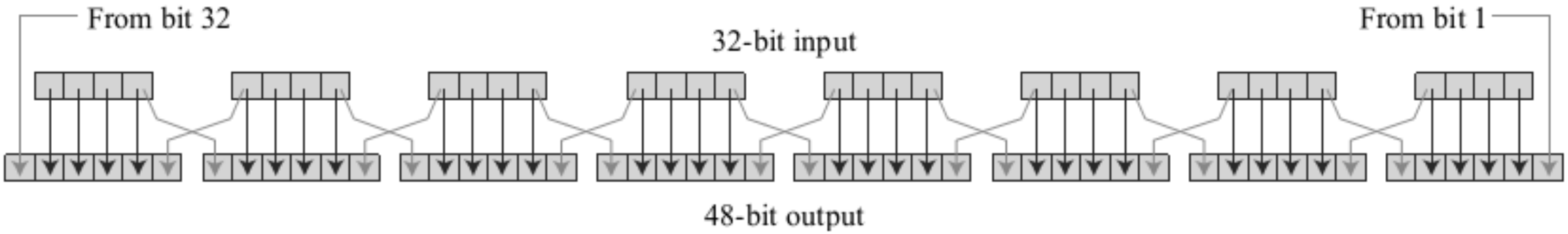


DES Function

- The heart of DES is the DES function.
- The DES function applies a 48-bit key to the rightmost 32 bits (R_{I-1}) to produce a 32-bit output.
- This function is made up of four sections:
  - an expansion P-box,
  - a whitener (that adds key),
  - a group of S-boxes, and
  - a straight P-box



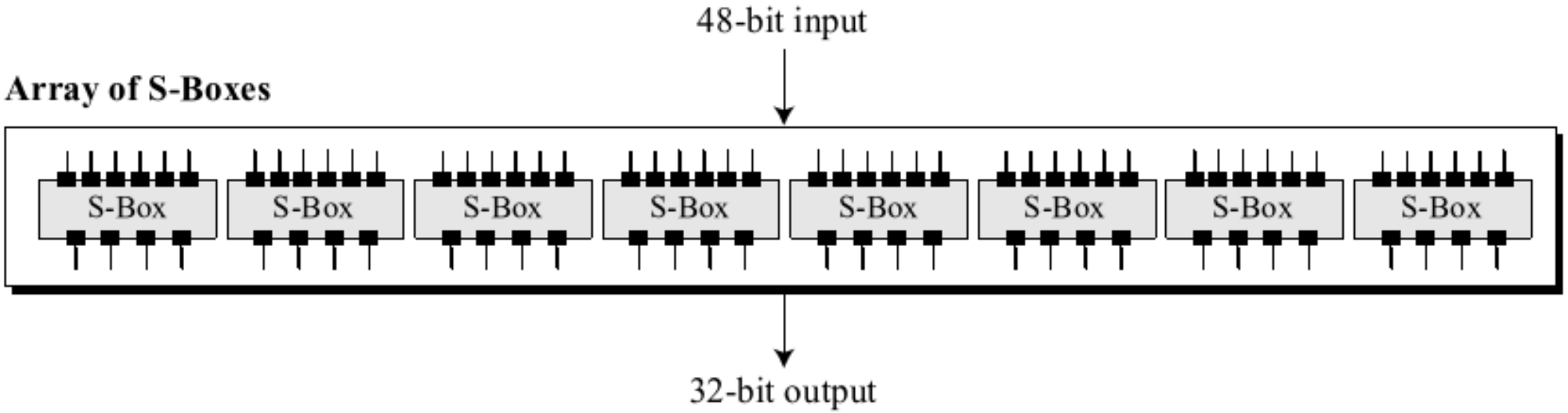
Expansion permutation and Expansion P-box table



32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	01

S-Boxes

The S-boxes do the real mixing (confusion). DES uses 8 S-boxes, each with a 6-bit input and a 4-bit output.



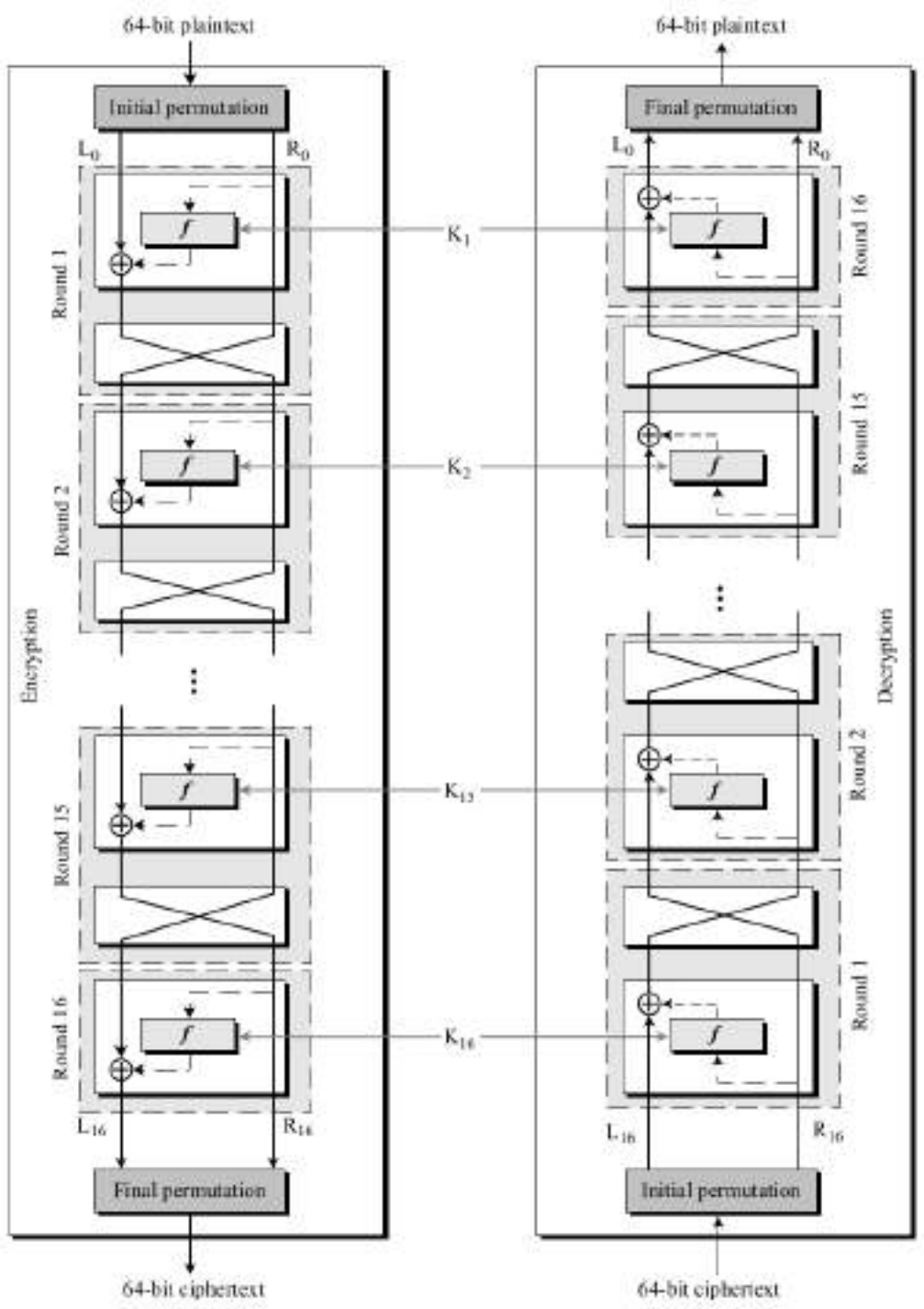
Straight Permutation

- The last operation in the DES function is a straight permutation with a 32-bit input and a 32-bit output
- The seventh bit of the input becomes the second bit of the output

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Cipher and Reverse Cipher

- Using mixers and swappers, we can create the cipher and reverse cipher, each having 16 rounds.
- The cipher is used at the encryption site; the reverse cipher is used at the decryption site.
- The whole idea is to make the cipher and the reverse cipher algorithms similar.
- Many Approaches to Implement DES



DES cipher and reverse cipher for the first approach

Pseudocode for DES cipher

```
Cipher (plainBlock[64], RoundKeys[16, 48], cipherBlock[64])
{
    permute (64, 64, plainBlock, inBlock, InitialPermutationTable)
    split (64, 32, inBlock, leftBlock, rightBlock)
    for (round = 1 to 16)
    {
        mixer (leftBlock, rightBlock, RoundKeys[round])
        if (round!=16) swapper (leftBlock, rightBlock)
    }
    combine (32, 64, leftBlock, rightBlock, outBlock)
    permute (64, 64, outBlock, cipherBlock, FinalPermutationTable)
}
```

Pseudocode for DES cipher

```
mixer (leftBlock[32], rightBlock[32], RoundKey[48])
{
    copy (32, rightBlock, T1)
    function (T1, RoundKey, T2)
    exclusiveOr (32, leftBlock, T2, T3)
    copy (32, T3, rightBlock)
}
```

```
swapper (leftBlock[32], rigthBlock[32])
{
    copy (32, leftBlock, T)
    copy (32, rightBlock, leftBlock)
    copy (32, T, rightBlock)
}
```

Pseudocode for DES cipher

```
function (inBlock[32], RoundKey[48], outBlock[32])
{
    permute (32, 48, inBlock, T1, ExpansionPermutationTable)
    exclusiveOr (48, T1, RoundKey, T2)
    substitute (T2, T3, SubstituteTables)
    permute (32, 32, T3, outBlock, StraightPermutationTable)
}
```

Pseudocode for DES cipher

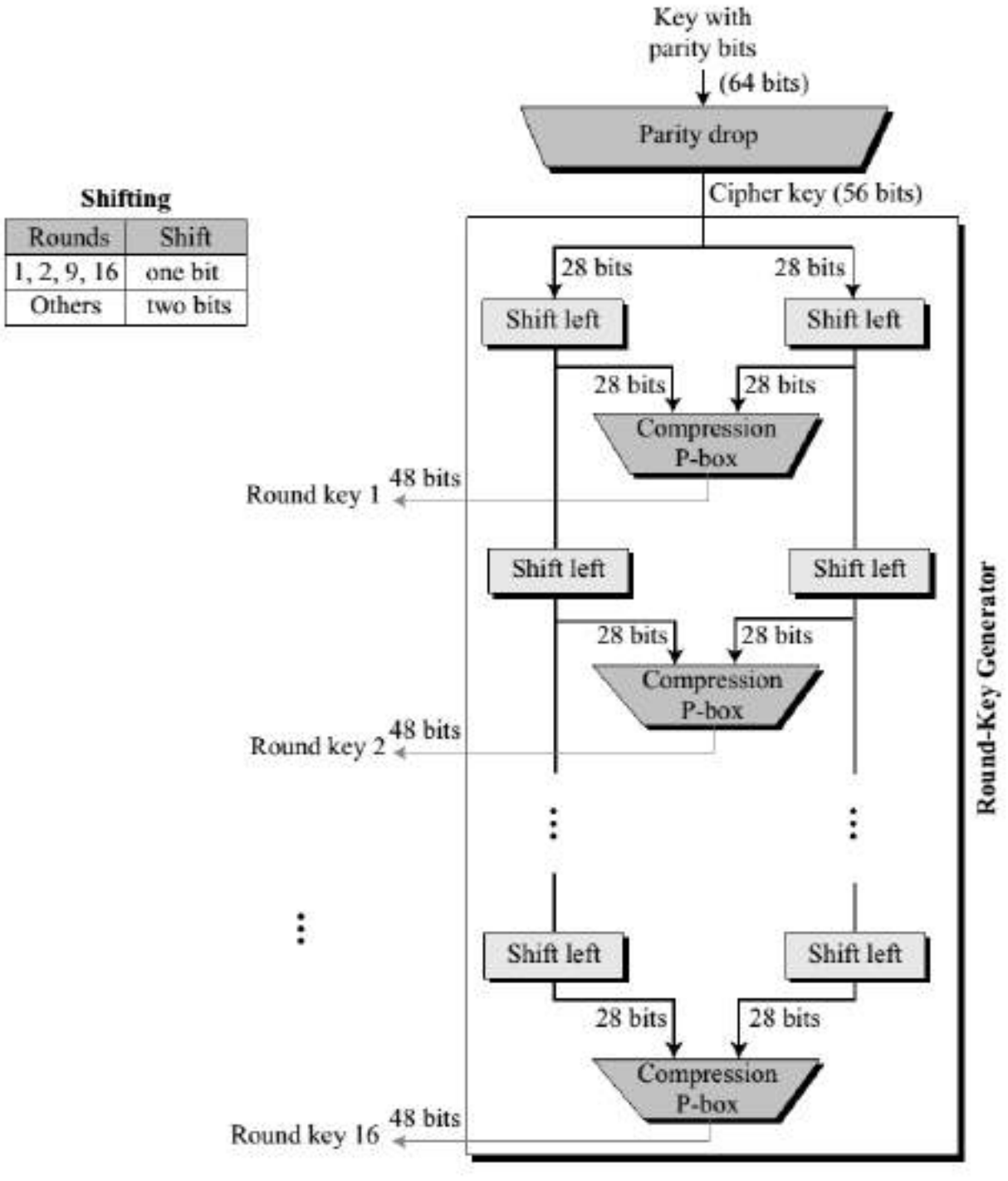
```
substitute (inBlock[32], outBlock[48], SubstitutionTables[8, 4, 16])
{
    for (i = 1 to 8)
    {
        row ← 2 × inBlock[i × 6 + 1] + inBlock [i × 6 + 6]
        col ← 8 × inBlock[i × 6 + 2] + 4 × inBlock[i × 6 + 3] +
              2 × inBlock[i × 6 + 4] + inBlock[i × 6 + 5]

        value = SubstitutionTables [i][row][col]

        outBlock[[i × 4 + 1] ← value / 8;      value ← value mod 8
        outBlock[[i × 4 + 2] ← value / 4;      value ← value mod 4
        outBlock[[i × 4 + 3] ← value / 2;      value ← value mod 2
        outBlock[[i × 4 + 4] ← value
    }
}
```

Key Generation

- The round-key generator creates sixteen 48-bit keys out of a 56-bit cipher key.
- However, the cipher key is normally given as a 64-bit key in which 8 extra bits are the parity bits, which are dropped before the actual key-generation process.



Shifting

Rounds	Shift
1, 2, 9, 16	one bit
Others	two bits

Algorithm for round-keys generation

```
Key_Generator (keyWithParities[64], RoundKeys[16, 48], ShiftTable[16])
{
    permute (64, 56, keyWithParities, cipherKey, ParityDropTable)
    split (56, 28, cipherKey, leftKey, rightKey)
     for (round = 1 to 16)
     {
          shiftLeft (leftKey, ShiftTable[round])
          shiftLeft (rightKey, ShiftTable[round])
          combine (28, 56, leftKey, rightKey, preRoundKey)
          permute (56, 48, preRoundKey, RoundKeys[round], KeyCompressionTable)
     }
}
```

DES ANALYSIS

- Critics have used a strong magnifier to analyze DES.
- Tests have been done to measure the strength of some desired properties in a block cipher.
- Properties
  - Two desired properties of a block cipher are the avalanche effect and the completeness.
- Avalanche Effect
  - Avalanche effect means a small change in the plaintext (or key) should create a significant change in the ciphertext.
  - DES has been proved to be strong with regard to this property.
- Completeness effect
  - Completeness effect means that each bit of the ciphertext needs to depend on many bits on the plaintext.
  - The diffusion and confusion produced by P-boxes and S-boxes in DES, show a very strong completeness effect.

DES Weaknesses

- Weaknesses in Cipher Design
  - S-boxes
    - In S-box 4, the last three output bits can be derived in the same way as the first output bit by complementing some of the input bits.
    - Two specifically chosen inputs to an S-box array can create the same output.
    - It is possible to obtain the same output in a single round by changing bits in only three neighboring S-boxes.
  - P-boxes
    - It is not clear why the designers of DES used the initial and final permutations; these have no security benefits.
    - In the expansion permutation (inside the function), the first and fourth bits of every 4-bit series are repeated.

DES Weaknesses

- Weakness in the Cipher Key
  - DES is in its key size (56 bits). To do a brute-force attack on a given ciphertext block, the adversary needs to check 2^{56} keys.
  - With available technology, it is possible to check one million keys per second.
  - If we can make a computer with one million chips (parallel processing), then we can test the whole key domain in approximately 20 hours.
  - Computer networks can simulate parallel processing. In 1977 a team of researchers used 3500 computers attached to the Internet to find a key challenged by RSA Laboratories in 120 days. The key domain was divided among all of these computers, and each computer was responsible to check the part of the domain.
  - If 3500 networked computers can find the key in 120 days, a secret society with 42,000 members can find the key in 10 days

DES Weaknesses

- Weak Keys
  - Four out of 2^{56} possible keys are called weak keys
  - The round keys created from any of these weak keys are the same and have thesame pattern as the cipher key.

Keys before parities drop (64 bits)	*Actual key (56 bits)*
0101 0101 0101 0101	0000000 0000000
1F1F 1F1F 0E0E 0E0E	0000000 FFFFFFF
E0E0 E0E0 F1F1 F1F1	FFFFFFF 0000000
FEFE FEFE FEFE FEFE	FFFFFFF FFFFFFF

DES Weaknesses

- Semi-weak Keys
  - There are six key pairs that are called semi-weak keys.
  - A semi-weak key creates only two different round keys and each of them is repeated eight times.

First key in the pair	*Second key in the pair*
01FE 01FE 01FE 01FE	FE01 FE01 FE01 FE01
1FE0 1FE0 0EF1 0EF1	E01F E01F F10E F10E
01E0 01E1 01F1 01F1	E001 E001 F101 F101
1FFE 1FFE 0EFE 0EFE	FE1F FE1F FE0E FE0E
011F 011F 010E 010E	1F01 1F01 0E01 0E01
E0FE E0FE F1FE F1FE	FEE0 FEE0 FEF1 FEF1

DES Weaknesses

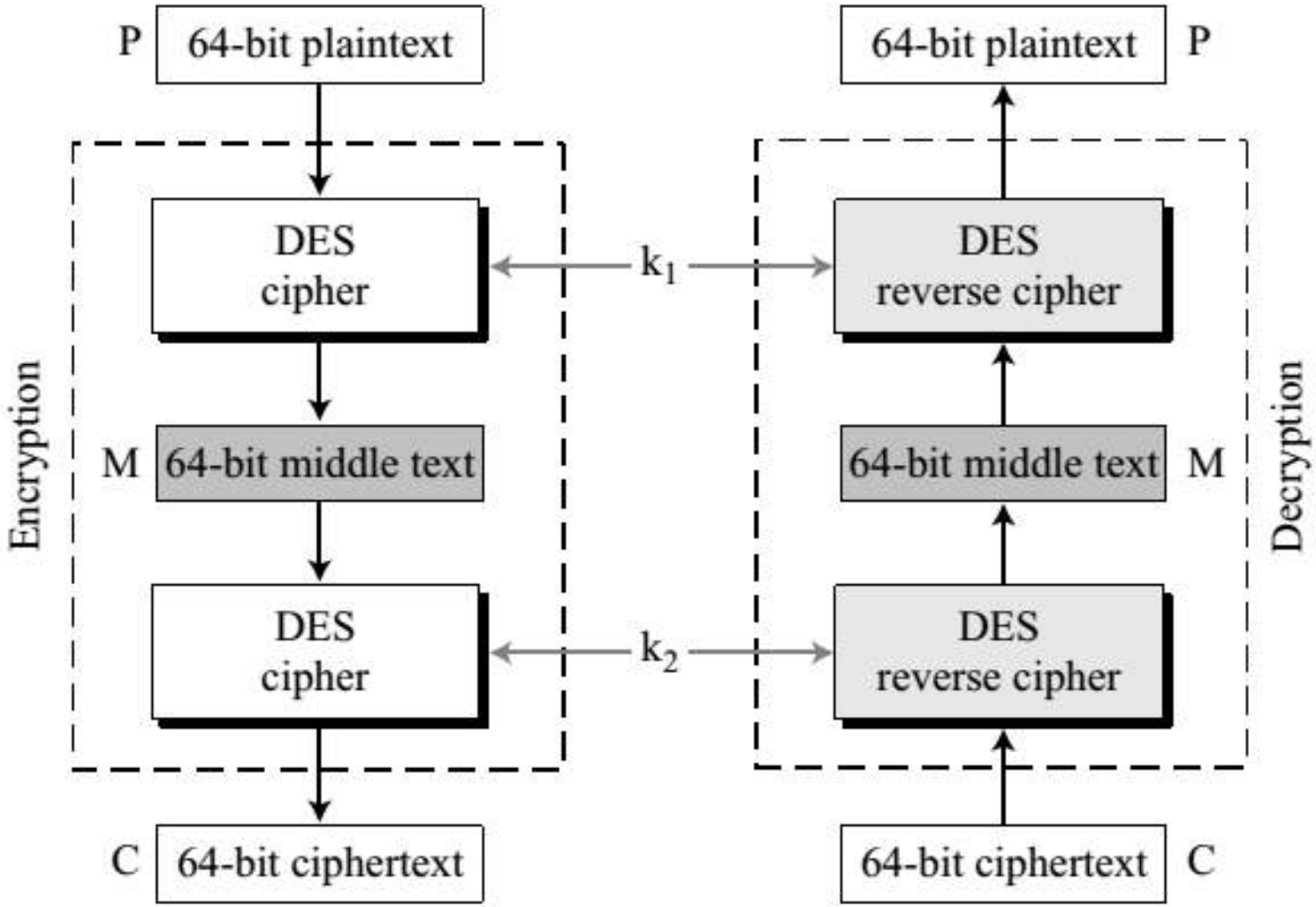
- Semi-weak Keys
  - There are six key pairs that are called semi-weak keys.
  - A semi-weak key creates only two different round keys and each of them is repeated eight times.

First key in the pair	*Second key in the pair*
01FE 01FE 01FE 01FE	FE01 FE01 FE01 FE01
1FE0 1FE0 0EF1 0EF1	E01F E01F F10E F10E
01E0 01E1 01F1 01F1	E001 E001 F101 F101
1FFE 1FFE 0EFE 0EFE	FE1F FE1F FE0E FE0E
011F 011F 010E 010E	1F01 1F01 0E01 0E01
E0FE E0FE F1FE F1FE	FEE0 FEE0 FEF1 FEF1

MULTIPLE DES

- The major criticism of DES regards its key length.
- With available technology and the possibility of parallel processing, a brute-force attack on DES is feasible.
- One solution to improve the security of DES is to abandon DES and design a new cipher, with the advent of AES.
- The second solution is to use multiple (cascaded) instances of DES with multiple keys
  - Double DES
  - Triple DES

Double DES


P
64-bit plaintext
DES cipher
k_1
M
64-bit middle text
DES cipher
k_2
C
64-bit ciphertext
Encryption
64-bit plaintext
P
DES reverse cipher
64-bit middle text
M
DES reverse cipher
64-bit ciphertext
C
Decryption

Meet-in-the-Middle Attack

- Using a known-plaintext attack called meet-in-the-middle attack proves that double DES improves this vulnerability slightly (to 2^{57} tests), but not tremendously (to 2^{112}).

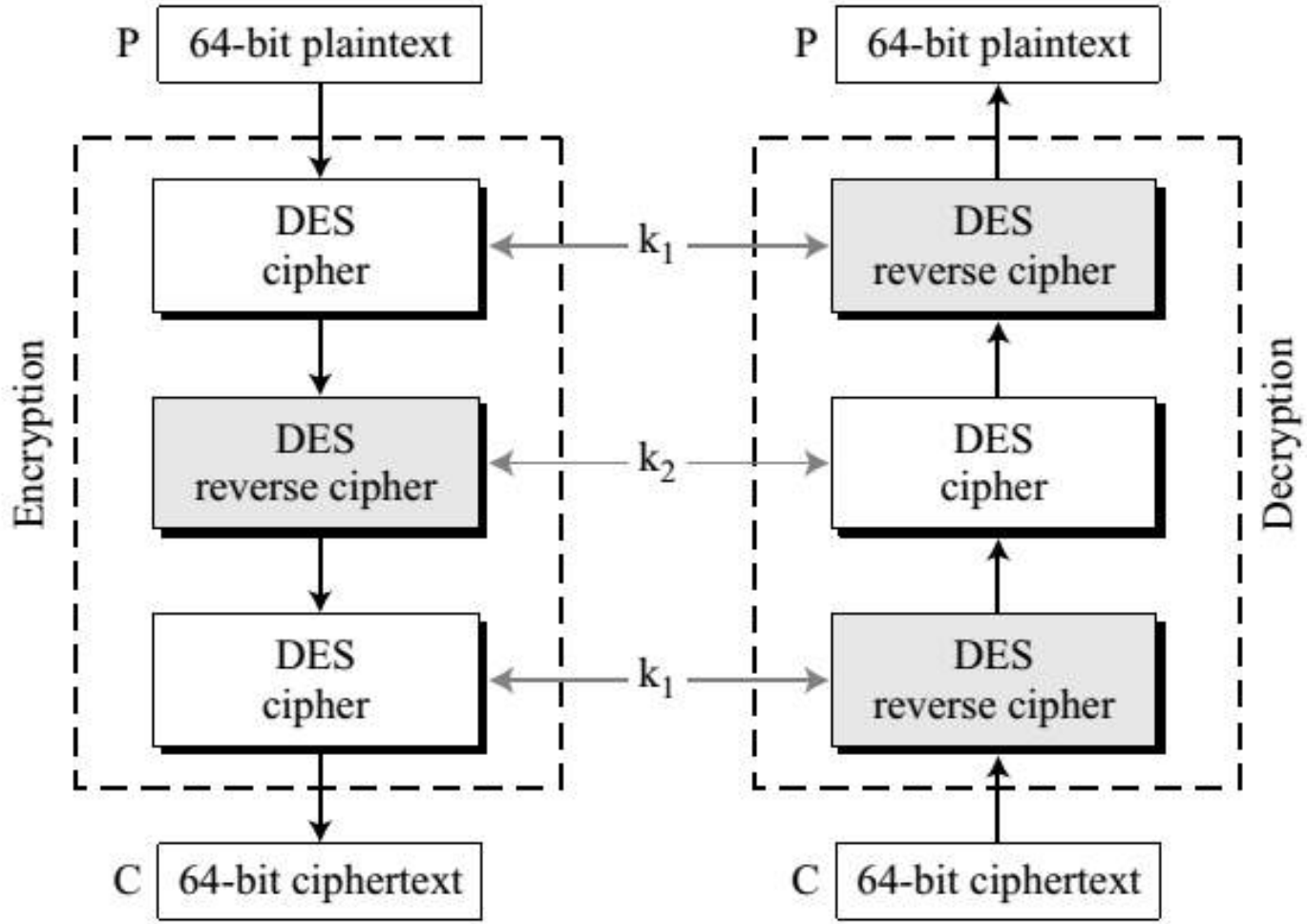
$$M = E_{k_1}(P) \qquad \text{and} \qquad M = D_{k_2}(C)$$

- Eve encrypts P using all possible values (2^{56}) of k1 and records all values obtained for M.
- Eve decrypts C using all possible values (2^{56}) of k2 and records all values obtained for M.
- This means that instead of using 2^{112} key-search tests, Eve uses 2^{56} key-search tests two times

Triple DES

- To improve the security of DES, triple DES (3DES) was proposed.
- This uses three stages of DES for encryption and decryption.
- Two versions of triple DES are in use today:
  - triple DES with two keys and
  - triple DES with three keys.

Triple DES with two keys

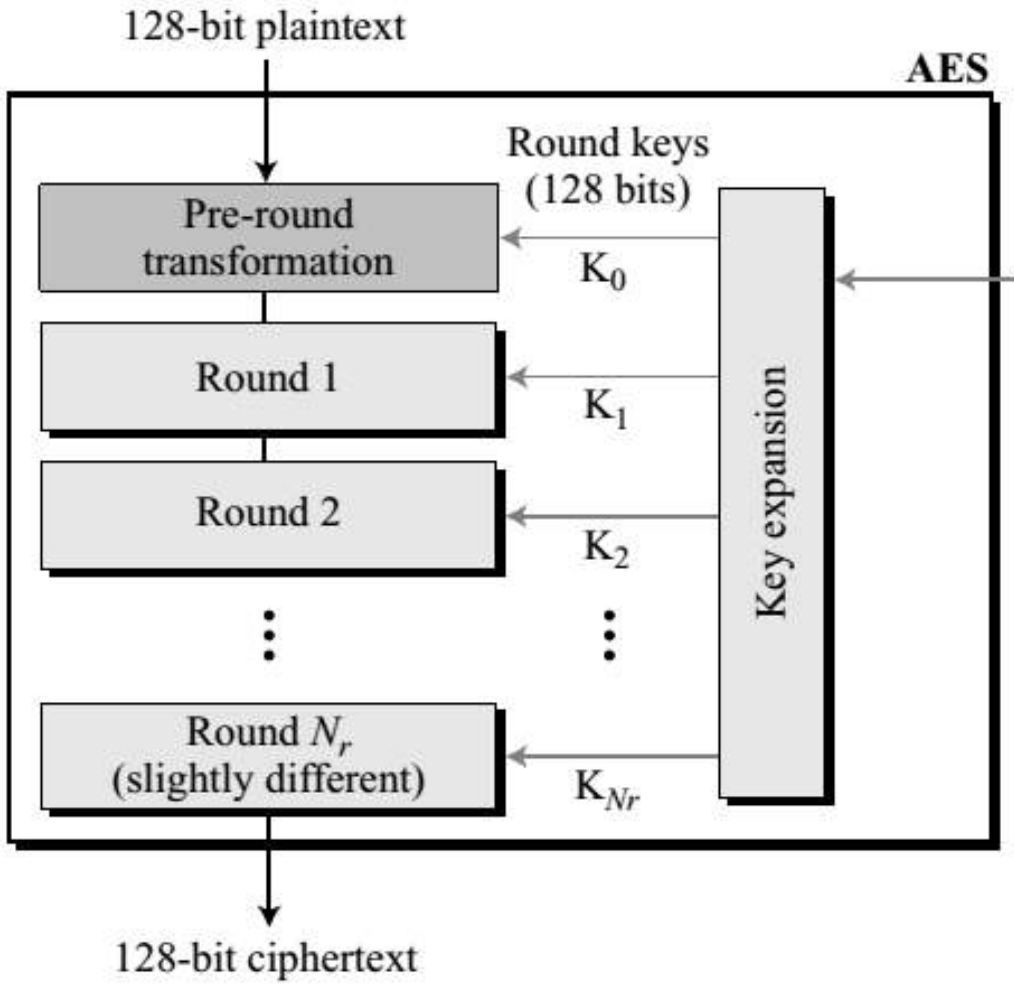


Advanced Encryption Standard (AES)

INTRODUCTION

- The Advanced Encryption Standard (AES) is a symmetric-key block cipher published by the National Institute of Standards and Technology (NIST) in December 2001.
- The criteria defined by NIST for selecting AES fall into three areas: security, cost, and implementation.
- AES has defined three versions, with 10, 12, and 14 rounds.
- Each version uses a different cipher key size (128, 192, or 256), but the round keys are always 128 bits.

General design of AES encryption cipher



Cipher key
(128, 192, or 256 bits)

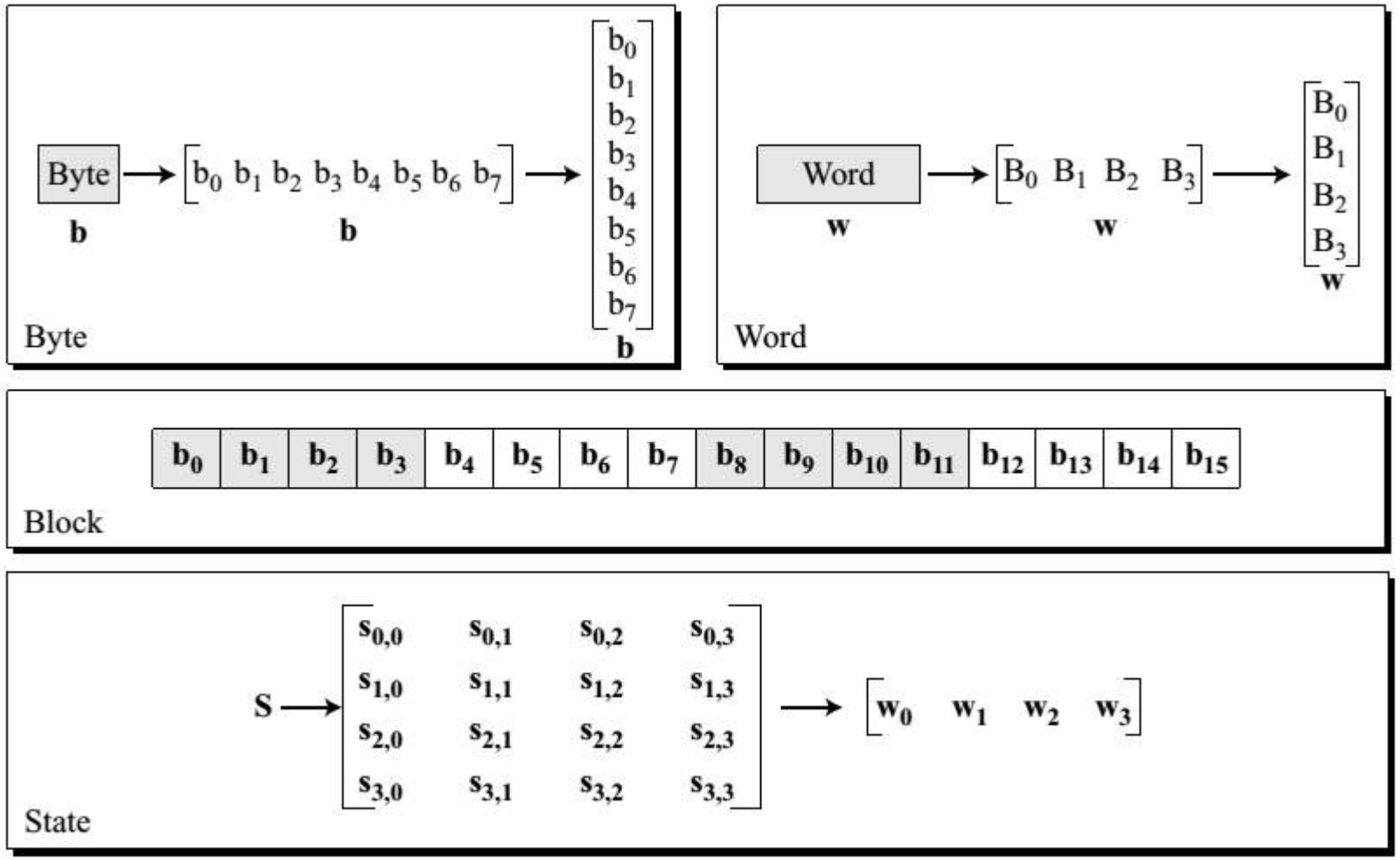
Nr	Key size
10	128
12	192
14	256

Relationship between number of rounds and cipher key size

Number of round keys = $N_r + 1$

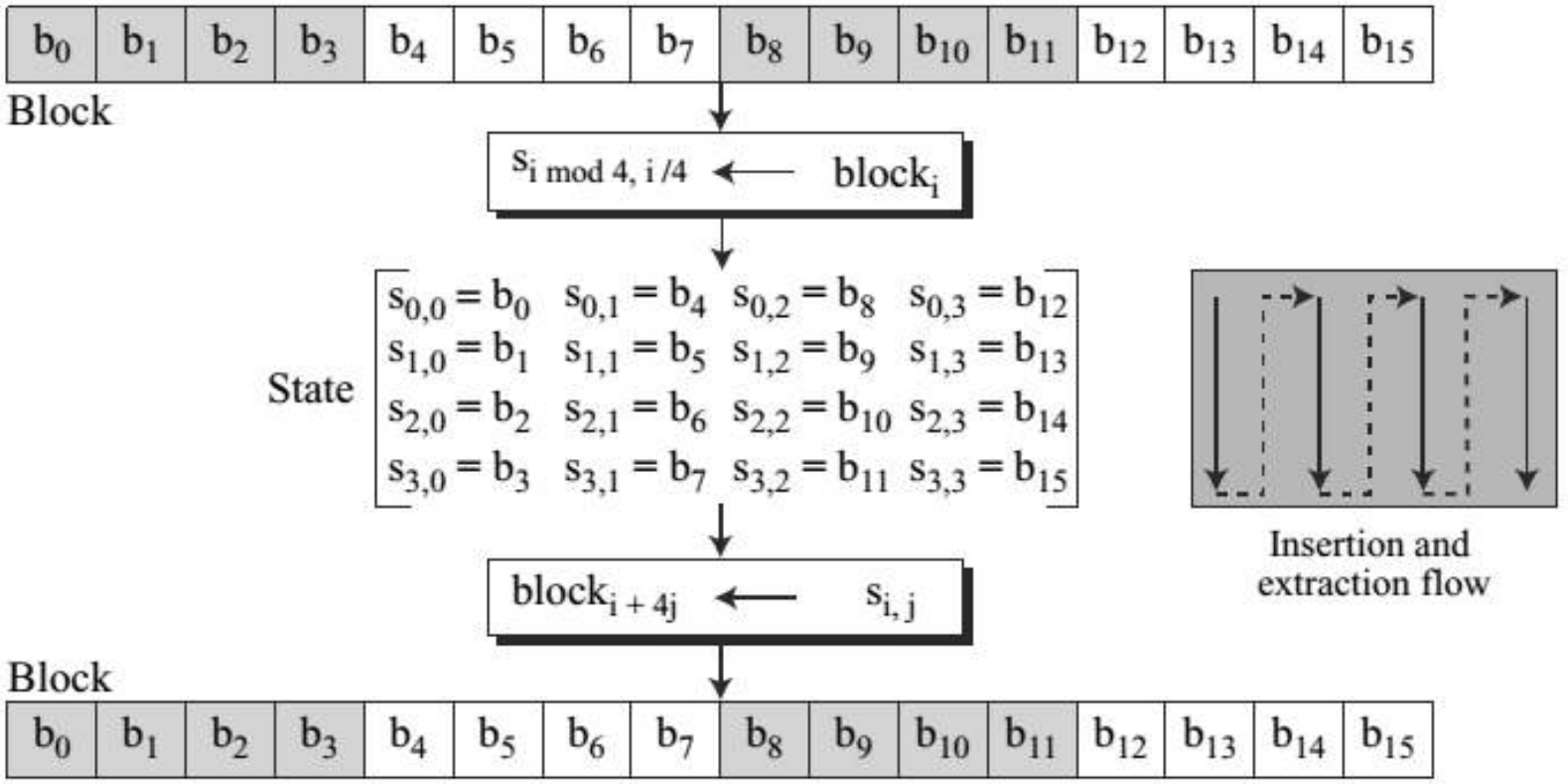
Data Units

- AES uses five units of measurement to refer to data: bits, bytes, words, blocks, and state.
- The bit is the smallest and atomic unit; other units can be expressed in terms of smaller ones.



Block-to-state and state-to-block transformation

- Although the states in different stages are normally called S, we occasionally use the letter T to refer to a temporary state.



Example #1

- Assume that the text block is “AES uses a matrix”.
- We add two bogus characters at the end to get “AESUSESAMATRIXZZ”

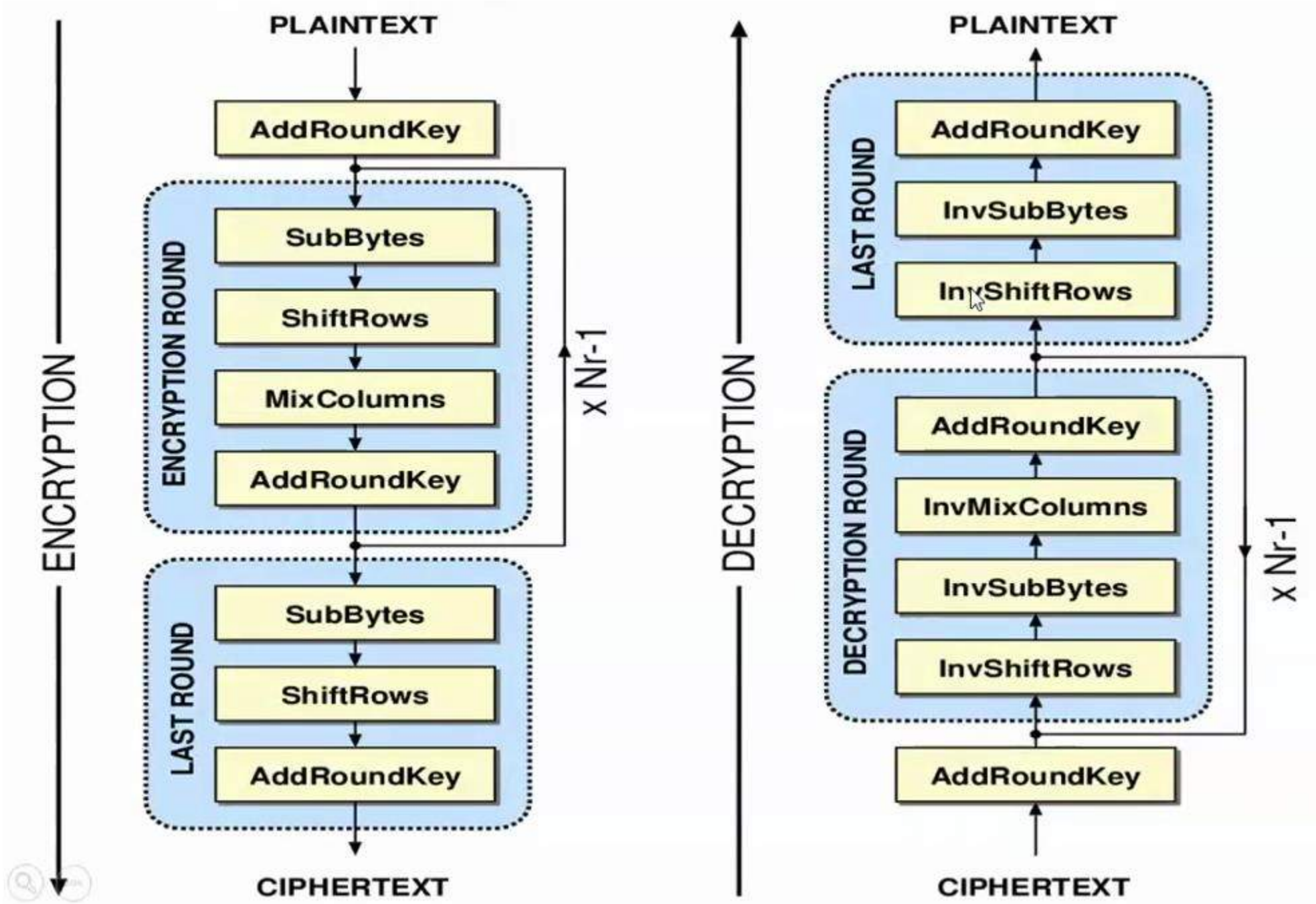
Text	A	E	S	U	S	E	S	A	M	A	T	R	I	X	Z	Z
Hexadecimal	00	04	12	14	12	04	12	00	0C	00	13	11	08	23	19	19

$$\begin{bmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 23 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{bmatrix}$$ State

e

	DEC	HEX		DEC	HEX
A	00	00	N	13	0D
B	01	01	O	14	0E
C	02	02	P	15	0F
D	03	03	Q	16	10
E	04	04	R	17	11
F	05	05	S	18	12
G	06	06	T	19	13
H	07	07	U	20	14
I	08	08	V	21	15
J	09	09	W	22	16
K	10	0A	X	23	17
L	11	0B	Y	24	18
M	12	0C	Z	25	19

AES


ENCRYPTION
PLAINTEXT
AddRoundKey
ENCRYPTION ROUND
SubBytes
ShiftRows
MixColumns
AddRoundKey
x Nr-1
LAST ROUND
SubBytes
ShiftRows
AddRoundKey
CIPHERTEXT
DECRYPTION
PLAINTEXT
LAST ROUND
AddRoundKey
InvSubBytes
InvShiftRows
DECRYPTION ROUND
AddRoundKey
InvMixColumns
InvSubBytes
InvShiftRows
x Nr-1
AddRoundKey
CIPHERTEXT

TRANSFORMATIONS

- To provide security, AES uses four steps for each round:
  - substitution,
  - permutation,
  - mixing, and
  - key-adding.
- The above steps are also called AES Transformation function

Substitution

- First, the substitution is done for each byte.
- Second, only one table is used for transformation of every byte, which means that if two bytes are the same, the transformation is also the same.
- Third, the transformation is defined by either a table lookup process or mathematical calculation in the GF(2^8) field.
- AES uses two invertible transformations.
  - SubBytes
  - InvSubBytes

Substitution :SubBytes

- The SubBytes operation involves 16 independent byte-to-byte transformations.
- Substitution table (S-box) for SubBytes transformation
- For example, two bytes, $5A_{16}$ and $5B_{16}$, which differ only in one bit (the rightmost bit) are transformed to BE_{16} and 39_{16}, which differ in four bits.

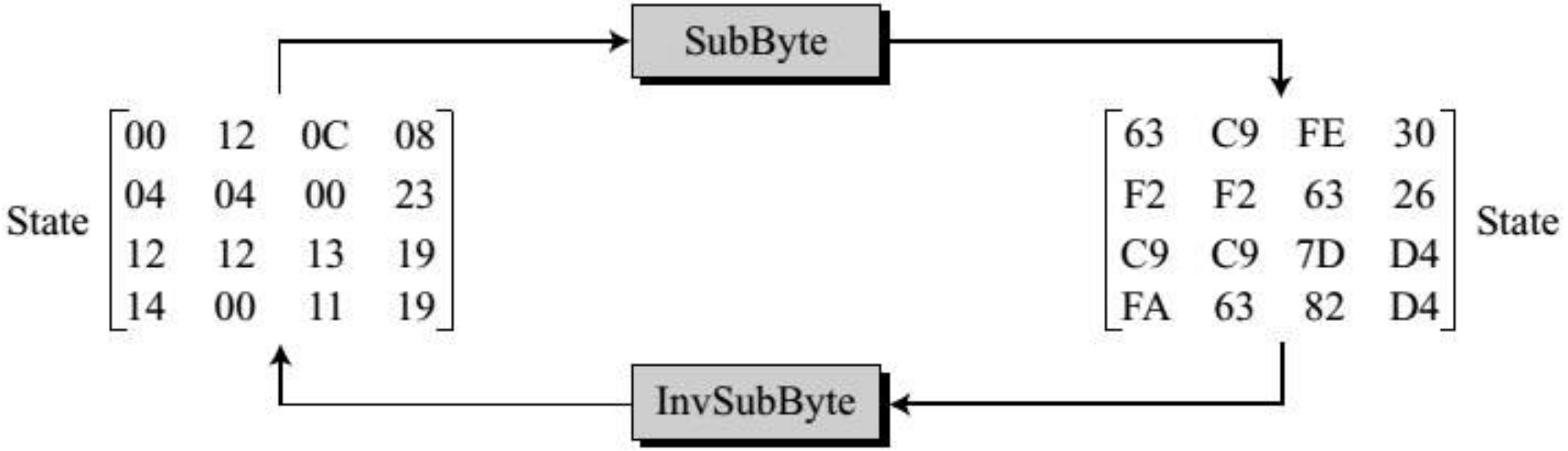
	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*A*	*B*	*C*	*D*	*E*	*F*
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	CB	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Substitution :InvSubBytes

- InvSubBytes is the inverse of SubBytes

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

SubBytes transformation for Example #2



If the two bytes have the same values, their transformation is also the same.
For example, the two bytes 04_{16} and 04_{16} in the left state are transformed to $F2_{16}$ and $F2_{16}$ in the right state and vice versa.

Pseudocode for SubBytes transformation

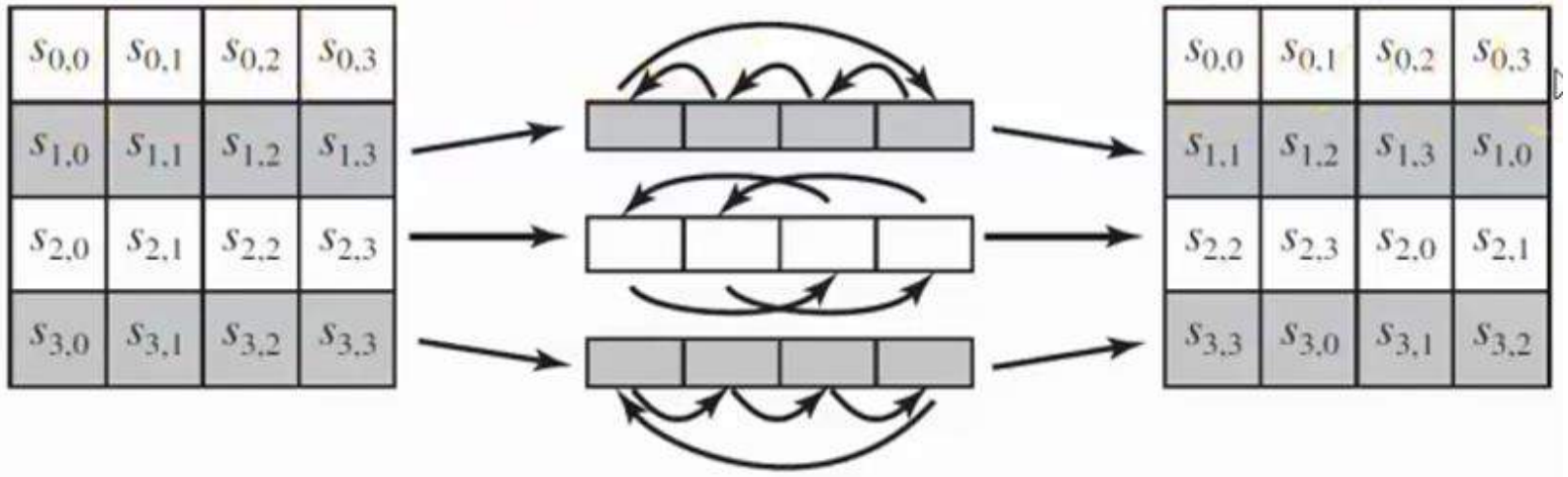
```
SubBytes (S)
{
   for (r = 0 to 3)
      for (c = 0 to 3)
            S_{r,c} = subbyte (S_{r,c})
}
```

```
subbyte (byte)
{
   a ← byte^{-1}                    // Multiplicative inverse in GF(2^8) with inverse of 00 to be 00
    ByteToMatrix (a, b)
    for (i = 0 to 7)
    {
        c_i ← b_i ⊕ b_{(i+4)mod 8} ⊕ b_{(i+5)mod 8} ⊕ b_{(i+6)mod 8} ⊕ b_{(i+7)mod 8}
        d_i ← c_i ⊕ ByteToMatrix (0x63)
   }
    MatrixToByte (d, d)
    byte ← d
}
```

Permutation

- Another transformation found in a round is shifting, which permutes the bytes.
- Unlike DES, in which permutation is done at the bit level, shifting transformation in AES is done at the byte level; the order of the bits in the byte is not changed.
- The ShiftRows and InvShiftRows transformations are inverses of each other

- Rules of shifting rows,
  - Row 1 → No Shifting
  - Row 2 → 1 byte left shift
  - Row 3 → 2 byte left shift
  - Row 4 → 3 byte left shift

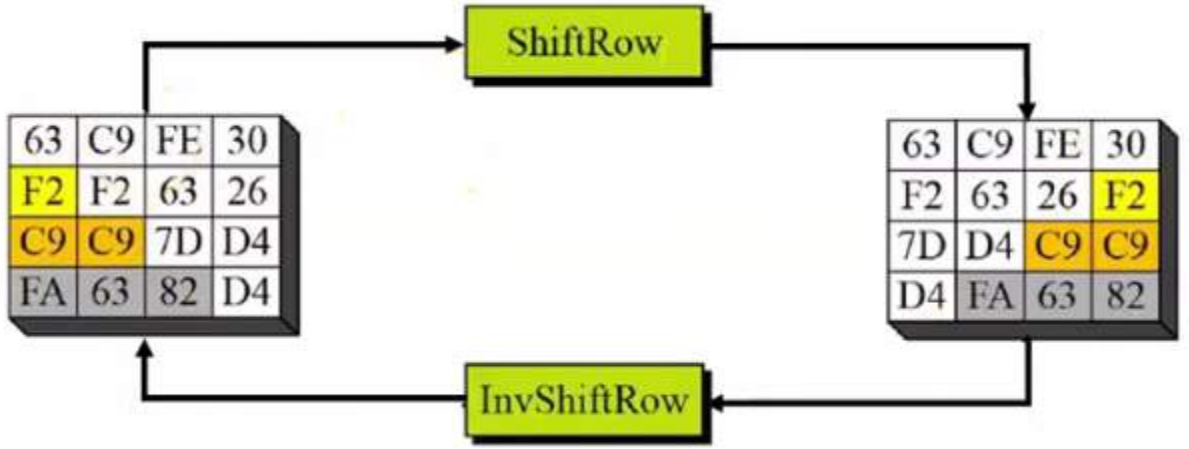


Permutation : Shift Rows

```
ShiftRows (S)
{
    for (r = 1 to 3)
        shiftrow (s_r, r)                    // s_r is the rth row
}
```

```
shiftrow (row, n)                            // n is the number of bytes to be shifted
{
   CopyRow (row, t)                          // t is a temporary row
   for (c = 0 to 3)
        row_((c – n) mod 4) ← t_c
}
```

Example #3

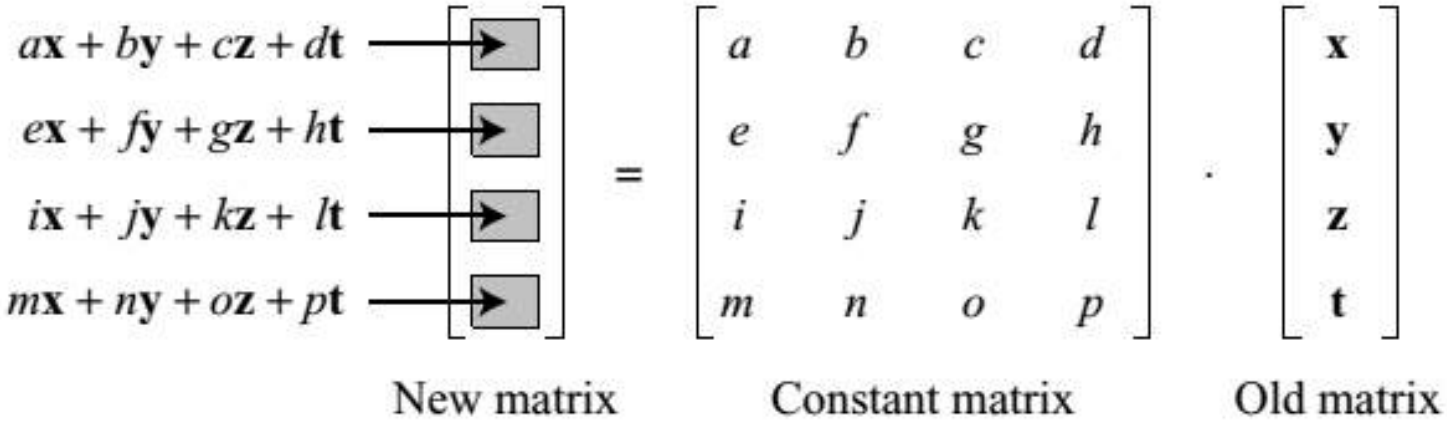


Mixing

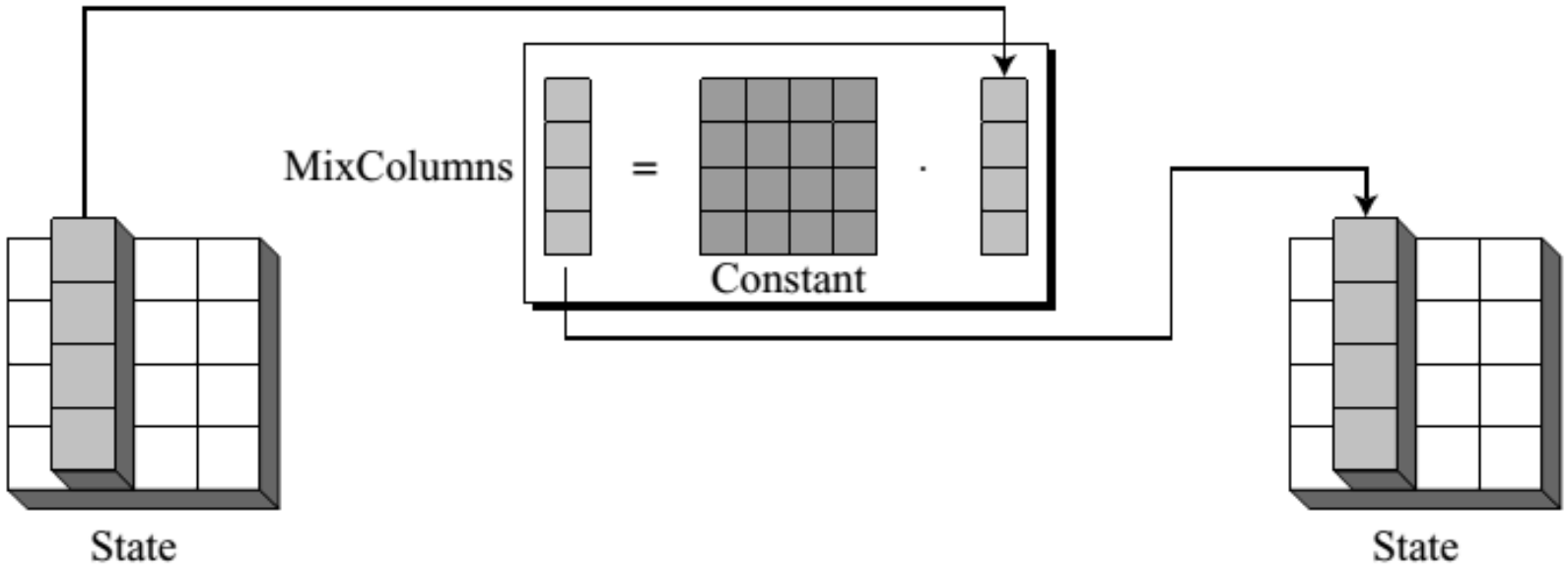
- The substitution provided by the SubBytes transformation changes the value of the byte based only on original value and an entry in the table;
  - The process does not include the neighboring bytes.
  - We can say that SubBytes is an intrabyte transformation.
- The permutation provided by the ShiftRows transformation exchanges bytes without permuting the bits inside the bytes.
  - We can say that ShiftRows is a byte-exchange transformation.
- We also need an interbyte transformation that changes the bits inside a byte, based on the bits inside the neighboring bytes.
- We need to mix bytes to provide diffusion at the bit level.

Mixing

- The mixing transformation changes the contents of each byte by taking four bytes at a time and combining them to recreate four new bytes.
- To guarantee that each new byte is different (even if all four bytes are the same), the combination process first multiplies each byte with a different constant and then mixes them.
- The mixing can be provided by matrix multiplication.



MixColumns & InvMixColumns



$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \quad \text{Inverse} \quad \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix}$$

C $\qquad$ C^{-1}

Pseudocode for MixColumns transformation

```
MixColumns (S)
{
    for (c = 0 to 3)
        mixcolumn (s_c)
}
```

```
mixcolumn (col)
{
   CopyColumn (col, t)                    // t is a temporary column

   col_0 ← (0x02) • t_0 ⊕ (0x03 • t_1) ⊕ t_2 ⊕ t_3

   col_1 ← t_0 ⊕ (0x02) • t_1 ⊕ (0x03) • t_2 ⊕ t_3

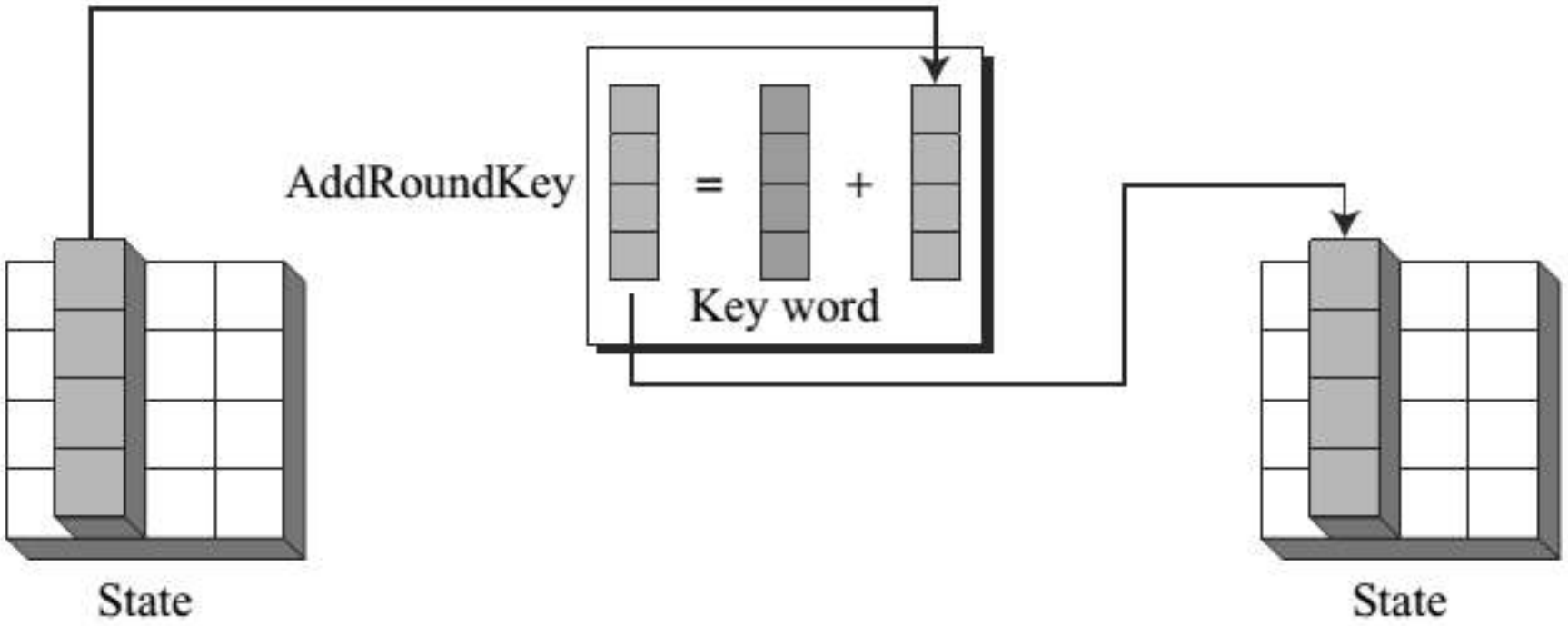
   col_2 ← t_0 ⊕ t_1 ⊕ (0x02) • t_2 ⊕ (0x03) • t_3

   col_3 ← (0x03 • t_0) ⊕ t_1 ⊕ t_2 ⊕ (0x02) • t_3
}
```

Key Adding: AddRoundKey

- AddRoundKey also proceeds one column at a time.
- It is similar to MixColumns in this respect.
- MixColumns multiplies a constant square matrix by each state column;
- AddRoundKey adds a round key word with each state column matrix.
- The operation in MixColumns is matrix multiplication; the operation in AddRoundKey is matrix addition.
- Since addition and subtraction in this field are the same, the AddRoundKey transformation is the inverse of itself.

AddRoundKey transformation

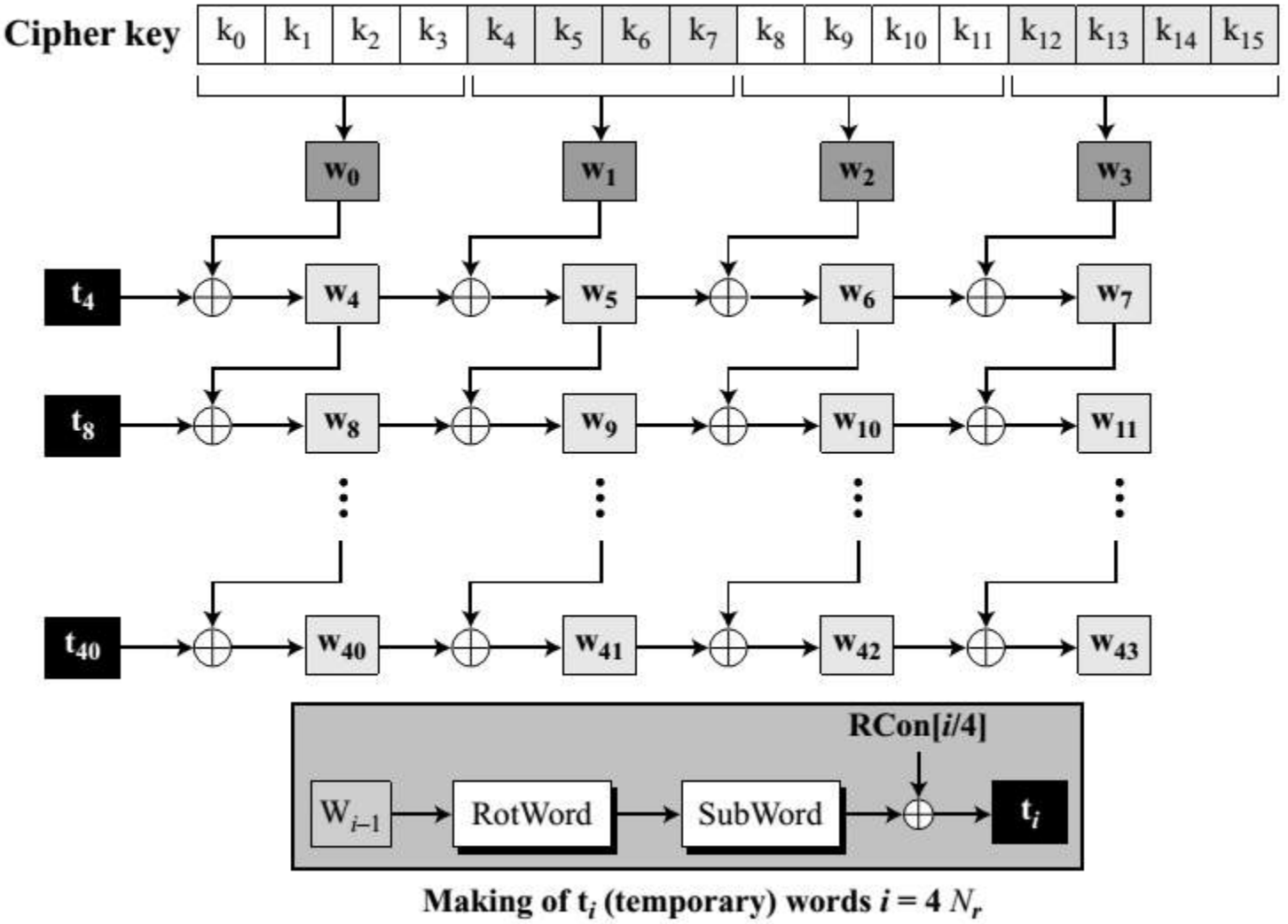


Pseudocode for AddRoundKey transformation

```
AddRoundKey (S)
{
    for (c = 0 to 3)
        s_c ← s_c ⊕ w_4 round + c
}
```

KEY EXPANSION

- In the AES-128 version (10 rounds), there are 44 words;
- In the AES-192 version (12 rounds), there are 52 words; and
- In the AES-256 version (with 14 rounds), there are 60 words.
- Each round key is made of four words.

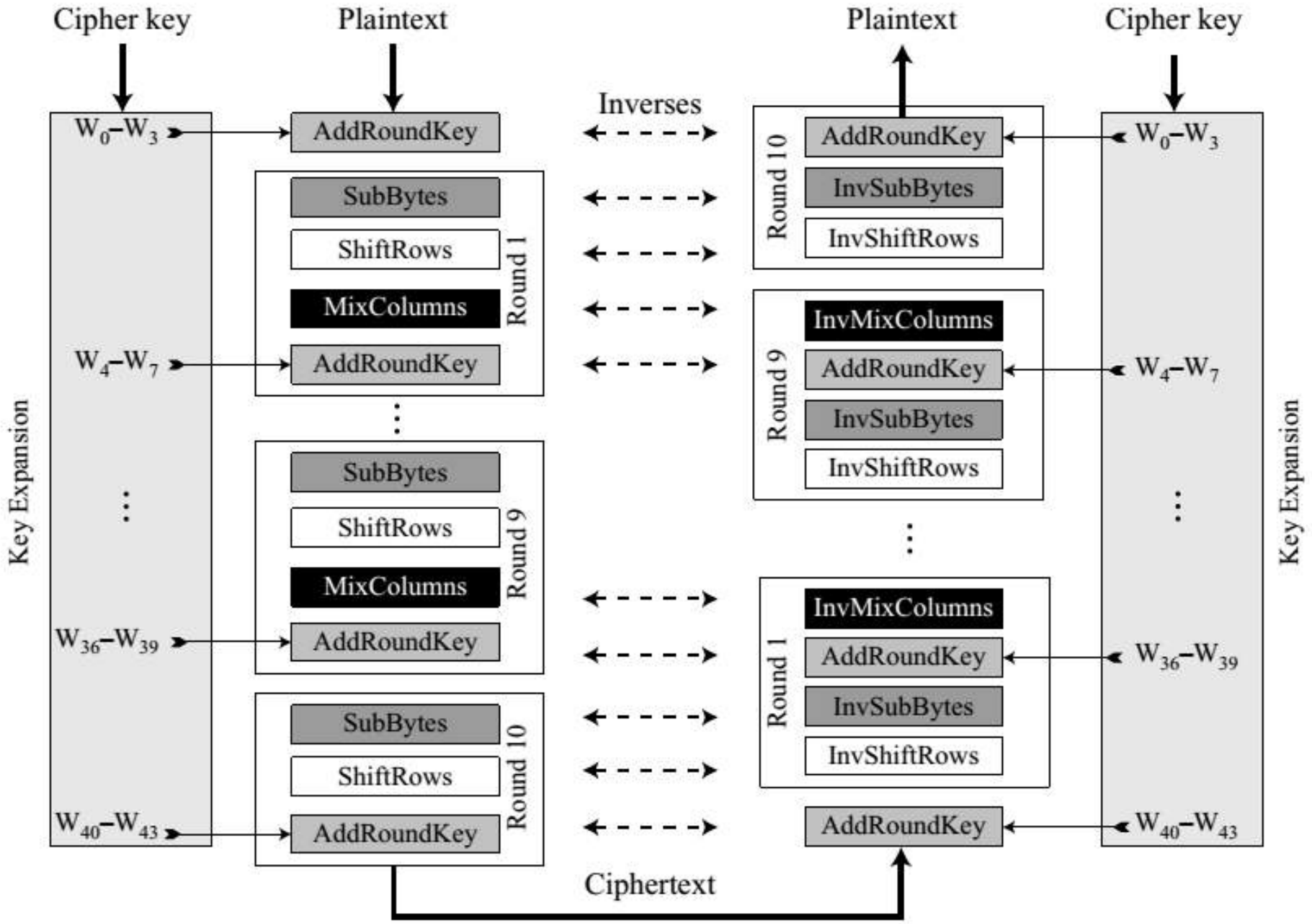


Making of t_i (temporary) words $i = 4\ N_r$

Pseudocode for key expansion in AES-128

```
KeyExpansion ([key0 to key15], [w0 to w43])
{
    for (i = 0 to 3)
        wi ← key4i + key4i+1 + key4i+2 + key4i+3

    for (i = 4 to 43)
    {
        if (i mod 4 ≠ 0)    wi ← wi–1 + wi–4
        else
        {
            t ← SubWord (RotWord (wi–1)) ⊕ RConi/4        // t is a temporary word
            wi ← t + wi–4
        }
    }
}
```



Cipher and inverse cipher of the original design

```
Cipher (InBlock [16], OutBlock[16], w[0 … 43])
{
    BlockToState (InBlock, S)

    S ← AddRoundKey (S, w[0…3])
     for (round = 1 to 10)
     {
          S ← SubBytes (S)
          S ← ShiftRows (S)
           if (round ≠ 10)   S ← MixColumns (S)
          S ← AddRoundKey (S, w[4 × round, 4 × round + 3])
      }

   StateToBlock (S, OutBlock);
}
```

Pseudocode for cipher in the original design

ANALYSIS OF AES

- Security
  - AES was designed after DES. Most of the known attacks on DES were already tested on AES; none of them has broken the security of AES so far.
- Brute-Force Attack
  - For DES we need 2^{56}(ignoring the key complement issue) tests to find the key;
  - For AES we need 2^{128} tests to find the key.
  - This means that if we can break DES in t seconds, we need ($2^{72} \times t$) seconds to break AES. This would be almost impossible.
  - AES provides two other versions with longer cipher keys. The lack of weak keys is another advantage of AES over DES.

ANALYSIS OF AES

- Statistical Attacks
  - The strong diffusion and confusion provided by the combination of the SubBytes, ShiftRows, and MixColumns transformations removes any frequency pattern in the plaintext.
  - Numerous tests have failed to do statistical analysis of the ciphertext.
- Differential and Linear Attacks
  - AES was designed after DES.
  - Differential and linear cryptanalysis attacks were no doubt taken into consideration.
  - There are no differential and linear attacks on AES as yet.
- Simplicity and Cost
  - The algorithms used in AES are so simple that they can be easily implemented using cheap processors and a minimum amount of memory.